

All Chapter Summaries (Condensed)

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Contents

Preface	4
Introduction	4
1 Chapter 1 — Reinforcement Learning Basics	7
1.1 How Reinforcement Learning Became a Discipline	7
1.2 Core Principles — Agent, Action, Environment	7
1.3 The MDP Formulation	8
1.4 Information Sets — When You Cannot See Everything	8
1.5 Dynamic Programming — When You Have the Model	9
1.6 Temporal-Difference Learning — Learning Without a Model	9
1.7 DQN — Deep Q-Networks	10
1.8 PPO — Proximal Policy Optimization	10
1.9 Practical Validation — Custom Implementations vs Stable-Baselines3	11
2 Chapter 2 — Game Theory & CFR Basics	14
2.1 From Single-Agent RL to Multi-Agent Strategic Interaction	14
2.2 Extensive-Form Games and Information Sets	14
2.3 Minimax Theorem	14
2.4 Nash Equilibrium	15
2.5 Regret Matching — The Building Block	15
2.6 Counterfactual Regret Minimization (CFR)	16
2.7 The Mathematics of Poker	17
2.8 Kuhn Poker — The Simplest Testbed	17
2.9 Exploitability — Measuring Distance from Nash	18
2.10 Connections to Chapter 1 and Forward Pointers	19
2.11 Empirical Visualizations	19
3 Chapter 3 — CFR Variants & Monte Carlo Methods	23
3.1 Why Vanilla CFR Breaks at Scale	23
3.2 Monte Carlo Tree Search as Inspiration	23
3.3 Markov Chains and the Law of Large Numbers	23
3.4 Leduc Poker — Scaling Up the Benchmark	24
3.5 CFR+ — The Regret Flooring Trick	25
3.6 MCCFR — Sampling the Game Tree	27
3.7 The Variance-Speed Tradeoff	29
3.8 When MCCFR Wins — The Crossover Point	32
3.9 Connections to Chapter 2 and Forward Pointers	34
4 Chapter 4 — Game Abstraction & Scaling Imperfect-Information Games	35
4.1 Why Abstraction Is Needed	35
4.2 Routes to Abstraction	35
4.3 Axes of Abstraction	37
4.4 The Exploitability Gap	37
4.5 Earth Mover’s Distance: A Primer	38
4.6 The Merging Criterion	38
4.7 The Error Budget	39
4.8 Choosing an Abstraction in Practice	40
4.9 Build-Time Decisions	40
4.10 Runtime Patching	41

4.11	Architecture: Blueprint + Live Patches	42
4.12	Practical Validation	43
4.13	Connections and Forward Pointers	45
5	Chapter 5 — Neural Networks for Imperfect-Information Games	47
	Why Neural Networks	47
5.1	Fundamentals	47
5.2	Architecture & Composition	48
5.3	Neural Networks in CFR	54
5.4	Other Neural Applications	59
6	Chapter 6 — End-to-End Game AI Architectures	61
6.1	Introduction	61
6.2	DeepStack (2017)	62
6.3	Libratus (2017/2018)	67
6.4	Pluribus (2019)	72
6.5	ReBeL (2020)	78
6.6	Student of Games / SoG (2023)	84
6.7	Synthesis	91
7	Chapter 7 — Opponent Modeling in Imperfect-Information Games	96
7.1	Why Opponent Modeling	96
7.2	The Bayesian Core — belief, evidence, response	98
7.3	Seeing Through a Keyhole — the partial-observability problem	100
7.4	Three Models, One Interface	101
7.5	The Consistency Problem and the Sequence-Form Fix	102
7.6	When a Model Is Confident and Wrong	104
7.7	From Model to Money — best response, the ceiling, and a self-inflicted leak	105
7.8	Adapting to Change — non-stationary opponents	107
7.9	Connections and Forward Pointers	109
8	Chapter 8 — Safe Exploitation in Imperfect-Information Games	110
8.1	Why Safe Exploitation — the other half of the dial	110
8.2	Exploitation as Constrained Optimization	111
8.3	What “Safe” Means — three definitions, in increasing realism	113
8.4	Sequence-Form LP and Constraint Generation	115
8.5	Restricted Nash Response and the Bang-Bang Switch	115
8.6	Best Equilibrium on Kuhn — safe, but only a little more profitable (the core result)	117
8.7	Real-Time Safety — subgame gadgets (SES)	119
8.8	At Scale — Kuhn works; Leduc breaks globally, holds locally	120
8.9	Connections and Forward Pointers	123
9	Chapter 9 — Multi-Agent Reinforcement Learning: Coordination, Competition, and Communication	124
9.1	Why multi-agent RL is a different problem	124
9.2	Markov games — the bridge from Chapters 2–8 to MARL	125
9.3	The family of methods	126
9.4	Independent learning and its failure modes	128
9.5	Centralized training, decentralized execution (CTDE)	131
9.6	PSRO — game theory over a population of policies	134
9.7	Learned communication (CommNet)	138

9.8	LOLA — modeling the opponent as a learner	139
9.9	Honest notes, limitations, and where this hands off	140
9.10	Key takeaways for the thesis synthesis	141
10	Chapter 10 — Population-Based Training and Evolutionary Game Theory	142
10.1	Why populations — and why structure decides everything	142
10.2	The family of methods	142
10.3	Replicator dynamics — where a population can rest	143
10.4	The spinning top — transitive vs cyclic structure	146
10.5	The AlphaStar-style PBT league	148
10.6	Diversity — is the population actually diverse?	150
10.7	EGTA — evaluating the population, and a surprise	151
10.8	Honest notes, limitations, and where this hands off	154
10.9	Key takeaways for the thesis synthesis	154
11	Chapter 11 — Coalition Formation and Detection	156
11.1	Why the third player changes everything	156
11.2	The coalition detector — reading alliances off the moves	157
11.3	Shapley credit for a competitive game	158
11.4	Coalition-aware MAPPO — and when the coalition score actually rises	160
11.5	EGTA and the spinning top — is SLS a wheel or a ladder?	163
11.6	Honest notes, limitations, and where this hands off	164
11.7	Key takeaways for the thesis synthesis	164
12	Chapter 12 — Sequence Models and LLM Agents in Strategic Settings	166
12.1	Why reframe reinforcement learning as sequence prediction	166
12.2	The luck-versus-skill trap	166
12.3	ARDT — conditioning on what you can guarantee	166
12.4	What return conditioning actually does in poker	168
12.5	Language models as strategic agents	169
12.6	Exploitation, adaptation, and the limits of the toy game	172
12.7	Key takeaways for the thesis synthesis	173
13	Chapter 13 — Behavioral Analysis of Real Hand Histories	175
13.1	From solving games to reading logs	176
13.2	The data and the parser	176
13.3	Describing a player: statistics and how many hands they need	177
13.4	How far real players are from a strong reference	179
13.5	Predicting decisions: behavioral cloning and the baseline trap	180
13.6	A behavioral fingerprint: player2vec for poker	182
13.7	Types, stability and online inference	184
13.8	Collusion: detection on injected colluders in real sessions	186
13.9	Can the pipeline find a bot?	188
13.10	Honest notes, limitations, and where this hands off	189
13.11	Key takeaways for the thesis synthesis	189
14	Chapter 14 — Evaluation Frameworks and Exploitability Metrics	191
14.1	Why evaluating an adaptive agent is hard	191
14.2	The framework: three layers and a fourth question	192
14.3	Layer 2 — rankings disagree, in predictable ways	194
14.4	Layer 3 — confidence, and what AIVAT taught	196

14.5 The joint protocol on Kuhn and Leduc	198
14.6 Approximate best responses and the RRPS score	200
14.7 Three and four players	201
14.8 Where existing evaluation breaks — a checklist	202
14.9 Honest notes, limitations, and where this hands off	203
14.10 Key takeaways for the thesis synthesis	203

Preface

Introduction

Research Context and Significance

Multi-agent systems operating under conditions of imperfect information represent one of the most challenging and practically relevant areas of artificial intelligence research. In such systems, autonomous agents must make strategic decisions without complete knowledge of the environment state or the intentions of other participants — a setting that arises naturally in cybersecurity (adversarial network defense), financial markets (competing algorithmic traders), autonomous system coordination, and fraud detection.

The mathematical framework for reasoning about such interactions is provided by extensive-form games with imperfect information, a branch of game theory that models sequential decision-making when players hold private information. Over the past decade, substantial progress has been achieved in computing equilibrium strategies for large-scale imperfect-information games. Landmark systems include Libratus (Brown and Sandholm, 2017) and Pluribus (Brown and Sandholm, 2019), which defeated professional human players in two-player and six-player poker respectively, as well as more general architectures such as ReBeL (Brown et al., 2020) and Student of Games (Schmid et al., 2023), which unify game-solving approaches across perfect- and imperfect-information domains.

Despite these advances, a critical limitation persists: state-of-the-art systems compute fixed equilibrium strategies that do not adapt to the specific behavioral patterns exhibited by encountered opponents. In practice, real-world adversaries rarely play optimally, and the ability to detect and exploit systematic deviations from equilibrium — while maintaining guarantees against being exploited in return — constitutes a fundamental open problem. This problem, known as safe opponent exploitation, has been investigated primarily in two-player zero-sum settings. Its extension to multiplayer environments, where coalitions may form and dissolve dynamically, remains largely unexplored in the literature.

The present study plan is organized as a progressive program of fifteen study steps, grouped into seven thematic phases:

- **Foundation** (Chapters 1–2): Reinforcement learning and game-theoretic fundamentals required by all subsequent work.
- **Scaling the Toolbox** (Chapters 3–4): Monte Carlo CFR variants and game abstraction techniques for handling larger game instances.
- **Neural Methods for Games** (Chapters 5–6): Neural network approximations of equilibrium strategies and end-to-end game AI architectures.

- **Opponent Modeling and Exploitation** (Chapters 7–8): Inference from behavioral traces and safe exploitation algorithms — the thesis-critical core.
- **Multi-Agent Dynamics** (Chapters 9–11): Multi-agent reinforcement learning, population-based training, and coalition dynamics in competitive settings.
- **Data-Driven Approaches** (Chapters 12–13): Sequence models and behavioral analysis pipelines connecting theory to real-world data.
- **Integration** (Chapters 14–15): Evaluation framework construction and research frontier mapping.

Throughout the study plan, Kuhn Poker and Leduc Hold'em serve as the primary implementation testbeds. These games are chosen deliberately as pedagogical vehicles: their small state spaces (12 and approximately 936 information sets, respectively) and known analytical equilibria permit exact verification of every algorithm, while their imperfect-information structure retains the strategic complexity that the thesis demands. By working within well-understood domains, each step concentrates on algorithmic concepts rather than domain-specific engineering, maximizing the volume of theoretical material covered within the allotted timeframe. In later phases, the study plan validates generality beyond poker by applying the developed methods to matrix games, Goofspiel, So Long Sucker, and anonymized real-world behavioral data.

The planned thesis contributions are formulated in domain-agnostic terms — none is specific to poker or to any other single game. The Behavioral Adaptation Framework (Contribution 1) targets arbitrary imperfect-information games; the Multi-Agent Safe Exploitation heuristics (Contribution 2) will be defined over general N-player extensive-form structures; and the Evaluation Methodology (Contribution 3) is intended for cross-domain applicability.

The study plan is aligned with the milestones of the university individual plan. Upon completion of each thematic phase, the corresponding material will be incorporated into Chapter I of the dissertation — covering the state-of-the-art analysis and the formulation of relevance, objectives, tasks, and thesis statements. In this way, the literature review and foundational exposition accumulate incrementally rather than being composed retrospectively. Chapter 15 produces a detailed Chapter I outline and publication pipeline.

Research Objective and Expected Contributions

The primary objective of this doctoral research is to develop an adaptive AI agent to be used in various computer games. The agent will start from a safe strategy but will be able to improve its outcome against sub-optimal opponents, thus removing the limitations of previous systems which rarely dynamically adapt their behavior. The research program is structured around three planned contributions:

Contribution 1 — Behavioral Adaptation Framework. The goal is to develop a general method for inferring and adapting to opponent strategies from observed action sequences in real time, applicable to arbitrary imperfect-information games. This contribution aims to address the gap between static equilibrium computation and dynamic, opponent-aware play.

Contribution 2 — Multi-Agent Safe Exploitation. This contribution will explore tractable heuristic

approaches — including KL-regularized exploitation (constraining the exploitative policy to remain close to a safe reference strategy via a Kullback–Leibler divergence penalty) and equal-share baselines — for safe exploitation in small N-player games (three-player Kuhn and Leduc poker variants), seeking to extend existing results from the two-player zero-sum case. The scope is limited to the empirical validation of practical heuristics rather than a general safety theorem for arbitrary N-player settings.

Contribution 3 — Evaluation Methodology. The aim is to design a domain-agnostic framework for measuring agent adaptability and robustness across different game environments and opponent populations, with emphasis on identifying failure modes of existing evaluation approaches and demonstrating where standard metrics may provide misleading assessments.

1 Chapter 1 — Reinforcement Learning Basics

1.1 How Reinforcement Learning Became a Discipline

Reinforcement learning grew from three threads that were largely unaware of each other for decades: trial-and-error learning from animal psychology,¹ dynamic programming and the value function from optimal control,² and temporal-difference methods.³⁴ Watkins’s Q-learning (1989)⁵ fully joined them, and by Sutton and Barto’s textbook (1998, updated 2018)⁶ RL had become a distinct framework for sequential decision-making under uncertainty, not just “machine learning with rewards.” DQN brought deep networks to it in 2013–2015,⁷ and PPO followed in 2017.⁸

1.2 Core Principles — Agent, Action, Environment

The entire RL framework rests on a simple loop. An **agent** observes the current state of an **environment**, selects an **action**, and receives a **reward** signal plus a new state. Then it repeats. The agent’s goal is to learn a **policy** — a mapping from states to actions — that maximizes the cumulative reward over time.

- **State** (s): a description of the environment at a particular moment — in CartPole the cart position and velocity and the pole angle and angular velocity; in poker the cards dealt and the betting history.
- **Reward** (r): a scalar feedback signal, sparse (only at the end of a game) or dense (every timestep).
- **Policy** (π): deterministic ($\pi(s) = a$) or stochastic ($\pi(a|s)$ gives a probability distribution over actions).
- **Value function** ($V^\pi(s)$): the expected cumulative reward from state s onward, if the agent follows policy π — how good a state is in the long run, not just immediately.
- **Discount factor** (γ): $\gamma = 0.99$ means the agent is patient; $\gamma = 0.5$ means it is myopic.

The critical design challenge in RL is the **exploration-exploitation tradeoff**: the agent must explore unfamiliar actions to discover potentially better strategies, yet exploit its current knowledge to collect

¹Thorndike, E.L. (1911). *Animal Intelligence: Experimental Studies*. Macmillan.

²Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.

³Samuel, A.L. (1959). “Some Studies in Machine Learning Using the Game of Checkers.” *IBM Journal of Research and Development*, 3(3), 210–229.

⁴Sutton, R.S. (1988). “Learning to Predict by the Methods of Temporal Differences.” *Machine Learning*, 3(1), 9–44. PhD thesis: Sutton, R.S. (1984). *Temporal Credit Assignment in Reinforcement Learning*. University of Massachusetts Amherst.

⁵Watkins, C.J.C.H. (1989). *Learning from Delayed Rewards*. PhD thesis, University of Cambridge. Convergence was proved in Watkins, C.J.C.H. & Dayan, P. (1992). “Q-learning.” *Machine Learning*, 8(3–4), 279–292.

⁶Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press. Ch. 1 “Introduction”; Ch. 3 “Finite Markov Decision Processes”; Ch. 4 “Dynamic Programming”; Ch. 5 “Monte Carlo Methods”; Ch. 6 “Temporal-Difference Learning”; §11.3 “The Deadly Triad”; Ch. 13 “Policy Gradient Methods”. <http://incompleteideas.net/book/the-book-2nd.html>

⁷Mnih, V. et al. (2013). “Playing Atari with Deep Reinforcement Learning.” arXiv:1312.5602; Mnih, V. et al. (2015). “Human-level control through deep reinforcement learning.” *Nature*, 518(7540), 529–533. DOI 10.1038/nature14236

⁸Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017). “Proximal Policy Optimization Algorithms.” arXiv:1707.06347.

reward. Too much exploration wastes time; too much exploitation gets stuck at local optima.

Value-based methods (like DQN⁷) learn the value of states or state-action pairs and derive a policy from those values; **policy-based** methods (like PPO⁸) learn the policy directly.⁶

1.3 The MDP Formulation

A **Markov Decision Process** (MDP) is the formal framework most RL algorithms are built on: a set of states S , a set of actions A , a transition function $P(s'|s, a)$, a reward function $R(s, a, s')$ and a discount factor $\gamma \in [0, 1)$.

The “Markov” part means that the future depends only on the current state, not on the history of how you got there. This is a strong assumption — and one that does not hold in many interesting settings (like poker, where the history of bets matters). Section 1.4 deals with that.

The objective is to find a policy π^* that maximizes the expected discounted return:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

The key recursive relationship is the **Bellman equation**:

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^{\pi}(s')]$$

This says: the value of a state equals the expected immediate reward plus the discounted value of wherever you end up. All of dynamic programming, TD learning, and deep RL build on variations of this equation.⁶

1.4 Information Sets — When You Cannot See Everything

Standard MDPs assume the agent can fully observe the state. Real problems often violate this. In poker, you do not see the opponent’s cards. In real-time strategy games, you have fog of war. These are **partially observable** settings.

The formal extension is the **Partially Observable MDP** (POMDP)⁹, which adds an observation function — the agent sees observations that are noisy or incomplete views of the true state. But POMDPs are notoriously hard to solve optimally.

In game theory, the same idea is captured through **information sets**. An information set groups together all game states that a player cannot distinguish between, given what they have observed. In Kuhn Poker

⁹Kaelbling, L.P., Littman, M.L. & Cassandra, A.R. (1998). “Planning and acting in partially observable stochastic domains.” *Artificial Intelligence*, 101(1–2), 99–134. DOI 10.1016/S0004-3702(98)00023-X

(3 cards, 2 players), when you hold a Jack and no betting has occurred, you are in an information set — you know your card but not your opponent’s, so you cannot tell if the true state is “I have Jack, opponent has Queen” or “I have Jack, opponent has King.”¹⁰

Algorithms that work in these settings — like CFR (Counterfactual Regret Minimization, covered in Chapter 2) — operate over information sets rather than individual states. This is a fundamentally different computational problem than standard RL, and it is why game-playing AI requires its own set of tools beyond what DQN or PPO provide out of the box.

1.5 Dynamic Programming — When You Have the Model

Dynamic programming (DP) solves an MDP when you have complete knowledge of the environment (P and R) — in practice, when you know the rules of the game perfectly. **Policy iteration** alternates between evaluating a fixed policy with the Bellman equation and making the policy greedy with respect to the new values, until the policy stops changing. **Value iteration** combines evaluation and improvement into a single update. Both give exact solutions with clean convergence guarantees, but they need the full model, and the computation scales with the number of states. Every subsequent method — Monte Carlo, TD learning, DQN, PPO — can be understood as an approximation to the DP solution when the model is unknown or the state space is too large.⁶

1.6 Temporal-Difference Learning — Learning Without a Model

Temporal-difference (TD) learning is the breakthrough that made RL practical. Unlike DP, it needs no model of the environment; unlike Monte Carlo methods, it does not wait until the end of an episode. It updates value estimates after every single step, using a bootstrap: the current estimate of the next state’s value stands in for the true future return.

Its control version is **Q-learning** (Watkins, 1989):⁵

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

The term in brackets is the **TD error** — the difference between what happened (reward + estimated future value) and what was predicted. If the TD error is positive, the state was better than expected; if negative, worse. The max operator makes Q-learning **off-policy** — it learns about the optimal policy

¹⁰Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press. DOI 10.1017/CBO9780511811654. Ch. 3 “Introduction to Noncooperative Game Theory: Games in Normal Form”; Ch. 4 “Computing Solution Concepts of Normal-Form Games” (§4.1, the LP for two-player zero-sum games); Ch. 5 “Games with Sequential Actions: Reasoning and Computing with the Extensive Form” (§5.2, imperfect-information games and information sets; §5.2.3, the sequence form underneath every LP in Chapter 8); Ch. 7 “Learning and Teaching”, the learning-in-repeated-games framing, including the tension that your actions both *exploit* and *teach* the opponent. Free: <http://www.masfoundations.org/download.html>

even while following an exploratory one. The bootstrapping introduces some bias, but TD learns online and with lower variance than Monte Carlo; it is the foundation DQN and most modern value-based algorithms are built on.⁶

1.7 DQN — Deep Q-Networks

DQN (Mnih et al., 2015)⁷ is the algorithm that proved deep neural networks could work as function approximators in RL at scale: it learned to play 49 Atari games from raw pixel input.

How it works. DQN replaces the Q-table with a neural network that takes a state as input and outputs Q-values for all actions. The network is trained by minimizing the squared TD error: the difference between the predicted Q-value and the target (reward + discounted max Q-value of the next state). At each step, the agent picks the action with the highest Q-value (exploitation) or a random action with probability ϵ (exploration).

Function approximation with Q-learning alone was known to be unstable (the “deadly triad”⁶ of off-policy + function approximation + bootstrapping). Two tricks make DQN stable enough to work:

1. **Experience replay.** Instead of training on sequential transitions (which are highly correlated), transitions are stored in a large buffer and sampled randomly in mini-batches. This breaks the correlation and lets each experience be reused multiple times. It is the same principle as shuffling your training data in supervised learning.
2. **Target network.** A separate copy of the Q-network is maintained and updated only periodically. The TD target is computed using this frozen copy. Without it, the target shifts with every training step — you are chasing a moving target, which causes oscillation and divergence.

Limitations. DQN handles only discrete action spaces and can overestimate Q-values (addressed by Double DQN¹¹). And fundamentally, it learns a greedy deterministic policy — there is no natural way to express stochastic policies, which matters in games where mixing strategies is optimal.

In our implementation, the most impactful hyperparameters were network size, epsilon minimum, and target network sync frequency.

1.8 PPO — Proximal Policy Optimization

PPO (Schulman et al., 2017)⁸ **learns the policy directly** — a neural network outputs a probability distribution over actions and is trained to increase the probability of actions that lead to high returns.

The problem PPO solves. Policy gradient methods are notoriously unstable in practice: the basic

¹¹van Hasselt, H., Guez, A. & Silver, D. (2016). “Deep Reinforcement Learning with Double Q-Learning.” *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1). DOI 10.1609/aaai.v30i1.10295. arXiv:1509.06461

REINFORCE algorithm¹² suffers from high variance and can take destructively large steps that ruin a good policy. TRPO (Schulman et al., 2015)¹³ constrained how much the policy can change per update, but its second-order step is complicated to implement.

How PPO works. PPO keeps the stability idea from TRPO but replaces the constraint with a much simpler mechanism: **clipping**. The key quantity is the probability ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$ — how much more (or less) likely the new policy is to take the same action the old policy took. PPO clips this ratio to the range $[1 - \epsilon, 1 + \epsilon]$ (typically $\epsilon = 0.2$), which means:

- If an action turns out to be good (positive advantage) and the ratio grows beyond $1 + \epsilon$, the gradient is zeroed — the policy is not allowed to become too eager for that action.
- If an action turns out to be bad (negative advantage) and the ratio drops below $1 - \epsilon$, the gradient is also zeroed — preventing the policy from overcorrecting.

This is the **clipped surrogate objective**: TRPO-like stability with plain first-order stochastic gradient steps.

Architecture. PPO is actor-critic: a **policy network** (actor) outputs action probabilities, and a **value network** (critic) estimates state values, which enable low-variance advantage estimates through Generalized Advantage Estimation (GAE).¹⁴

Why PPO matters compared to DQN: PPO handles both discrete and continuous action spaces and can learn stochastic policies, which is critical in game settings where mixing strategies is necessary. Being on-policy, it avoids stale data but is less sample-efficient than DQN, needs careful tuning (GAE lambda, entropy bonus, network architecture) and can still struggle with sparse rewards.

In our implementation, PPO solved LunarLander-v3 (100-episode average of 202.2 against a 200 target) at ~264K environment steps. The decisive change between the two recorded runs was network capacity (doubling from [64,64] to [128,128], with a larger budget).

1.9 Practical Validation — Custom Implementations vs Stable-Baselines3

To verify that our from-scratch implementations actually work, we compared them against Stable-Baselines3 (SB3)¹⁵ — a widely used, well-tested RL library. For PPO, SB3 was given our hyperparameters and the same 500K-step budget. For DQN, the SB3 run shown uses the RL Baselines3 Zoo’s tuned CartPole hyperparameters¹⁶ and a 100K-step budget, so that comparison is against a tuned baseline

¹²Williams, R.J. (1992). “Simple statistical gradient-following algorithms for connectionist reinforcement learning.” *Machine Learning*, 8(3–4), 229–256. DOI 10.1007/BF00992696

¹³Schulman, J., Levine, S., Abbeel, P., Jordan, M. & Moritz, P. (2015). “Trust Region Policy Optimization.” *Proceedings of the 32nd International Conference on Machine Learning*, 1889–1897.

¹⁴Schulman, J., Moritz, P., Levine, S., Jordan, M. & Abbeel, P. (2016). “High-Dimensional Continuous Control Using Generalized Advantage Estimation.” *ICLR 2016*. arXiv:1506.02438

¹⁵Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. & Dormann, N. (2021). “Stable-Baselines3: Reliable Reinforcement Learning Implementations.” *Journal of Machine Learning Research*, 22(268), 1–8.

¹⁶Raffin, A. (2020). *RL Baselines3 Zoo*. GitHub repository. <https://github.com/DLR-RM/rl-baselines3-zoo>; CartPole-v1 settings in `hyperparams/dqn.yml`.

rather than at matched settings.

1.9.1 DQN on CartPole-v1

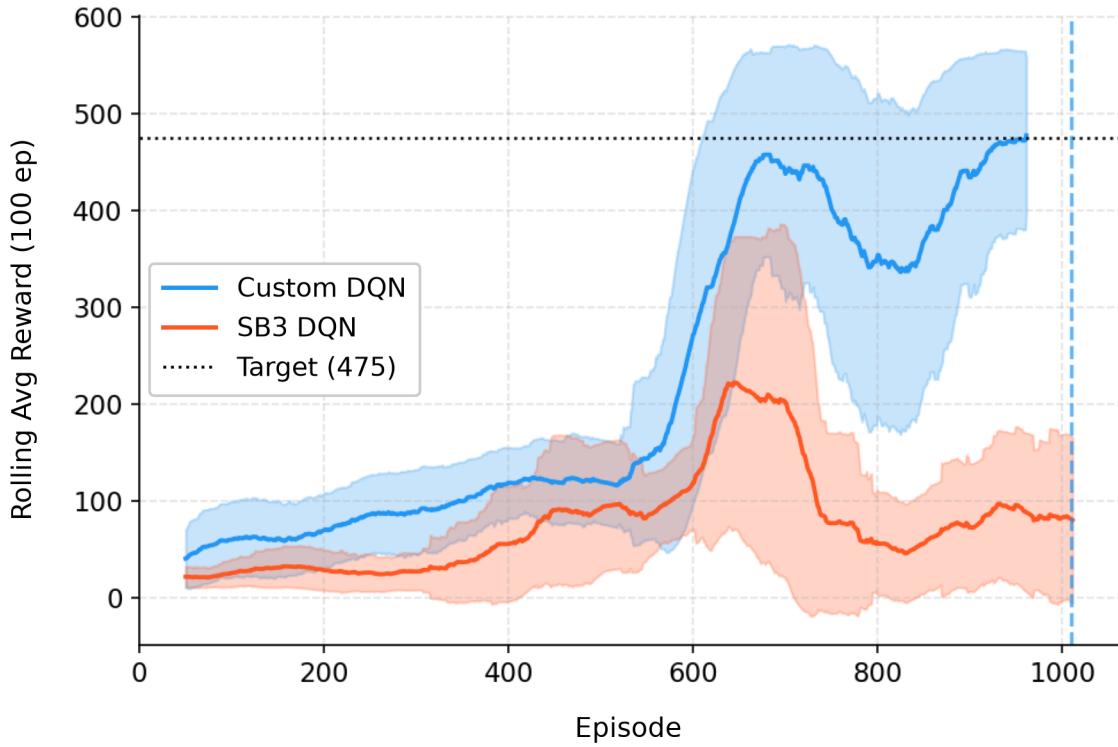


Figure 1: DQN on CartPole-v1: our implementation against SB3 (first 1,061 of SB3’s 1,255 episodes shown). Rolling 100-episode reward average, $\pm$ one standard deviation shaded; dotted line: the 475 target.

Our custom DQN solves CartPole by episode ~ 1011 , reaching a 100-episode rolling average of 477.5 (above the 475 target). The SB3 DQN shown (run of 6 April 2026) uses the Zoo’s settings with a 100K-step budget, twice the 50K the Zoo trains them for. Over its 1,255 episodes its rolling average peaks at 222.4 at episode 695 (about 51K steps) and ends at 134.3, never approaching the target.

Because SB3 ran with tuned rather than matched settings, this is not a like-for-like comparison, and no cause for the gap has been established. Two differences are untested: the rewards plotted come from training episodes, in which SB3 still acts at random 4% of the time (final $\epsilon = 0.04$) while our ϵ had decayed below 1%; and SB3 makes half as many updates per environment step as our agent. Each implementation was run once, ours without a fixed seed, so SB3’s shortfall needs a second seed before it is read as a property of SB3 rather than of this run.

1.9.2 PPO on LunarLander-v3

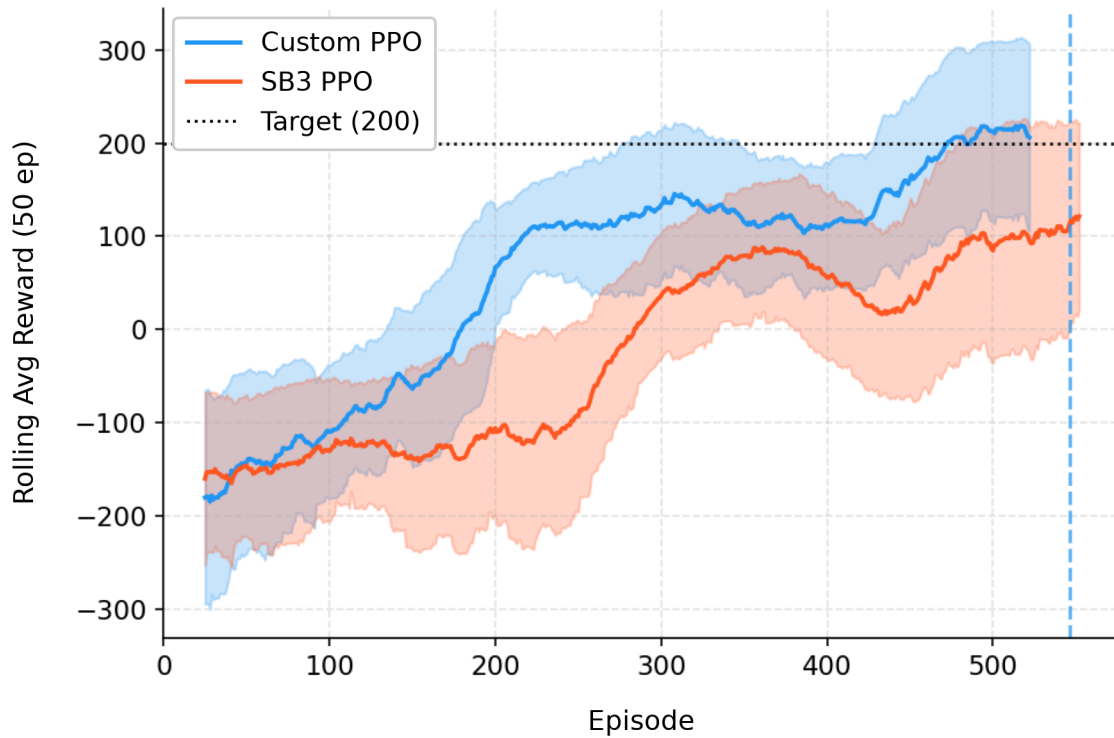


Figure 2: PPO on LunarLander-v3: our implementation (stopped at episode 547) against SB3 (first 577 of its 1,224 episodes shown). Rolling 50-episode reward average, $\pm$ one standard deviation shaded; dotted line: the 200 target.

Our custom PPO crosses the 200 target around episode 543 (264K steps). SB3’s PPO with the same 500K-step budget peaks at a best rolling-100 average of 131.2 — still climbing but not converged.

The gap is narrower here than with DQN, with identical PPO hyperparameters. The remaining difference may come from SB3’s orthogonal weight initialization and from its normalizing advantages per mini-batch rather than per rollout; with one run per implementation it may also be chance. Given 1–2M steps, SB3 PPO would likely reach the target.

1.9.3 Takeaway

A clean from-scratch implementation is 300–500 lines versus SB3’s ~10K. The tradeoff: fewer features but complete transparency and debuggability. For research purposes — particularly the opponent exploitation work planned for Chapters 7–8 — custom implementations allow surgical modifications (e.g., injecting belief-state observations) that would require deep SB3 subclassing. That practical flexibility is why we built from scratch.

2 Chapter 2 — Game Theory & CFR Basics

2.1 From Single-Agent RL to Multi-Agent Strategic Interaction

Chapter 1 treated environments as passive — the cart-pole doesn't fight back, the lander doesn't try to crash you. The agent optimized against fixed dynamics. Chapter 2 introduces a fundamentally different problem: the environment includes another *strategic* decision-maker whose actions depend on yours.

In a multi-agent setting, the “best” strategy depends on what the opponent does, and the opponent's best strategy depends on what you do. This circularity is the core challenge of game theory, and it shifts the solution concept from “optimal policy” to **Nash equilibrium**.¹⁷

2.2 Extensive-Form Games and Information Sets

An **extensive-form game** represents strategic interaction as a tree: each node is a decision point for one player (or “chance” for random events like card deals), edges are actions, and leaves carry payoffs. In **perfect-information** games (chess, Go) each node is its own information set, and minimax with alpha-beta pruning solves them. In **imperfect-information** games (poker) several game states are *indistinguishable* to the acting player.

An **information set** (Section 1.4) groups the game states a player cannot tell apart, so the player must use the same strategy for all of them. This is what makes imperfect-information games fundamentally harder: you must pick one action that works well *on average* across all states in the information set.

In Kuhn Poker, the information set “2pb” contains two game states: “I hold Queen, opponent holds Jack, history is pass-bet” and “I hold Queen, opponent holds King, history is pass-bet.” The player holding Queen cannot distinguish these and must use the same strategy (call probability) for both.¹⁷

2.3 Minimax Theorem

The Minimax Theorem (von Neumann, 1928)¹⁸ states that in every finite two-player zero-sum game, with mixed strategies, the payoff a player can guarantee (maximin) equals the payoff the opponent can hold them to (minimax); this common value is the “value of the game.” In two-player zero-sum games a

¹⁷Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Ch. 3 (games in normal form; §3.3 best response and Nash equilibrium, §3.4.1 maxmin and minmax strategies); §4.1 (computing Nash equilibria of two-player zero-sum games); Ch. 5 (extensive-form games; §5.2 imperfect information and information sets, §5.2.3 the sequence form); §7.5 “No-regret learning and universal consistency”. Free: <http://www.masfou.ndations.org/download.html>

¹⁸von Neumann, J. (1928). “Zur Theorie der Gesellschaftsspiele.” *Mathematische Annalen*, 100(1), 295-320. English translation: “On the Theory of Games of Strategy,” in *Contributions to the Theory of Games*, Vol. 4 (1959), 13-42.

minimax strategy profile is equivalent to a Nash equilibrium, so discovering it guarantees safety against any exploitative strategy the opponent might employ.

2.4 Nash Equilibrium

A **Nash equilibrium** is a strategy profile where each player’s strategy is a best response to every other player’s strategy. Formally, for each player i :

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(\sigma_i, \sigma_{-i}^*) \quad \forall \sigma_i$$

No player can improve their expected payoff by unilaterally deviating. This does not mean everyone is happy with the outcome (Prisoner’s Dilemma), just that no one can do better by changing only their own strategy. Nash (1950)¹⁹ proved that every finite game has at least one Nash equilibrium (possibly in mixed strategies) — a foundational theorem that does not help with computation.

For 2-player zero-sum games (where one player’s gain is the other’s loss), Nash equilibria are **minimax optimal**: playing your Nash strategy guarantees you at least the game value, regardless of what the opponent does. This is the safety guarantee that makes Nash equilibrium the baseline for poker AI.

In N-player or general-sum games, computing a Nash equilibrium is PPAD-complete, even for two-player general-sum games, and is therefore believed to be intractable.^{20,21} Furthermore, the equilibria lose their safety guarantee: playing a Nash strategy in a 3-player game does not protect against two opponents who might deviate from equilibrium, potentially forming temporary coalitions that exploit the third player.²² This dynamic necessitates entirely different evaluation and training frameworks (such as those discussed in Chapters 9 and 11) since the direct equivalence between Nash and minimax optimality no longer holds.

2.5 Regret Matching — The Building Block

Before CFR can minimize regret across a game tree, we need a mechanism to minimize regret at a single decision point. **Regret matching** is that mechanism.

The idea: after many rounds of play, for each action a , compute the **cumulative regret** — how much better you would have done if you had always played a instead of your actual mixed strategy. Then set your next strategy proportional to the *positive* regrets:

¹⁹Nash, J.F. (1950). “Equilibrium Points in N-Person Games.” *Proceedings of the National Academy of Sciences*, 36(1), 48–49.

²⁰Daskalakis, C., Goldberg, P. W. & Papadimitriou, C. H. (2009). “The Complexity of Computing a Nash Equilibrium.” *SIAM Journal on Computing*, 39(1), 195–259.

²¹Chen, X., Deng, X. & Teng, S.-H. (2009). “Settling the Complexity of Computing Two-Player Nash Equilibria.” *Journal of the ACM*, 56(3), 1–57.

²²Szafron, D., Gibson, R. & Sturtevant, N. (2013). “A Parameterized Family of Equilibrium Profiles for Three-Player Kuhn Poker.” *Proc. AAMAS 2013*, 247–254. In three-player Kuhn poker one player can transfer utility from one opponent to the other while staying within the equilibrium family.

$$\sigma^{T+1}(a) = \begin{cases} \frac{R^{T,+}(a)}{\sum_{a'} R^{T,+}(a')} & \text{if } \sum > 0 \\ \frac{1}{|A|} & \text{otherwise} \end{cases}$$

Convergence guarantee: Blackwell (1956)²³ and Hart & Mas-Colell (2000)²⁴ proved that regret matching ensures average regret converges to zero at rate $O(1/\sqrt{T})$ (cumulative regret grows no faster than $O(\sqrt{T})$). In a two-player zero-sum game, if both players use regret matching at their respective decision points, the average strategy profile converges to a Nash equilibrium.

A simple example is Rock-Paper-Scissors. If you play regret matching against a fixed opponent who always plays Rock, your cumulative regret for Paper will grow (Paper beats Rock), while regret for Scissors turns negative and has no effect on the strategy. The strategy quickly moves to Paper, the correct best response.²⁵

2.6 Counterfactual Regret Minimization (CFR)

CFR (Zinkevich et al., 2007)²⁶ extends regret matching to extensive-form games: it decomposes the total regret into **local regrets** at each information set and minimizes each independently with regret matching. If each information set’s average regret converges to zero, the overall *average* strategy converges to a Nash equilibrium.

The “counterfactual” part is crucial. At each information set I , the regret for action a is not simply “how much better a would have been.” It is the *counterfactual* regret — the improvement assuming the player had intentionally played to reach I (reach probability = 1 for the player) but everything else (opponent strategy, chance events) stayed the same. Formally:

$$R^T(I, a) = \sum_{t=1}^T \pi_{-i}^t \cdot (v^t(I, a) - v^t(I))$$

where π_{-i}^t is the opponent’s reach probability and $v^t(I, a)$ is the counterfactual value of action a at I on iteration t .

Why opponent reach probability? The regret at an information set matters more when the opponent is likely to have played such that we reach that information set. Weighting by π_{-i} ensures that regret is proportional to the frequency with which the information set is “relevant” to the game outcome.

²³Blackwell, D. (1956). “An Analog of the Minimax Theorem for Vector Payoffs.” *Pacific Journal of Mathematics*, 6(1), 1-8.

²⁴Hart, S. & Mas-Colell, A. (2000). “A Simple Adaptive Procedure Leading to Correlated Equilibrium.” *Econometrica*, 68(5), 1127-1150.

²⁵Neller, T.W. & Lanctot, M. (2013). “An Introduction to Counterfactual Regret Minimization,” §2-3 (version of 9 July 2013).

²⁶Zinkevich, M., Johanson, M., Bowling, M. & Piccione, C. (2007). “Regret Minimization in Games with Incomplete Information.” *Advances in Neural Information Processing Systems 20*, 1729-1736.

In the implementation used here (chance sampling, as in Neller & Lanctot, 2013²⁵), each iteration samples one card deal, traverses the tree with regret matching at every information set, and updates regrets weighted by the opponent’s reach and strategy sums weighted by the player’s own. The output is the **average** strategy, not the final iteration’s.

Convergence: the exploitability of the average strategy falls as $O(1/\sqrt{T})$ in the number of iterations T , with a bound linear in the number of information sets (Theorem 4, Zinkevich et al. 2007);²⁶ a tighter bound was proved later.²⁷

2.7 The Mathematics of Poker

Poker combines three sources of complexity that rarely co-occur: **hidden information** (the same information set can correspond to many underlying game states), **stochastic elements** (card deals add chance nodes, so a strategy must account for all possible deals) and **strategic deception** (a player who always bets with strong hands and checks with weak ones is trivially exploitable, so Nash equilibrium requires **randomized bluffing at mathematically precise frequencies**, which Chen & Ankenman derive analytically for toy games²⁸).

2.8 Kuhn Poker — The Simplest Testbed

Kuhn Poker (Kuhn, 1950) is the standard minimal example for imperfect-information game algorithms. With only 3 cards, 2 players, and 12 information sets, it is small enough to solve analytically yet rich enough to exhibit the key phenomena: bluffing, information asymmetry, mixed equilibria.

Rules: 3 cards {J, Q, K}, each player antes 1 chip, each receives 1 private card. Player 0 acts first: pass or bet. Player 1 responds: pass or bet. If Player 0 passed and Player 1 bet, Player 0 gets to respond again. Showdown with higher card winning.

Nash equilibrium (parameterized by $\alpha \in [0, 1/3]$):

- **Player 0 with J:** Bet (bluff) with probability α . If facing bet after passing, always fold.
- **Player 0 with Q:** Always pass. If facing bet after passing, call with probability $1/3 + \alpha$.
- **Player 0 with K:** Bet with probability 3α . If facing bet after passing, always call.
- **Player 1 with J:** After pass, bet (bluff) with probability $1/3$. After bet, always fold.
- **Player 1 with Q:** After pass, always pass. After bet, call with probability $1/3$.
- **Player 1 with K:** Always bet, always call.

²⁷Lanctot, M., Waugh, K., Zinkevich, M. & Bowling, M. (2009). “Monte Carlo Sampling for Regret Minimization in Extensive Games.” *Advances in Neural Information Processing Systems* 22, 1078–1086.

²⁸Chen, B. & Ankenman, J. (2006). *The Mathematics of Poker*. ConJelCo — analytical Nash derivations for toy poker games.

(Player 1’s strategy is the same in every equilibrium; only Player 0’s depends on α . The strategy CFR reaches is shown in the last figure under “Empirical Visualizations”).

Game value: Player 0’s expected payoff at Nash is $-1/18 \approx -0.0556$. Player 0 has a structural disadvantage from acting first. The first figure under “Empirical Visualizations” shows how training approaches this value.

Why Kuhn matters: every key concept in imperfect-information game solving appears here in miniature. If your algorithm cannot solve Kuhn correctly, it cannot solve anything.²⁹

2.9 Exploitability — Measuring Distance from Nash

Exploitability is the primary evaluation metric for strategies in imperfect-information games. It measures how much a strategy can be exploited by a worst-case opponent:

$$\text{exploit}(\sigma) = BR_0(\sigma_1) + BR_1(\sigma_0)$$

where $BR_i(\sigma_{-i})$ is the expected payoff of player i ’s best response to the opponent’s strategy. In a two-player zero-sum game this sum is also called NashConv; OpenSpiel reports half of it as “exploitability”, so the two conventions differ by a factor of 2. At Nash equilibrium, exploitability is exactly zero; for approximate Nash equilibria, it quantifies the approximation quality.

Best response computation in imperfect-information games is subtle: the best-responding player must choose the same action at all states within an information set. Picking the best action per game state (using knowledge of the opponent’s private card) computes an “oracle” value, not a true best response. For Kuhn Poker, enumerating all $2^6 = 64$ pure strategies suffices.

Convergence rate: In theory, the exploitability of CFR’s average strategy decreases as $O(1/\sqrt{T})$ ²⁶ (see the second figure under “Empirical Visualizations”). On a log-log plot, this appears as a line with slope ≈ -0.5 . Over five seeds, with checkpoints from 100 to 100,000 iterations, the measured slope is -0.52 ± 0.02 , consistent with the $O(1/\sqrt{T})$ bound, which is an upper limit rather than a prediction of the exact rate.

Why not just game value? A strategy can achieve the correct game value while still being exploitable. Consider a Kuhn strategy where Player 1 always bluffs with J (instead of 1/3 of the time). The average game value might be close to $-1/18$, but the strategy is highly exploitable — Player 0 can always call with Q and profit. Exploitability catches this; game value alone does not.

This metric recurs throughout the thesis. Chapters 7–8 (opponent modeling, safe exploitation) use exploitability as the primary measure. It is also the starting point of the thesis’s evaluation methodology (Contribution #3).

²⁹Kuhn, H.W. (1950). “Simplified Two-Person Poker.” *Contributions to the Theory of Games*, Vol. 1, Annals of Mathematics Studies 24, Princeton University Press, 97–103.

2.10 Connections to Chapter 1 and Forward Pointers

Local optimization $\rightarrow$ global convergence. In Chapter 1, Q-learning reduces the TD error one state at a time, yet in the tabular case the Q-function provably converges to the optimum. In Chapter 2, CFR minimizes regret at each information set independently, yet the overall *average* strategy converges to Nash. Both demonstrate the same principle: local updates, when properly structured, achieve global objectives.

Strategy accumulation $\approx$ experience replay. CFR outputs the *average* strategy, not the last iteration’s strategy. This averaging stabilizes convergence, just as experience replay in DQN stabilizes learning by preventing the agent from over-fitting to recent transitions.

Forward: for larger games even chance-sampled CFR is too expensive;³⁰ Chapter 3 samples parts of the tree (Monte Carlo CFR), Chapter 4 reduces the game size through abstraction, and Chapter 5 replaces tabular strategies with neural networks.

2.11 Empirical Visualizations

To validate the theoretical guarantees of CFR and its application to Kuhn Poker, several metrics were tracked during the training process:

³⁰Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Heads-up limit hold’em poker is solved.” *Science*, 347(6218), 145–149. Used CFR+ to solve heads-up limit Texas Hold’em — the first non-trivial imperfect-information game played competitively by humans to be (essentially weakly) solved.

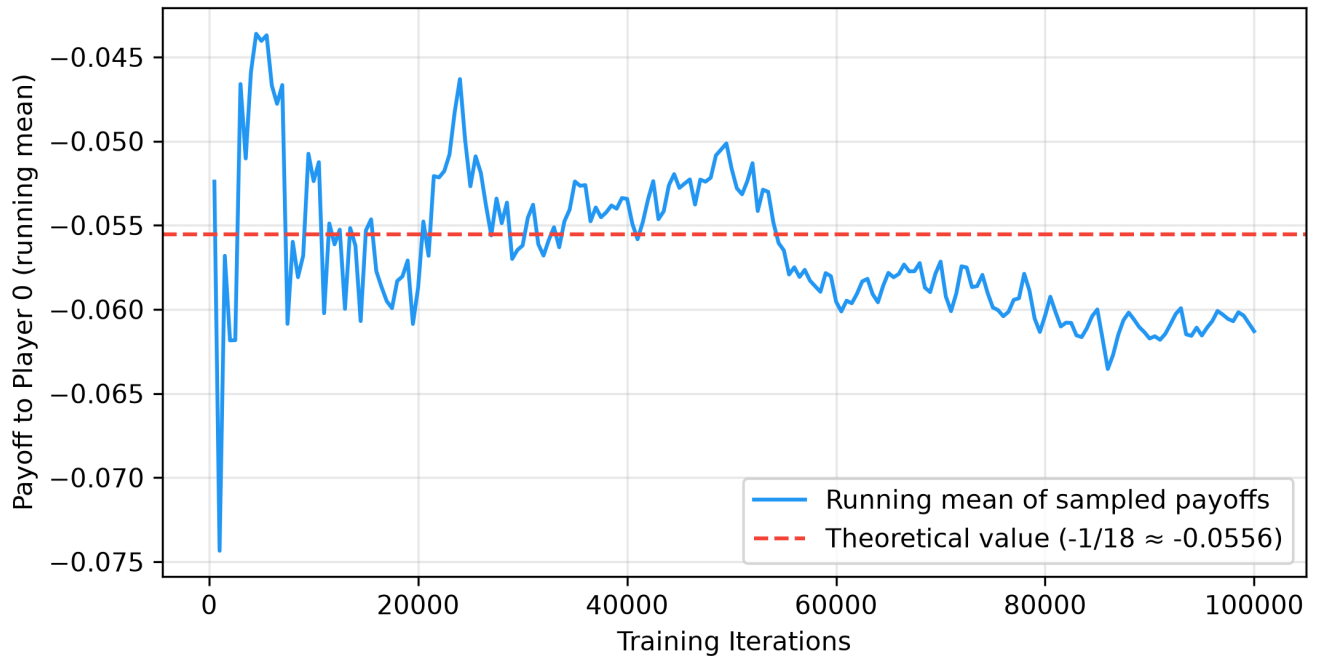


Figure 3: The running mean of Player 0's payoff over the sampled deals, averaged from the start of training, fluctuates around the theoretical value $-1/18$ within sampling noise. The exact value of the average strategy after 100,000 iterations is -0.0555 .

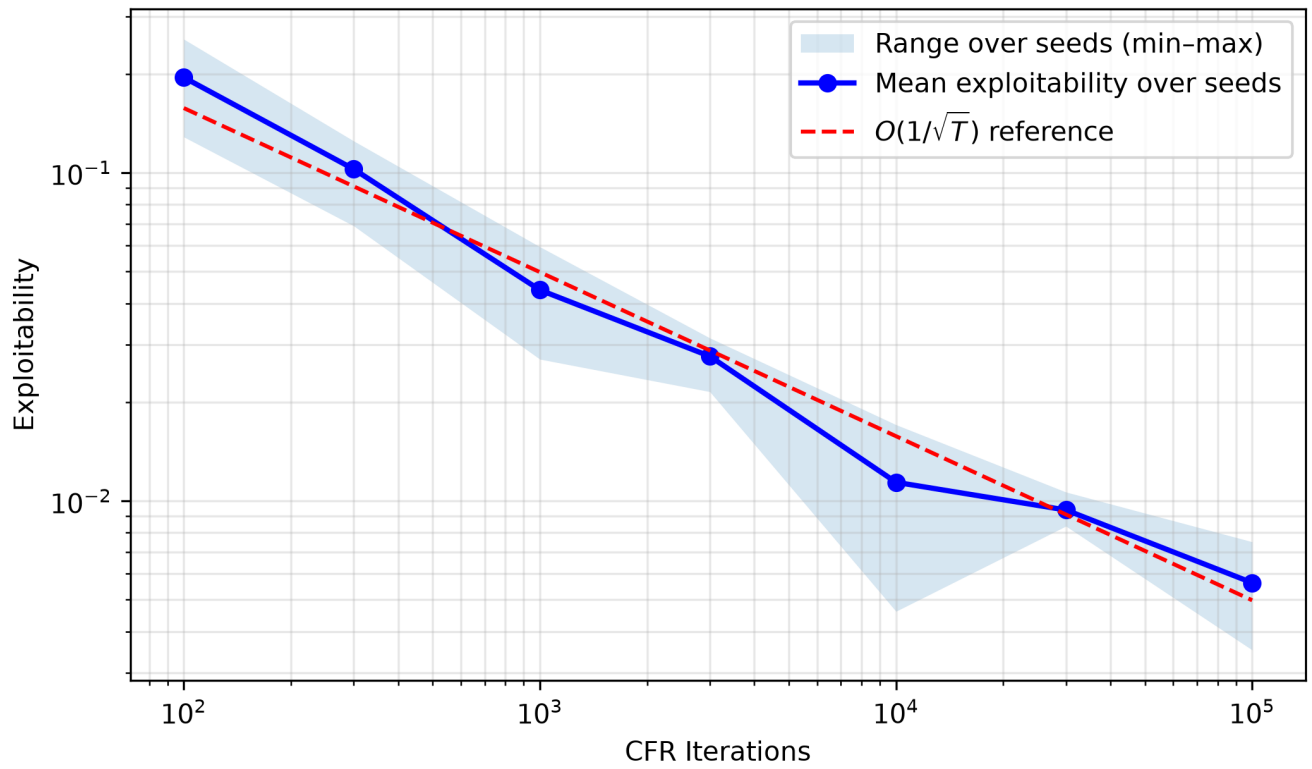


Figure 4: Exploitability of the average strategy against the number of iterations, log-log scale, mean over five seeds; the shaded band is their range. The measured points roughly follow the $O(1/\sqrt{T})$ reference slope (dashed line), consistent with the theoretical bound.

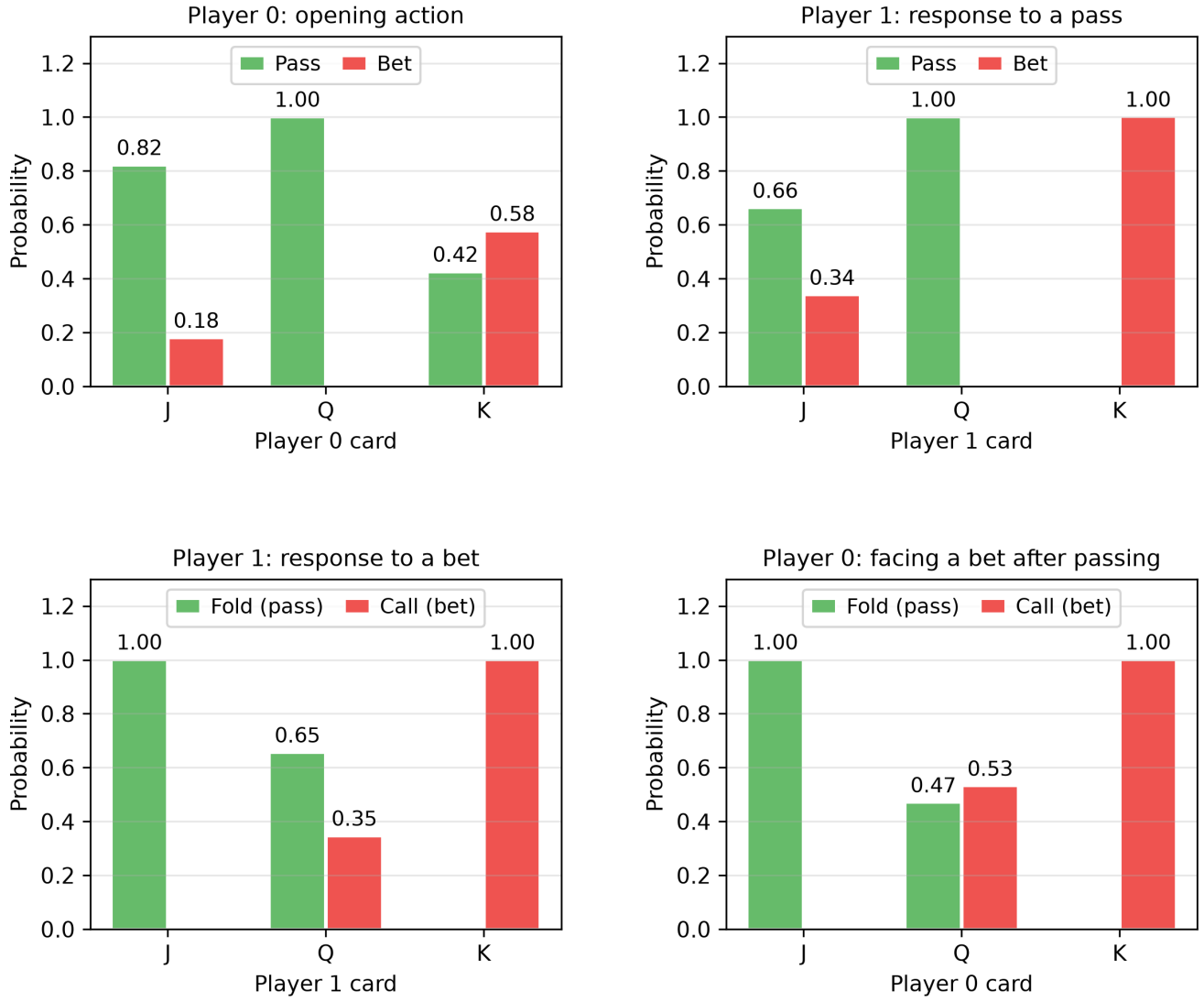


Figure 5: The average strategy CFR finds for Kuhn poker after 100,000 iterations: action probabilities at each information set. The Jack's bluffing and the Queen's calling frequencies are close to the Nash equilibrium family ($\alpha \approx 0.18$).

3 Chapter 3 — CFR Variants & Monte Carlo Methods

3.1 Why Vanilla CFR Breaks at Scale

Chapter 2 established a working Vanilla CFR solver on Kuhn Poker — 12 information sets, 58 nodes (30 terminal), solvable in milliseconds. That experience is deceptive. Vanilla CFR requires a **full traversal of the game tree on every iteration**, and real games are not Kuhn. Leduc Poker has 936 information sets ($\sim 78\times$ larger); Limit Texas Hold'em has over 10^{14} , and a single traversal of its tree took about an hour on 4,800 CPU cores (see Section 3.8).

Chapter 3 closes the scale gap through two complementary mechanisms: **CFR+** (Tammelin, 2014),³¹ which converges dramatically faster and was the algorithm that “solved” Heads-Up Limit Texas Hold'em in 2015, and **Monte Carlo CFR** (Lanctot et al., 2009),³² which samples a small portion of the tree instead of traversing all of it — each iteration orders of magnitude cheaper, at the cost of noisier updates. The other route, game abstraction, is the subject of Chapter 4.

For the thesis, these solvers compute the baseline Nash strategy — the “play it safe” anchor from which an adaptive agent deviates based on opponent observations (Contribution #1). On games of Kuhn and Leduc size that is CFR+ (see Section 3.7); MCCFR becomes necessary on larger games.

3.2 Monte Carlo Tree Search as Inspiration

Monte Carlo Tree Search (MCTS) uses **random sampling** to evaluate positions in a game tree without exhaustive enumeration — “evaluate some branches to full depth instead of all branches to fixed depth.” It grows an asymmetric tree, playing random games from promising nodes and propagating their results back up the visited path, whereas minimax with alpha-beta pruning evaluates every relevant node to a fixed depth and applies a heuristic at the leaves. For Go (branching factor ~ 250 , no good evaluation function before neural networks), MCTS was the breakthrough; the same “sample instead of enumerate” principle carries over to imperfect-information game trees via MCCFR.³³

3.3 Markov Chains and the Law of Large Numbers

The “Monte Carlo” in MCCFR refers to a broad family of computational methods that use **repeated random sampling** to obtain numerical results. The key theoretical guarantee is the **Law of Large**

³¹Tammelin, O. (2014). “Solving Large Imperfect Information Games Using CFR+.” arXiv:1407.5042. The Heads-Up Limit Hold'em result is Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Heads-up limit hold'em poker is solved.” *Science*, 347(6218), 145-149.

³²Lanctot, M., Waugh, K., Zinkevich, M. & Bowling, M. (2009). “Monte Carlo Sampling for Regret Minimization in Extensive Games.” *Advances in Neural Information Processing Systems* 22, 1078-1086.

³³Browne, C. et al. (2012). “A Survey of Monte Carlo Tree Search Methods.” *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1-43.

Numbers: if enough trajectories are drawn independently, the average of the sampled values converges to the true expected value. For MCCFR, at a fixed strategy profile:

$$\frac{1}{T} \sum_{t=1}^T \hat{v}_I^{(t)} \xrightarrow{T \rightarrow \infty} v_I$$

where $\hat{v}_I^{(t)}$ is the sampled counterfactual value at information set I on iteration t , and v_I is the true value that vanilla CFR computes exactly. The sampled values are **unbiased estimators** — their expected value equals the true value — and, with sampling probabilities bounded away from zero, this gives with probability at least $1 - p$ the same $O(1/\sqrt{T})$ decrease of average regret as full-traversal CFR.³²

The cost is **variance**: individual samples can deviate substantially from the true value, requiring more iterations to reach the same precision. That variance-for-speed tradeoff is the central theme of this chapter.³⁴

3.4 Leduc Poker — Scaling Up the Benchmark

Kuhn Poker captures the essence of imperfect information in the smallest possible game tree. Leduc Poker scales that up by roughly two orders of magnitude — still solvable exactly, but large enough that algorithm differences become meaningful:

Table 1: Kuhn and Leduc Poker compared: cards, betting rounds, information sets and tree size.

Property	Kuhn Poker	Leduc Poker
Cards	3 (J, Q, K)	6 ($\{J, Q, K\} \times \{\spadesuit, \heartsuit\}$)
Rounds	1	2 (pre-flop + community card)
Community card	None	1 revealed between rounds
Hand ranking	High card only	Pair (private = community) beats high card
Bet sizes	Fixed (1 chip)	Variable (2 in round 1, 4 in round 2)
Max raises/round	1	2
Information sets	12	936
Game tree nodes	58	10,200
Chance outcomes	6	120

³⁴Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press. Ch. 1 (the field); Ch. 3 (finite Markov decision processes); Ch. 4 (dynamic programming); Ch. 5 (Monte Carlo methods); Ch. 6 (temporal-difference learning). <http://incompleteideas.net/book/the-book-2nd.html>

Three qualitatively new features appear: information changes between rounds (the community card reveal), a Jack can become the best hand if a Jack is dealt on the board, and the 4-chip raise in round 2 makes later decisions carry more weight.

Despite being $\sim 78\times$ larger than Kuhn, Leduc remains small enough for exact computation (a full tree traversal takes ~ 50 ms). That makes it the ideal benchmark: large enough for performance differences to be meaningful, small enough that all algorithms can be run to near-convergence within minutes.³⁵

3.5 CFR+ — The Regret Flooring Trick

CFR+ makes three small changes to vanilla CFR, each trivial to implement but dramatic in effect.

Regret flooring. In vanilla CFR, cumulative regrets can become arbitrarily negative; CFR+ floors them at zero after every update:

$$R^{T+1}(I, a) = \max(R^T(I, a) + r^T(I, a), 0)$$

This prevents actions from accumulating large negative regret that takes many iterations to “pay off” before the action is reconsidered. The analogy to ReLU activations in neural networks is direct: both clip negative values to zero.

Linear strategy averaging. CFR+ weights iteration t ’s strategy by t in the running average, so later iterations — which have lower regret and better strategies — contribute more, and early, noisy iterations are down-weighted.

Alternating updates. CFR+ updates the two players in turn within each iteration, so the second player’s update already sees the first player’s new strategy.

Combined effect: empirical convergence improves from $O(1/\sqrt{T})$ to approximately $O(1/T)$. On Leduc, CFR+ reaches 2.0×10^{-4} after about 1,100 iterations; at vanilla CFR’s measured rate the same value would take on the order of two million iterations (extrapolated). The $O(1/T)$ rate is consistently observed but unproven; what is proven is the family-wide $O(1/\sqrt{T})$ bound,³⁶ whose original proof was corrected by Burch et al.³⁷

Exploitability here is the average best-response gain against the two players, $(BR_0 + BR_1)/2$; NashConv is their sum, i.e. twice as large. On Leduc after 5,000 iterations OpenSpiel’s CFR+ reaches exploitability $\approx 1.8 \times 10^{-5}$ while vanilla CFR is still at 3.6×10^{-3} — a $\sim 190\times$ improvement from two of the three modifications (both OpenSpiel solvers already alternate updates).³¹

³⁵Southey, F. et al. (2005). “Bayes’ Bluff: Opponent Modelling in Poker.” *UAI*. arXiv:1207.1411.

³⁶Tammelin, O., Burch, N., Johanson, M. & Bowling, M. (2015). “Solving Heads-Up Limit Texas Hold’em.” *Proc. IJCAI*, 645–652.

³⁷Burch, N., Moravčík, M. & Schmid, M. (2019). “Revisiting CFR+ and Alternating Updates.” *Journal of Artificial Intelligence Research*, 64, 429–443. DOI 10.1613/jair.1.11370.

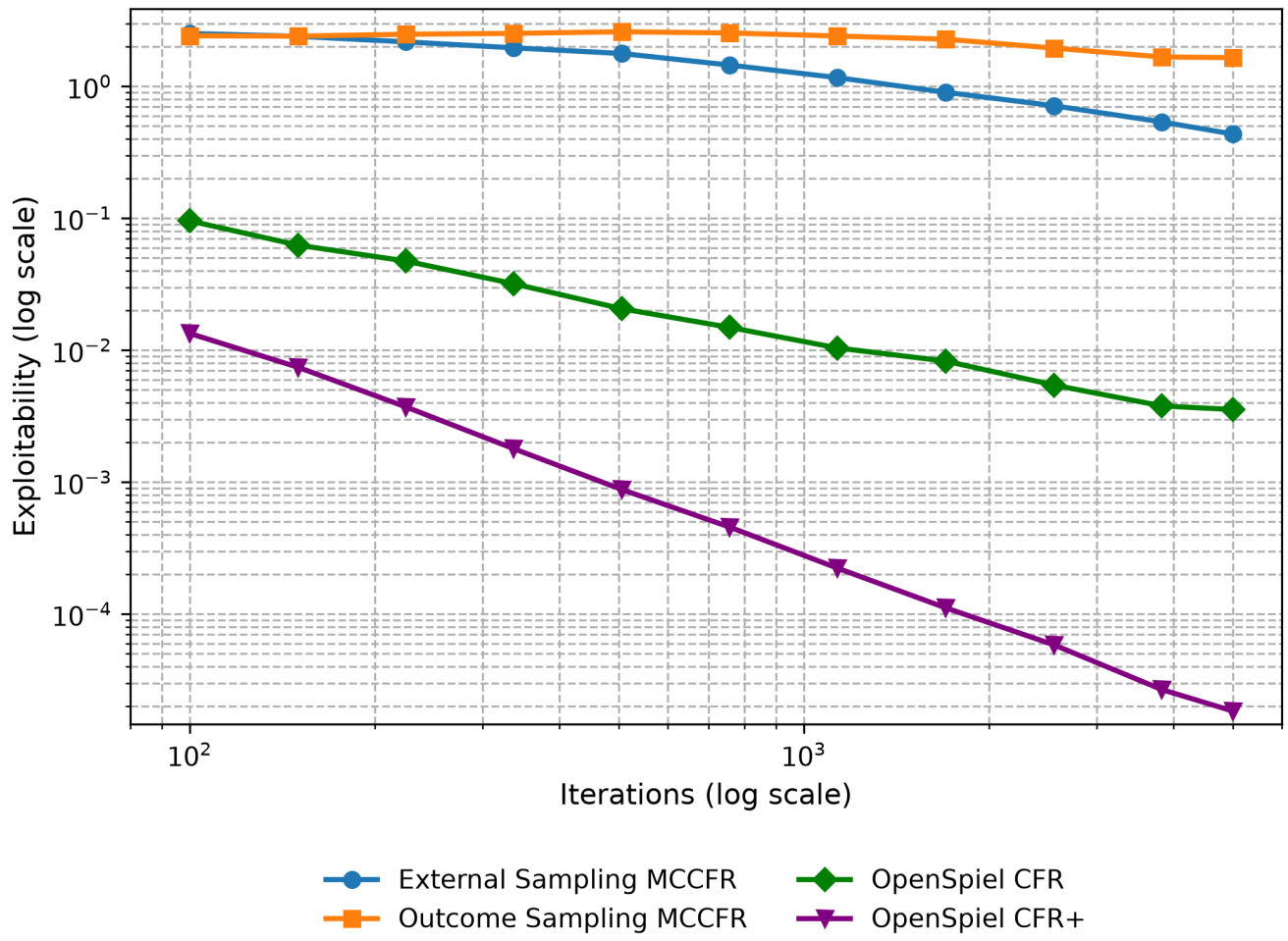


Figure 6: Leduc Poker: exploitability vs iterations for the four OpenSpiel solvers

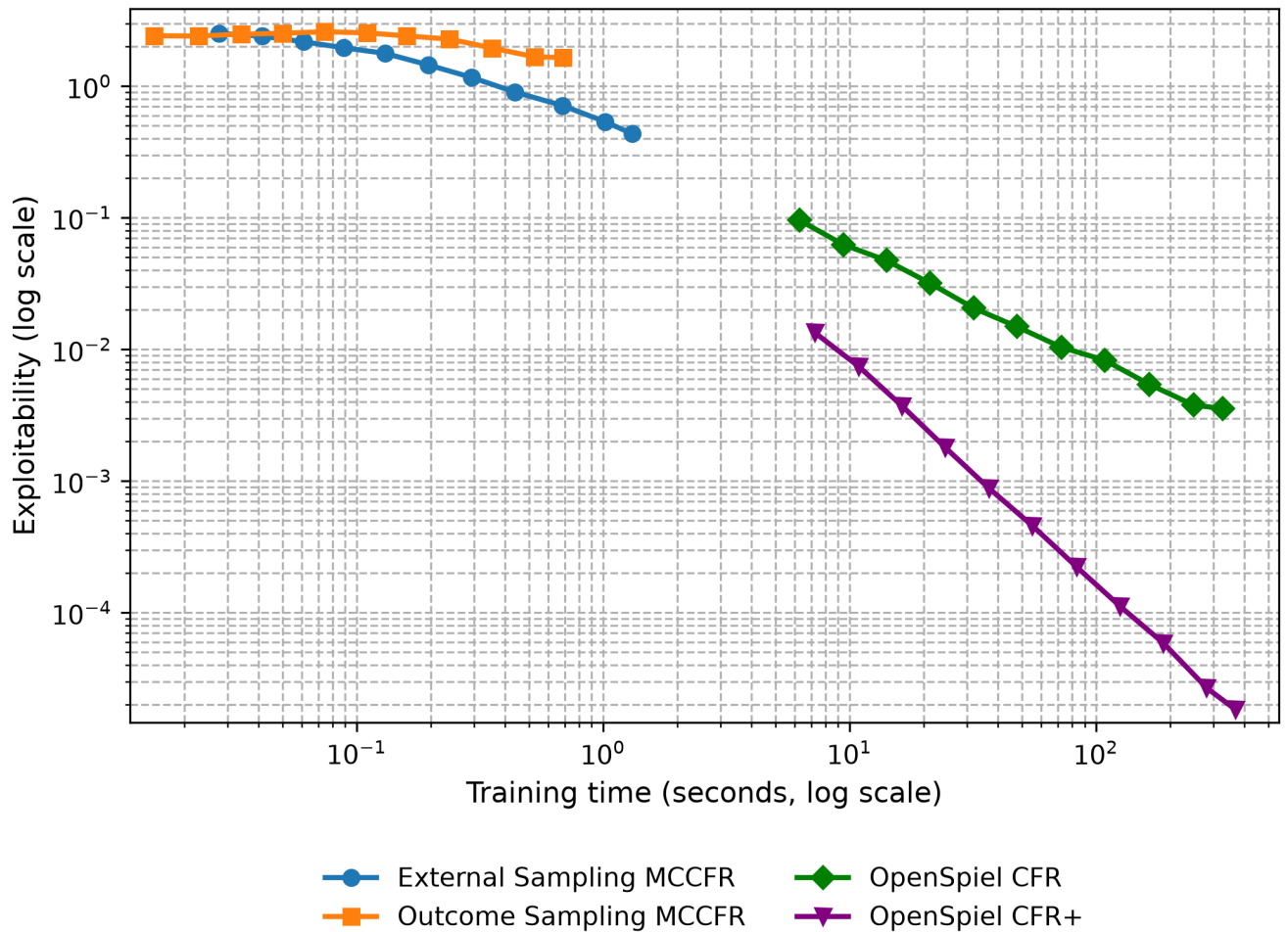


Figure 7: Leduc Poker: exploitability vs training time for the four OpenSpiel solvers

3.6 MCCFR — Sampling the Game Tree

MCCFR replaces full tree traversal with partial sampling. Two variants sit on the same variance-speed curve at different points:

Table 2: External and outcome sampling: what each variant samples at every node type, and the resulting cost per iteration.

Variant	Chance nodes	Traverser's nodes	Opponent's nodes	Cost per iteration
External Sampling	Sample one deal	Explore all actions	Sample one action	~42 nodes (~945× faster than full)
Outcome Sampling	Sample one deal	Sample one action (ε-on-policy)	Sample one action	~5.5 nodes (~2,228× faster)

External Sampling explores every action at the updating player’s nodes but samples at chance and opponent nodes, so each iteration touches a tiny slice of the tree. **Outcome Sampling** draws a single root-to-terminal trajectory; because actions at the updating player’s nodes are also sampled, the regret update must be corrected by **importance sampling** — the ratio of the true reach probability to the sampling probability. An ϵ -on-policy mixture (with probability ϵ , choose uniformly; otherwise follow the current strategy) ensures all actions are explored even when the current strategy assigns zero probability.

For both, sampled counterfactual values are **unbiased estimators** of the true values (Lanctot et al., Lemma 1); together with their regret bounds (Theorems 4–5), this guarantees convergence to an approximate Nash equilibrium despite the sampling noise. The cost is variance: each sampled update reflects only one possible deal, not the expectation over all deals.

On Kuhn Poker every algorithm runs in seconds, and the ordering is already Leduc’s: after 5,000 iterations CFR+ is at exploitability $\approx 3 \times 10^{-5}$ and vanilla CFR at $\approx 2 \times 10^{-4}$, external and outcome sampling at ≈ 0.013 and ≈ 0.04 (the chance-sampled Chapter 2 solver at ≈ 0.006):

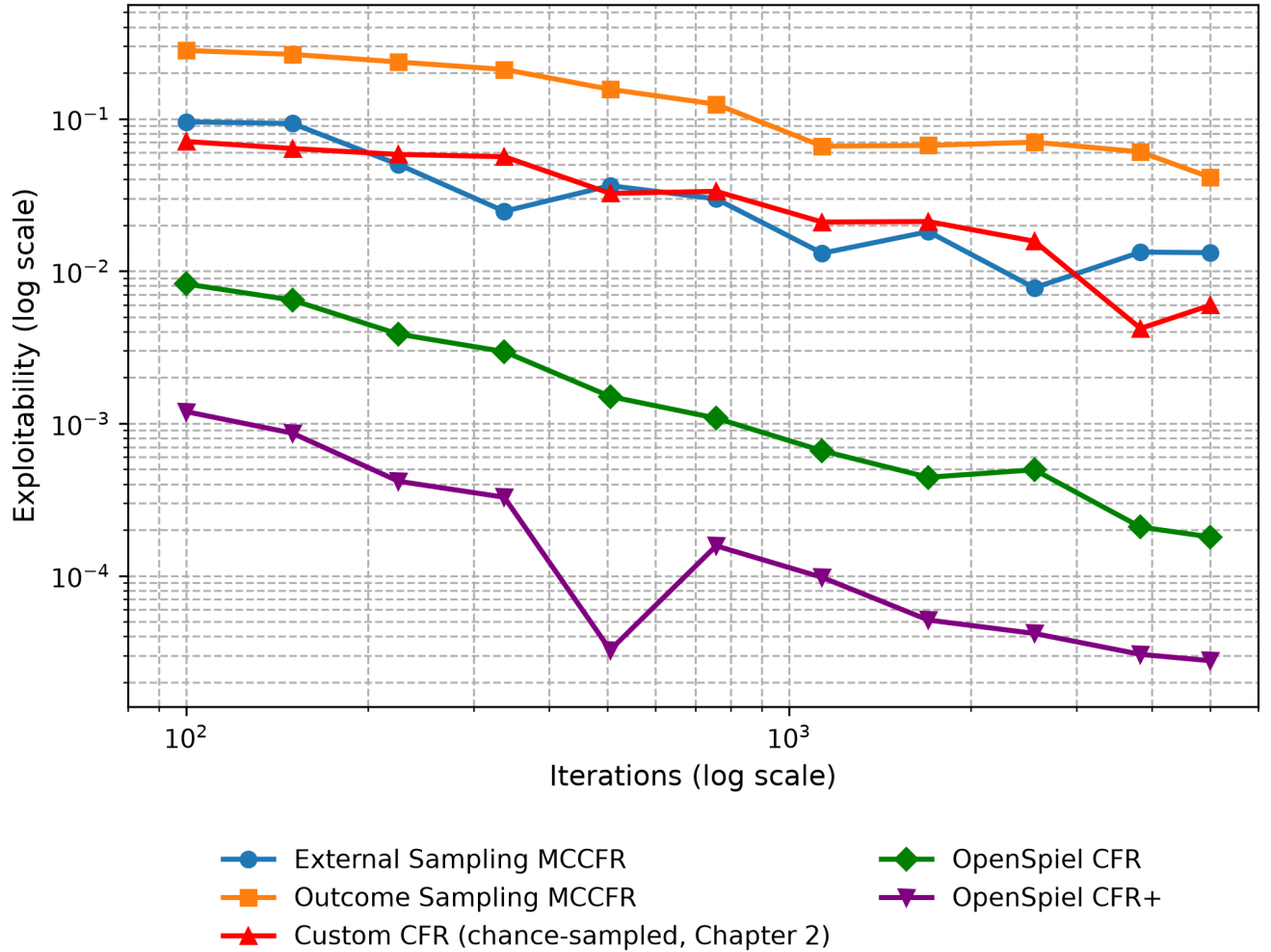


Figure 8: Kuhn Poker: exploitability vs iterations

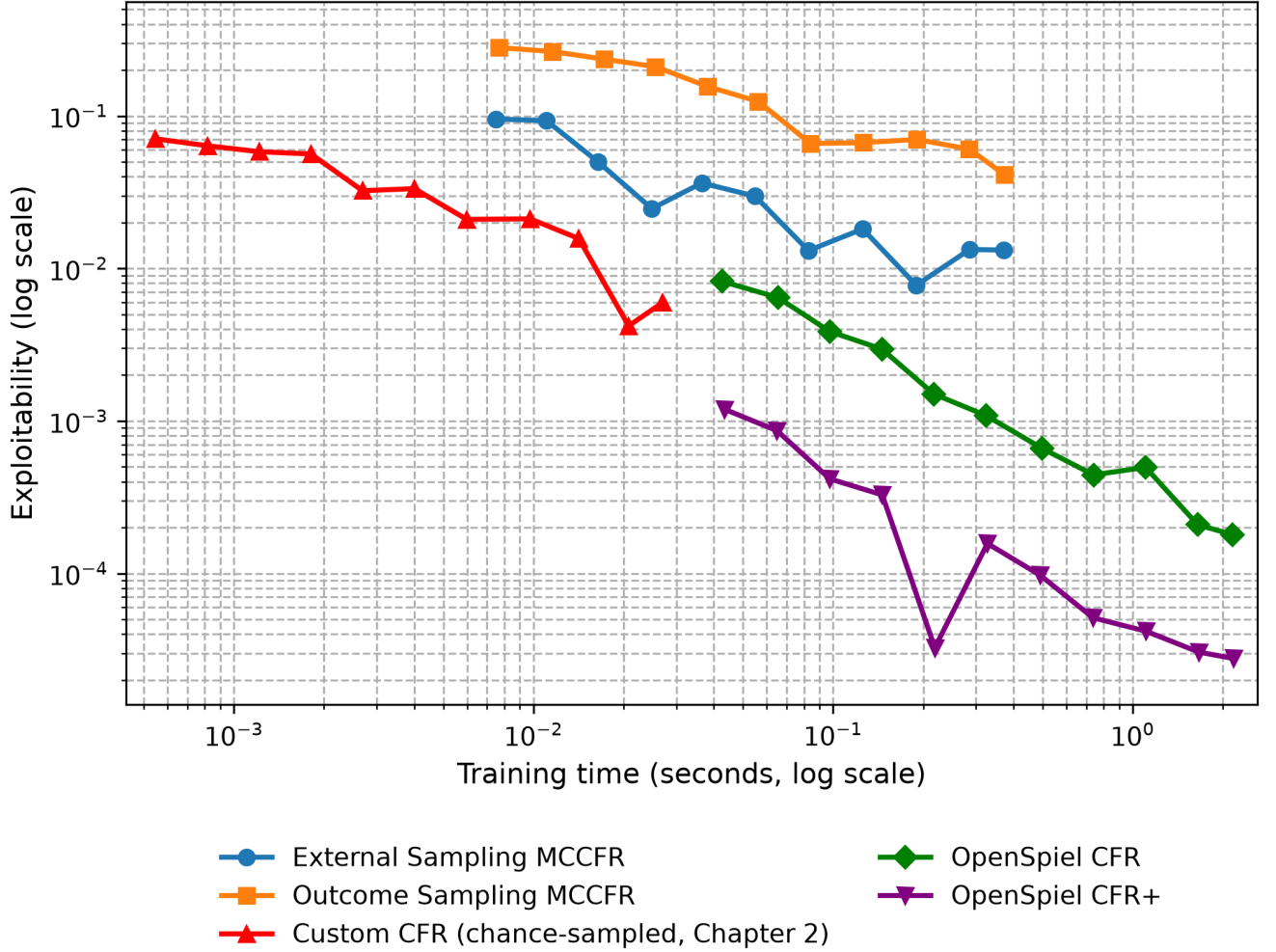


Figure 9: Kuhn Poker: exploitability vs training time

To see the tradeoff emerge, the game tree has to be large enough that per-iteration cost matters. That is Leduc’s role.³²

3.7 The Variance-Speed Tradeoff

All four algorithms have a guaranteed $O(1/\sqrt{T})$ upper bound on exploitability — CFR,³⁸ MCCFR with probability at least $1 - p$,³² and CFR+.^{36,37} Their measured rates differ: both MCCFR variants follow a slope of about -0.4 to -0.5 , vanilla CFR about -0.8 and CFR+ about -1.8 . The model $\epsilon(T) = C/\sqrt{T}$ below is therefore an approximation (for vanilla CFR, $\epsilon\sqrt{T}$ falls from 0.94 to 0.27) and does not apply to CFR+. Converting to wall-clock via $C_w = C/\sqrt{\text{speed}}$:

³⁸Zinkevich, M., Johanson, M., Bowling, M. & Piccione, C. (2007). “Regret Minimization in Games with Incomplete Information.” *Advances in Neural Information Processing Systems* 20.

Table 3: Measured convergence constants for the four solvers, per iteration and in wall-clock terms.

Algorithm	C_{iter}	Speed (iter/s)	C_w (wall-clock)	C_w relative to vanilla
Vanilla	0.34	20.6	0.075	1.0×
CFR				
CFR+	–	20.6	–	not $C/\sqrt{T}$ (see text)
MCCFR	107	19,470	0.767	10.2× ($\approx 105\times$ more time)
External				
MCCFR	245	45,901	1.143	15.3× ($\approx 233\times$ more time)
Outcome				

The wall-clock constant is what determines real-world performance. Despite being 945× faster per iteration, External Sampling’s variance constant (107 vs 0.34) is 315× larger. The speed advantage **does not overcome the variance penalty**:

$$\frac{99,000 \times \text{more iterations needed}}{945 \times \text{faster per iteration}} \approx 105 \times \text{more wall-clock time}$$

Why does the variance exist? Vanilla CFR computes **exact** counterfactual values by enumerating all 120 deals. Its regret update has zero variance. MCCFR samples one deal per iteration. Even if it explored the full subtree for that deal, it would still have variance across deals — a consequence of sampling the chance event (the deal) itself, not of how the tree is explored within a deal.

A formal 180-second timed benchmark of all four variants on Leduc makes the wall-clock comparison concrete. Per-iteration view (sampling variants complete millions of updates, full-traversal variants a few thousand):

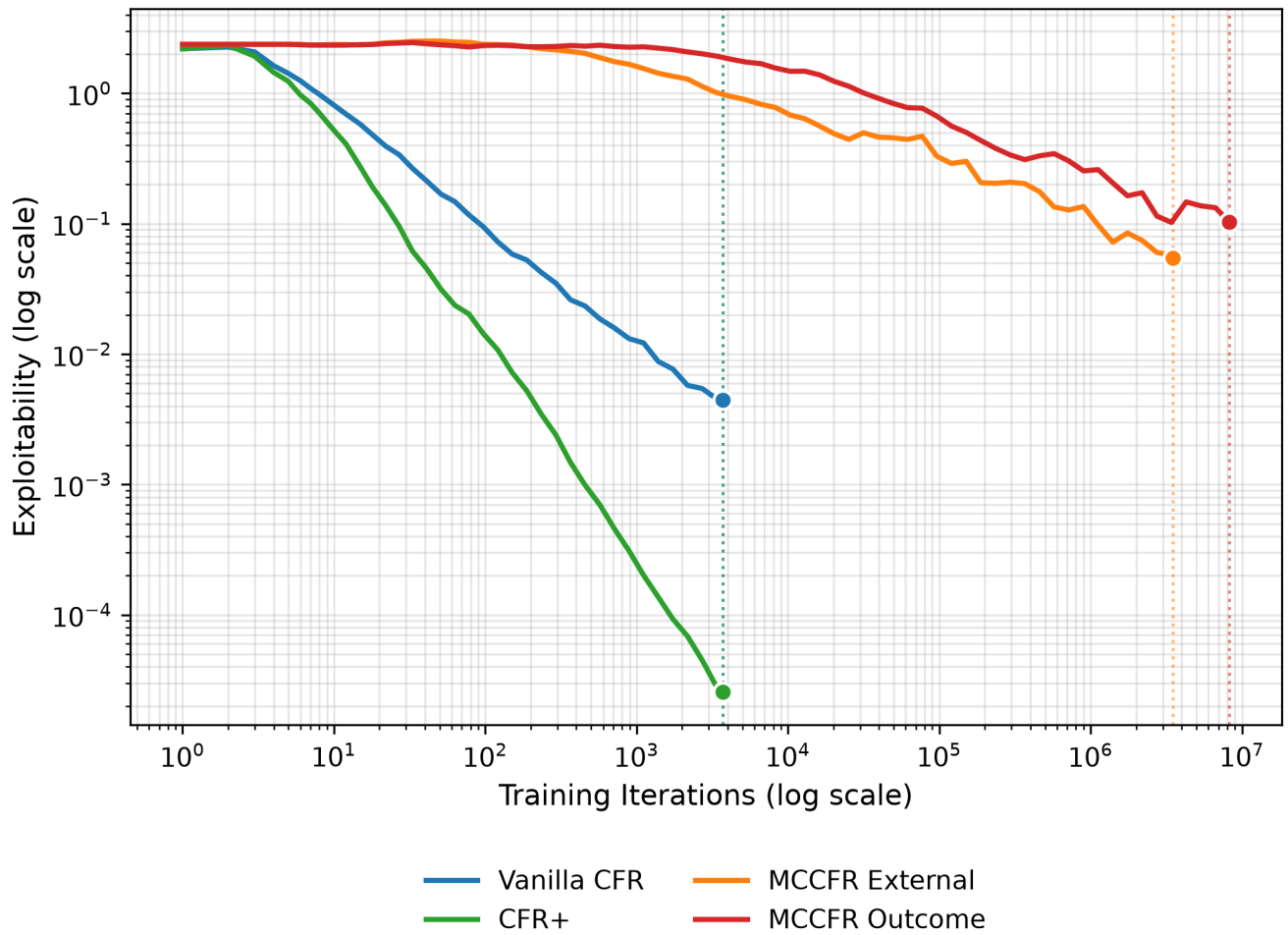


Figure 10: Leduc Poker, own implementations with a 180 s budget per algorithm: exploitability vs iterations

Wall-clock view — the fair comparison (given equal compute, which algorithm produces the best strategy?):

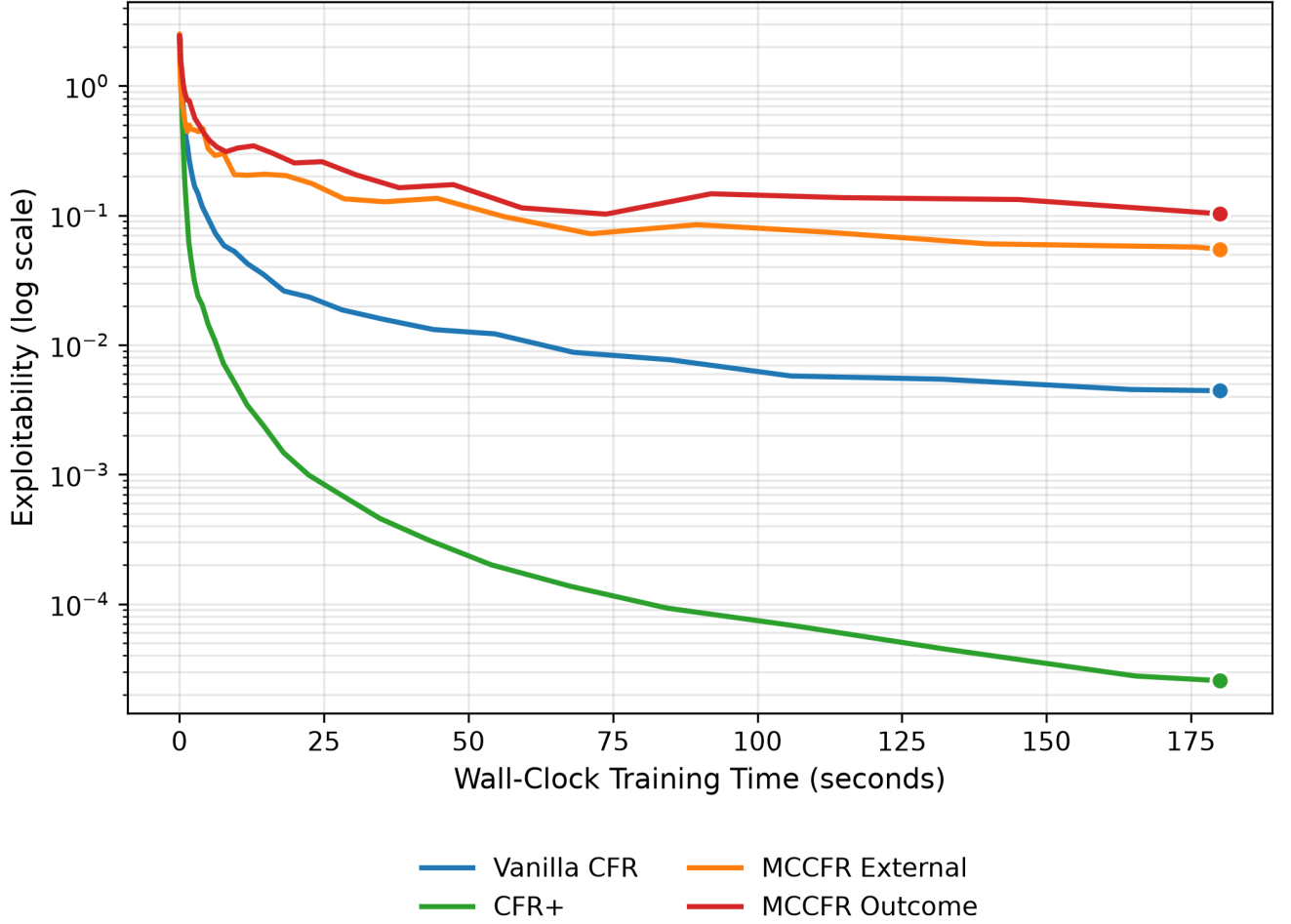


Figure 11: Leduc Poker, own implementations with a 180 s budget per algorithm: exploitability vs wall-clock time

CFR+ reaches exploitability 2.6×10^{-5} — nearly exact Nash — in 3 minutes. Vanilla CFR follows at 4.4×10^{-3} . Both MCCFR variants trail vanilla CFR by 12–23 $\times$ and CFR+ by three to four orders of magnitude on this game size.

3.8 When MCCFR Wins — The Crossover Point

The pattern above raises an obvious question: if MCCFR is always worse on Leduc, why was it invented? The answer is that vanilla CFR’s per-iteration cost grows linearly in the tree size $|N|$, while MCCFR’s grows far more slowly (roughly as $\sqrt{|N|}$ for external sampling³²). At sufficient scale the linear cost wins. The critical tree size is:

$$|N|_{\text{crossover}} = |N|_{\text{Leduc}} \cdot \left(\frac{C_{mc}}{C_v} \right)^2 \cdot \frac{\text{speed}_v}{\text{speed}_{mc}}$$

Plugging in measured values:

Table 4: Game size at which MCCFR overtakes vanilla CFR, per sampling variant.

Variant	$ N _{\text{crossover}}$
External Sampling	~1.1 million nodes
Outcome Sampling	~2.4 million nodes

Compared against actual games:

Table 5: The crossover threshold applied to three real games.

Game	$ N $	External wins?	Outcome wins?
Leduc Poker	10,200	No (105× too small)	No (233× too small)
Heads-up limit Texas Hold'em	$\sim 3 \times 10^{17}$	Yes by the model ($\sim 10^{11}$ × above), yet solved by full-traversal CFR+	Yes by the model
No-Limit Texas Hold'em	up to $\sim 6 \times 10^{164}$	Yes	Yes

On Leduc, full traversal dominates because the entire game tree fits in memory and can be enumerated in milliseconds. The estimate is an order-of-magnitude figure for this implementation: the constants are measured on Leduc only, vanilla CFR is modelled as $C/\sqrt{T}$ although it converges faster, and if MCCFR’s cost per iteration grows as $\sqrt{|N|}$ the same data put the threshold near 10^8 nodes.

At real poker scales the conclusion reverses, though not uniformly. Heads-up limit hold'em (3.16×10^{17} states) was still solved by full-traversal CFR+ — 1,579 iterations of about 61 minutes each on 4,800 cores³⁹ — and its authors report CFR+ needing considerably less computation than sampling CFR. No-limit hold'em (up to 6.31×10^{164} states in the ACPC formats⁴⁰) is beyond any full traversal; there sampling — MCCFR combined with abstraction and variance-reduction extensions⁴¹ — is the practical route.

³⁹Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Heads-up limit hold'em poker is solved.” *Science*, 347(6218), 145–149. DOI 10.1126/science.1259433. Used CFR+ to solve heads-up limit Texas Hold'em — the first non-trivial imperfect-information game played competitively by humans to be essentially solved.

⁴⁰Johanson, M. (2013). “Measuring the Size of Large No-Limit Poker Games.” arXiv:1302.7008.

⁴¹Schmid, M., Burch, N., Lanctot, M., Moravčík, M., Kadlec, R. & Bowling, M. (2019). “Variance Reduction in Monte Carlo Counterfactual Regret Minimization (VR-MCCFR) for Extensive Form Games Using Baselines.” *Proc. AAAI*, 33(01), 2157–2164.

3.9 Connections to Chapter 2 and Forward Pointers

Same principle, different regime. Chapter 3 keeps the principle of Chapter 2 — minimizing regret locally at each information set drives the global strategy to Nash equilibrium — but introduces two independent refinements: CFR+ modifies how regrets accumulate (flooring + linear weighting), and MCCFR modifies how regrets are measured (sampled estimator instead of exact expectation). Both normally output the **average** strategy, although the landmark CFR+ solve of heads-up limit hold'em used the players' current strategies instead.³⁹

Forward:

- **Chapter 4 (Game Abstraction & Scaling)** — reduces the tree itself through state and action abstraction; combined with MCCFR, this makes real poker tractable.
- **Chapter 5 (Neural Networks for Imperfect-Information Games)** — replaces tabular strategy storage with neural function approximators, using MCCFR's sampling framework as the data-generation engine. The variance properties established here explain why Deep CFR samples externally and why its outcome-sampling successor DREAM adds a learned baseline to control variance.
- **Chapter 6 (End-to-End Game AI Architectures)** — builds on both CFR+ and MCCFR for complete agents. The Leduc engine built for this chapter remains the benchmark environment in later chapters.⁴²

⁴²Brown, N. & Sandholm, T. (2017). "Safe and Nested Subgame Solving for Imperfect-Information Games." *NeurIPS* — bridges tabular CFR with subgame decomposition for real poker.

4 Chapter 4 — Game Abstraction & Scaling Imperfect-Information Games

4.1 Why Abstraction Is Needed

Chapters 1–3 built the algorithms. Each was demonstrated on a game small enough to enumerate exactly: Kuhn (12 information sets), Leduc (936). That toolkit is complete — but it only works when the entire game tree fits in memory.

The size of a real game grows with the hidden cards, with the branching of bet sizes and with the depth of the betting: heads-up limit hold'em reaches 3.16×10^{17} states,⁴³ and the combined number of information sets in heads-up no-limit Hold'em is $\sim 10^{161}$ — more than there are atoms in the observable universe by a factor of $\sim 10^{80}$. None of Chapter 3's algorithms can run on that tree.⁴⁴

The mechanism that bridges the gap is **abstraction**: deliberately collapse parts of the game so the same algorithms can run on a smaller, structurally simpler proxy, and measure what that costs in strategy quality. The central question of every practical poker AI since 2007 is:

How much can the game be shrunk before the abstract Nash strategy stops being a good strategy in the real game?

The answer is a Pareto curve: the size of the abstracted game against the **exploitability gap** — how much worse the abstract strategy is than the real game's exact Nash.

The recipe is the same across every approach: *build a smaller game whose strategies translate back into playable strategies for the real game, run the Chapter 3 algorithms on that smaller game, then bound the damage*. That recipe has two routes.

4.2 Routes to Abstraction

In this chapter the word “abstraction” covers two operationally different things; the literature usually presents neural approximation as an alternative to abstraction rather than a kind of it.⁴⁵ This chapter implements the **explicit** route end-to-end; since no implicit work is performed here, the implicit counterpart of each phase is only listed in the table below.

Both routes share the same **intuition** — compress while preserving value — captured by the Information Bottleneck Lagrangian (by analogy with the information bottleneck method⁴⁶), with β as the exchange

⁴³Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Heads-up limit hold'em poker is solved.” *Science*, 347(6218), 145–149. DOI 10.1126/science.1259433. Used CFR+ to solve heads-up limit Texas Hold'em — the first non-trivial imperfect-information game played competitively by humans to be essentially solved.

⁴⁴Johanson, M. (2013). “Measuring the Size of Large No-Limit Poker Games.” Technical report, University of Alberta; arXiv:1302.7008.

⁴⁵Brown, N., Lerer, A., Gross, S. & Sandholm, T. (2019). “Deep Counterfactual Regret Minimization.” *ICML*.

⁴⁶Tishby, N., Pereira, F. C. & Bialek, W. (1999). “The Information Bottleneck Method.” arXiv:physics/0004057.

rate between memory and value:

$$\mathcal{L} = \underbrace{\text{Complexity}(Z)}_{\text{memory cost}} - \beta \cdot \underbrace{\text{Value}(\pi_Z)}_{\text{strategic worth}}$$

When $\beta = 0$, the algorithm compresses the entire game into one state and plays terribly. When $\beta \rightarrow \infty$, it refuses to merge anything and uses the full game. Every algorithm in the abstraction pipeline is one specific instantiation of that knob:

- **Lossless merging** — corner case $\beta \rightarrow \infty$: only merges when $\text{Value}(\pi_Z) = \text{Value}(\pi)$ exactly.
- **Bounded lossy** — middle of the curve with a guarantee: pick β such that $|\text{Value}(\pi_Z) - \text{Value}(\pi)| \leq \varepsilon_{\text{abs}}$.
- **Empirical similarity (HSD + EMD)** — measures the curve itself: EMD between abstract and full distributions is a proxy for the value-loss term.
- **Runtime patching** — orthogonal: accept the value loss from any chosen β , then patch it at runtime via subgame solving.

Table 6: The explicit and implicit routes to abstraction, compared on every practical axis.

Aspect	Explicit / tabular	Implicit / IB-style (Deep RL)
Where compression lives	A partition over info sets / a finite chosen set of actions	A continuous latent vector $z = f_\theta(s)$
The “knob”	k in k-means buckets, the bet-set, the suit-isomorphism rule	β multiplying $I(S; Z)$ in the loss
Guarantee	A formal bound on the exploitability gap only for lossless and bounded abstractions; none for empirical (k-means + EMD) ones	Information-theoretic bound on $I(S; Z)$; no Nash-preservation theorem
Counterpart of each phase	The four phases of this chapter	Deep Sets, information bottlenecks, learned embeddings; rate-distortion curves and test loss; no separate build phase; depth-limited search with neural value functions (DeepStack, ReBeL)
Where it appears in this thesis	This chapter (4)	Chapter 5 (Deep CFR), chapter 6 (end-to-end), chapter 12 (sequence models)

4.3 Axes of Abstraction

Every concrete technique falls onto one of two orthogonal axes — *what states the agent treats as the same* (information abstraction) and *what moves the agent considers* (action abstraction); runtime refinement has its own section below.

4.3.1 Information Abstraction

Group together info sets that the agent will treat as the same state; the merging criterion, the error budget and the build-time pipeline below are all about *how* and *when* to do it. In vanilla CFR only abstracting the **traversal**, not just the **node map**, produces a wall-clock speedup.

4.3.2 Action Abstraction and the Translation Problem

Restrict the set of actions the solver considers, run CFR on the restricted game, then handle whatever the real opponent does that lies outside that restriction. For continuous bet sizes in no-limit poker this is *mandatory* for tabular methods — CFR cannot run on a continuous tree without first collapsing it to a finite proxy (typical poker grid: {fold, call, 0.5×pot, 1×pot, 2×pot, all-in}).

The translation problem is then **acute**: when the opponent plays a 0.7×pot bet but the abstraction only contains 0.5×pot and 1×pot, the agent must convert that bet to a node it has trained on. Three translators in common use:

1. **Nearest-action** — round to the closest abstract bet; worst when bets cluster between abstract sizes.
2. **Probability-split (linear)** — split the mass between the two nearest abstract bets by distance.
3. **Pseudo-harmonic mapping** — a randomized mapping derived from the equilibrium of a simplified poker game; in its authors’ comparisons markedly less exploitable than earlier heuristic mappings, and the leading static translator before nested subgame solving (below).⁴⁷

Translation errors compound across betting rounds — an opponent who detects rounding can systematically bet just below or above grid points to coerce wrong-sized responses. Such exploitation has been observed in practice (the agent Tartanian1, 2007),⁴⁷ and it is the operational reason runtime patching (below) exists.

4.4 The Exploitability Gap

For a strategy played in game G , exploitability is the standard metric from Chapter 3. Chapter 4 introduces a derived metric, the **exploitability gap**: how much worse an abstract strategy is than the

⁴⁷Ganzfried, S. & Sandholm, T. (2013). “Action Translation in Extensive-Form Games with Large Action Spaces: Axioms, Paradoxes, and the Pseudo-Harmonic Mapping.” *IJCAI*, 120–127.

real game’s exact Nash, *measured in the real game*.

Let G be the real game and $\hat{G}$ its abstraction. Let $\hat{\sigma}^*$ be the Nash of $\hat{G}$, and let $T(\hat{\sigma}^*)$ be its translation back into a playable strategy for G (identity for a pure information abstraction). Then

$$\Delta_{\text{abs}}(\hat{G}) = \text{exploit}_G(T(\hat{\sigma}^*)) - \text{exploit}_G(\sigma_G^*)$$

with σ_G^* the exact Nash of G . By definition $\Delta_{\text{abs}} \geq 0$, with equality if the abstraction is lossless. Every Pareto plot in this chapter has Δ_{abs} on one axis; where computing it outright is too expensive, the error budget below supplies a bound and a proxy in its place.

4.5 Earth Mover’s Distance: A Primer

What it measures. Given two probability distributions over the same support, EMD — also called the **Wasserstein-1 distance**⁴⁸ — is the minimum amount of “work” needed to transform one into the other, where *work* = *mass moved* × *distance moved*. Imagine each distribution as piles of sand on a number line — EMD is the smallest total effort to reshape pile A into pile B. Unlike a comparison of means, it respects the *geometry* of where mass lives: two distributions concentrated at opposite ends are far apart, regardless of where their means happen to coincide.

In this chapter’s context. Each information set has a *Hand Strength Distribution (HSD)* — the histogram of end-of-game win probabilities computed by rolling out unseen cards. Two HSDs with the same mean can have completely different shapes: a peaked “stable medium hand” vs a bimodal “boom-or-bust” hand. Expected-hand-strength clustering merges these together; EMD does not.

Cheap on 1-D histograms. When the support is one-dimensional and ordered (binned win probability $\in [0, 1]$), EMD reduces to the L1 distance between cumulative distributions — a single linear sweep over the bins. This is why HSD + EMD is fast enough to cluster millions of info sets.

4.6 The Merging Criterion

When can two information sets be collapsed into one? Three nested levels of strictness, strictest at the top.

4.6.1 Level 1 — Lossless

Merge two info sets only when they are *strategically identical*: same probability of being reached, same recursive structure, and same utility consequences against every possible opponent continuation.⁴⁹ When

⁴⁸Rubner, Y., Tomasi, C. & Guibas, L. J. (2000). “The Earth Mover’s Distance as a Metric for Image Retrieval.” *International Journal of Computer Vision*, 40(2), 99–121.

⁴⁹Gilpin, A. & Sandholm, T. (2007). “Lossless Abstraction of Imperfect Information Games.” *Journal of the ACM*, 54(5), art. 25 — GameShrink, and the Rhode Island Hold’em result.

this holds, the merge is free: any optimal strategy in the abstract game lifts to an optimal strategy in the original game with **zero** exploitability cost.

Example: A red Jack and a black Jack in a game where suits don’t matter (no flushes possible) are losslessly mergeable. A red Jack in a game *with* flushes is not — the suit silently affects the opponent’s flush odds through card removal.

4.6.2 Level 2 — Bounded lossy

Relax “identical utilities” to “utilities differ by at most ε ”. Each merge then contributes a slack budget that propagates into a bound on overall exploitability — *controlled* forgetting, not arbitrary.

Example: Putting J and Q into a single “low card” bucket forces the agent to play them identically, even though optimally they differ slightly. The system absorbs that mistake in exchange for a much smaller game.

4.6.3 Level 3 — Empirical similarity

When the inputs are too high-dimensional to enumerate (Texas hold’em has about 2.4×10^9 canonical private/public card combinations on the last round⁵⁰), drop the per-merge bound: compute an HSD per info set, use EMD between them as the distance,⁵¹ and merge within clusters. There is no formal guarantee — quality is checked *after* solving, and even refining an abstraction does not guarantee lower exploitability.⁵²

4.7 The Error Budget

Three quantification tools: the analytical bound prices a Level 2 merge, the EMD proxy scores a Level 3 one, and the third measures the finished abstraction whichever way it was built.

4.7.1 Tool 1 — Analytical bound

Sum the per-merge slack constants, weighted by how often each info set is actually reached during play, to get an upper bound on the exploitability gap.⁵³ The reach-weighting is the key idea: a sloppy merge in a rare endgame scenario costs almost nothing overall, while sloppy merges in dense, frequently-visited regions are catastrophic.

⁵⁰Johanson, M., Burch, N., Valenzano, R. & Bowling, M. (2013). “Evaluating State-Space Abstractions in Extensive-Form Games.” *AAMAS*, 271–278.

⁵¹Ganzfried, S. & Sandholm, T. (2014). “Potential-Aware Imperfect-Recall Abstraction with Earth Mover’s Distance in Imperfect-Information Games.” *AAAI*, 28(1).

⁵²Waugh, K., Schnizlein, D., Bowling, M. & Szafron, D. (2009). “Abstraction Pathologies in Extensive Games.” *AAMAS*, 781–788.

⁵³Kroer, C. & Sandholm, T. (2014). “Extensive-Form Game Abstraction with Bounds.” *ACM EC*, 621–638; and Kroer, C. & Sandholm, T. (2016). “Imperfect-Recall Abstractions with Bounds in Games.” *ACM EC*, 459–476.

4.7.2 Tool 2 — EMD proxy

EMD between the hand-strength histograms of two info sets, measured without enumerating leaves. It is a *proxy*, not a bound, and carries no formal guarantee.⁵⁰ In this chapter’s runs it does not track exploitability: it is zero for the $k = 3$ and $k = 5$ abstractions, whose exploitability stays at 0.38–0.57.

4.7.3 Tool 3 — CFR-BR direct evaluator

The strongest measurement: for perfect-recall abstractions it finds the strategy within the abstraction that is least exploitable in the full game, *isolating* abstraction error from solving error. CFR-BR strategies show exploitability as low as 1/3 of the plain-CFR strategies on the *same* abstraction — part of the measured “abstraction error” is the choice of the abstract game’s equilibrium as the target, not the abstraction itself.⁵⁰

4.8 Choosing an Abstraction in Practice

The decision order. First take every lossless merge available; then accept bounded lossy merges only when the error budget is tolerable; otherwise cluster with HSD + EMD and verify the resulting abstraction empirically.

Global optimisation is off the table. Picking the partition that minimises the analytical bound is **NP-complete**, even for a tiny single-player game two levels deep. Practical pipelines therefore work level by level; under reasonable conditions a single level reduces to k -centre clustering, which has polynomial-time approximation algorithms with constant-factor guarantees.⁵³

Distribution-aware beats expectation-based. On equal information-set budgets, once the abstraction is large enough, HSD + EMD beats expected-strength clustering on both exploitability and head-to-head play; in very small abstractions the expectation-based one can be stronger.⁵⁰ Together with the recall result below, this is why the build-time pipeline defaults to imperfect recall + HSD + EMD.

4.9 Build-Time Decisions

Two complementary pipelines, one per criterion-strictness level.

4.9.1 Pipeline 1 — GameShrink (lossless)

An exhaustive merger that walks the *signal tree* bottom-up, merging every pair of sibling subtrees that passes the strategic-equivalence test. The signal tree is smaller than the game tree because it collapses all betting paths leading to the same public + private signals into one node: on Rhode Island Hold’em it is ~6.6M nodes vs ~3.1B game-tree nodes. This pipeline + linear programming solved Rhode Island Hold’em in 2007.⁴⁹

4.9.2 Pipeline 2 — HSD + EMD + k-means + imperfect recall (lossy)

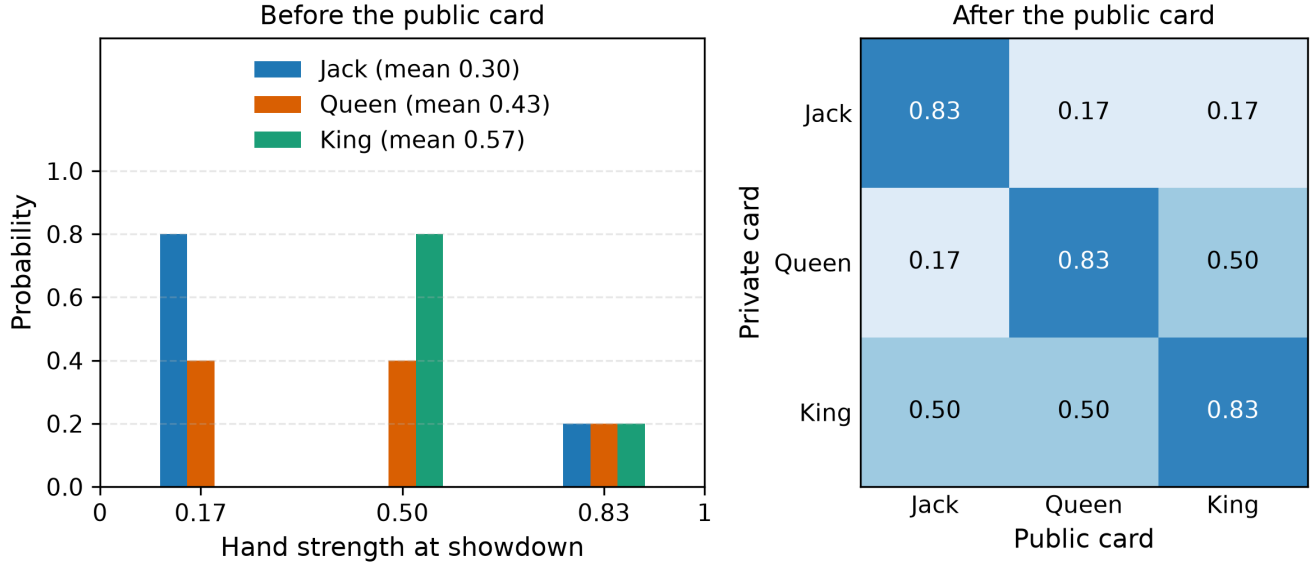


Figure 12: Leduc hand-strength distributions (HSD). Left: each private card before the public card. Right: the nine (private, public) combinations give three distinct HSDs; colour is the EMD k-means bucket, the same for $k = 3$ and 5 .

When the lossless merger has run to convergence and the game is still too large, switch to clustering: compute an HSD for each information set, cluster those distributions with EMD as the distance metric, and map each information set to a bucket id. Under *perfect recall* the bucket identity includes the past bucket trail; under *imperfect recall* the later-round bucket can forget it, freeing more buckets for the round where new information matters most.

In Texas hold'em, imperfect recall wins at a fixed bucket budget — capacity is spent on rounds that matter rather than on remembering history.⁵⁰ The Leduc runs of this chapter do not reproduce it: at a similar size (108 vs 132 information sets) imperfect recall is no better (0.574 vs 0.571; see the fixed-limit Leduc figure under Practical Validation).

The output. The frozen result of both pipelines — the strategy obtained by solving the abstract game, and the starting point for everything that happens at runtime — is referred to throughout the rest of this chapter as the **blueprint**.

4.10 Runtime Patching

When play descends into a subgame and the abstract blueprint is too coarse, re-solve the subgame at higher fidelity *in real time*. Two patches matter — together they were the load-bearing components of

Libratus, the first AI to defeat top humans in heads-up no-limit Texas hold'em.⁵⁴

4.10.1 Why subgame solving cannot be done in isolation (Coin Toss)

A simple counterexample called *Coin Toss*: a coin lands Heads or Tails with equal probability, only P_1 sees the outcome. P_1 chooses *Sell* (with payoff that depends on the coin) or *Play* (where P_2 guesses the side). The optimal P_2 strategy in the *Play* subgame is **not** a function of the *Play* subgame alone — it depends on the value P_1 would have gotten by choosing *Sell* instead. Change *Sell*'s payoff and the optimal *Play* strategy flips, even though the *Play* subgame itself is unchanged.⁵⁵

The fix: solve an *augmented subgame* that includes the original subgame plus extra “alternative-payoff” nodes encoding what each player could have achieved by *not entering* this subgame.⁵⁶

4.10.2 Patch 1 — Safe subgame solving

The augmented subgame is anchored to blueprint values: each top-of-subgame information set gets an alternative payoff equal to what the blueprint promised that player at this point. Solving it yields a refined strategy whose exploitability is provably no higher than the blueprint's.⁵⁵ Replacing the conservative blueprint payoffs with *estimates* of equilibrium value drops the strict guarantee but typically lowers real-play exploitability.

4.10.3 Patch 2 — Nested subgame solving (the action-translation killer)

When the *opponent* plays an action a outside the abstraction, do not round a to a known action via a translator — *re-solve a fresh subgame that contains a* , with the safe-subgame scaffold, and append the new sub-strategy to the blueprint; the blueprint grows only where play actually goes. In a no-limit flop hold'em test game, this was about 10× less exploitable than pseudo-harmonic translation (119–150 vs 1,465 mbb/hand).⁵⁵ Static translators remain the right choice only when latency cannot afford a live CFR solve.

4.11 Architecture: Blueprint + Live Patches

The full pipeline adds up to a single architectural pattern, used by the leading no-limit poker AIs since 2017 — Libratus⁵⁴ and Modicum heads-up, Pluribus⁵⁷ six-handed (DeepStack⁵⁸ is the exception: it computes no whole-game strategy in advance): abstract and solve the game at build time, freeze the **blueprint**, and re-solve at runtime wherever the blueprint is coarse or the opponent leaves the abstraction.

⁵⁴Brown, N. & Sandholm, T. (2018). “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals.” *Science*, 359(6374), 418–424.

⁵⁵Brown, N. & Sandholm, T. (2017). “Safe and Nested Subgame Solving for Imperfect-Information Games.” *NeurIPS*.

⁵⁶Burch, N., Johanson, M. & Bowling, M. (2014). “Solving Imperfect Information Games Using Decomposition.” *AAAI* — re-solving and the augmented subgame.

⁵⁷Brown, N. & Sandholm, T. (2019). “Superhuman AI for multiplayer poker.” *Science*, 365(6456), 885–890.

⁵⁸Moravčík, M. et al. (2017). “DeepStack: Expert-level artificial intelligence in heads-up no-limit poker.” *Science*, 356(6337), 508–513.

4.12 Practical Validation

The implementation phase built a small Leduc-family pipeline — suit isomorphism, card bucketing, action abstraction on Mini-NL Leduc, a larger Extended Leduc (four ranks, two suits) and their combination — evaluated by exploitability gaps, EMD proxies, OpenSpiel cross-validation and a Pareto frontier.

The main empirical lesson matches the theory: lossless abstraction is almost free strategically and very useful computationally, while lossy information and action abstraction introduce persistent exploitability floors. Under 180-second CFR+ budgets, suit isomorphism reduced fixed-limit Leduc from 936 to 288 information sets and reached lower exploitability than the full game because it completed many more iterations. On Extended Leduc, the same idea reduced the game from 10,304 to 2,968 information sets and improved final exploitability from about 2.7×10^{-2} to about 1.3×10^{-3} .

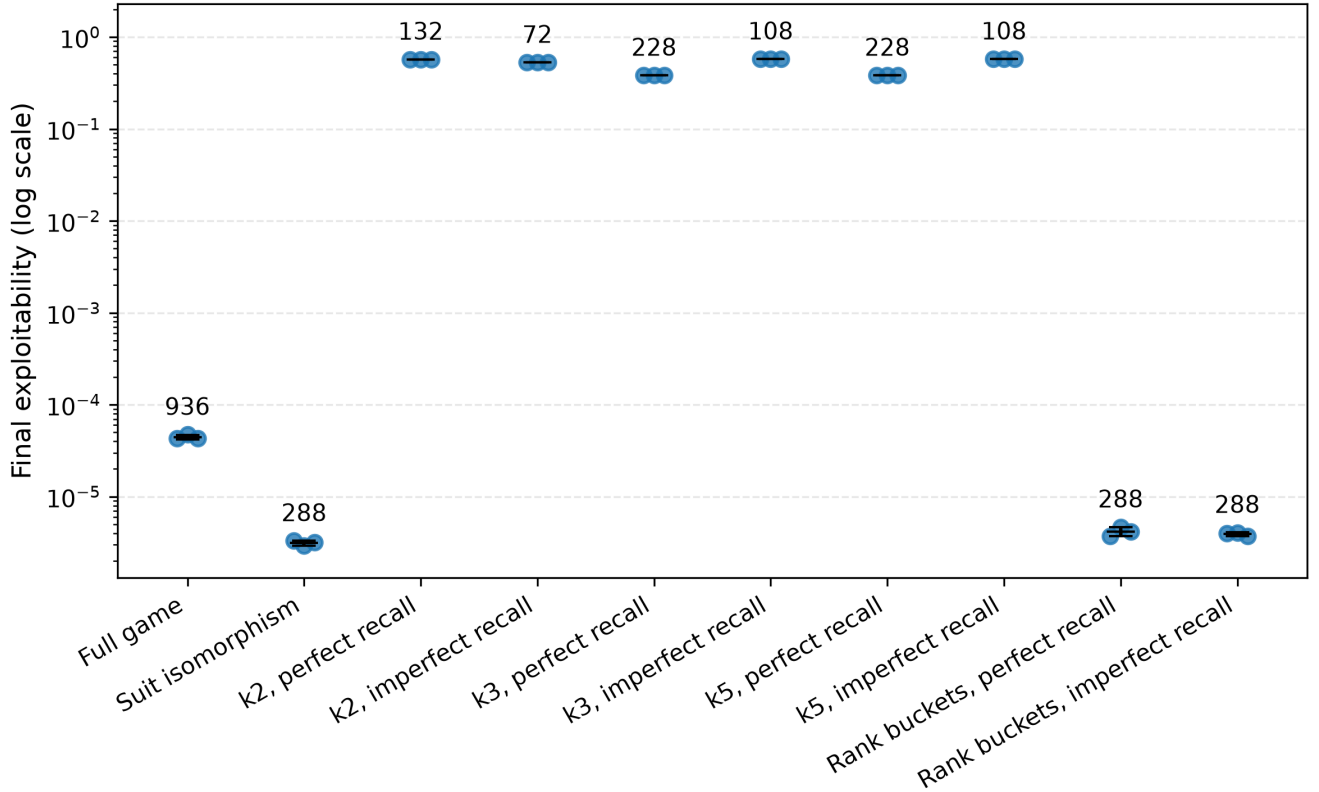


Figure 13: Fixed-limit Leduc: final exploitability after 180 s of CFR+ for each abstraction configuration (three runs, log scale); the number above each group is its information-set count.

The lossy bucket runs show the other side of the tradeoff. Smaller bucketed games train faster, but the error does not disappear with more CFR+ iterations because the strategy is solving the wrong game. In fixed-limit Leduc, coarse bucket abstractions remained around 0.38-0.57 exploitability, even though they completed more iterations than full CFR+. This is the practical meaning of the exploitability gap:

abstraction error is not optimizer error.

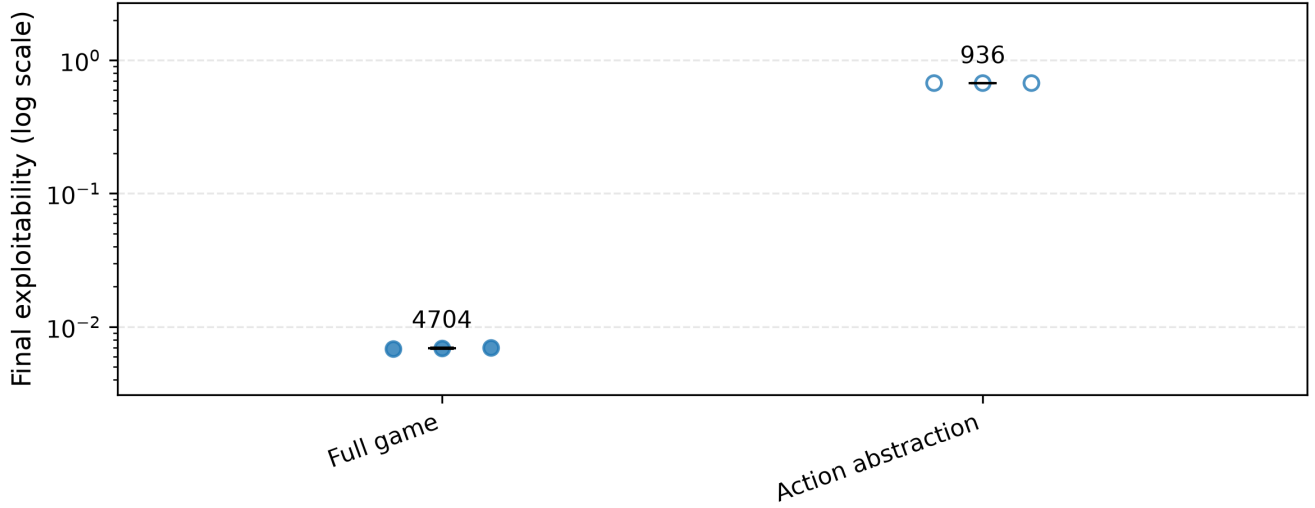


Figure 14: Mini-NL Leduc: final exploitability after 180 s of CFR+, full game against action abstraction (hollow markers, reflecting the current deployment rule; see text); numbers are information-set counts.

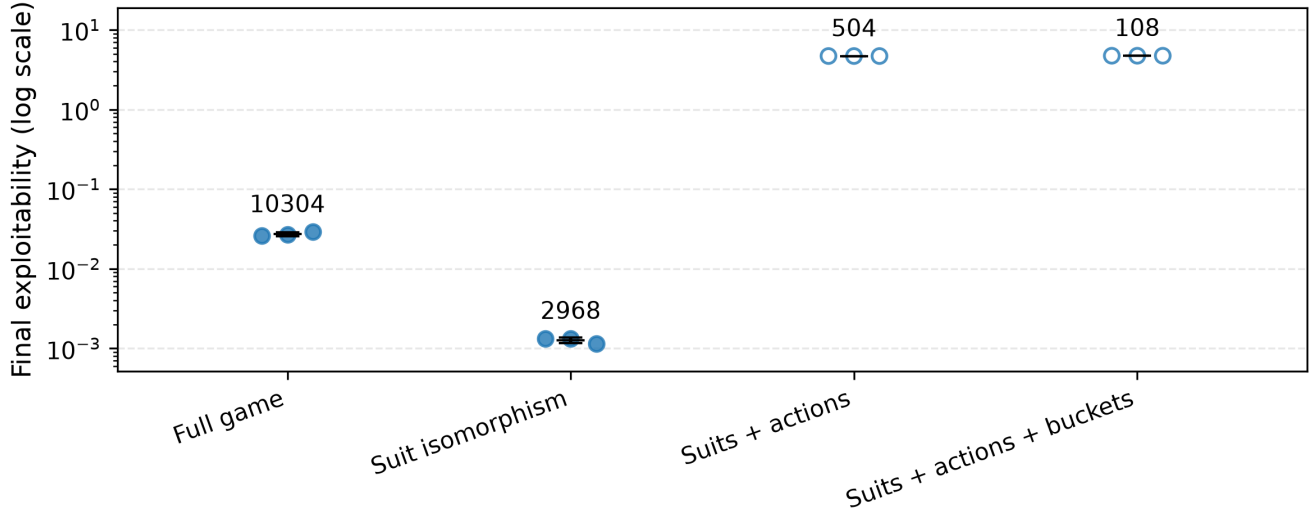


Figure 15: Extended Leduc: final exploitability after 180 s of CFR+ for the full game, suit isomorphism and the combined abstractions; hollow markers include action abstraction (see text); numbers are information-set counts.

Action abstraction was the riskiest part of the chapter. In Mini-NL Leduc, restricting the action set reduced the information-set count from 4,704 to 936 and produced many more CFR+ iterations under the same time budget, but exploitability stayed high. In Extended Leduc, adding action abstraction on top of suit isomorphism produced a compact tree, but the deployed strategy was highly exploitable. These numbers measure the current deployment rule — every abstract small bet is played as the large bet, and opponent large bets are read as small — rather than the three translators, which never engage in this

harness and return identical values. They show how much a naive mapping between the abstract and the real action set can cost; the literature’s answer to off-tree actions is nested subgame solving.⁵⁵

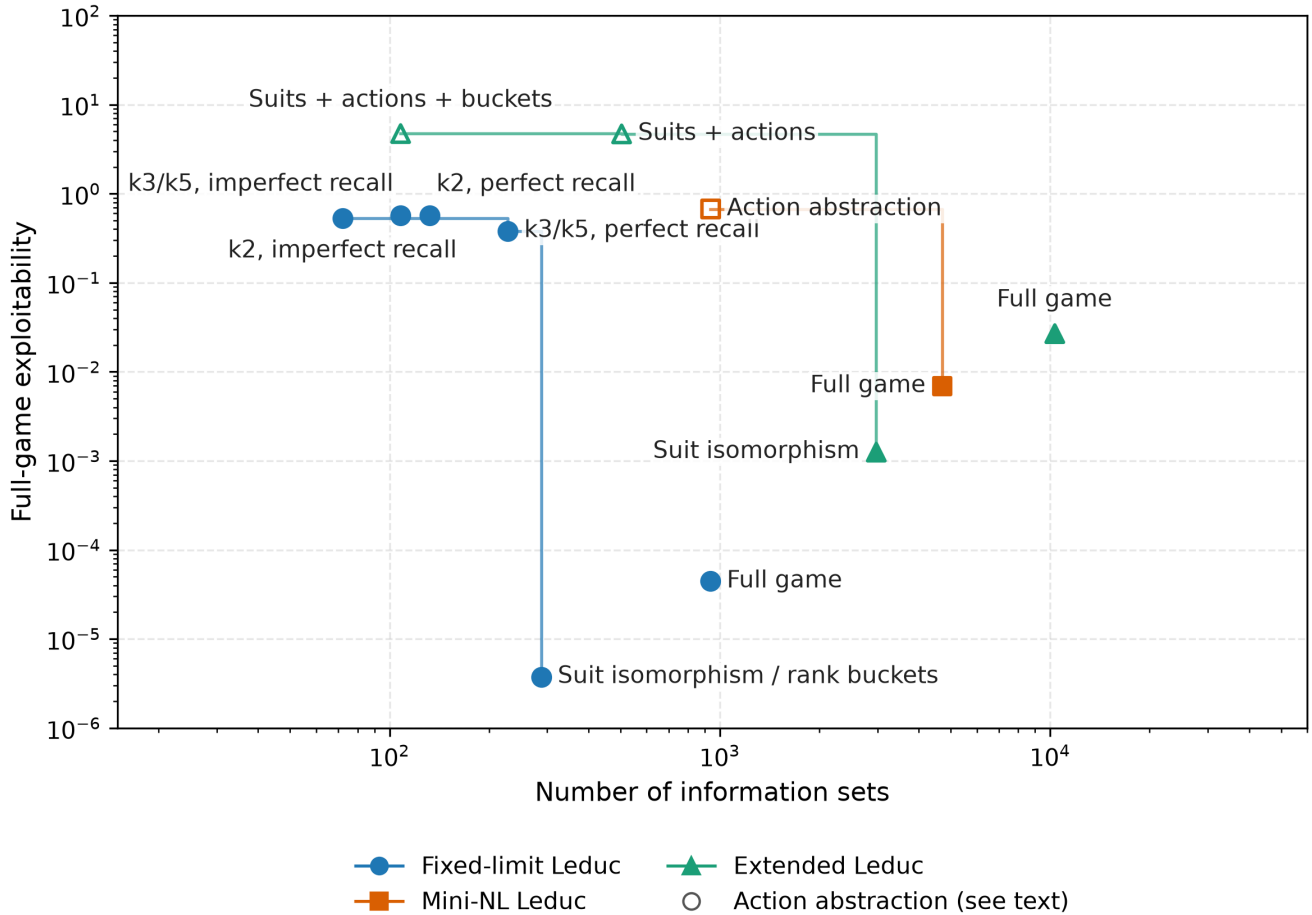


Figure 16: Pareto frontier: information-set count against full-game exploitability after 180 s of CFR+ (mean of three runs); lines join non-dominated points; hollow markers are action-abstraction configurations under the current deployment rule.

The Pareto view is the right final diagnostic. Lossless suit isomorphism lands on the attractive part of the frontier. Coarse buckets and action abstraction can reduce the game further, but they move onto a different regime where smaller size is paid for with strategy quality.

4.13 Connections and Forward Pointers

Chapter 4 changes the input game itself, asking which states and actions can be merged without destroying the strategic signal. The bridge to Chapter 5 is the implicit route: Deep CFR replaces the hand-built and clustering-built partitions with learned function approximators. The bridge to Chapter 6 is the blueprint architecture: Chapter 4 supplies the vocabulary — blueprint, action translation, exploitability

gap, Pareto frontier, and safe/nested refinement — and Chapter 6 turns those pieces into complete game-playing systems.

For the thesis, abstraction matters because opponent adaptation only works at the resolution the representation preserves. If the abstraction merges two strategically distinct opponent-facing states, no downstream opponent model can recover that distinction. Conversely, a representation that is too fine may be too expensive to solve or evaluate. The Chapter 4 Pareto frontier therefore becomes part of the evaluation methodology: strategy quality must be reported together with the size and granularity of the game representation that produced it.

5 Chapter 5 — Neural Networks for Imperfect-Information Games

This is a condensed survey of neural-network methods for imperfect-information games — how networks are built, trained and deployed when players cannot see the full state — and serves as a quick reference while reading the later chapters. Chapter 5 is a deliberate one-off deviation from the usual learning cycle: the from-scratch implementation phase is deferred and the theory is broadened in its place, so this document is a breadth-first map of the field rather than a record of code. Sections on learned abstraction, evaluating neural policies, experimental architectures and a closing decision map are planned but not included in this version.

Why Neural Networks

Tabular CFR and its Monte Carlo variants (Chapters 2–4) store a regret value and a strategy value at each information set, so their memory cost grows with the number of information sets.⁵⁹ The experiments in this chapter quantified this limit on Leduc poker, a game small enough to hold in a table: tabular MCCFR reached an exploitability of 0.097, whereas Deep CFR under a comparable wall-clock budget stayed above 1.1 at every network size tried. At this scale the neural method offers no advantage, because the table already fits.

A neural network replaces the table with a function f_θ that maps a state’s features to a value or an action distribution and generalizes to states never visited during training. This inverts the cost structure: memory becomes independent of the number of information sets, at the price of more expensive iterations and weaker convergence guarantees. The two regimes meet at a crossover in game size — below it, enumeration is cheaper and exact; above it, the table cannot be constructed and function approximation is the only option. The games this research targets lie above the crossover.

5.1 Fundamentals

5.1.1 Neural Network Fundamentals

The chapter assumes the standard deep-learning toolkit. A feed-forward network stacks layers $h = \sigma(Wx + b)$ — an affine map followed by a nonlinearity such as ReLU — into a function f_θ . It is trained by gradient descent on a loss (mean squared error for values, cross-entropy for action distributions), with backpropagation computing the gradient and Adam, the default optimizer here, adapting the step size of each parameter. Weight decay, dropout and early stopping counter overfitting. From reinforcement learning, *bootstrapping* — updating a value toward the network’s own estimate at the next state (Chapter 1) — makes learning sample-efficient but less stable. Full treatments are in Sutton & Barto (2nd ed.), chapters 9 and 11, and in any standard deep-learning reference.

⁵⁹Zinkevich, M., Johanson, M., Bowling, M. & Piccione, C. (2007). “Regret Minimization in Games with Incomplete Information.” *Advances in Neural Information Processing Systems 20*, 1729-1736.

5.1.2 Putting the Fundamentals Together on Leduc Poker

These pieces combine into the simplest neural solver for a game. Take Leduc poker, the testbed from Chapters 3–4: its information state — the player to act, the private card, the public card and the betting history — encodes as a vector of thirty numbers. A small network maps that vector to a value for each action; backpropagation with Adam fits it to target values generated during play, and light regularization prevents it from memorizing the noise in those targets. Used well, the network provides two capabilities the table lacks: it generalizes, so a value learned at one information set informs similar unseen ones, and its memory cost is fixed by the number of parameters rather than by the number of information sets.

The same machinery degrades when overdriven. On Leduc the smallest network tested reached a lower exploitability than a network several times larger, probably because at so small a budget the larger networks see too few distinct samples to train (a one-seed hypothesis). A learning rate set too high destabilizes the descent; excessive regularization flattens the sharp, near-deterministic strategies CFR converges to, since heavy weight decay or dropout pulls the output toward the uniform distribution. Capacity and regularization must be matched to the amount and quality of the data rather than maximized by default — a point Section 5.2 develops into explicit sizing guidance.

5.1.3 Non-Stationarity and the Deadly Triad

Combining these components introduces instability. Sutton & Barto identify a *deadly triad*: when function approximation, bootstrapping and off-policy learning are used together, the parameters can diverge rather than converge, because each update’s bootstrapped target depends on the same parameters being modified. Any one component is safe in isolation; the combination is not. The practical remedies — a target network and a replay buffer, which made Deep Q-Networks trainable (Chapter 1) — reappear in the neural game solvers of Sections 5.3 and 5.4.

Games introduce a second source of instability. In supervised learning the data distribution is fixed, and in single-agent reinforcement learning it stabilizes once the policy converges. In a game the opponent is also adapting, so the function being approximated is itself moving. This non-stationarity is not incidental — it is the central difficulty of the setting this research addresses. Several later methods are best understood as responses to it: self-play against frozen or time-averaged opponents restores a stationary target, population-based methods replace the moving target with an explicit set of fixed opponents, and the opponent-modeling and fast-adaptation techniques of Chapters 7 and 12 attempt to track it directly.

5.2 Architecture & Composition

5.2.1 Inductive Bias: Matching Architecture to Game Structure

Every architecture encodes assumptions about the structure of its input before it sees any data; these built-in assumptions are its *inductive bias*. A fully connected network assumes nothing, which is general but data-hungry. Choosing an architecture whose bias matches the game’s structure reduces the parameters and data needed and improves generalization to states absent from training; the architecture decisions in the rest of Section 5.2 are, at bottom, choices of inductive bias. Three correspondences cover most

imperfect-information games. A spatial state — a board or a grid — calls for a convolutional network, whose *translation equivariance* lets a feature learned in one region transfer to others.⁶⁰ An unordered collection — a hand of cards, or the set of other players — calls for a *permutation-invariant* network, so that capacity is not spent learning that the order is irrelevant. A history revealed over time, as under partial observability, calls for a recurrent network or an attention mechanism that carries forward what earlier observations implied.

5.2.2 Layer Types in Practice: MLP, CNN, RNN, Attention/Transformer

The families differ in their *wiring* — which units connect to which — and in whether the connecting weights are *shared* across positions or time; the neuron, the nonlinearity and backpropagation are identical. In the convolution panel, edge colour denotes the weight, so a repeated colour marks a shared weight; in the other panels colour only distinguishes kinds of connection.

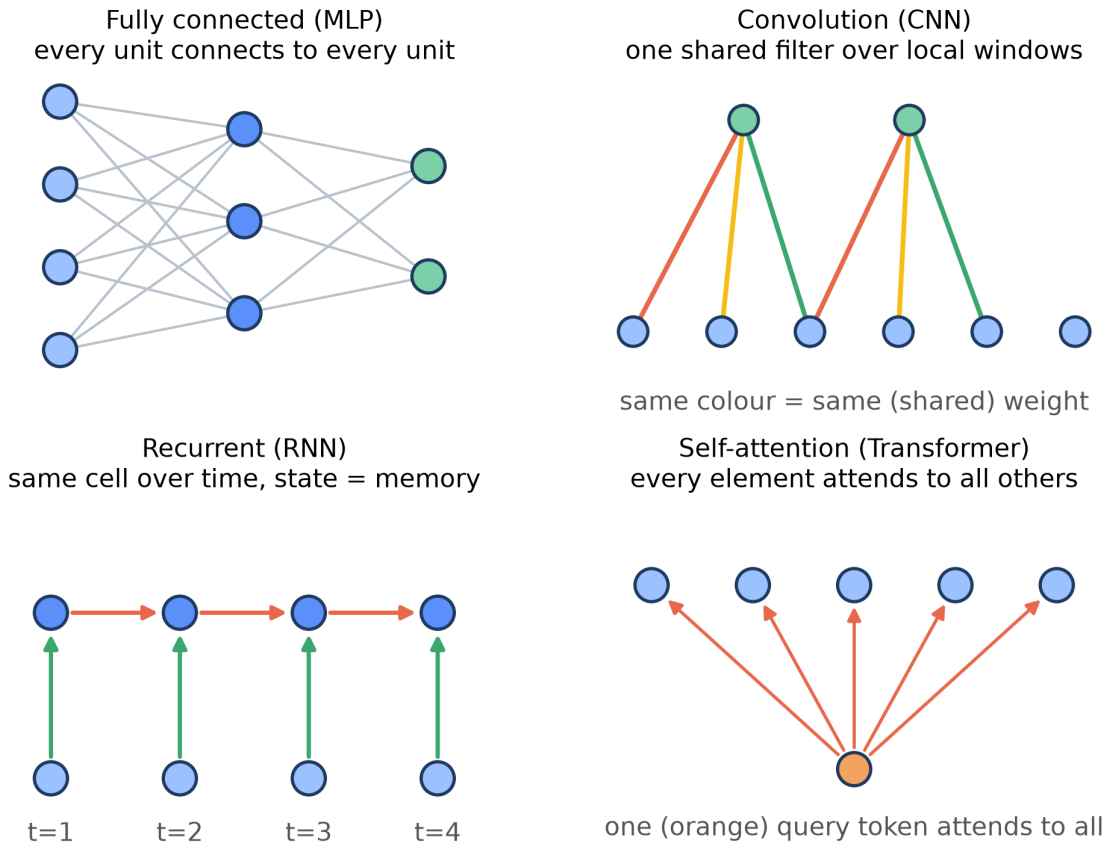


Figure 17: Connectivity of the four layer families: a fully connected MLP, a convolution that reuses one filter across local windows, a recurrent cell reused across time, and self-attention.

A **fully connected layer** — the multilayer perceptron, or MLP — connects every unit to every unit in the next layer, each connection carrying its own weight. It assumes no structure, its number of weights

⁶⁰LeCun, Y., Bengio, Y. & Hinton, G. (2015). “Deep learning.” *Nature* 521, 436–444. DOI 10.1038/nature14539.

is the product of the two layer sizes, and all structure must be learned from data. It is the right choice for a flat feature vector with no spatial or sequential meaning, such as a poker information state.

A **convolutional layer** (CNN) connects each output unit to only a small local window of the input and applies the same small set of weights, the *filter*, at every position. The number of weights then depends on the filter size rather than the input size, and a pattern learned at one location is recognized at every location; stacked convolutions combine local patterns into larger ones. This is the standard choice for boards and grids.

A **recurrent layer** (RNN) applies the same cell along a sequence, combining each input with the previous step’s *hidden state*, which acts as a memory of everything seen so far — the property needed when the state is a history revealed over time. Gated variants (LSTM, GRU) learn what to keep and what to discard, which lets them retain information across longer spans.

A **self-attention layer** (the Transformer) lets every element interact with every other directly: the output for an element is a weighted average of all elements’ values, with the weights set by the similarity between its query and each key. It is permutation-equivariant, so order must be supplied through positional encodings when it matters, and it captures long-range dependencies at a cost that grows with the square of the number of elements. It suits variable-length histories and set-valued inputs.

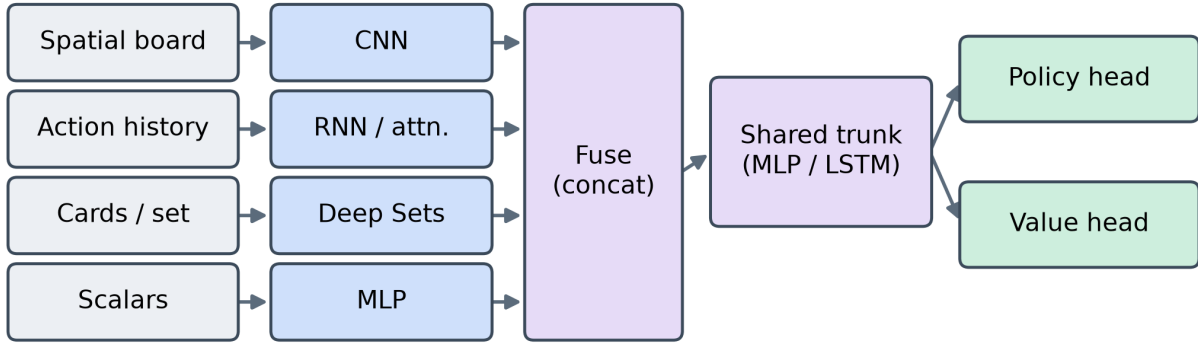
A genuinely unordered input calls for **permutation invariance**, which comes not from a new layer type but from a layout: apply the same small network to each element, then combine the results with an order-independent operation such as a sum, mean or maximum. This is the recipe behind Deep Sets.

In practice a network rarely uses one family alone. The usual design is a pipeline: a separate *encoder* matched to each part of the input, the encoder outputs *fused* by concatenation into a shared *trunk*, and one or more *heads* that read predictions off the trunk — commonly a policy head and a value head. A Deep Recurrent Q-Network (DRQN) composes a convolutional encoder, an LSTM and an output MLP — the natural template for a fog-of-war grid agent.⁶¹ AlphaStar encodes a set of units with a Transformer, the spatial minimap with a CNN and scalar statistics with an MLP, then fuses all three and passes them through an LSTM core before its action heads.⁶² “Which architecture” is therefore usually “which encoder for each input, and how to combine them,” rather than a single choice.

⁶¹Hausknecht, M. & Stone, P. (2015). “Deep Recurrent Q-Learning for Partially Observable MDPs.” *arXiv:1507.06527*.

⁶²Vinyals, O. et al. (2019). “Grandmaster level in StarCraft II using multi-agent reinforcement learning.” *Nature* 575, 350–354. DOI 10.1038/s41586-019-1724-z.

General pattern: encoders → fuse → shared trunk → heads



Example 1 — DRQN (memory for partial observability)



Example 2 — AlphaStar

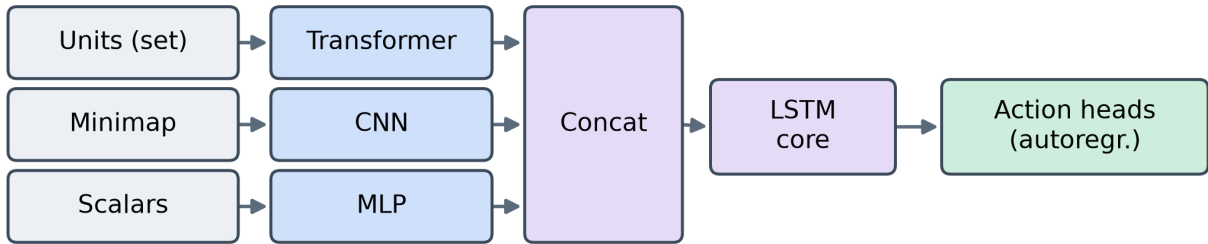


Figure 18: The general encoder / trunk / heads pattern (top); two instances below — DRQN and AlphaStar.

5.2.3 Encoding Game State & History

Before any of these architectures can run, the information state — everything the acting player may legally observe — has to become a fixed-shape tensor, and its layout determines which architecture can exploit it. Categorical facts are encoded either as *one-hot* vectors — a length- k vector with a single 1 marking which of k categories holds — or, when k is large, as *embeddings*: short learned vectors looked up per category, which compress a large vocabulary and let similar categories share structure. Leduc poker is small enough for one-hot throughout: the player to act, the private card, the public card and the betting history each become one-hot fields, producing OpenSpiel’s thirty-dimensional vector; the pot is not a separate input because it follows from the betting history.

Richer inputs call for richer layouts. A spatial state is encoded as a stack of grids, one *channel* per entity type, so that a convolution can read it directly. Pommerman — a grid-based, Bomberman-style game — splits its 11x11 board into a separate binary plane for each entity type (passages, walls, bombs, flames, power-ups, the acting agent and each opponent), with scalars such as remaining ammunition appended as

a short vector. A history whose order matters is encoded as a *sequence* of per-step vectors for a recurrent layer or a Transformer; an unordered collection is encoded element by element and pooled. In every case the encoding is where the inductive bias is actually applied: a grid is presented as a grid precisely so that a convolution’s assumptions hold. Continuous features such as the pot or a stack size are normalized to a comparable scale, and a network can also learn to compress a long history into a compact summary — the idea behind learned abstractions and learned belief states (Chapter 6).

One discipline is specific to imperfect-information games and easy to violate: the encoding must contain only what the player can observe, and nothing hidden. If any privileged information — the opponent’s private card, or the true contents of a fogged cell — leaks into the input tensor, the network will learn to use it and effectively see through the fog, yielding a policy that cannot be deployed against a real opponent and an exploitability figure that is meaningless. This is why frameworks such as OpenSpiel define an explicit per-player information-state representation; getting that representation right is a correctness requirement, not a tuning choice.⁶³

5.2.4 Sizing & Capacity

A network’s *capacity* — set by its width, the number of units per layer, and its depth, the number of layers — is the size of the family of functions it can represent. Too little capacity and the network *underfits*; too much and it *overfits*, fitting the noise in a limited sample rather than the underlying structure. The right capacity is the one matched to how much informative data training actually supplies — which, in a game solver, is often far less than the raw number of states suggests, because the values being fit are themselves noisy estimates.

The experiments in this chapter showed this directly. Running Deep CFR on Leduc under a fixed budget but with different network sizes, the smallest network — two hidden layers of thirty-two units — reached a lower exploitability (1.14) than a three-layer network four times wider (1.49). The likely reason is that a few thousand traversals are too few to train the larger networks; with one seed and near-flat curves this is a hypothesis, not a measured effect. A third run, two layers of sixty-four units, is left out: it stays at 1.70 at every checkpoint, the value the solver gave before its bug was fixed (Section 5.2.5), so it awaits a rerun. The lesson is not that smaller is always better, but that capacity must track the data budget: on a larger game, or with far more traversals, the ranking would likely reverse.

⁶³Lanctot, M. et al. (2019). “OpenSpiel: A Framework for Reinforcement Learning in Games.” *arXiv:1908.09453*.

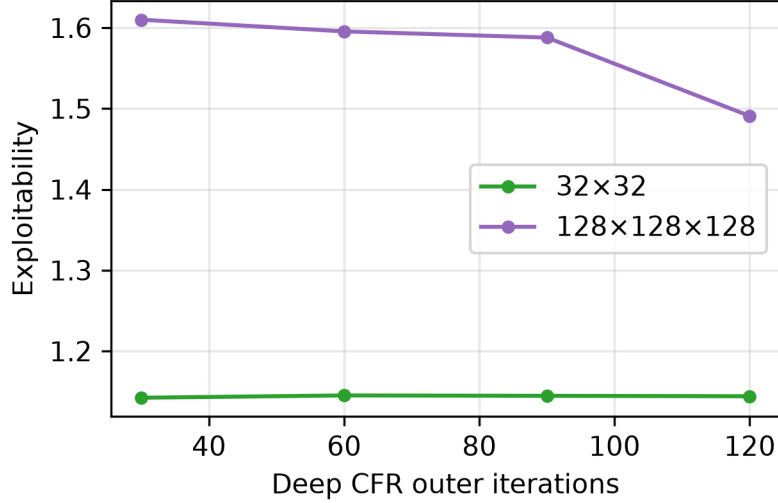


Figure 19: Deep CFR exploitability on Leduc for two network sizes (32×32 and $128 \times 128 \times 128$; one seed). The 64×64 run is left out until it is rerun.

A workable recipe is to start small and increase width and depth only while a held-out metric — validation loss, or exploitability for a solver — keeps improving, stopping when it plateaus. Width lets a network represent more distinct patterns at a given level of abstraction; depth lets it compose simple patterns into more abstract ones, which is why spatial and sequential problems reward several layers while a flat feature vector often does not.

Width need not stay constant across a network. A *funnel* shrinks toward the output, compressing the input into a small summary; an *hourglass* adds an expanding half, and its narrow middle — the *bottleneck* — forces all information through a low-dimensional code, so the network keeps only what matters: the seed of a learned abstraction or belief state. The same pinch in an otherwise uniform stack is a *collar*.

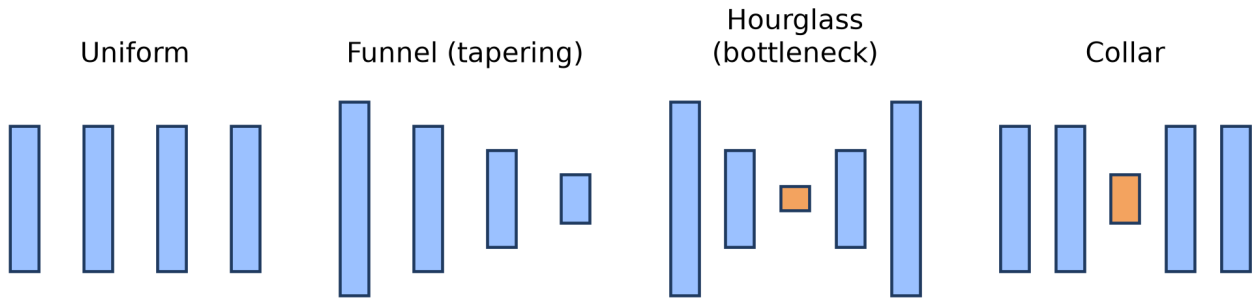


Figure 20: Common network shapes: a uniform stack, a tapering funnel, an hourglass with a central bottleneck (orange), and a uniform stack with one narrow collar (orange). Bar height is layer width.

Skip (residual) connections address depth rather than width: adding a layer’s input to its output lets gradients reach deep layers, so very deep networks become trainable — the idea behind ResNets and the

deep value networks of Chapter 6.⁶⁴

5.2.5 Training Stability & Diagnostics

Besides the target network and the replay buffer, two stabilizers recur in the game solvers ahead. *Reservoir sampling* keeps a fixed-size sample in which every item seen so far is equally likely to be retained, so an average over the entire history of training stays unbiased without storing all the data; Deep CFR uses it so that its strategy network trains on an unbiased sample across all iterations (Section 5.3).⁶⁵ *Variance reduction* — subtracting a baseline from sampled returns, or using control variates⁶⁶ — lowers the noise that sampling injects into the gradient, so the network converges from fewer samples. Input normalization and sensible weight initialization keep activations and gradients in a usable range from the first step.

Even with these in place, a neural solver fails quietly more often than loudly, so reading the diagnostics matters. A loss that keeps moving while the evaluation metric — exploitability, for a solver — stalls is the signature of a network that is *alive but not converged*: training is working, but the budget is too small for the average strategy to settle, which is what the networks in this chapter showed on Leduc at a few thousand traversals. The more dangerous case is training that is not running at all while appearing to: in the exploration, a one-character bug in an OpenSpiel routine left the advantage networks untrained, yet the program ran to completion and produced plausible output, with exploitability simply frozen near 1.69 whatever the iteration count (a uniform random policy scores 2.37). Even after the fix, the 64×64 run sits at 1.70 at every checkpoint, the same level, so this chapter does not rely on its numbers until the run is repeated. The lesson is to confirm that the loss actually moves and that the evaluation metric responds before trusting any result — a silent no-op is indistinguishable from a hard problem unless the curves are checked.

5.3 Neural Networks in CFR

5.3.1 From Tabular CFR to Function Approximation

Tabular CFR keeps two tables at every information set: cumulative regrets, from which regret matching derives the current strategy, and a cumulative strategy, whose running average is the Nash approximation the algorithm returns (Chapters 2–4). Neural CFR replaces the two tables with two networks that take an information-state tensor as input and predict what the table would have stored; because a network generalizes across similar information states, it never has to enumerate them. The methods below differ mainly in *which* table they approximate and *how* they generate the data to train it: Deep CFR learns advantages from sampled tree traversals, while NFSP learns the average strategy from self-play without traversing the tree at all.⁶⁵

⁶⁴He, K., Zhang, X., Ren, S. & Sun, J. (2016). “Deep Residual Learning for Image Recognition.” *CVPR*, 770–778. DOI 10.1109/CVPR.2016.90.

⁶⁵Brown, N., Lerer, A., Gross, S. & Sandholm, T. (2019). “Deep Counterfactual Regret Minimization.” *ICML*.

⁶⁶Schmid, M., Burch, N., Lanctot, M., Moravčík, M., Kadlec, R. & Bowling, M. (2019). “Variance Reduction in Monte Carlo Counterfactual Regret Minimization (VR-MCCFR) for Extensive Form Games Using Baselines.” *AAAI* 33, 2157–2164.

5.3.2 Deep CFR and Its Single-Network Variants

Deep CFR (Brown et al., 2019) is the direct neural translation of MCCFR. On each iteration it traverses the game tree by external sampling, and at every information set it visits it computes the sampled counterfactual advantage of each action — how much better that action did than the current strategy on average. These (information-state, advantage) pairs go into a reservoir buffer, and an *advantage network*, one per player, is trained to predict them; the current strategy is then read off the predicted advantages by regret matching, exactly as the table would have been used. A separate *strategy network*, trained from its own reservoir buffer, learns the average strategy across all iterations — the Nash approximation returned at the end.

The one equation that distinguishes Deep CFR from tabular MCCFR is the advantage network’s objective: a mean squared error between its prediction and the sampled advantages,

$$L(\theta) = \frac{1}{|B|} \sum_{(I, \tilde{a}) \in B} \|f_{\theta}(I) - \tilde{a}\|^2,$$

where B is a batch drawn from the reservoir, I an information-state tensor, $\tilde{a}$ the vector of sampled advantages for its actions, and f_{θ} the advantage network (in practice the samples are weighted by iteration, mirroring linear CFR). Everything else carries over unchanged from the tabular algorithm.

Refinements followed. *Single Deep CFR* drops the separate strategy network and recovers the average strategy from the stored advantage networks of every iteration, which removes the approximation error of the second network; it matched or slightly beat Deep CFR in exploitability on Leduc and beat it head-to-head in the larger 5-Flop Hold’em.⁶⁷ *DREAM* makes the method model-free: it replaces external sampling with outcome sampling and adds a learned baseline as a control variate to absorb the extra variance (Section 5.2).⁶⁸ *ESCHER* drops importance sampling altogether, estimating regret from a learned history value function; in dark chess it beat DREAM and NFSP head-to-head in over 90% of games.⁶⁹

⁶⁷Steinberger, E. (2019). “Single Deep Counterfactual Regret Minimization.” *arXiv:1901.07621*.

⁶⁸Steinberger, E., Lerer, A. & Brown, N. (2020). “DREAM: Deep Regret Minimization with Advantage Baselines and Model-free Learning.” *arXiv:2006.10410*.

⁶⁹McAleer, S., Farina, G., Lanctot, M. & Sandholm, T. (2023). “ESCHER: Eschewing Importance Sampling in Games by Computing a History Value Function to Estimate Regret.” *ICLR*; arXiv:2206.04122.

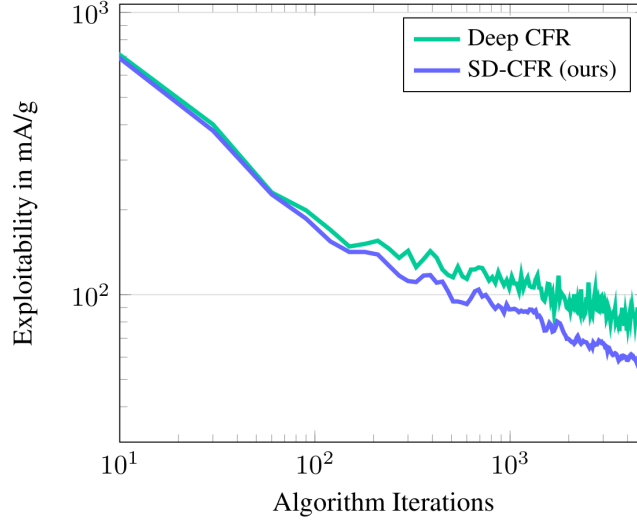


Figure 21: Single Deep CFR (SD-CFR) matches or slightly beats Deep CFR on Leduc (milli-antes per game) without an average-strategy network. Reproduced from Steinberger (2019), Fig. 1a; not own work.

On Leduc the exploration confirmed both the promise and the catch. Deep CFR was training — its advantage losses moved (and grew, because the targets are weighted by iteration) — but at 120 iterations it was still above 1.1 at every network size (1.14 at best), against 0.097 for tabular MCCFR at comparable wall-clock time. This is not in itself a defect: Leduc is small enough that the table wins outright, and Deep CFR needs a far larger sampling budget to approach Nash (40 traversals per iteration here, against 1,500 in Steinberger’s Leduc setup). Its advantage appears only when the game is too large for the table to exist — the regime Section 5.3.4 makes precise.

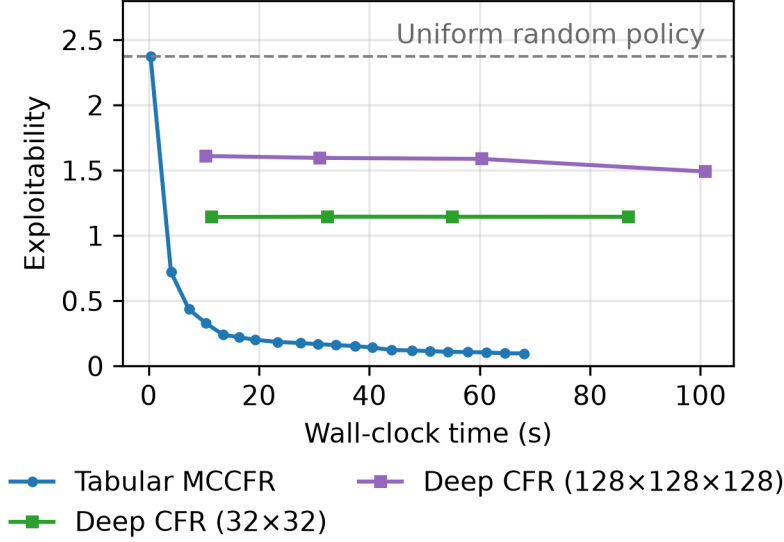


Figure 22: Exploitability of Deep CFR (32x32 and 128x128x128) and of tabular MCCFR on Leduc against wall-clock time; the dashed line is a uniform random policy.

5.3.3 NFSP — Neural Fictitious Self-Play

NFSP — Neural Fictitious Self-Play (Heinrich & Silver, 2016) — reaches an approximate Nash equilibrium from the reinforcement-learning side, without ever traversing the game tree. Each player keeps two networks. A *best-response network*, a DQN, learns the greedy best response to the opponent’s current behaviour; an *average-policy network*, trained by supervised learning on the player’s own past actions, learns the time-average of those best responses. The average policy is what converges toward Nash — the same averaging that CFR performs, reached through play rather than regret. The *anticipatory parameter* η ties the two together: in each episode the agent plays its best-response network with probability η and its average-policy network otherwise, so best-response play keeps generating fresh, on-distribution data for the average policy, while the average policy supplies a stable opponent for the best response.

The price is sample efficiency. On Leduc the exploration ran NFSP for tens of thousands of episodes and saw exploitability drift around 2.5, near the uniform-random level of 2.37, rather than fall, because reaching a usable equilibrium even on this small game needs on the order of a million episodes or more. NFSP scales to large games, where its model-free simplicity is an asset, but on a teaching benchmark it converges far more slowly than either tabular CFR or Deep CFR.⁷⁰

⁷⁰Heinrich, J. & Silver, D. (2016). “Deep Reinforcement Learning from Self-Play in Imperfect-Information Games.” *arXiv:1603.01121*.

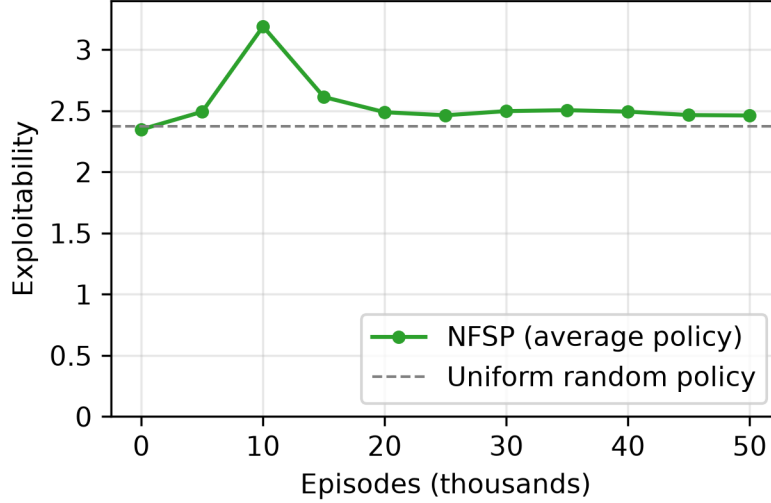


Figure 23: NFSP exploitability on Leduc over 50,000 training episodes: the curve stays around the level of a uniform random policy (dashed, ≈ 2.37).

5.3.4 Trade-offs: When Neural CFR Pays Off

Neural networks buy CFR generalization and bounded memory: a network shares structure across similar information states and stores a strategy in a fixed number of parameters, so equilibrium computation no longer requires visiting every information set. The cost is steep per-iteration overhead and weaker guarantees: each iteration trains networks rather than incrementing counters, and the clean monotonic behaviour of tabular CFR becomes approximate. The Leduc numbers are the small-game face of this trade-off: where the table fits, it wins.⁶⁵ As a game grows, the table eventually does not fit in memory at all, while a network’s footprint is unchanged; beyond that crossover the neural method is the only option, which is why Deep CFR and its relatives matter for full-scale poker even though they lose on Leduc.

One structural caveat bounds where regret-based neural methods apply. CFR computes counterfactual values by reasoning about the whole subtree below an information set, which is cheap when the tree is *wide but shallow* — poker has an enormous number of hands but ends after a few betting rounds. In games that are *deep* — hundreds of sequential moves, as in a gridworld — that reasoning becomes prohibitively expensive, and the model-free self-play methods of Section 5.4 become the more natural fit. The regret-based family keeps growing within its niche — NeuRD, for instance, changes the softmax policy-gradient update by one line so that it follows the replicator dynamics; in the tabular case it is equivalent to softmax CFR⁷¹ — but its sweet spot remains wide, shallow, imperfect-information games.

⁷¹Hennes, D. et al. (2020). “Neural Replicator Dynamics: Multiagent Learning via Hedging Policy Gradients.” *AAMAS*, 492–501; arXiv:1906.00190.

5.4 Other Neural Applications

5.4.1 The Four SOTA Algorithm Families

Approximating CFR (Section 5.3) is one of four broad families this chapter uses to organise current methods, and choosing among them is largely a matter of matching the family to the game’s size, tree shape and number of players.⁷² The table places each; the sections that follow take the three not yet covered in turn.

Table 7: Families of neural methods for imperfect-information games: core idea, where each fits, and its cost.

Family	Core idea	Best for	Cost
Regret-based (Deep CFR, DREAM, NeuRD)	Approximate CFR with networks	Wide, shallow games; unexploitability	High
Population-based (PSRO, NFSP, XFP)	Best-respond to a growing opponent pool	n-player; opponent modeling	Medium
Search + RL (ReBeL, Student of Games)	Learned value plus online search at play	High-stakes; inference-time compute	High
Model-free self-play (PPO, DQN, MAPPO, QMIX)	Optimize reward via self-play	Spatial, long-horizon (grids)	Low–medium

5.4.2 Population-Based: PSRO, NFSP, XFP

Population-based methods reframe equilibrium-finding as an iterative tournament: the members of a pool of policies play one another to form a *meta-game*, a new policy is trained as an approximate best response to the current population and added to the pool, and the process repeats. This is the idea behind PSRO (Policy-Space Response Oracles); fictitious self-play, including the NFSP of Section 5.3, is the special case where the best response targets the time-average of the pool. Because the population is an explicit set of opponents rather than a single moving target, these methods extend naturally to the n -player, opponent-modeling setting this thesis is concerned with, and population training of this kind is at the core of the league with which AlphaStar reached grandmaster level in StarCraft II.⁶² The price is running and storing many agents, and the equilibrium guarantees are weaker than CFR’s.⁷³ Chapters 9 and 10 treat these methods in depth.

5.4.3 Search + RL: ReBeL & Student of Games

Search-plus-RL methods are the AlphaZero idea adapted to hidden information: the searching player does not know the true state, so the search must range over a distribution of possible states. ReBeL (Brown

⁷²Rudolph, M. et al. (2026). “Reevaluating Policy Gradient Methods for Imperfect-Information Games.” *ICLR*; arXiv:2502.08938.

⁷³Lanctot, M. et al. (2017). “A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning.” *NeurIPS* (PSRO).

et al., 2020)⁷⁴ makes this precise with the *public belief state* — a probability distribution over the players’ private information given everything public — and runs CFR over a small, depth-limited subgame at play time, with a learned value network supplying the leaf values. Student of Games (Schmid et al., 2023; preprint title Player of Games)⁷⁵ generalizes the recipe into one algorithm for both perfect and imperfect information. These systems underlie the strongest poker results but depend on having compute available *during* play, not only during training; they are the subject of Chapter 6.

5.4.4 Model-Free Self-Play: PPO, DQN, MAPPO, QMIX

Model-free self-play methods ignore the game tree entirely and optimize the reward signal directly, improving a policy from episodes of play against copies of itself: PPO⁷⁶ and DQN⁷⁷ for the single-agent core, and the multi-agent extensions MAPPO⁷⁸ and QMIX⁷⁹ for cooperative settings. Because they never reason about the subtree below a state, they are unbothered by the depth that defeats regret methods, which makes them the natural choice for spatial, long-horizon games — gridworlds, Pommerman, and the like; partial observability is handled by the recurrent and attention architectures of Section 5.2. Naive self-play with a single-agent algorithm can cycle and stay highly exploitable, which is why newer model-free methods change the learning dynamics (NeuRD,⁷¹ R-NaD in DeepNash,⁸⁰ magnetic mirror descent⁸¹) so that play approaches equilibrium. A large study over five two-player zero-sum games found that the specialised methods NFSP, PSRO, ESCHER and R-NaD did not outperform generic policy-gradient methods such as PPO and MMD in exploitability.⁷² Exploitability therefore has to be measured for every model-free agent, not assumed.

⁷⁴Brown, N., Bakhtin, A., Lerer, A. & Gong, Q. (2020). “Combining Deep Reinforcement Learning and Search for Imperfect-Information Games.” *NeurIPS 33*; arXiv:2007.13544.

⁷⁵Schmid, M. et al. (2023). “Student of Games: A unified learning algorithm for both perfect and imperfect information games.” *Science Advances* 9(46), eadg3256. DOI 10.1126/sciadv.adg3256.

⁷⁶Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017). “Proximal Policy Optimization Algorithms.” arXiv:1707.06347.

⁷⁷Mnih, V. et al. (2013). “Playing Atari with Deep Reinforcement Learning.” arXiv:1312.5602; Mnih, V. et al. (2015). “Human-level control through deep reinforcement learning.” *Nature*, 518(7540), 529–533.

⁷⁸Yu, C. et al. (2022). “The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games.” *NeurIPS Datasets and Benchmarks*; arXiv:2103.01955.

⁷⁹Rashid, T. et al. (2018). “QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning.” *ICML*; arXiv:1803.11485.

⁸⁰Perolat, J. et al. (2022). “Mastering the game of Stratego with model-free multiagent reinforcement learning.” *Science* 378, 990–996. DOI 10.1126/science.add4679.

⁸¹Sokota, S. et al. (2023). “A Unified Approach to Reinforcement Learning, Quantal Response Equilibria, and Two-Player Zero-Sum Games.” *ICLR*; arXiv:2206.05825.

6 Chapter 6 — End-to-End Game AI Architectures

6.1 Introduction

Chapter 6 is the keystone of this study plan. Chapters 1–5 assembled the parts in isolation — the game-theoretic vocabulary of extensive-form games and Nash equilibria, counterfactual regret minimization (CFR) and its Monte-Carlo variants, game abstraction, and the neural function approximators that replace tabular storage once a game outgrows it — and this chapter is where those parts converge into complete, competition-grade systems. The treatment is deliberately **architectural**: rather than re-deriving the mathematics, which lives in the technical chapters and the cited papers, each system is studied at the level of how it is *put together* — what it computes offline, what it computes in real time, where learning sits, and what it can actually guarantee. A short per-system scorecard opens each section so the same nine dimensions line up at a glance.

The five systems span seven years and, read in order, trace the evolution of superhuman game AI: **DeepStack** (2017), **Libratus** (2017/2018), **Pluribus** (2019), **ReBeL** (2020), and **Student of Games** (2023). It is tempting to read such a list as a leaderboard — each entry strictly stronger than the last — but that is neither what happened nor how this chapter is organized. The progression is a sequence of *deliberate trades*: each system bought a new capability by giving something else up. Read this way, the chapter is a study of what each advance cost, not a ranking of winners.

Beneath the individual trades, three axes of motion run through all five systems and give the chapter its spine. The first is **representational**: the move from hand-built *abstraction* — bucketing similar hands and allowing only a few bet sizes — toward *learned neural approximation*, in which a network generalizes across situations a table could never enumerate. The second is **temporal**: the move from *solving the entire game offline* into a stored strategy toward *learning a compact model and searching with it in real time*. The third axis appears only at the very end, with Student of Games: the unification of **perfect- and imperfect-information** play in one sound algorithm, joining the two great traditions of game AI — the minimax / Monte-Carlo-tree-search / AlphaZero line and the CFR / poker line.

One idea ties these threads together and deserves to be named before the systems themselves, precisely because it is *not* one of them: **depth-limited solving**. Formalized by Brown, Sandholm & Amos (2018)⁸², it is the principle that one may search only a little way ahead in an imperfect-information game and substitute a *learned or precomputed value* for the remainder — provided the substitution is done so that hidden information does not render it unsound. It is the theory that places DeepStack’s continual re-solving and Libratus’s nested subgame solving in a common framework, that underwrites Pluribus’s continuation strategies, and that, in belief-state form, becomes the inner loop of ReBeL and Student of Games. We treat it as connective tissue — referenced wherever a system instantiates it — rather than as a sixth entry.

Finally, it is worth flagging at the outset the single thread the synthesis returns to. Every system here is, by

⁸²Brown, N., Sandholm, T. & Amos, B. (2018). “Depth-Limited Solving for Imperfect-Information Games.” *NeurIPS* 31; arXiv:1805.08195.

deliberate design, **opponent-blind**: each computes a strategy that is hard to beat *in the worst case* and then plays it without regard to who is actually across the table — because an exploitative deviation can itself be counter-exploited and, in the words of Pluribus’s authors, because existing opponent-exploitation techniques “require too many samples to be competitive with human ability outside of small games”; both papers name conservative (safe) exploitation as the only exception.⁸³⁸⁴ This robustness-first stance is the field’s great strength and, for a dissertation about *adaptive* play, its defining limitation: it is exactly the opponent-awareness these systems omit that Chapters 7–12 set out to add.

6.2 DeepStack (2017)

DeepStack (Moravčík et al., 2017)⁸⁵, from the University of Alberta computer-poker group with collaborators in Prague, was the first program to defeat professional poker players at heads-up no-limit Texas hold’em (HUNL) — professionals, though not HUNL specialists — with statistical significance, and the first to put heuristic search — the engine behind chess and Go — on a theoretically sound footing in a game of imperfect information. Over a four-week study it beat a pool of 33 professionals by 492 milli-big-blinds per game (mbb/g, the standard poker win-rate unit; 50 mbb/g is a sizable professional edge) across 44,852 hands, and no known technique could find a flaw in its play. Its guiding *what-if* is the one that frames this whole chapter: **could a program play poker the way AlphaGo plays Go — searching locally from the current situation and trusting a learned value function for everything beyond the horizon — even though, in poker, the situation is itself partly hidden?**

Table 8: DeepStack (2017) at a glance.

At a glance	DeepStack (2017)
Players	2 (heads-up)
Game type	HUNL — heads-up no-limit Texas hold’em (2-player zero-sum)
Blueprint (offline)?	No — offline work trains value nets, not a stored strategy
Neural component	Deep counterfactual value networks (flop, turn, aux); value-only
Search mechanism	Continual re-solving (depth-limited CFR look-ahead, every decision)
Abstraction?	None constrains play; 1,000-bucket clustering only at the net input, plus sparse betting + river action bucketing in look-ahead
Perfect-info too?	No (imperfect-information only)
Compute	Offline-heavy (~175 CPU-core-years to label the turn network); play-time runs on one GPU, < 5 s/decision
Key innovation	Continual re-solving + learned counterfactual values: the first <i>sound</i> heuristic search for imperfect-information games

⁸³Brown, N. & Sandholm, T. (2019). “Superhuman AI for multiplayer poker.” *Science*, 365(6456), 885–890.

⁸⁴Brown, N. & Sandholm, T. (2018). “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals.” *Science*, 359(6374), 418–424.

⁸⁵Moravčík, M. et al. (2017). “DeepStack: Expert-level artificial intelligence in heads-up no-limit poker.” *Science*, 356(6337), 508–513.

6.2.1 The gap it closed

Every prior game-AI milestone — backgammon, chess, Go — rested on *local search*: from the current position, look a few moves ahead and substitute a heuristic value for the rest. That recipe assumes the position is known, and poker breaks the assumption — the right action depends on the distribution over the opponent’s hidden cards, revealed only through their betting. The open question DeepStack set out to answer is whether heuristic search can be made *sound* under hidden information at the scale of HUNL, a game with roughly 10^{160} decision points.

For nearly two decades the dominant answer had been something else entirely: **abstraction plus offline equilibrium plus translation**, as in Chapter 4 — shrink HUNL’s 10^{160} situations into roughly 10^{14} abstract ones, solve that smaller game offline with CFR into a stored *blueprint*, and at play time *translate* each real situation and opponent bet into the nearest abstract one. The compression is lossy, and the loss shows up as exploitability — how much a worst-case opponent can win, the field’s quality metric, zero at a Nash equilibrium. In 2015 the abstraction-based program Claudico lost to professionals by 91 mbb/g,⁸⁵ and a local-best-response probe (LBR, a tractable lower bound on exploitability) later showed top competition bots exploitable by more than 3,000 mbb/g — four times worse than folding every hand.⁸⁶ DeepStack closes this gap by discarding the whole edifice: it never builds a full-game abstraction and never stores a blueprint, reasoning about each situation *as it actually arises* and replacing only the distant remainder of the game with a learned estimate.

6.2.2 Architecture

DeepStack splits cleanly into an **offline** phase that learns intuition and an **online** phase that searches with it (see the figure below).⁸⁵

⁸⁵Lisý, V. & Bowling, M. (2017). “Equilibrium Approximation Quality of Current No-Limit Poker Bots.” *AAAI Workshop on Computer Poker* — local best response (LBR); arXiv:1612.07547.

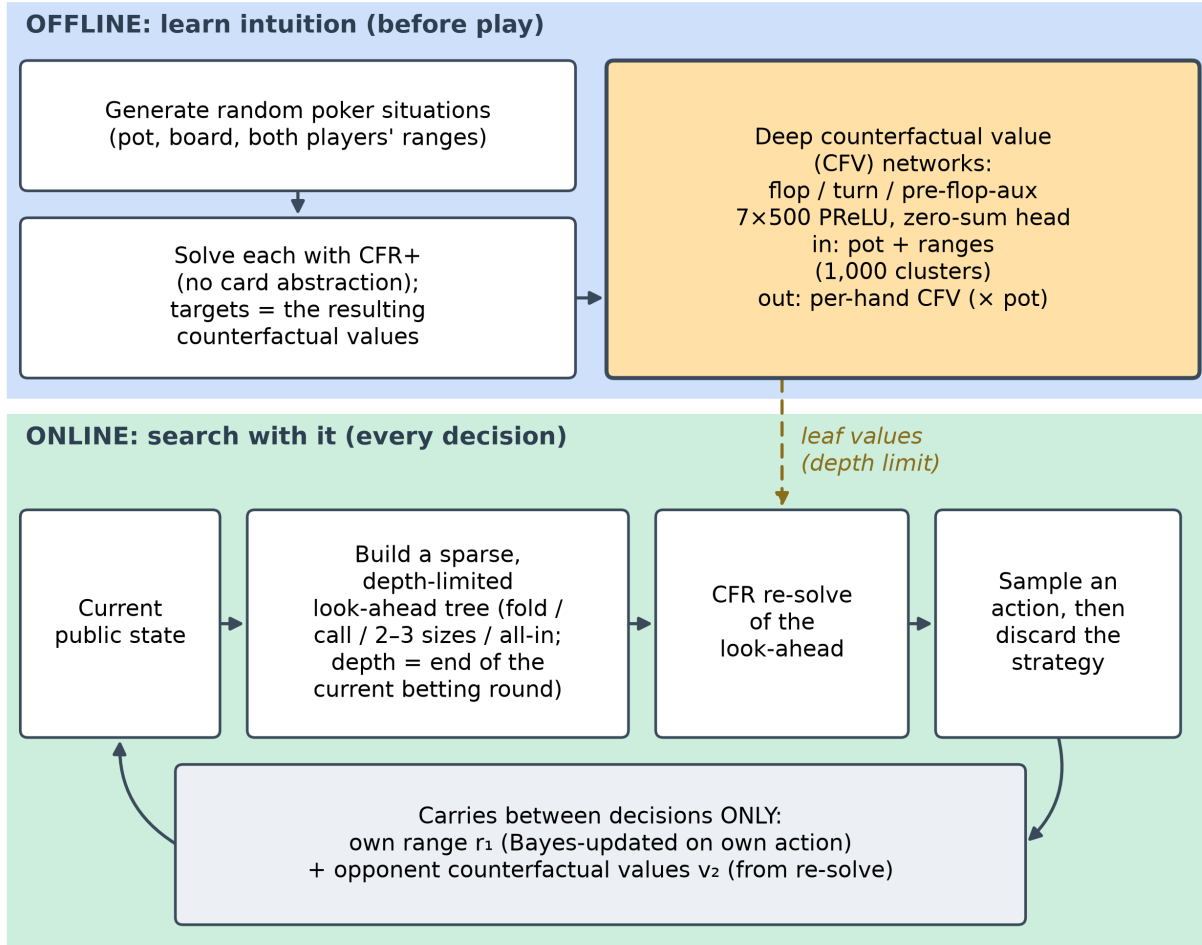


Figure 24: DeepStack architecture: offline intuition-building (top) feeding a shared deep counterfactual-value network, reused as the leaf evaluator inside the online continual-re-solving loop (bottom).

Offline, the system generates millions of random poker situations and solves them with a CFR solver to obtain target *counterfactual values* — conditional “what-if” payoffs for holding each possible hand — and these (situation $\rightarrow$ value) pairs train the **deep counterfactual value (CFV) networks**. Online, DeepStack carries no blueprint: between decisions it remembers only two vectors — its own *range* (the distribution over the hands it could be holding) and the opponent’s *counterfactual values* — and at each turn it runs CFR over a small look-ahead tree rooted at the true current state, using a CFV network to supply leaf values at the depth limit. The neural network sits *only* at the depth limit as a leaf evaluator; CFR does the searching; and — crucially — there is no full-game abstraction in the loop.

6.2.3 Key innovation: continual re-solving with learned counterfactual values

DeepStack’s contribution is the marriage of two ideas, each of which makes the other practical.

The first is **continual re-solving**: reconstructing a fresh local strategy at every decision from the maintained range and opponent counterfactual-value vector, then discarding it, so the agent never stores or

commits to a global strategy. Classical re-solving (Burch et al., 2014) shows that to reconstruct a strategy for a subgame you do not need the whole strategy — you need only your own range entering the subgame and a vector of the opponent’s counterfactual values.⁸⁷

What makes this work is the bookkeeping. After each event DeepStack updates its two vectors by simple rules: on its **own action** it swaps in the re-solved counterfactual values for the chosen action and Bayes-updates its range; on a **chance event** it swaps in that card’s values and zeroes the now-impossible hands; and on the **opponent’s action** it does *nothing at all*. That last point is the quiet masterstroke: because it tracks the opponent’s *values* rather than their *range*, and never needs the opponent’s specific action to maintain those values, it sidesteps the action-translation step that crippled abstraction-based bots. The opponent can make any bet of any size; DeepStack simply re-solves from the state that bet produced.

The second idea is the **learned counterfactual value network** — DeepStack’s “intuition.” In a perfect-information game a leaf evaluator maps one state to one number; under imperfect information it must map a *whole public state plus both players’ ranges* to a *vector* of counterfactual values, one per hand, because the values at a node shift with the ranges that reach it. With this network supplying values at the end of the current betting round, the depth limit shrinks the re-solved game from 10^{160} decision points to at most 10^{17} , and the sparse action set described below to about 10^7 — small enough to solve in under five seconds on a single GPU.

The pairing is provably sound.⁸⁵ If the value network’s error is at most ϵ and the re-solve runs T CFR iterations, the resulting strategy’s exploitability is bounded by

$$\text{exploitability} < k_1 \epsilon + k_2 / \sqrt{T},$$

with game-specific constants k_1, k_2 . The first term is the price of imperfect intuition; the second is ordinary CFR convergence. This bound is the theoretical heart of the paper — the guarantee that heuristic search can be carried into imperfect information without the strategy quietly becoming exploitable.

6.2.4 Caveats, dead-ends, and what the paper under-describes

The most important asterisk is that **the deployed system is not the one the theorem covers**. To play at human speed DeepStack restricts its look-ahead to a sparse betting set (fold, call, two or three bet sizes, all-in), and the paper states plainly that this “voids the soundness property of Theorem 1.” So the shipped guarantee is empirical — supported by the LBR results below — not proven. A second deviation compounds this: the soundness proof assumes *best-response* constraint values, but DeepStack actually uses *self-play* values, which lack a theoretical justification yet were less exploitable in early tests. The proven algorithm and the winning algorithm are, strictly, different algorithms.

Abstraction also creeps back at the margins. DeepStack advertises that it uses no card abstraction to constrain play — but it *does* cluster hands into 1,000 buckets at the value network’s input, and on the

⁸⁷Burch, N., Johanson, M. & Bowling, M. (2014). “Solving Imperfect Information Games Using Decomposition.” *AAAI*, pp. 602–608 — re-solving and the augmented subgame.

river it abandons the network entirely, solving to the end of the game while using a **bucketed action abstraction** for tractability. Neither undermines the result, but together they show that the headline “sound, abstraction-free search” is an aspiration the implementation approximates rather than attains.

6.2.5 Compute & accessibility

DeepStack’s cost is almost entirely **offline, and almost entirely in the CFR solving used to label the value networks**: the turn network alone consumed about **175 CPU-core-years** on a 6,144-core cluster, while at play time DeepStack runs on **one commodity GPU at under five seconds per decision**.⁸⁵ A from-scratch build was thus within reach only of a well-resourced lab at the time, though a public reference implementation on Leduc lowers the entry barrier.

6.2.6 Strengths and limitations

DeepStack’s central strength is **soundness with low exploitability**: it is the first imperfect-information search method with a real guarantee, and empirically LBR — which exposes competition bots as losing thousands of mbb/g — cannot find any way to beat it, itself losing by over 350 mbb/g.⁸⁵ It needs **no full-game abstraction and no action translation**, so off-tree opponent bets are handled exactly rather than rounded. A neat bonus is evaluation synergy — DeepStack’s own value function is exactly what the AIVAT variance-reduction estimator needs, cutting the human-study standard deviation by 85% and making significance achievable in only a few thousand hands.

The limitations set the agenda for the rest of the chapter. DeepStack is **two-player zero-sum only**; its soundness leans on that structure. It **re-solves from scratch** at every decision, and it was validated only against humans and LBR — never head-to-head against the strongest abstraction bots.

6.2.7 Legacy and modern relevance

Strip away the poker specifics and DeepStack’s core idea is **depth-limited search with a learned value function at the leaves, adapted to hidden information** — the imperfect-information counterpart of the value-guided search behind AlphaGo and AlphaZero. That idea did not age into obsolescence; it became the template: ReBeL (2020)⁸⁸ recast the leaf-value learning around public belief states with AlphaZero-style self-play, and Student of Games (2023)⁸⁹ — which shares several DeepStack authors — unified perfect- and imperfect-information play in a single algorithm. **AIVAT-style variance reduction**, which turns a learned value model into a control variate for low-variance evaluation of any stochastic agent, remains directly reusable; the hand-engineered machinery *around* the idea — the 1,000-bucket k-means clustering, the separate per-round networks, the hand-tuned re-solving gadget — is what ReBeL and Student of Games replace. The honest verdict: **as a deployed poker system DeepStack is superseded, but it is far from a mere stepping stone** — its central paradigm won and is now

⁸⁸Brown, N., Bakhtin, A., Lerer, A. & Gong, Q. (2020). “Combining Deep Reinforcement Learning and Search for Imperfect-Information Games.” *NeurIPS* 33, 17057–17069; arXiv:2007.13544.

⁸⁹Schmid, M. et al. (2023). “Student of Games: A unified learning algorithm for both perfect and imperfect information games.” *Science Advances*, 9(46), eadg3256. doi:10.1126/sciadv.adg3256 (arXiv:2112.03178, first posted in 2021 as “Player of Games”).

mainstream, and for this thesis specifically its (range, opponent counterfactual values) state and its error-propagation bound are direct seeds for belief-based opponent modelling (Contribution 1) and safe-exploitation analysis (Contribution 2).

6.3 Libratus (2017/2018)

DeepStack answered its *what-if* by throwing the abstraction-and-blueprint edifice away — yet it still kept a sparse betting abstraction inside its own look-ahead, and it was never tested head-to-head against the strongest prior bots or against HUNL specialists in a long, rigorous match. Libratus (Brown & Sandholm, 2017; Carnegie Mellon)⁸⁴ was built independently and announced the same year, and it made the opposite bet: keep the abstraction-and-blueprint paradigm and cure its one fatal disease. Its guiding question is the mirror image of DeepStack’s: **what if we solve a coarse blueprint of the whole game offline, then repair it in real time wherever the abstraction is too crude — with a provable guarantee that the repair never leaves us more exploitable?** In January 2017 Libratus became the first program to beat top human HUNL specialists in a long match, defeating four professionals by 147 mbb/g over 120,000 hands at 99.98% significance — and, unlike DeepStack, it first dismantled the prior best poker AI head-to-head.

Table 9: Libratus (2017/2018) at a glance.

At a glance	Libratus (2017/2018)
Players	2 (heads-up)
Game type	HUNL — heads-up no-limit Texas hold’em (2-player zero-sum)
Blueprint (offline)?	Yes — an abstracted full-game strategy solved offline with MCCFR (detailed early, coarse late)
Neural component	None — purely tabular CFR / abstraction (no neural networks anywhere)
Search mechanism	Nested safe subgame solving (real-time CFR+ re-solve of the late game, re-run for every off-tree opponent bet)
Abstraction?	Yes — card abstraction (turn/river, blueprint only) + asymmetric action abstraction; dissolved to <i>no card abstraction</i> inside the real-time subgames
Perfect-info too?	No (imperfect-information only)
Compute	Offline- <i>and</i> online-heavy: ~25M CPU core-hours on the Bridges supercomputer; ~50 nodes and tens of seconds per late decision; no GPUs
Key innovation	Blueprint + real-time nested <i>safe</i> subgame solving + self-improvement: exact, provably-safe responses to off-tree bets in place of action translation

6.3.1 The gap it closed

The previous section laid out the paradigm DeepStack discarded — abstraction plus offline equilibrium plus translation — and named its fatal flaw. Libratus targets exactly that flaw without discarding the paradigm. The blueprint is solved in advance over a *fixed* menu of bet sizes; at play time any opponent bet

that is not on that menu is *translated* — rounded to the nearest size the bot knows. That rounding is the single largest exploitable seam in abstraction-based poker: in 2015 Libratus’s own predecessor Claudico lost the first *Brains vs. AI* match to professionals by 91 mbb/g, in good part because opponents could feel out and punish its translation boundaries.⁹⁰ The same year, then, produced two opposite cures for the same disease — one neural and blueprint-free, one tabular and blueprint-based — and Libratus is the proof that the older paradigm, properly repaired, was still enough to reach superhuman play.

6.3.2 Architecture

Libratus is a pipeline of three modules that operate on three different timescales — **offline** (before the match), **online** (during each decision), and **overnight** (between days of play) — and, unlike every other system in this chapter, it contains **no neural network at all** (see the figure below).⁸⁴

⁹⁰Ganzfried, S. (2016). “Reflections on the First Man vs. Machine No-Limit Texas Hold ’em Competition.” *ACM SIGecom Exchanges*, 14(2), 2–15; arXiv:1510.08578.

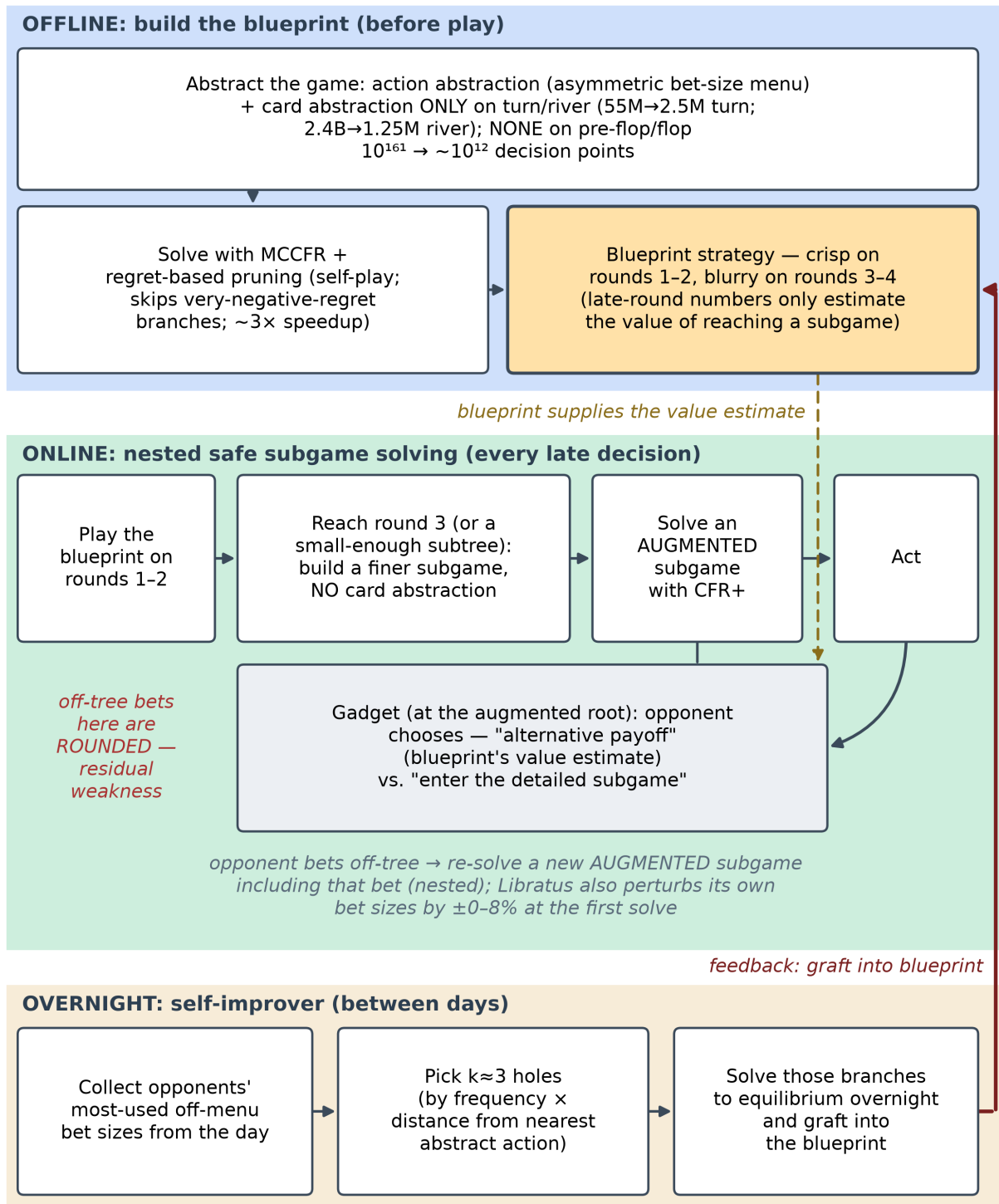


Figure 25: Libratus’s three-module pipeline: an offline blueprint (top), the online nested safe-subgame solver that plays it (middle), and the overnight self-improver that grafts solved holes back into the blueprint (bottom, feedback arrow).

Module 1 — the blueprint (offline). Libratus first compresses HUNL’s roughly 10^{161} decision points

to about 10^{12} with an *action abstraction* that keeps only a discrete menu of bet sizes and a *card abstraction* that groups strategically similar hands, used only on the turn and river and only in the blueprint.⁹¹ It then solves this abstract game by self-play with an improved **Monte Carlo counterfactual regret minimization (MCCFR)** that probabilistically prunes very-negative-regret branches. The result is the **blueprint**: a complete but uneven strategy — detailed early, coarse late — whose late-round numbers are used not to play but only to *estimate the value of reaching a subgame*.

Module 2 — nested safe subgame solving (online). Libratus plays the blueprint only in the early rounds. On reaching the third betting round — or any earlier point where the rest of the hand is small enough — it discards the coarse late-game blueprint and instead **builds a fresh, finer-grained subgame with no card abstraction and solves it in real time** with a heavily optimized **CFR+** (a fast, deterministic CFR variant). When the opponent makes a bet that is not on the blueprint’s menu, rather than round it, Libratus solves a new subgame that *contains that exact bet*, and repeats this for every subsequent off-menu action — *nested* subgame solving.

Module 3 — the self-improver (overnight). Because real-time solving is skipped on the first two rounds, off-menu opponent bets *there* are still rounded. The self-improver narrows this residual seam between days: it computes proper game-theoretic responses to a few of the most damaging holes overnight and grafts those branches into the blueprint. Pointedly, this is **not opponent exploitation** — it never tries to model and punish the humans’ mistakes (which would expose Libratus to counter-exploitation). Instead it uses the opponents’ bets only as a hint about *which of Libratus’s own holes to patch*, and the patches are universal, improving play against any future opponent.

6.3.3 Key innovation: nested safe subgame solving

Libratus’s central contribution is making **real-time subgame solving both safe and nested** — strong enough to beat humans and provably unable to make the strategy much more exploitable than the blueprint it refines.

As in Chapter 4, an imperfect-information subgame *cannot be solved in isolation*: its right strategy depends on the strategies in *other, unreached* subgames, and earlier real-time solvers that simply assumed the opponent had played the blueprint up to this point — **unsafe** subgame solving — could be punished by any deviation. Libratus’s **safe** subgame solving removes that assumption with a small gadget, the *augmented subgame*. At the root of the subgame the opponent is handed a choice for every hand they might hold: either take a fixed **alternative payoff** — the blueprint’s estimate of what that hand is worth here — or *enter* the detailed subgame and play it out. Solving this augmented game forces Libratus’s refined strategy to make the opponent **no better off than that estimate for every possible hand**, which is exactly the meaning of “safe”. Libratus sharpens this with a variant the authors call **Estimated-Maxmargin** — it maximizes the *smallest* safety margin across all opponent hands — using blueprint *estimates* of opponent values instead of conservative *upper bounds* and de-emphasizing hands the opponent could only hold by having made an earlier mistake.

⁹¹Brown, N. & Sandholm, T. (2017). “Libratus: The Superhuman AI for No-Limit Poker (Demonstration).” *IJCAI-17*. <https://www.ijcai.org/proceedings/2017/0772>

The pairing comes with a guarantee that parallels DeepStack’s.⁹² If σ^* is the least-exploitable strategy that differs from the blueprint only inside the solved subgames, and the blueprint’s estimate of the opponent’s subgame values is off by at most Δ , then the refined strategy’s exploitability obeys

$$\text{exploitability}(\sigma_{\text{refined}}) \leq \text{exploitability}(\sigma^*) + 2\Delta.$$

The 2Δ term is the price of imperfect value *estimates*, and it is the structural twin of DeepStack’s $k_1\epsilon$ term — both bound how much an error in the values handed to the solver can cost in worst-case exploitability.

“Nested” is the second half. Rather than expand the blueprint’s bet menu (which would balloon the offline solve), Libratus crafts a **distinct response to each off-menu bet in real time**: when the opponent bets a size it has never enumerated, it builds an augmented subgame whose alternative payoff is the best *in-menu* action the opponent could have taken instead, solves it, and does so afresh for every later off-menu bet down the hand. In the controlled experiments this beat the previous standard — rounding the bet to the nearest menu size — by more than an order of magnitude in worst-case exploitability (119 versus 1465 mbb/g in the reported small-game test).⁹²

6.3.4 Caveats, dead-ends, and what the paper under-describes

The most revealing caveat is that **the blueprint alone is not superhuman — it does not even beat the prior bot**. Against Baby Tartanian8, the 2016 competition winner, Libratus’s raw blueprint did not beat it (-8 ± 15 mbb/g, 95% CI); only when nested subgame solving was switched on did the same system win, by 63 ± 28 mbb/g.⁸⁴ The offline strategy, in other words, is a scaffold, and essentially all of Libratus’s edge comes from real-time search.

Several other asterisks matter. **Action translation is not eliminated, only contained**: on the first two betting rounds Libratus still rounds off-menu opponent bets, and the entire self-improver module exists to chip away at that residual seam, three holes per night, never closing it. **Safe subgame solving had to be re-engineered to be usable at all** — for three years it was considered impractical because, in head-to-head play, textbook-safe solving lost to the theoretically unjustified *unsafe* variant — and even so Libratus deliberately uses *unsafe* solving once, at its first entry into the third round, because it is cheaper and empirically fine there. Using estimates rather than upper bounds can in principle push exploitability *above* the blueprint’s, bounded only by the 2Δ of the theorem. Finally, the true compute surfaces only in the secondary sources, and the **code was never released**, leaving independent verification to rest on the published pseudocode.

6.3.5 Compute & accessibility

Where DeepStack’s bill is paid almost entirely offline and its play is cheap, Libratus is **expensive at both ends**. The project consumed roughly **25 million CPU core-hours** on the Bridges supercomputer

⁹²Brown, N. & Sandholm, T. (2017). “Safe and Nested Subgame Solving for Imperfect-Information Games.” *NeurIPS* 30, 689–699; arXiv:1705.02955.

at the Pittsburgh Supercomputing Center over a year — of which about 6 million went to building and solving the blueprint, about 3 million to real-time subgame solving during the match, about 3 million to the self-improver, and the remaining ~13 million to exploratory experiments and evaluation.⁹¹ Each **real-time subgame solve used about 50 nodes and took on the order of tens of seconds**, with no GPUs anywhere,⁹³ which made Libratus, as deployed, essentially impossible to reproduce outside a major supercomputing centre.

6.3.6 Strengths and limitations

Libratus’s signal strength is **decisive, rigorously demonstrated superhuman play**: it beat each of the four professionals individually and, unlike DeepStack, also beat the **prior best poker AI head-to-head** by 63 mbb/g, disentangling its strength from the question of human skill. Its limitations — two-player zero-sum only, abstraction-based, still translating off-menu bets on the early rounds, no neural generalization, and supercomputer-scale play — set up the rest of the chapter, and because both humans and AI adapted over the match, even the headline significance is, strictly, an “as-if-independent” figure, though a 147 mbb/g margin over 120,000 hands leaves no real doubt.⁸⁴

6.3.7 Legacy and modern relevance

Strip away the abstraction machinery and Libratus’s enduring idea is **real-time search layered on a coarse precomputed strategy, made safe under hidden information**: solve the local situation exactly at decision time, consistently with a cheap global plan, and respond to whatever the opponent actually does rather than to a rounded approximation of it. Libratus’s nested safe subgame solving is the direct ancestor of the real-time search in **Pluribus** (2019), and the same “refine a global value estimate with a local solve” pattern reappears, now with *learned* values replacing the tabular blueprint, in **ReBeL** (2020). The honest verdict is that Libratus is *superseded as an architecture but vindicated as a thesis*: its bet that **real-time search matters more than a bigger precomputed strategy** was exactly right, even as its bet on tabular abstraction was overtaken. For this thesis specifically, Libratus contributes two seeds: its safe-subgame-solving exploitability bound is a template for **safe exploitation under value error (Contribution 2)**, and its deliberate refusal to exploit — “fix your own weaknesses, do not model the opponent” — is the precise foil against which a *bounded, deliberate* opponent adaptation (Contribution 1) can be defined.

6.4 Pluribus (2019)

Libratus settled two-player no-limit hold’em, but its safety guarantees — and indeed the very meaning of “solving” the game — rested on two-player zero-sum structure. Poker as humans actually play it, though, seats six. Pluribus (Brown & Sandholm, 2019; Carnegie Mellon and Facebook AI)⁸³ confronted the multiplayer question directly: **what becomes of the blueprint-plus-real-time-search recipe when you remove the two-player crutch and sit at a six-handed table — a setting where a Nash**

⁹³Sandholm, T. (2021). “State-of-the-art for two-player no-limit Texas hold’em: Libratus.” Lecture 13 slides, 15-888 *Computational Game Solving* (Fall 2021), Carnegie Mellon University.

equilibrium is neither unique, nor efficiently computable, nor even a guarantee that you will not lose? Its answer was empirical and emphatic. Across two formats — five professionals seated with one copy of Pluribus, and one professional against five copies — it beat fifteen elite professionals, winning by about **48 milli-big-blinds per game** against five humans at once (roughly five big blinds per hundred hands, a decisive six-handed margin) at 95% statistical significance. And it did so after training for **eight days on a single 64-core server for about \$150 of cloud compute** — on the order of a thousandth of what the supercomputer behind Libratus consumed.

Table 10: Pluribus (2019) at a glance.

At a glance	Pluribus (2019)
Players	6 (six-max) — the first superhuman AI in any benchmark game with more than two players/teams
Game type	6-max NLHE — six-player no-limit Texas hold'em (imperfect-information; multiplayer, <i>not</i> 2-player zero-sum)
Blueprint (offline)?	Yes — a full-game blueprint solved offline by Linear MCCFR; played <i>directly</i> only on the first betting round, a scaffold thereafter
Neural component	None — purely tabular CFR / abstraction (no neural networks anywhere, as in Libratus)
Search mechanism	Real-time <i>depth-limited</i> search with $k=4$ continuation strategies at the leaves (nested <i>unsafe</i> solving, re-solved from the start of each betting round); Linear CFR inside subgames
Abstraction?	Yes — action (1–14 blueprint bet sizes; 1–6 in search) + information abstraction (lossless first round; lossy buckets later, finer in search than in the blueprint)
Perfect-info too?	No (imperfect-information only)
Compute	Famously cheap: blueprint ~12,400 core-hours / 8 days / < 512 GB on one 64-core server (~\$144); play on 2 CPUs (28 cores), < 128 GB, no GPUs, 1–33 s/decision
Key innovation	Superhuman six-player play via blueprint + depth-limited search with continuation strategies — won <i>empirically</i> , on a tiny budget, with no N-player safety guarantee

6.4.1 The gap it closed

Every superhuman game AI before Pluribus — checkers, chess, Go, and both prior poker programs — shared a hidden assumption: two players, zero sum. In that setting a Nash equilibrium carries a property that makes it the obvious target: any player who adopts one is *guaranteed not to lose* in expectation, whatever the opponent does, and if two players independently compute equilibria, their strategies still combine into an equilibrium. With more than two players, finding — even approximating — a Nash

equilibrium is computationally intractable in general; there are typically *many* equilibria; and, fatally, if each player independently selects one, the resulting joint strategy need not be an equilibrium at all, so a player can *lose* while playing an impeccable equilibrium strategy. The conclusion is stark — in multiplayer poker a Nash equilibrium is *neither unique nor a safety guarantee*, and the field’s two-decade definition of success simply evaporates.

Pluribus closes this gap by changing the goal and then meeting it: it aims only to *empirically and consistently defeat elite humans*, and it accepts up front that its algorithms carry no guarantee of converging to an equilibrium outside two-player zero-sum play. That concession is the entire point — and it is the precise gap this thesis’s Contribution 2 sets out to fill. The other half of the gap is mechanical: with six players, solving every late-game subgame *to the end* in real time, as Libratus did, becomes infeasible, so Pluribus needed a real-time search that looks only a little way ahead and stops.

6.4.2 Architecture

Like Libratus, Pluribus splits into an offline phase that builds a blueprint and an online phase that searches — but the balance of power between them is inverted, and, again like Libratus, there is **no neural network anywhere** (see the figure below).

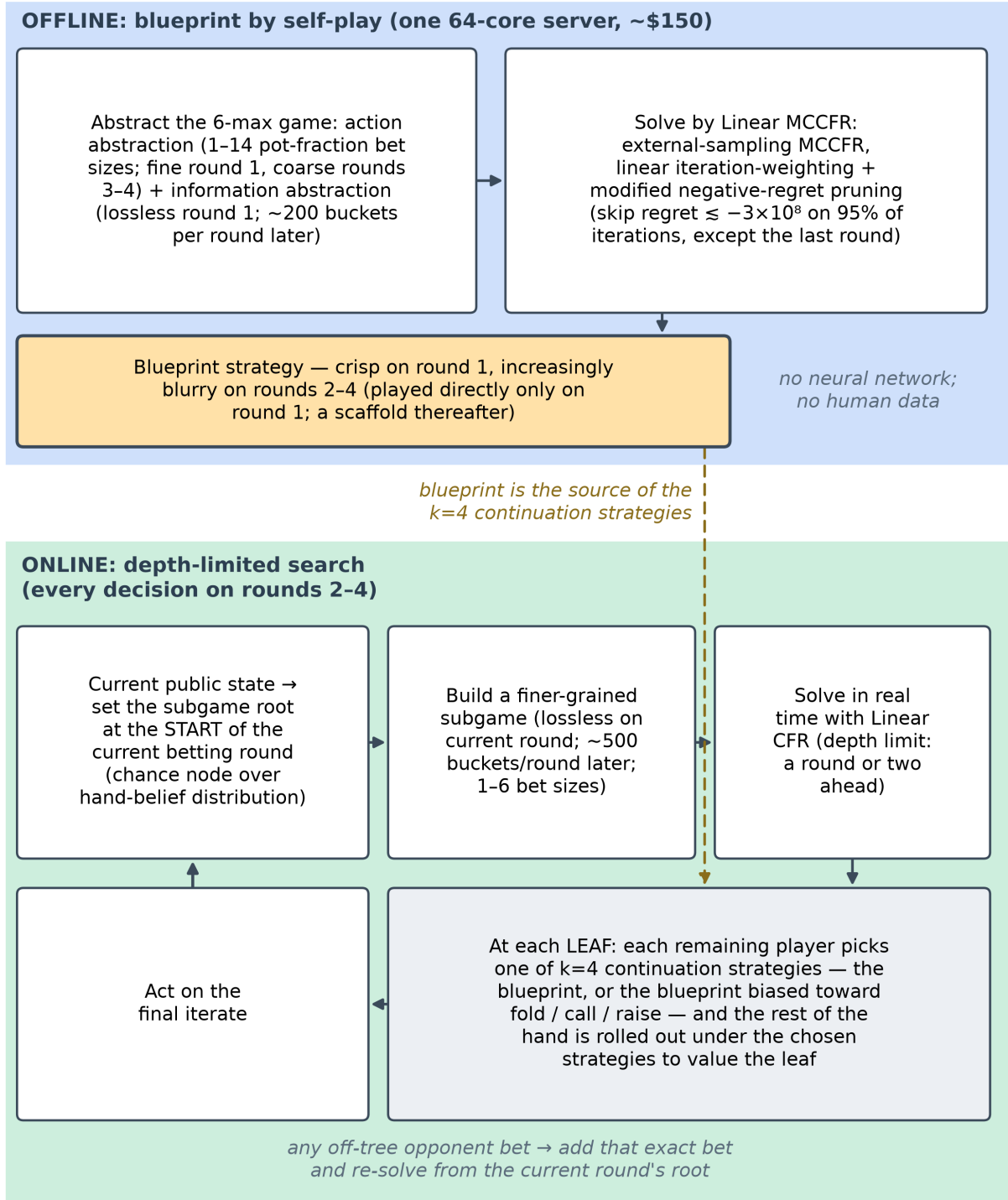


Figure 26: Pluribus’s two-phase architecture: an offline Linear-MCCFR blueprint (top) reused as the source of $k=4$ continuation strategies inside the online depth-limited search (bottom).

Offline, Pluribus computes a blueprint for the whole game by self-play with an **external-sampling Monte Carlo CFR (MCCFR)**, abstracting the game only coarsely, and it plays this blueprint *directly* in just the first of the four betting rounds. Everywhere else — flop, turn, river — Pluribus discards the

coarse blueprint locally and **searches in real time** for a finer strategy, exactly as Libratus did, with one decisive difference: it does not solve to the end of the game. It looks only a round or two ahead to a depth limit and stops, which is what makes six players affordable. Unlike Libratus, there is **no self-improver**: Pluribus never patches its blueprint between sessions, because depth-limited search on three of the four rounds already does the repairing.

6.4.3 Key innovation: depth-limited search with continuation strategies

Heuristic search in chess or Go works by looking a fixed distance ahead and reading a value off the horizon, on the assumption that both sides play well from there. That assumption fails completely under hidden information, and Brown and Sandholm make the failure vivid with a one-shot sequential Rock–Paper–Scissors: if the searcher assumes the opponent will play the equilibrium (each throw one-third of the time) beyond the horizon, then every action looks equally valued at zero, so the searcher might settle on “always play Rock” — whereupon the opponent switches to always Paper and the true value plummets. A leaf in an imperfect-information game therefore has *no single value*: its worth depends on the strategy the searcher will adopt, which is precisely what the search is trying to determine.

Pluribus’s central contribution is the way it gives leaves a value anyway. When search reaches the depth limit, instead of freezing one continuation, **each player still in the hand chooses among four different continuation strategies** for the remainder of the game, and may mix over them. The four are deliberately simple: the blueprint itself, and three biased copies that multiply the probability of *folding*, of *calling*, and of *raising* (each then renormalized). Because an opponent can always switch to whichever continuation punishes the searcher most, an unbalanced strategy — the poker equivalent of always playing Rock — is no longer rewarded, and the searcher is driven toward balance. This idea was first proven in a two-player precursor, Modicum (Brown, Sandholm & Amos, 2018), which beat two former champion bots while running on a 4-core laptop with 16 GB of memory.⁸²

Pluribus generalizes the idea from two players to six. In Modicum only the *opponent* chose among continuation strategies while the searcher always played the blueprint; Pluribus lets the **searcher choose among the continuation strategies too**, which balances the players. It also re-solves not from the current decision point but from the *start of the current betting round*, holding fixed only the actions it has already taken. This “unsafe” search — so named because, unlike Libratus’s, it carries no exploitability guarantee — is cheaper and, because it begins just after a high-branching chance event, turns out to be hard to exploit in practice.

What is conspicuously missing from all of this is a *guarantee*. DeepStack bounded its exploitability by $k_1\epsilon + k_2/\sqrt{T}$ ⁸⁵ and Libratus by 2Δ ;⁹² Pluribus offers no such bound, and the omission is principled rather than careless. CFR’s engine still does, in any finite game, drive each player’s *average regret* to zero,

$$\frac{R_i^T}{T} \rightarrow 0 \quad (\text{no-regret}),$$

but the inference that carried the two predecessors from no-regret to safety — *no-regret play converges*

to a Nash equilibrium, and a Nash equilibrium cannot be beaten — holds only when there are two players and the game is zero-sum. With six players the first implication fails (self-play need not approach an equilibrium) and the second is meaningless (an equilibrium is not unbeatable). The very iterations that *proved* DeepStack and Libratus safe therefore buy Pluribus only empirical strength: superhuman in practice, with nothing certified.

6.4.4 Caveats, dead-ends, and what the paper under-describes

The most consequential admission is that Pluribus uses **unsafe** subgame solving — it assumes opponents have played the strategy it computes *for* them — which, the authors state plainly, “lacks theoretical guarantees on performance even in two-player zero-sum games and there are cases where it leads to highly exploitable strategies.”⁹⁴ Safe alternatives exist, but in head-to-head play they did worse, so Pluribus takes the empirical win and mitigates the risk only by always re-solving from the start of the betting round. A second seam is inherited and never fully closed: on the *first* betting round, opponent bets too far off the blueprint’s menu are still **rounded** by action translation, and Pluribus, having no self-improver, simply lives with it.

Pluribus’s headline innovations are **never individually ablated** — the authors concede that measuring each one’s contribution is “too expensive” — but the two-player precursor paper does show that a *single* blueprint continuation at the leaves lost to Baby Tartanian8 (-10 ± 8 mbb/g) and did not beat Slumbot (-1 ± 15), whereas the four-continuation version beat both ($+6 \pm 5$ and $+11 \pm 9$);⁸² the continuation-strategy set is doing real work, not decoration. Smaller curiosities round out the picture: Pluribus plays its **final** search iterate rather than the usual time-average, to avoid residual bad actions; it learned to **abandon “limping”** during self-play yet **“donk-bets” far more than humans do**; and — like Libratus — its **code was never released**, leaving only pseudocode for independent verification, because poker is played commercially.

6.4.5 Compute & accessibility

The blueprint was trained in **eight days on a single 64-core server** for about **12,400 core-hours** and under 512 GB of memory — roughly **\$144** at cloud spot prices — and at the table Pluribus runs on **two CPUs (28 cores) and under 128 GB, with no GPUs at any point**.⁸³ Where DeepStack concentrated its cost offline and Libratus paid heavily at both ends, Pluribus is **cheap at both ends**. The collapse is not magic but algorithmic: the compounding of depth-limited search (which the authors estimate saves at least five orders of magnitude over solving to the end), Linear CFR, and aggressive pruning and memory thrift. With no code released, though, “accessible” describes the *method*, demonstrated at laptop scale by Modicum, more than a downloadable program.

⁹⁴Brown, N. & Sandholm, T. (2019). Supplementary Materials for “Superhuman AI for multiplayer poker.” *Science*, 365(6456).

6.4.6 Strengths and limitations

Pluribus’s signal strength is simply that it is **first**: the first AI to reach superhuman performance in any widely recognized benchmark game with more than two players or two teams, and in poker’s most popular form.⁸³ The win was decisive and rigorous — **+48 mbb/g (p = 0.028)** against five elite pros at the table, and **+32 mbb/g (p = 0.014)** with five copies against a lone pro — measured with the **AIVAT** variance reducer (which cut variance about ninefold) over 20,000 hands.

The limitations are precisely what it gave up to get there. Pluribus has **no safety guarantee whatsoever** in the six-player setting — no Nash convergence, no exploitability bound — so its superhuman status is an *empirical* fact about fifteen strong humans over 20,000 hands, not a theorem. The paper claims only that “there are large-scale, complex multiplayer imperfect-information settings in which a carefully constructed self-play-with-search algorithm can produce superhuman strategies”.⁸³ And its 48-mbb/g six-handed win rate, though decisive, is not the same currency as Libratus’s 147 heads-up — different game, five opponents, higher variance.

6.4.7 Legacy and modern relevance

Two ideas outlast the poker specifics. The first is technical: **depth-limited search made sound under hidden information by giving each leaf a small menu of selectable continuation strategies rather than a single value**. The second is economic: Pluribus is the canonical demonstration that **a harder problem can be solved with a thousandfold less compute through better algorithms and search at decision time**, rather than more hardware. In the architectural arc of this chapter, Pluribus is the point where the *player-count* barrier falls; the *generalization* barrier it leaves standing — everything is still tabular and hand-abstracted — is what **ReBeL** and **Student of Games** dismantle next, by replacing the blueprint with learned belief-state values. For this dissertation specifically, Pluribus is the keystone of Contribution 2 — the empirical proof that Nash-and-search methods *work* in N-player imperfect-information games while offering **no safety guarantee at all**, which is exactly the gap a theory of multi-agent safe exploitation must close.

6.5 ReBeL (2020)

Pluribus broke the player-count barrier while remaining exactly what Libratus was — a giant, hand-abstracted lookup table with nothing learned that transfers from one situation to the next — and it bought its six-handed win by giving up safety altogether. ReBeL (Brown, Bakhtin, Lerer & Gong, 2020; Facebook AI Research)⁸⁸ steps back from six players to two and asks the opposite question: **what if, instead of hand-crafting an abstraction and precomputing a blueprint, we ran AlphaZero — self-play reinforcement learning plus search, at both training and test time — in a game of hidden information, recovering the very guarantees Pluribus discarded?** Its answer, *Recursive Belief-based Learning*, is, by its authors’ account, the first algorithm to make reinforcement learning *and* search provably sound in imperfect-information games. The trick is to recast the game as a “perfect-information” game over **public belief states** — probability distributions over what each player might be holding, given common knowledge — on which value and policy functions are well-defined; an AlphaZero-

style loop then trains a neural value (and policy) network on those states, with counterfactual regret minimization (CFR) solving depth-limited subgames at the leaves. ReBeL provably converges to a Nash equilibrium in any two-player zero-sum game, beat Dong Kim — a heads-up professional who had done best of the four humans against Libratus — by 165 mbb/g over 7,500 hands while using *far less* domain knowledge than any prior poker AI, and, unlike the closed Libratus and Pluribus, its implementation (for Liar’s Dice) was **open-sourced**.

Table 11: ReBeL (2020) at a glance.

At a glance	ReBeL (2020)
Players	2 (heads-up) — <i>guarantees</i> are two-player zero-sum; the formalism is N-player, but guarantees and experiments are two-player only
Game type	General 2p0s imperfect-information; evaluated on HUNL poker + Liar’s Dice (and turn endgame hold’em). Reduces to an AlphaZero-like algorithm in perfect-information games
Blueprint (offline)?	No stored blueprint — the offline product is a <i>learned PBS value (+ policy) network</i> from self-play, not a strategy table
Neural component	PBS value network + (optional) PBS policy network; MLP (GeLU/LayerNorm), 6×1536 hidden for poker, input = belief over each player’s 1,326 hands + board + pot + bet flag
Search mechanism	CFR (CFR-D / CFR-AVG; also FP/FLOP) solving a depth-limited subgame rooted at a PBS, with the learned value net supplying (iteration-dependent) leaf values — at both training and test time
Abstraction?	None — no card/information abstraction (lossy or lossless); the value net replaces it. Keeps only a small (≤ 9) hand-chosen bet-size menu, with off-tree bets added live
Perfect-info too?	No (presented/evaluated as imperfect-information), with the nuance that it <i>degenerates</i> to AlphaZero-style search if private information is removed — full unification is Student of Games’ claim
Compute	GPU-trained: full HUNL used ~90 DGX-1 nodes × 8 V100 GPUs for self-play data generation (a contrast with Pluribus’s CPU-only ~\$150); CFR on a single CPU thread; play < 2 s/hand, ≤ 5 s/decision
Key innovation	Public belief states + an AlphaZero-style self-play loop training a PBS value/policy net with CFR run in belief space: by its authors’ account the first <i>sound</i> RL+Search for imperfect-information games, recovering a provable 2p0s Nash guarantee with no abstraction or blueprint

6.5.1 The gap it closed

Libratus and Pluribus were, underneath, *tabular* objects — a strategy or blueprint stored over a hand-engineered abstraction; DeepStack alone had a neural component, and even it leaned on a 1,000-bucket clustering at the network’s input and an abstraction on the river. Meanwhile AlphaZero’s marriage of self-play reinforcement learning with search, the most successful paradigm in game AI, had been *unavailable* for imperfect information: prior RL+Search algorithms were “not theoretically sound in imperfect-information games and have not been shown to be successful in such settings”.⁸⁸

The reason is the conceptual crux of the whole chapter. AlphaZero assumes each state has a single well-defined value, and hidden information destroys that assumption. The paper’s example is a modified Rock–Paper–Scissors in which scissors wins (or loses) double: the equilibrium throws rock and paper 40% of the time and scissors 20%, and at that equilibrium every action has expected value zero. A one-ply chess-style search that substitutes the equilibrium value at the leaves therefore finds every move equally good and might settle on “always rock” — whereupon the opponent switches to “always paper” and rock’s true value collapses from 0 to -1 . **In an imperfect-information game the value of an action depends on the probability with which it is played**, so a state defined by the sequence of actions alone has no unique value. Pluribus had patched the symptom with selectable continuation strategies at the leaves⁸²; ReBeL cures the disease by redefining the state to include the probability distribution over the hidden information — a *public belief state* — on which values become well-defined again. The hand-crafted abstraction and the precomputed blueprint both disappear, replaced by a value function learned purely from self-play.

6.5.2 Architecture

ReBeL is best read as **AlphaZero for imperfect information**: a self-play loop that trains neural value and policy networks, where the “search” used during both training and play is CFR solving a depth-limited subgame, and everything operates on *public belief states* rather than raw game states (see the figure below).⁸⁸⁹⁵

⁹⁵Bakhtin, A. (2020, 3 Dec.). “ReBeL: A general game-playing AI bot that excels at poker and more.” *Meta AI blog*.

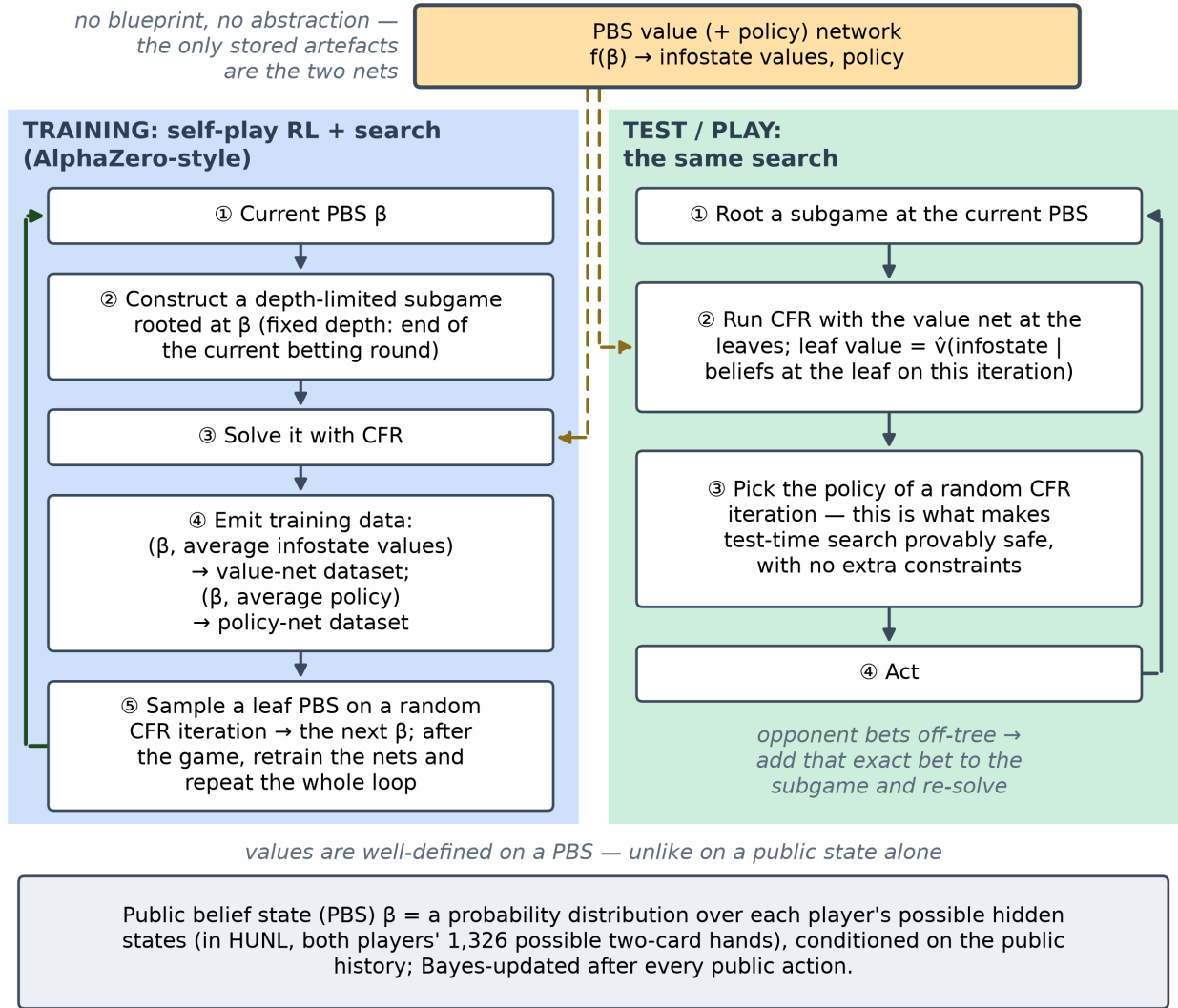


Figure 27: ReBeL’s AlphaZero-style loop: self-play training (left) and test-time play (right) share one PBS value/policy network and both solve depth-limited subgames rooted at the public belief state.

Starting from a root PBS, ReBeL constructs a depth-limited subgame, solves it with CFR (specifically CFR-D, “CFR with decomposition”, or its variant CFR-AVG), records the solved values and policy as training targets, samples a leaf to become the next root, and continues to the end of the game — then retrains the networks and iterates, exactly as AlphaZero alternates self-play with network training. On every CFR iteration each leaf’s value comes from the **learned PBS value network**, so the leaf values shift from iteration to iteration, which is precisely what keeps the search sound under hidden information; an optional policy network warm-starts CFR to cut the number of iterations. There is **no precomputed blueprint** — the only artefacts carried out of training are the two networks — and **no abstraction** of any kind: ReBeL computes a distinct policy for every infostate, feeding the network only the raw belief distribution over both players’ 1,326 possible hands plus the board, pot, and a single “has anyone bet this round” flag. CFR (Chapter 3) is the search engine and neural value approximation (Chapter 5) the leaf

evaluator, now fused into a single self-play-plus-search loop.

6.5.3 Key innovation: public belief states and learned values in belief space

ReBeL’s contribution is one conceptual move with three technical consequences: stop searching over *states* and start searching over *beliefs about states*.

The **public belief state (PBS)** is the heart of it, and the Meta AI blog gives the cleanest intuition.⁹⁵ Take a card game and modify it so the players cannot see their own cards — only an impartial referee can; on each turn a player announces, for every card they *might* hold, the probability with which they would take each action, and the referee samples the move for the player’s true card. Because all players’ strategies are assumed common knowledge, everyone can track, via Bayes’ rule, the probability that each player holds each possible hand. This modified game is **strategically identical** to the original, yet it contains *no private information*: its state — the vector of those probabilities — is fully observed by everyone. ReBeL calls that state a public belief state: formally, a joint probability distribution over the players’ possible infostates, given the common public observations.⁸⁸ Viewing imperfect-information games as continuous-state perfect-information games is an old idea (it goes back to work on decentralized multi-agent POMDPs); ReBeL’s achievement is being the first to combine it with self-play reinforcement learning in an *adversarial* setting.

The first consequence is that **values become well-defined again**: in two-player zero-sum games every PBS β carries a unique value $V_i(\beta)$ with $V_1(\beta) = -V_2(\beta)$, defined by both players playing a Nash equilibrium in the subgame from β . This formalizes and generalizes DeepStack’s belief-conditional counterfactual values, the first instance of a PBS value function. The second is that the AlphaZero loop can run with **CFR as the search algorithm**: the belief representation is a very high-dimensional continuous space in which tree search is hopeless, but in two-player zero-sum games it poses a *convex* optimization problem and CFR is effectively a gradient-based solver for it — the one substantive difference from AlphaZero.

The third consequence, and the headline, is that the whole thing is **provably sound**. During self-play ReBeL can assume both players’ policies are common knowledge, so it always knows the true PBS; against a real opponent it does not, which naively breaks search. ReBeL’s fix is to run *the same algorithm* at test time and **act according to the policy of a randomly chosen CFR iteration**. The paper proves (Theorem 3) that this yields safe search — a Nash equilibrium *in expectation* — with no extra constraints, where every prior “safe search” method bolted on constraints so costly they were never fully used in a competitive bot. With a PBS value network whose error is at most δ and T CFR iterations per subgame, ReBeL plays an ε -Nash equilibrium — one no opponent can exploit for more than ε — with

$$\varepsilon = \delta C_1 + \frac{\delta C_2}{\sqrt{T}},$$

for game-specific constants C_1, C_2 . The structure echoes DeepStack’s $k_1\epsilon + k_2/\sqrt{T}$, but where DeepStack’s bound governed a single re-solve, ReBeL’s governs *convergence to a Nash equilibrium of the whole game*, and both terms scale with the learned value’s error δ : as the value network improves, ReBeL’s play

approaches an exact Nash equilibrium. This is the precise sense in which ReBeL returns to two-player zero-sum and recovers the guarantees Pluribus gave up.

6.5.4 Caveats, dead-ends, and what the paper under-describes

The *theory* is in the main text; the *engineering honesty* is in the appendices. Appendix D catalogues what ReBeL deliberately **throws away**. It uses *no* information abstraction, lossy or lossless. Where DeepStack trained its value net on *randomly generated* PBSs drawn from a hand-tuned sampler, ReBeL generates its training PBSs purely from self-play, arguing that random sampling “would be like learning a value function for Go by randomly placing stones on the board” — and it shows (Figure 2 of the paper) that a value net trained on random beliefs “fails to learn anything valuable.” It also drops the precomputed all-in equity tables and the solving to the end of the game on the third betting round, always solving only to the end of the *current* round. The upshot is double-edged: ReBeL must learn *six* “layers” of values where DeepStack needed three, which *increases* the surface for error propagation. And it is not domain-knowledge-free: it keeps a hand-chosen menu of at most eight or nine bet sizes, with off-tree bets added to the subgame and re-solved, à la Libratus and Pluribus.

The clean theorems rest on **idealizations**: Theorem 2’s convergence assumes a perfect function approximator. The variant the agent actually runs for efficiency, a modified **CFR-AVG**, is by the authors’ own admission *not known to be theoretically sound* — “whether or not this modified form of CFR-AVG is theoretically sound remains an open question”⁸⁸ — even though it performs well in poker, a familiar gap between the proven algorithm and the shipped one. And the safe-search guarantee depends on picking a *random* CFR iteration, which could land on a terrible early one; this is mitigated only by Linear CFR, which down-weights early iterations.

6.5.5 Compute & accessibility

Where Pluribus’s blueprint came from CPU-only CFR on a single server for about \$150, ReBeL’s intuition is a neural network trained at **GPU-cluster scale**: the full HUNL agent used **about 90 DGX-1 nodes, each with eight 32 GB Nvidia V100 GPUs**, to generate self-play data, while the CFR search itself runs on a single CPU thread and play takes under two seconds per hand. Its **implementation was open-sourced** (for Liar’s Dice), where Libratus and Pluribus published only pseudocode,^{88,483} but the poker agent was withheld because ReBeL “can compute a policy for arbitrary stack sizes and arbitrary bet sizes in seconds”, making it a ready-made cheating tool — the open artefact is the research game, not the superhuman poker bot.

6.5.6 Strengths and limitations

ReBeL backs its theory with results: it beat the prior champions Slumbot (+45 mbb/g) and BabyTartanian8 (+9 mbb/g), drove the local-best-response (LBR) probe⁸⁶ to a large loss, and beat Dong Kim by 165 mbb/g over 7,500 hands, with no card abstraction. The same algorithm, unchanged, converges to approximate Nash in Liar’s Dice, and in perfect-information games it reduces to an AlphaZero-like method — a *framework*, not a poker program.

The limitations set up the final system. ReBeL’s guarantees are **two-player zero-sum only**: the unique PBS value, the convexity that licenses CFR-as-search, and the soundness proofs all lean on that structure (the notation is written for N agents, but every experiment and guarantee is two-player). Its most concrete scaling wall is that the **PBS grows with the amount of hidden information**: the network’s input scales with the number of infostates in a public state, so games with great strategic depth but little common knowledge — the paper names Recon Blind Chess — blow the representation up, and adding players only makes the belief space larger. It also assumes the **exact rules of the game are known** (a MuZero-style extension is flagged as future work), inherits a residual **value-approximation error** δ , and its strongest empirical result rests on a **single human opponent** over 7,500 hands.

6.5.7 Legacy and modern relevance

ReBeL’s enduring idea is representational: **convert hidden information into a belief state and the full power of self-play reinforcement learning plus search transfers across**. Its closest descendant is Student of Games, whose authors call ReBeL “the most closely related algorithm”⁸⁹; the RL-plus-search-with-learned-models lineage also runs into CICERO (2022)⁹⁶, a seven-player Diplomacy agent far outside ReBeL’s two-player guarantees. The PBS representation, the self-play-trained value/policy function on belief states, CFR-as-search with a learned leaf evaluator, the random-iteration “safe search for free” trick, and the fast equilibrium finder FLOP all remain reusable. For this thesis ReBeL cuts two ways: the PBS is the starting point for Contribution 1, which extends it to carry beliefs about the opponent’s *strategy type*, not just their cards; and ReBeL is the proof that the soundness Pluribus abandoned *can* be recovered — just not yet beyond two players, which is exactly the frontier Contribution 2 sets out to cross.

6.6 Student of Games / SoG (2023)

ReBeL recovered soundness for two-player zero-sum poker, but it remained an imperfect-information method that merely *reduced* to AlphaZero in perfect-information games, solved a *fixed* depth-limited subgame, and tied its test-time search to its training procedure. Student of Games (Schmid et al., 2023; Google DeepMind, with collaborators at the University of Alberta and EquiLibre Technologies in Prague) asks the most ambitious question of the chapter: **what if a single algorithm, learning from self-play with no human data, could play chess, Go, heads-up poker, and Scotland Yard — growing its own search tree as needed and remaining provably sound for perfect- and imperfect-information games alike?** Its engine is **Growing-Tree counterfactual regret minimization (GT-CFR)**: an anytime search that builds the game tree incrementally, guided by a policy network, with a learned value network evaluating the leaves it has not yet expanded. On perfect-information subtrees GT-CFR behaves like AlphaZero’s MCTS; on imperfect-information ones it behaves like CFR; and a single soundness theorem covers both. The system reaches **strong amateur-to-professional play in chess and Go, beats Slumbot — the strongest openly available heads-up no-limit bot — and defeats the state-of-the-art Scotland Yard agent**, all while being **proven to converge toward**

⁹⁶Meta Fundamental AI Research Diplomacy Team (FAIR), Bakhtin, A., Brown, N., Dinan, E. et al. (2022). “Human-level play in the game of Diplomacy by combining language models with strategic reasoning.” *Science*, 378(6624), 1067–1074. doi:10.1126/science.ade9097.

minimax-optimal play as computation and network accuracy grow. It shares several DeepStack authors (Bowling, Moravčík, Burch, Schmid) and first appeared in 2021 under the name *Player of Games*.

Table 12: Student of Games (2023) at a glance.

At a glance	Student of Games (2023)
Players	2 (two-player zero-sum) — guarantees <i>and</i> evaluation are 2p0s (Scotland Yard’s detectives count as one team); the underlying formalism is more general, but beyond 2p0s the Nash guarantee is “less meaningful”
Game type	Both perfect- and imperfect-information — the only unified system: chess + Go (perfect) and HUNL poker + Scotland Yard (imperfect)
Blueprint (offline)?	No — the offline product is a learned value+policy network, not a stored strategy table (as in ReBeL)
Neural component	A single counterfactual value-and-policy network (CVPN) : one net outputs per-infostate counterfactual <i>values</i> and a prior <i>policy</i> , for all game stages
Search mechanism	GT-CFR — alternates a CFR regret-update phase (CVPN values at the leaves) with an AlphaZero-style PUCT expansion phase that <i>grows</i> the public tree; run inside continual re-solving, at both training and test time
Abstraction?	None of the card/information kind (the CVPN replaces it); keeps only a small <i>randomized betting</i> (action) menu in poker ($\approx 20,000 \rightarrow 4\text{--}5$ actions), like ReBeL
Perfect-info too?	Yes — the distinguishing row; the only system in the chapter demonstrated on perfect-information games, with the <i>same</i> algorithm and soundness covering both classes
Compute	TPU-trained, deliberately matched to AlphaZero’s budget (Google TPUv4; Go the most expensive); reported <i>relative</i> to AlphaZero, no single dollar figure; search is $O(kT^2)$ ($O(T)$ for perfect-info); the full agent/code was not released
Key innovation	Growing-Tree CFR + sound self-play: one self-play-with-search algorithm, sound for both game classes , that grows its tree incrementally with a CVPN at the leaves and provably converges to Nash

6.6.1 The gap it closed

Since Samuel’s checkers program of the 1950s the two great traditions of game AI ran on separate tracks.⁸⁹ One — minimax, alpha–beta, Monte-Carlo tree search, and ultimately AlphaZero — conquered *perfect-information* games by combining search with a learned value function; the other — counterfactual regret minimization and the poker systems of this chapter — conquered imperfect-information poker through game-theoretic reasoning. AlphaZero could not play poker, because its tree search is *unsound* under

hidden information for the reason the ReBeL section made precise; the poker systems could not play Go, being built around the betting structure of one game; and even ReBeL was guaranteed only for imperfect information. The open question Student of Games answers is whether a *single* algorithm — one search procedure, one network architecture, one self-play loop — can be **sound and strong across both game classes at once**. The difficulty is not merely engineering: a perfect-information search wants to expand the one best line deeply and read a single value off each leaf, while an imperfect-information search must keep a *distribution* over actions (so as not to leak private information) and reason about a *belief* over hidden states at every leaf. Student of Games closes the gap with a search, GT-CFR, that grows a tree the way AlphaZero does — guided, asymmetric, anytime — while computing the game-theoretically sound quantities CFR does, built on ReBeL’s public-belief-state representation and trained through a *sound self-play* loop.

6.6.2 Architecture

Student of Games is, structurally, **AlphaZero’s self-play loop with Monte-Carlo tree search replaced by GT-CFR and with the value/policy network defined over public belief states**, the same search running both offline and online (see the figure below).

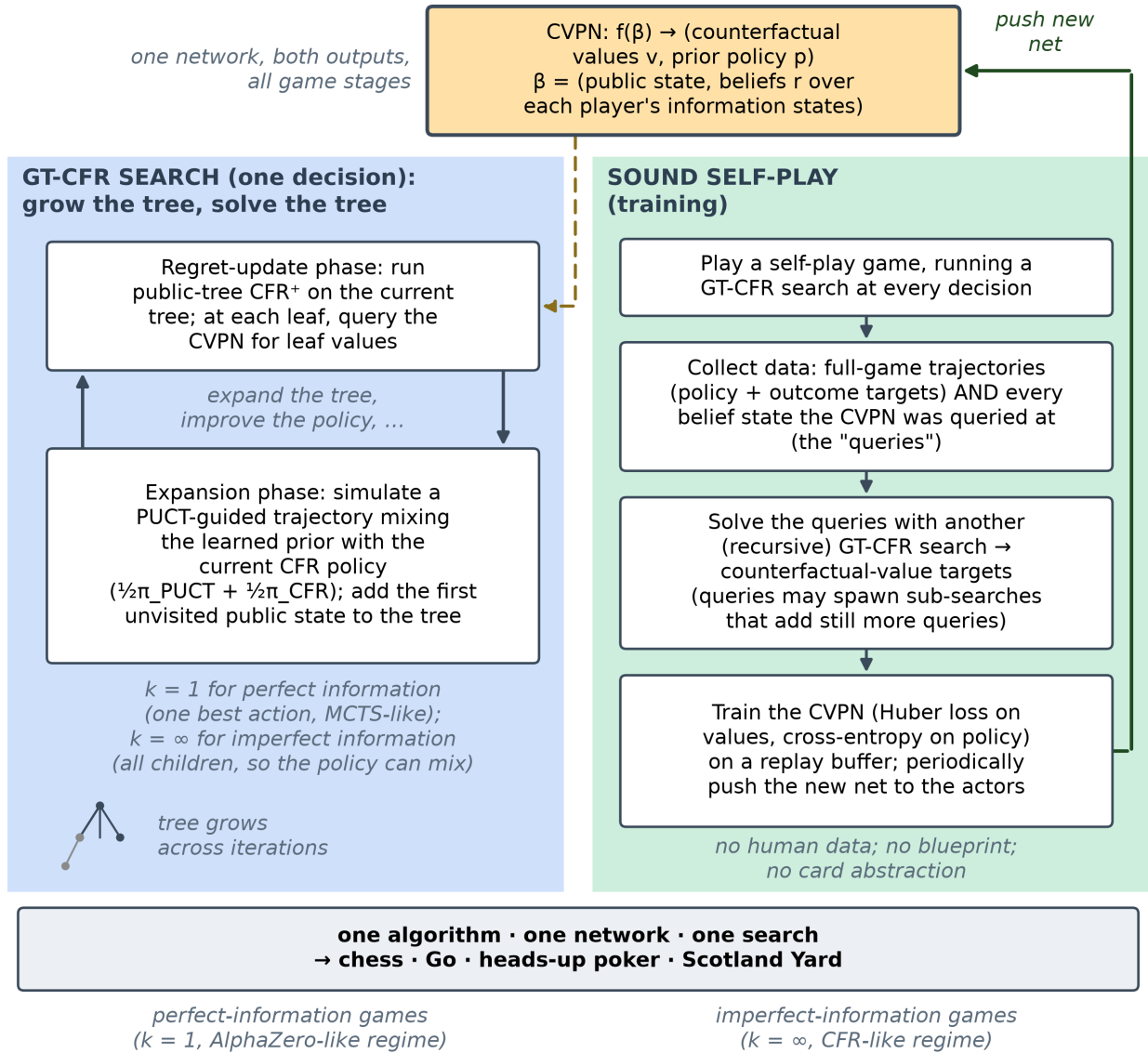


Figure 28: Student of Games: the GT-CFR search loop (left) and the sound self-play training loop that feeds it (right) share one CVPN; the footer names the perfect-/imperfect-information unification.

A **public belief state** $\beta = (s_{\text{pub}}, r)$ pairs the public state (in poker, the betting history and board) with a **range** r — a pair of distributions over the information states each player could privately occupy.⁸⁹ Perfect-information games are simply the degenerate case in which every public state has exactly *one* information state and the belief is a point mass, which is precisely why one representation can serve both classes. The **counterfactual value-and-policy network (CVPN)** is a *single* network that, given a belief state, emits both a vector of counterfactual values (one per information state, per player) and a prior policy, for every stage of the game, where DeepStack used separate value-only networks per round. Offline, actors generate self-play data with a GT-CFR search at every decision while trainers fit a new CVPN; online, the agent runs the very same search (via continual re-solving) to choose each move. CFR⁺

(Chapter 3) is the search’s inner loop and value-and-policy approximation (Chapter 5) is the CVPN; the binding novelty is GT-CFR, the search that *grows* its tree, and the sound self-play that keeps every search consistent with every other.⁸⁹

6.6.3 Key innovation: Growing-Tree CFR and sound self-play

Student of Games’ contribution is a search algorithm that does for imperfect information what Monte-Carlo tree search did for perfect information — grow a tree intelligently toward the lines that matter — *without* sacrificing the soundness that hidden information demands.

The first idea is **growing the tree by alternating two phases**. AlphaZero’s MCTS expands one node per simulation and never looks back, because in a perfect-information game the value of a node, once computed, never changes — the future does not alter the past. Under imperfect information this fails: as Schmid puts it, observing an opponent’s *future* action changes your *belief* about the private state they held in the past, so changing the strategy anywhere in the tree ripples everywhere, and after each expansion the *whole* tree must be re-solved. Each GT-CFR iteration therefore runs two phases in turn. The **regret-update phase** runs several iterations of public-tree CFR⁺ over the *current* tree and, at its frontier leaves, queries the CVPN for the counterfactual values of the subgame below, exactly as depth-limited solving uses a learned value at the horizon. The **expansion phase** then simulates a single trajectory from the root, choosing actions by a PUCT rule that *mixes* the network’s prior with the current CFR policy, and appends the first public state it reaches that is not yet in the tree. The result is an *anytime* search that, like MCTS, pours computation into the relevant lines, but that, unlike MCTS, is solving for a game-theoretically sound strategy at every step.⁸⁹

The second idea is the **single knob that adapts this one search to both game classes**. AlphaZero adds only the single most promising action when it expands, which is ideal when optimal play can be deterministic; optimal imperfect-information play is generally *stochastic*, so committing to one action is wrong. GT-CFR therefore expands the top- k actions by prior, with $k = 1$ for perfect-information games (the search is then essentially AlphaZero’s) and $k = \infty$ — all children — for imperfect-information games, where the search even gains a *finite-time* guarantee on policy quality. The same code thus reduces to MCTS-like search on a chess position and to CFR-like iteration on a poker decision.

The third element is **sound self-play**, which trains the CVPN. Like AlphaZero, the agent is trained to predict the result of its own search: each network query made during a search defines a subgame, that subgame is re-solved by *another* GT-CFR search, and the network is trained toward the search’s output (values by a Huber loss, policy by cross-entropy). The word *sound* is the load-bearing one: every search used to generate data must be consistent with the network and with the other searches along the trajectory (it must not be assembling fragments of two different equilibria), which is enforced by running the searches on a safe re-solving auxiliary game.

Theorem 1 completes the chapter’s family of soundness bounds: the regret of GT-CFR’s average policy after T iterations splits into a term that accumulates the value network’s ϵ -error over the tree’s *frontier* and an ordinary CFR regret term over the tree’s *interior*; dividing by T , the average policy’s exploitability

is bounded by

$$\text{exploitability}(\bar{\pi}^T) \lesssim \underbrace{|\mathcal{F}| \epsilon}_{\text{value-net error (frontier)}} + \underbrace{\frac{|\mathcal{N}| U \sqrt{A}}{\sqrt{T}}}_{\text{CFR convergence (interior)}},$$

with $|\mathcal{F}|$ the frontier size and $|\mathcal{N}|$ the number of information states in the interior, U the largest value gap, and A the maximum number of actions. The shape is the direct heir of DeepStack’s $k_1\epsilon + k_2/\sqrt{T}$ and ReBeL’s $\delta C_1 + \delta C_2/\sqrt{T}$, with two distinctions: the coefficients are the *sizes of the growing tree’s frontier and interior*, so adding nodes over time costs nothing in convergence order; and, decisively, **the same statement holds whether the tree is a perfect-information game tree or an imperfect-information public tree**. Theorem 2 adds that invoking GT-CFR through continual re-solving over a whole episode keeps the agent sound, with exploitability growing only *linearly* in the game length (a factor of roughly $5D + 2$ for D re-solving steps) — the property that lets Student of Games survive Scotland Yard’s twenty-four-round horizon. As the network improves ($\epsilon \rightarrow 0$) and search deepens ($T \rightarrow \infty$), play converges to a Nash equilibrium — for poker and chess alike.

6.6.4 Caveats, dead-ends, and what the paper under-describes

The main text states the theorems “only informally”; the architectures, hyperparameters, pseudocode, compute, and proofs live in the Supplementary Text. The headline caveat is openly owned: **Student of Games is weaker than AlphaZero in chess and Go given the same resources**, and the gap is not small in Go — its strongest configuration won just *2 of 400* games against a fully trained AlphaZero searching 8,000 simulations per move, even while crushing the classical program Pachi by over 1,100 Elo. The authors call this “the price of SoG’s generality” and attribute it to CFR being less efficient than Monte-Carlo tree search on perfect-information games: the unification is a proof of *soundness and competence* across classes, not of dominance in either.

Two scaling walls are the real limits. The first is the **belief-space blow-up**, which Schmid names as the main limitation: the CVPN must *enumerate the information states per public state*, manageable for poker’s 1,326 hands but “prohibitively expensive in some games”⁸⁹. This is ReBeL’s PBS-input limitation inherited intact; the paper floats a generative model that *samples* world states rather than enumerating them, but does not build one. The second is the **known-model requirement**: like AlphaZero and ReBeL, Student of Games needs a perfect simulator of the rules, unlike MuZero, which *learns* its model. Smaller asterisks: a **randomized betting abstraction** remains in poker (about twenty thousand actions reduced to four or five), the search is **quadratic in the number of iterations** ($O(kT^2)$ network calls), and the training-convergence argument holds only “asymptotically, as $T \rightarrow \infty$ and with very large (exponential) memory”. Together these mark Student of Games as a first, deliberately general proof of concept rather than a tuned, scalable product.

6.6.5 Compute & accessibility

Training is TPU-scale self-play, reported *relative to AlphaZero* rather than in absolute terms: the AlphaZero baseline used 3,500 concurrent actors each on a single Google TPuv4, and Student of Games “was trained using a similar amount of TPU resources”⁸⁹ — there is no single dollar figure of the kind Pluribus made famous. The full agent and its trained networks were **not released**, though the same group’s **OpenSpiel** ships the CFR family and the benchmark games: *an open method on an open substrate with a closed flagship*.

6.6.6 Strengths and limitations

The breadth backs the theory. In a single design Student of Games reaches expert-to-professional level in **chess and Go**, **beats Slumbot** (by about +7 mbb/g, which this paper writes as mbb/hand), and **defeats PimBot**, the state-of-the-art Scotland Yard agent, even when PimBot is given ten million search simulations to Student of Games’ few hundred — with no human data, no precomputed blueprint, and no card abstraction, and with exploitability falling as search and training grow, the empirical face of Theorem 1. The limitations are the shape of its ambition: it is **weaker than the specialists** in their home domains; its guarantees hold only for **two-player zero-sum** play (Scotland Yard’s detectives are pooled into a single team), so the *multiplayer* safety gap Pluribus exposed remains untouched even here; its **belief representation does not scale** to vast private-state spaces; it **requires a known model**; and the **flagship code was never released**.

6.6.7 Legacy and modern relevance

Student of Games lands all three of the chapter’s arcs at once: from hand-built abstraction to learned approximation, from offline solving to learning-and-searching, and finally **perfect and imperfect information in one algorithm**. The conceptual takeaway is that AlphaZero’s recipe (self-play, a learned value-and-policy network, and a tree grown by guided search) was never specific to perfect information; it only needed a *sound* search over *beliefs* rather than states. Swap Monte-Carlo tree search for GT-CFR, run it over public belief states, and the same recipe spans chess and poker with a single convergence guarantee.⁸⁹ Its unmet neighbours mark the frontier: **MuZero** learned the model Student of Games still assumes, and **CICERO** carries search-plus-learning into a seven-player, mixed-motive game — Student of Games unifies *information structure* but not *player count*. GT-CFR, the CVPN, and sound self-play are reusable in their own right.

For this thesis, Student of Games is both a ceiling and a signpost. Its belief — a distribution over the opponent’s possible information states — is exactly the substrate **Contribution 1** proposes to *enrich* with beliefs about the opponent’s strategy *type*, turning a Nash-seeking belief into an adaptive one. And its careful restriction of all guarantees to two-player zero-sum play — the paper is explicit that Nash is “less meaningful” beyond it — restates, from the most advanced vantage point in the literature, the exact gap **Contribution 2** targets: unifying the *information structure* of games did nothing to close the *multi-agent safety* gap that Pluribus first exposed.

6.7 Synthesis

The synthesis draws the five systems together — not to crown a winner, but to make explicit the shape of the arc, the shared machinery, and the open problems handed to the rest of the dissertation.

6.7.1 The arc in one read

Across the three axes of the introduction, every step forward on one axis was paid for somewhere else. DeepStack and Libratus attacked the lossy abstraction-and-blueprint paradigm from opposite directions.⁸⁵⁸⁴ Both stayed two-player and neither reported a result on the other’s headline metric (DeepStack played no head-to-head matches against other bots, and no LBR result was reported for Libratus)⁸⁸, so even at the start “better” was already multi-dimensional. Pluribus advanced the *player-count* axis to six by giving up safety entirely and remaining tabular; ReBeL returned to two players to *recover* a soundness guarantee while pushing the representational axis the furthest yet; and Student of Games added the third axis, unifying perfect- and imperfect-information play, at the cost of peak strength. The field traded *hand-built breadth* for *learned generality* and *offline precomputation* for *real-time search*, and no single system dominates on every axis at once. The table records what each system added and what it cost; the per-dimension values are in each section’s scorecard.

Table 13: The five systems in sequence: what each added, and what it gave up for it.

System (year)	What it added	What it gave up / what it cost
DeepStack (2017)	First <i>sound</i> depth-limited search under hidden information; learned counterfactual values in place of a stored strategy; discards the full-game abstraction-and-blueprint paradigm	Still keeps a sparse betting abstraction inside its look-ahead; never tested head-to-head against prior bots; ~175 CPU-core-years of offline value labelling
Libratus (2017/18)	First head-to-head <i>and</i> human win, via real-time nested safe subgame solving with a provable bound; an overnight self-improver that patches its own holes	Keeps and depends on abstraction + a petabyte blueprint; no neural generalization; supercomputer-scale (~25M core-hours)
Pluribus (2019)	First superhuman multiplayer (six-player) play; famously cheap (~\$150 on a single server)	Drops <i>all</i> safety guarantees (no N-player bound); relies on <i>unsafe</i> search; remains fully tabular and abstracted

System (year)	What it added	What it gave up / what it cost
ReBeL (2020)	“AlphaZero for imperfect information”: public belief states + learned value/policy + CFR in belief space; <i>recovers</i> the 2p0s guarantee; eliminates abstraction <i>and</i> blueprint; open-sources Liar’s Dice	Guarantees retreat to two players; the PBS blows up where common knowledge is scarce; GPU-cluster training; needs a known model
Student of Games (2023)	Unifies perfect- and imperfect-information play in one sound algorithm (GT-CFR + a single value-and-policy net + sound self-play)	Weaker than the specialists (markedly at Go); guarantees still 2p0s; belief-space blow-up; known-model requirement; flagship code unreleased

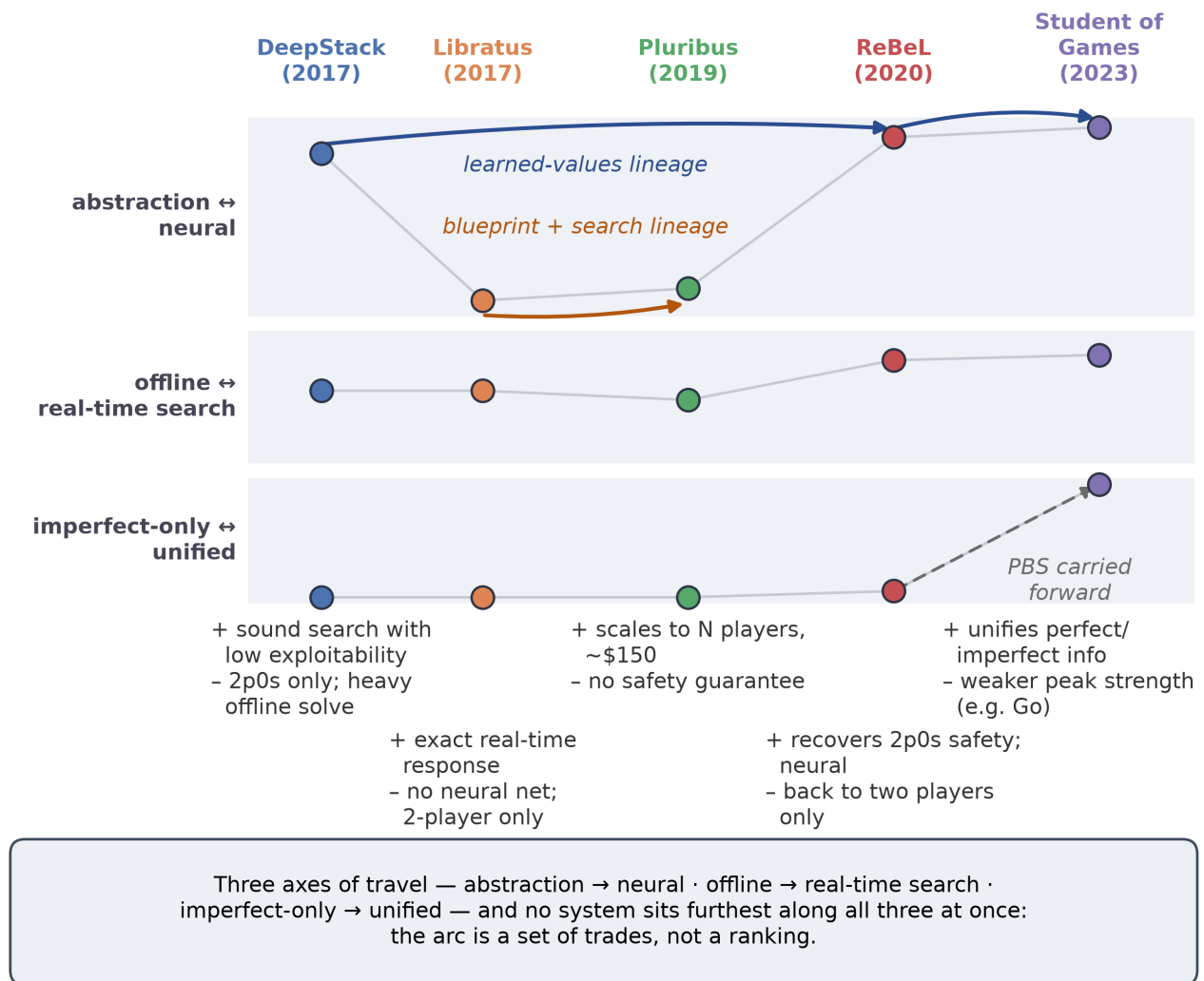


Figure 29: The seven-year arc as trade-offs: the five systems along three axes (abstraction → neural; offline → real-time search; imperfect-only → unified), with lineage arrows and gained/given-up tags.

6.7.2 What carries forward

Almost nothing in these systems is wholly new. **CFR and CFR⁺** (Chapter 3) are the inner solver of every system; **Monte-Carlo CFR** (Chapter 3) trains the blueprints of Libratus and Pluribus; **card and action abstraction** (Chapter 4) underpin Libratus and Pluribus and survive only as a vestigial betting menu in ReBeL and Student of Games; and **neural value — then value-and-policy — approximation** (Chapter 5) is what lets DeepStack, ReBeL, and Student of Games dissolve the abstraction the others depend on. Onto these the chapter layers its own primitives, each introduced by one system and reused by its successors: **depth-limited solving**, **continual re-solving**, **safe nested subgame solving**, the **public belief state**, and **Growing-Tree CFR**.

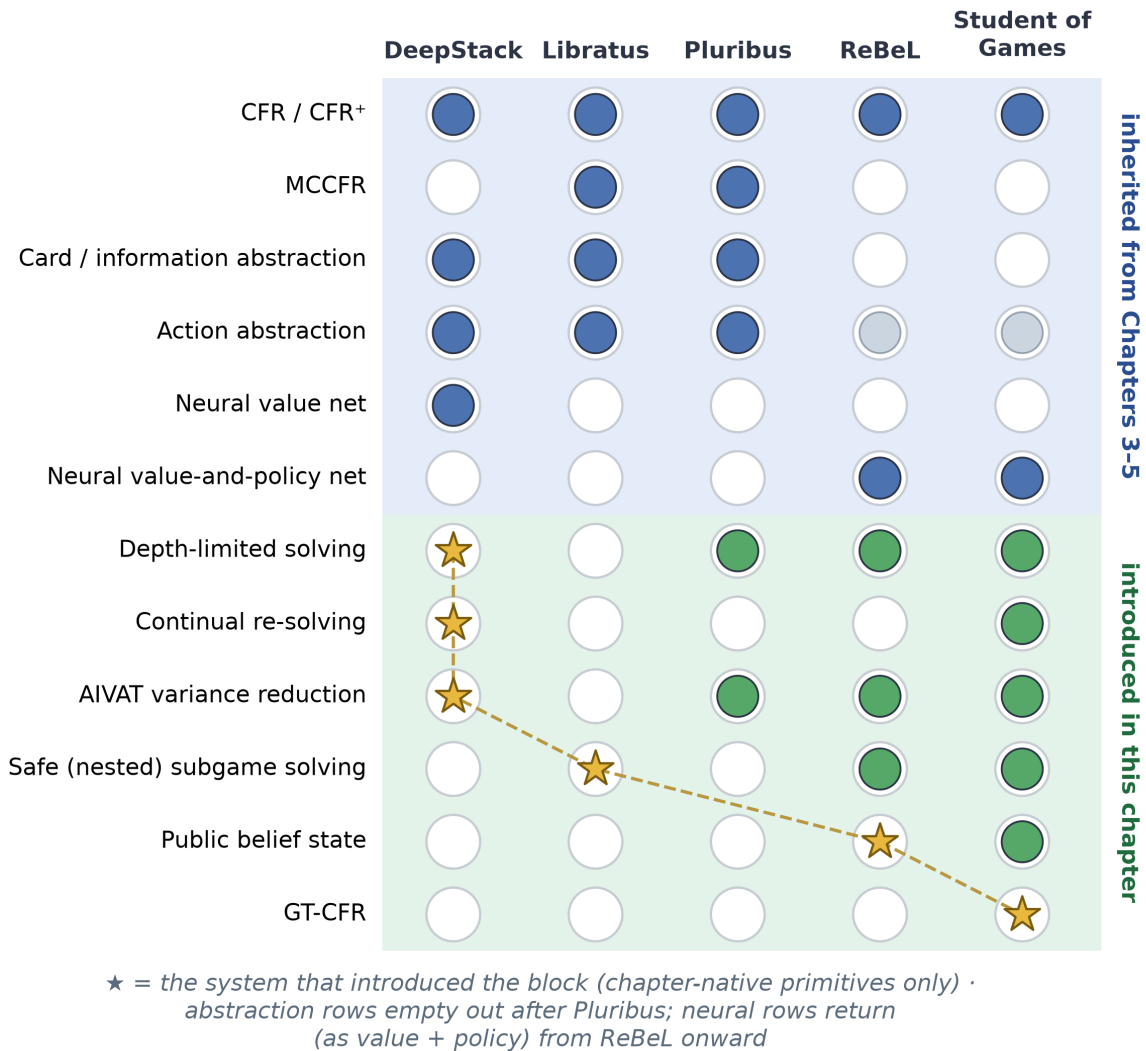


Figure 30: Component-reuse map: Chapters 3–5 building blocks and chapter-native primitives (rows) against the five systems (columns); stars mark the system that introduced each native primitive.

6.7.3 Why this matters for our research

Every system surveyed here is **opponent-blind by design**: each computes a worst-case-robust (approximately equilibrium) strategy and does not adapt it to the particular opponent. The Pluribus paper says so explicitly: Pluribus “plays a fixed strategy that does not adapt to the observed tendencies of the opponents”, does not know who its opponents are, and exploitative deviation is rejected because it “opens oneself up to exploitation”.⁸³ Five world-class systems thus share exactly one omission — none adapts to the specific, fallible opponent in front of it — and that omission is the opening this dissertation occupies.

For Contribution 1 (Behavioral Adaptation Framework). The chapter’s most direct gift is the **public belief state (PBS)** introduced by ReBeL and inherited by Student of Games, which Contribution 1 widens to carry beliefs about the opponent’s *strategy type*, not merely their cards. The PBS was invented to make *search sound*, and soundness is orthogonal to opponent modelling — so adaptation built on a PBS inherits the re-solving machinery and its convergence guarantees *unchanged*, letting an adaptive belief be updated inside a provably sound search rather than bolted on outside one. And every system shares a **two-tier architecture** — a cheap global policy (a blueprint, or a learned value network) refined by a real-time local solve — which is the dissertation’s own “start from a safe strategy and improve against weak opponents” with the second tier’s objective changed from *accuracy* to *bounded exploitation*.

For Contribution 2 (Multi-Agent Safe Exploitation). The keystone is Pluribus: superhuman six-player poker reached with *no* safety guarantee of any kind. A subtler reading turns the systems’ own caveat into the contribution’s justification. Pluribus’s paper argues that beyond two-player zero-sum a Nash equilibrium is neither unique nor efficiently computable, and that independently chosen equilibrium strategies need not form an equilibrium, so equilibrium play guarantees nothing;⁸³ Student of Games restricts its guarantees to two-player zero-sum play because beyond it the guarantee is “less meaningful”.⁸⁹ If equilibrium buys no safety in the multiplayer setting to begin with, then declining to exploit a visibly weak opponent forfeits a guarantee one never possessed — which reframes deliberate exploitation from a reckless departure from safety into the *rational objective* once safety is unattainable by equilibrium alone.

For Contribution 3 (Evaluation Methodology). The chapter standardizes on **exploitability** — the worst-case loss to a best-responding adversary — applied identically to all five systems, and that uniformity is the template for a cross-system evaluation framework. Two instruments carry over, repurposed. The **local-best-response (LBR) probe**, used throughout the chapter to *certify* unexploitability, becomes a *safety stress-test* when pointed at an adaptive agent — does chasing a weak opponent make you counter-exploitable? — which is the teaching-attack-style evaluation Chapter 8 calls for. And **AIVAT**, the learned-value variance reducer behind DeepStack’s and Pluribus’s reported results, makes a small but *growing* adaptation edge statistically visible where raw winnings would mislead.

Further leverage points. The following are compact extrapolations rather than claims made by the systems.

- **A reusable proof shape (C2).** The guarantees in the chapter share a similar form — a value-approximation term and (except for Libratus) a $1/\sqrt{T}$ convergence term (DeepStack: $k_1\epsilon + k_2/\sqrt{T}$; Libratus: 2Δ ; ReBeL: $\delta C_1 + \delta C_2/\sqrt{T}$; SoG: $|\mathcal{F}|\epsilon + |\mathcal{N}|U\sqrt{A}/\sqrt{T}$) — giving Contribution 2 a ready template for bounding the exploitability *incurred* by a deliberate deviation.

- **Depth-limited exploitation (C1/C2).** Pluribus’s multi-valued leaves (k selectable continuation strategies)⁸² are structurally a menu of opponent behaviours at the horizon; reinterpreting that menu as opponent *types* yields tractable exploitation past the search depth — the problem Milec et al. (2025), cited in Chapter 8, take up.
- **Abstraction transferred to type-space (C3).** Chapter 4’s lesson that lossy abstraction creates exploitable seams carries from state-space to opponent-*type* space: a too-coarse type model is exploitable the way a too-coarse card abstraction is, predicting a failure mode for Contribution 3 to catch.

6.7.4 Open problems and the hand-off to Chapters 7–12

Four open problems fall directly out of the synthesis, and each is the seed of a later chapter. The first is **opponent-blindness** itself: the entire frontier computes a fixed, worst-case-optimal strategy and plays it unconditionally, leaving the gap between static equilibrium and dynamic, opponent-aware play wide open — the subject of Chapter 7 (inferring opponent behaviour from action traces) and the inference half of Contribution 1. The second is **multiplayer safety**: Pluribus showed that self-play-plus-search methods *win* in N-player imperfect-information games although they have no known theoretical guarantees outside two-player zero-sum games⁸³, and Student of Games confirmed that even the most general sound-search framework still restricts its guarantees to two players — so a tractable theory of *safe exploitation beyond two-player zero-sum* (Chapter 8, Contribution 2) remains genuinely open. The third is **adaptation beyond the depth limit**: the chapter’s real-time solvers all look only a little way ahead, and carrying an opponent model soundly past that horizon — rather than discarding it at the leaf — is an unsolved problem; Chapter 8 only points to it (Milec et al., 2025). The fourth is **real-time compute budgets**: the systems’ costs differ by several orders of magnitude, and an adaptive agent that must re-solve *and* re-estimate an opponent online inherits the tightest version of that constraint.

These problems underlie Chapters 7 and 8, the thesis-critical core, and carry into multi-agent learning, populations and coalition formation (Chapters 9–11) and sequence models and LLM agents (Chapter 12); the cross-domain evaluation framework remains future work. The five systems are the platform this dissertation stands on and the foil it defines itself against: it inherits their belief-state representations, their real-time search, and their exploitability discipline, and it sets out to add the one capability they all, by design, leave out — the ability to notice that an opponent is not playing optimally, and to do something about it without becoming exploitable in turn.

7 Chapter 7 — Opponent Modeling in Imperfect-Information Games

This is a ground-up chapter on opponent modeling: the problem, the mathematics, the family of methods, and a set of controlled experiments on Kuhn Poker and Leduc Hold'em, bounded wherever possible by *exact* analytical references rather than simulated ones.

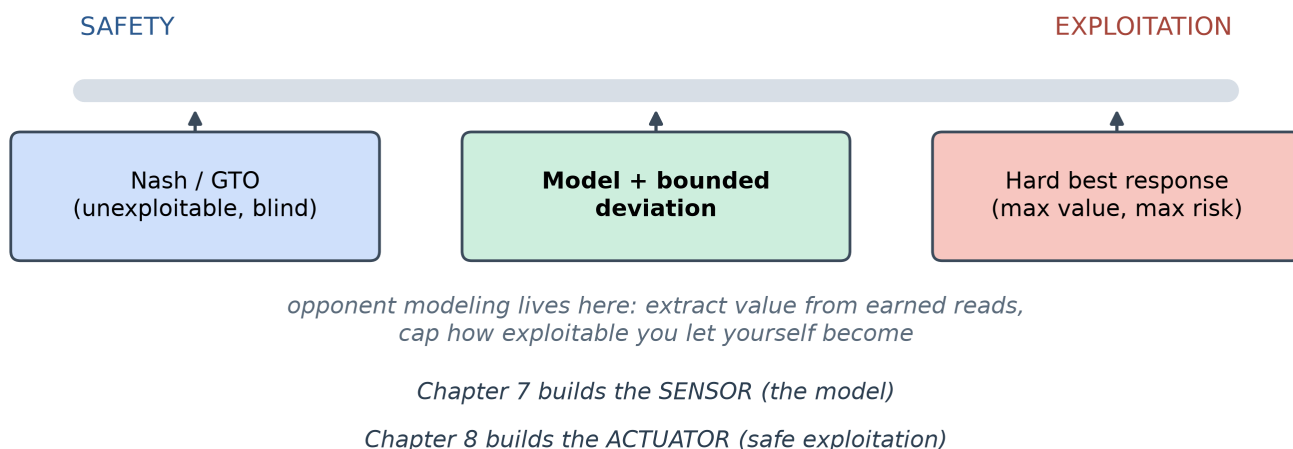
Where this sits in the thesis. A game-theoretic agent from the earlier chapters computes a *Nash equilibrium*. This chapter builds the opposite capability: a **sensor** that watches how a *specific* opponent actually plays and turns those observations into an estimate of their strategy, so the agent can deviate from equilibrium to punish that opponent's mistakes. That sensor is the first half of the thesis's Behavioral Adaptation Framework (Contribution #1); the second half, *safe* exploitation, is Chapter 8.

7.1 Why Opponent Modeling

A **Nash equilibrium** strategy is built never to lose in the long run, *no matter who it plays*. For the same reason it plays **identically** against a world champion and against someone who folds every single time you bet.

Opponent modeling is the act of watching a specific opponent's actions, forming a belief about their strategy, and **deviating from Nash to exploit the pattern** — bluffing more against someone who folds too much, value-betting thinner against someone who calls too much.

The cleanest way to see the value is a *rock-paper-scissors* picture. Your opponent secretly throws rock 70% of the time. The safe strategy is to randomize evenly and break even forever. But if you *notice* the bias, you throw paper more and start winning. Two things make this the whole chapter in miniature: you must **infer** the bias from a noisy stream of throws, and if you **over-commit** to paper you become predictable and they crush you with scissors. Inferring the bias is opponent modeling; knowing how far to lean is the exploitation-versus-safety tradeoff that runs through everything below.



7.1.1 How much is at stake — measured on Kuhn Poker

Table 14: Exploitation headroom in Kuhn: Nash EV, best-response EV, and the gap between them, per opponent type.

Against the three exploitable styles the gap is enormous — **about 0.21–0.23 per hand** (0.11–0.28 across both seats), on a game whose entire equilibrium value is about one twentieth of a chip. Against a Nash opponent the gap is essentially **zero**, exactly as theory demands: you cannot exploit an equilibrium, because best response cannot beat the game value. (The small negative Nash EVs are Kuhn’s known first-player disadvantage of $-1/18 \approx -0.056$; the equilibrium is a 20,000-iteration CFR approximation, so it can dip a few thousandths below.)

into an open pot, because the rock folds everything but the King).⁹⁷

7.2 The Bayesian Core — belief, evidence, response

Underneath every method in this chapter is one loop. You are a detective. You start with a hunch about how the opponent plays (a **prior**). Each action they take is a clue: it makes some explanations more likely and others less. You fold the clue into your suspicion (the **posterior**), and repeat. Opponent modeling *is* this loop:

belief about their strategy → see an action → update belief → best-respond → repeat.

Formally, the update is Bayes' rule. If σ denotes a candidate opponent strategy and a the action just observed,

$$P(\sigma \mid a) \propto P(\sigma) \cdot P(a \mid \sigma),$$

in words: **posterior** $\propto$ **prior** $\times$ **how well that strategy explains what they just did**. Over many hands the belief concentrates on the best-fitting explanation.

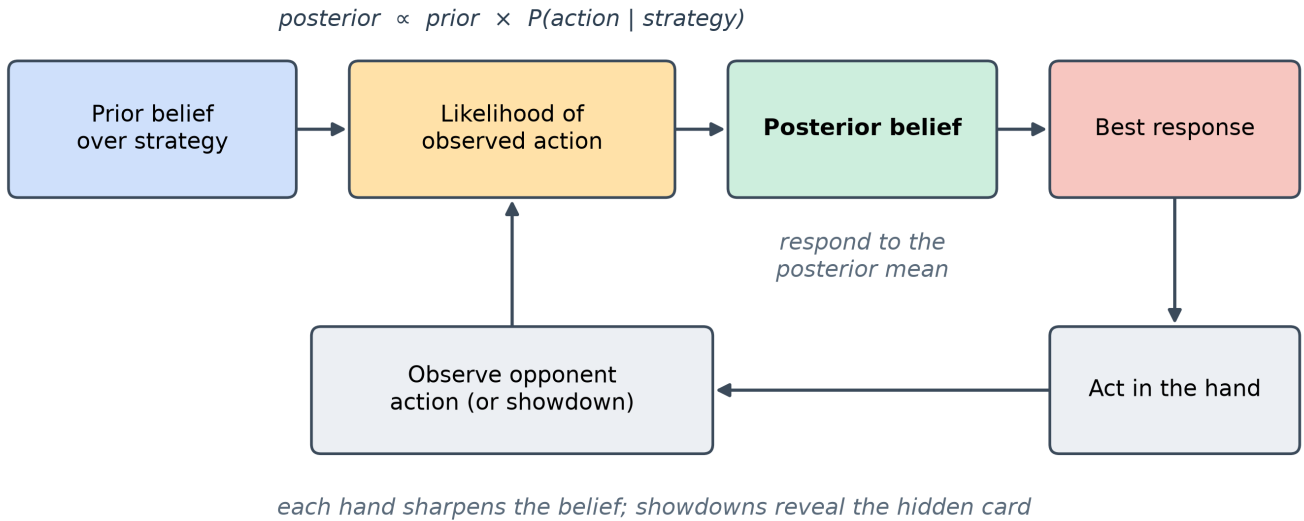


Figure 32: The opponent-modeling loop: the prior times the likelihood of each observed action gives the posterior, which drives a best response; the outcome feeds the next observation.

⁹⁷Southey, F. et al. (2005). “Bayes’ Bluff: Opponent Modelling in Poker.” *UAI*.

7.2.1 The type “zoo”

The simplest way to make this concrete is to reason over a small menu of predefined opponent **types**, a recurring cast that serves double duty — as opponents to play against, and as hypotheses the detector reasons over:

Table 15: The opponent type zoo: the behaviour of each fixed style.

Type	Behavior
Calling station	Never bets, never folds — checks when it can, calls any bet.
Rock (tight-passive)	Commits chips only with the best hand; folds everything else. The most exploitable type.
Maniac (loose-aggressive)	Always bets or calls, regardless of hand.
Nash	Balanced equilibrium play; mixes its actions; unexploitable.

7.2.2 The update, worked by hand

The mechanics are easiest to trust when you turn the crank once yourself. Suppose three candidate types bet a particular hand with probability 0.8, 0.5, and 0.1 respectively, and you start from a uniform prior $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$. You observe a **bet**:

$$\text{posterior} \propto \left(\frac{1}{3} \cdot 0.8, \frac{1}{3} \cdot 0.5, \frac{1}{3} \cdot 0.1\right) = (0.267, 0.167, 0.033) \xrightarrow{\text{normalize}} (0.571, 0.357, 0.071).$$

Observe a **second** bet and multiply by $(0.8, 0.5, 0.1)$ again:

$$(0.711, 0.278, 0.011).$$

The belief piles onto the bet-happy type, and it would do so *faster* the more the candidates disagree. That “how much they disagree” intuition returns in Section 6 as the reason some opponents are identified in four hands and others take two hundred.

7.2.3 Why Dirichlet, and why you only ever need the mean

When you estimate an action distribution by counting, the natural prior is the **Dirichlet** distribution, because it is the *conjugate* prior of the multinomial: the observed counts are simply **added** to the prior’s pseudo-counts, so the posterior mean of action i is

$$\mathbb{E}[p_i] = \frac{\alpha_i + n_i}{\sum_j (\alpha_j + n_j)},$$

where α_i are prior pseudo-counts (which double as *smoothing*, keeping no action at exactly zero) and n_i are observed counts. This closed form *is* the “continuous” model of Section 4.

To choose an optimal response you never need the whole posterior distribution over strategies; you only need its **mean**: your expected payoff against a *distribution* of opponent strategies equals your payoff against the single averaged strategy. (Section 5 revisits this: responding to the mean is payoff-optimal but can fail to *converge* to the opponent’s true strategy.)⁹⁸

7.3 Seeing Through a Keyhole — the partial-observability problem

In poker you observe **actions** — bet, call, fold — but **not** the private hand that produced them. The same “bet” can come from a monster or a bluff. The one exception is a **showdown**, where cards are revealed at the end of a hand that goes to the end without a fold. A showdown gives **cause and effect**: you see the actions *and* the private hand behind them, the gold-standard observation. A fold gives **effect only**: you saw *what* they did but not *what they held*.

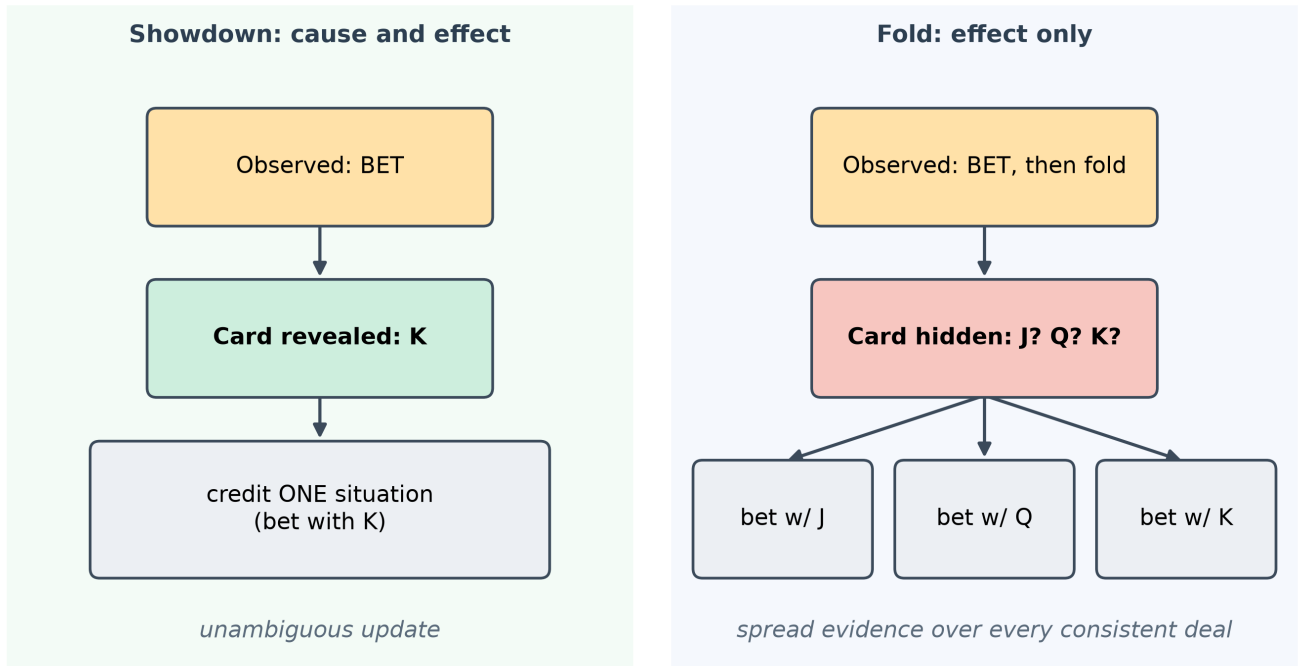


Figure 33: Partial observability: a showdown pins the action to the opponent’s revealed card; a fold hides it, spreading the evidence across every hand the opponent might have held.

The consequence is a genuine theoretical limit, not an implementation nuisance: **without ever observing the opponent’s private information, you can learn how often they take each action but not**

⁹⁸Ganzfried, S. & Sun, Q. (2018). “Bayesian Opponent Exploitation in Imperfect-Information Games.” *IEEE Conference on Computational Intelligence and Games (CIG)*, DOI 10.1109/CIG.2018.8490452; preprint arXiv:1603.03491 (2016). (Theorem 2.1: respond to the posterior mean.)

which hands produce it: many strategies yield the same action frequencies, and more hands cannot tell them apart.

How does a model cope with a fold? By reasoning over **all the hands the opponent might have held**: it replays the opponent’s decisions under each hypothetical private card and spreads the evidence across them, weighting each hypothesis by how plausible it is. This “marginalize over the hidden card” move is the technical heart of every model in the next two sections, and the formal object the consistency theory of Section 5 is built around.

7.4 Three Models, One Interface

There is no single “opponent model.” There is a family, trading off convergence speed, robustness to surprises, interpretability, and compute. This project implements three points on that spectrum behind **one shared interface** — each consumes a stream of observed hands and emits a predicted opponent strategy — so that the *same* downstream best response can be applied to all three.

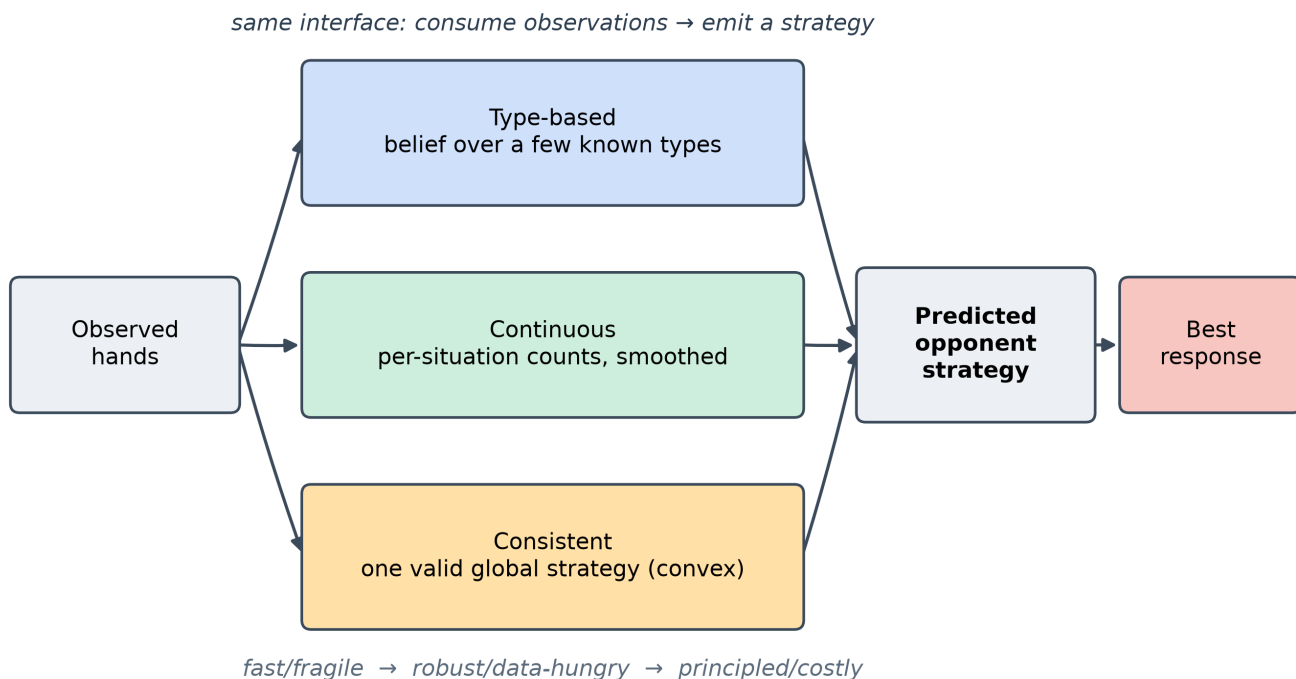


Figure 34: Three models on one interface: a belief over types, a per-situation count and a globally-consistent sequence-form estimate, all feeding the same best-response step.

- **Type-based.** A Bayesian belief over a fixed menu of types (Section 2); fastest and interpretable (“80% rock, 20% maniac”) **when the opponent is in the menu**, with no honest way to say so when it resembles none of them (Section 6).
- **Continuous.** Dirichlet-smoothed counts *at each situation* (Section 2); it can represent **any** strategy, but learns each situation independently and so needs many more observations — a hunger that, as

Section 7 shows, has teeth on the larger game.

- **Consistent.** A single *globally consistent* strategy, valid over the whole game tree; the most principled and most recent model (Section 5).

Table 16: The three opponent models compared on representation, convergence, robustness, interpretability and cost.

Model	Represents	Converges	Robust to out-of-menu?	Interpretable	Compute
Type-based	a few known types	fastest	no	high (named types)	cheap
Continuous	any per-situation strategy	slower (data-hungry)	yes	medium (per-situation)	cheap
Consistent	one valid global strategy	provably to the truth	yes	medium	expensive (optimization)

No single model wins, and that is itself a finding. The natural target is a **hybrid**: lean on structural priors (types) when data is sparse, and converge toward the consistent estimate as evidence accumulates.⁹⁹

7.5 The Consistency Problem and the Sequence-Form Fix

This section is the theoretical frontier of the chapter and the piece the thesis most directly extends. It answers a question the earlier models quietly beg: *if a model fits the observed behavior well, have we actually recovered the opponent’s true strategy?* The surprising answer is **not necessarily — even with infinite data** — and there is a principled fix.¹⁰⁰

7.5.1 The flaw: fitting the mean is not the same as finding the truth

The classical Bayesian recipe responds to the posterior **mean** strategy (Section 2). Call an opponent-modeling method **consistent** if its estimate converges to the opponent’s true strategy σ^* as observations accumulate against a fixed opponent. The classical recipe is not.

The cleanest counterexample is rock-paper-scissors. Suppose the modeler reasons over a set of *sampled* candidate strategies and always responds to some average of them. If the true strategy $\sigma^* = (0.8, 0.1, 0.1)$ lies **outside** the region those samples can combine to, the model — being always a weighted average of the samples — simply **cannot reach it**, no matter how much data arrives. The deeper result is that the method can fail **even when the truth lies inside** the achievable region: with the true strategy sitting

⁹⁹Bard, N., Johanson, M., Burch, N. & Bowling, M. (2013). “Online Implicit Agent Modelling.” *AAMAS*, 255–262 — the explicit-vs-implicit axis that frames this taxonomy.

¹⁰⁰Ganzfried, S. (2025). “Consistent Opponent Modeling in Imperfect-Information Games.” *arXiv:2508.17671*.

at the center of three samples that average to it, the posterior weight on the single best-fitting sample grows without bound relative to the others, so asymptotically the belief **collapses onto one sample** rather than settling on the true mixture.¹⁰⁰ Fitting the data ever more tightly, the model converges to the *wrong* strategy.

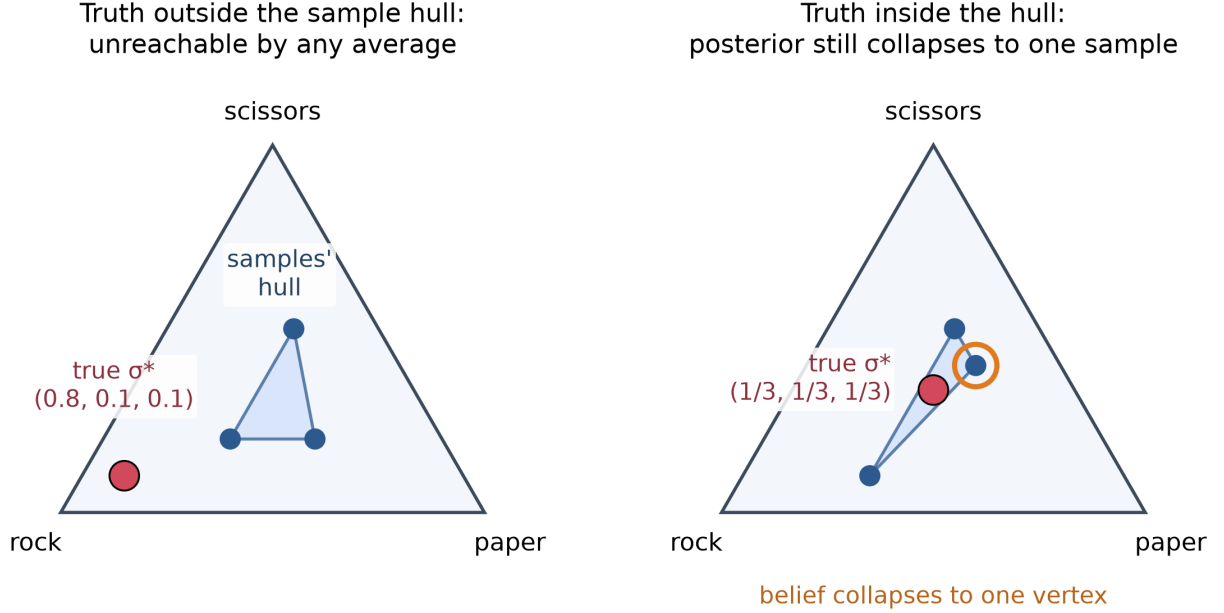


Figure 35: Fitting the mean can miss the truth: the modeler only produces points in its samples' hull. A truth outside is unreachable; one inside is abandoned as the posterior collapses onto a vertex.

This undercuts the intuitive safety net “just collect more data.” Consistency is not automatic; it has to be engineered.

7.5.2 The fix: a convex program in sequence form

The key change of variables is the **sequence form**: instead of per-situation action probabilities, represent the opponent's strategy by **realization weights** y_r over action *sequences*. The legal strategies are then exactly those satisfying a set of **linear** constraints $Fy = f$, $y \geq 0$ — polynomial in the size of the game tree, rather than exponential.

Partial observability (Section 3) enters through an **observability function**: for each observed hand, the set of trajectories consistent with what you actually saw. Putting a Dirichlet prior on the weights and maximizing the log-posterior gives the program

$$\max_y \sum_r (\alpha_r - 1) \log y_r + \sum_t \log \left(\sum_{r \in o(\ell_t)} q_r y_r \right) \quad \text{s.t.} \quad Fy = f, y \geq 0,$$

where $o(\ell_t)$ is the consistent-trajectory set for observation t and q_r are (normalized) chance probabilities.

When $\alpha_r \geq 1$ this objective is **concave** and the constraints are affine, so it is a **convex** problem with **no local optima**; a standard **projected gradient** scheme (a gradient step, then a projection back onto the constraint set) finds the global one.

This method returns the **mode** of the posterior (its single most probable point), not the mean. That is the crucial trade: the mode is *consistent* — under standard conditions (the true strategy lies in the interior of the set of valid strategies and has positive prior density, distinct strategies produce distinguishable observations, and every opponent situation is visited infinitely often) the estimate provably converges to σ^{*100} — whereas the payoff-optimal mean is generally intractable to compute exactly. So the design axis underneath the whole chapter is **mean versus mode**: payoff-optimal-but-intractable versus tractable-and-consistent.

7.5.3 What we found, honestly

We implemented this consistent estimator and verified it on Kuhn strategy recovery, where it performs as advertised: its recovered strategy sits very close to the truth (total-variation distance roughly **0.004 to 0.021**), matching or beating the continuous model’s recovery on the same game.

We did **not** run it inside the online exploitation loop, nor on the larger game. The estimate is the solution of an optimization that is **re-solved as observations accumulate**, and that cost grows with history — from a fraction of a second early on to many seconds per refit once tens of thousands of hands are in hand. We treat this as the *empirical answer* to a question the chapter poses explicitly — *is a per-update convex solve fast enough for real-time play?* — namely **not without incremental methods** (warm-starting each solve from the last, caching the per-hand terms). The understanding and the recovery result are what the thesis needs from this model now; the online engineering is future work.¹⁰⁰

7.6 When a Model Is Confident and Wrong

Section 5 was about a subtle failure with infinite data. This section is about a blunt failure with finite data, and it is the one that most directly motivates *safe* exploitation: a **misspecified menu** — an opponent the type-based model simply cannot represent.

We built a hidden opponent that is a 50/50 per-action blend of the rock and the maniac — deliberately **none** of the four candidate types. A reasonable guess is that the posterior would split its belief between the two nearest types. It does not. A product-of-likelihoods posterior concentrates on the *single best explanation*, so the belief lurches from one type to another and finally commits, hard, to **Nash** — the one candidate that assigns real probability to *both* betting and checking a middling hand, and so is never fatally contradicted. The model ends up **confident** (posterior near 1.0 on a single type) and **wrong** (that type is not what it is playing).

This exposes a trap worth stating plainly: the posterior is a **relative** quantity. “Ninety-five percent Nash” means “best fit *among these four candidates*,” not “good fit in absolute terms.” Measuring the winner’s

absolute fit gives the game away — the same Nash hypothesis that scores well against a genuine Nash opponent scores far worse against this blend, while reporting equal confidence on paper.

7.6.1 How long can it stay wrong? A robustness sweep

We stress-tested the phenomenon over **300 random seeds of 500 hands each**, separating two things a naive metric conflates: *slow convergence* from *falling after convergence*.

Table 17: Detector reliability over 300 seeds of 500 hands each.

Measurement (300 seeds x 500 hands)	Result
Correct long-run winner by hand 500	300 / 300 (100%)
Ever falls back after taking the lead for good	0 / 300 (never)
A wrong type still leading past hand 100	40 / 300 (~13%)
A wrong type still leading past hand 200	14 / 300 (~5%)
Hand at which the truth locks in for good	median 23 - 90th pct 125 - worst 461

The good news: the belief eventually lands on the closest representable strategy every time, and once it locks in it never falls. The sobering news is the middle rows: in a meaningful minority of runs the model held a **confident wrong belief for well over a hundred hands**. A model can be trusted *eventually*, but “eventually” is sometimes 200 hands away.

The honest fix is not a bigger menu — it is to lean on models that can represent blends (the continuous and consistent models of Sections 4-5), and to **scale how hard you exploit to how well-earned the read is**. That principle is the through-line into Chapter 8.

7.7 From Model to Money — best response, the ceiling, and a self-inflicted leak

To measure whether acting on a model wins, we feed each model’s estimate into the **same** exact best response and play full matches on both games, bracketing every result between two analytical yardsticks: the **Nash EV** (what safe equilibrium play earns against that opponent) and the **ceiling** (the exact best-response value, the most that is extractable if you knew the opponent perfectly).

Kuhn — both models reach the ceiling (rock: ceiling 0.167, type-based 0.168, continuous 0.168; maniac: 0.333, 0.337, 0.330; Nash: −0.055, −0.055, −0.053).

Leduc — the model class starts to matter. Leduc Hold’em (Section 3.4) is still exactly solvable, but large enough to separate the models.

Table 18: Leduc: the exploitation ceiling against what each model actually realised.

Opponent	ceiling	type-based	continuous
Level-1	3.056	3.061	2.672
Maniac	2.177	2.199	2.038
Calling station	1.464	1.451	1.434
Rock	0.937	0.912	0.848
Nash	-0.083	-0.085	-0.175

(“Level-1” is an opponent that best-responds to a uniformly random player; on Leduc the type-based model’s menu holds nine types, all shown in the next figure.)

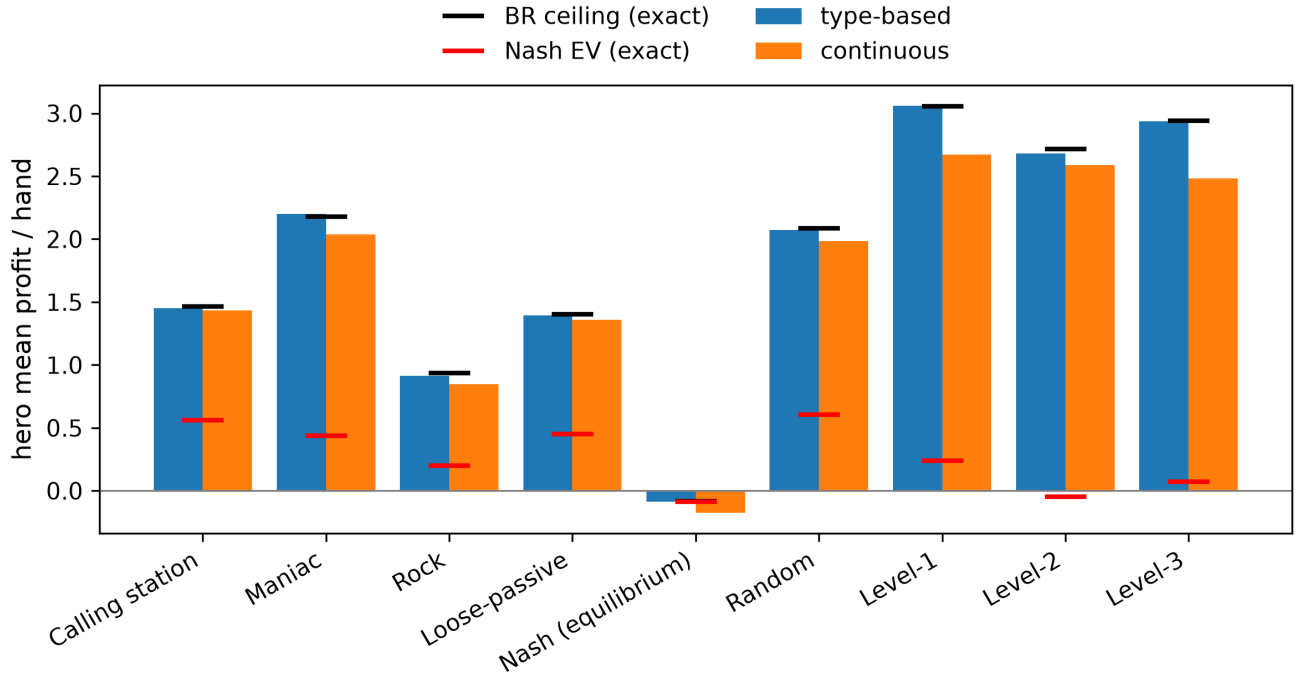


Figure 36: Leduc exploitation: the type-based model hugs the exact best-response ceiling; the continuous model sits below it for the hardest-to-fit types and below the safe baseline against Nash.

Two results carry the message, and the second is the important one:

1. **You cannot exploit an equilibrium.** Against the Nash opponent every model earns approximately the (negative) game value and never more. This confirms the exploitation elsewhere is real and not an artifact of the harness.
2. **A confident-but-underfit model makes *you* exploitable.** On Leduc the continuous model *loses* to Nash — -0.175 against a -0.083 ceiling. With imperfect data over Leduc’s many situations it

best-responds to a *wrong* estimate of an opponent who cannot be exploited at all, and in doing so opens a leak in its **own** play.

When the model class fits, best response extracts the full theoretical value; when it does not, the agent both leaves money on the table *and* — more dangerously — hands some back. That second cost is precisely what Chapter 8’s safety mechanism (bounding the deviation from Nash by the model’s own confidence) exists to prevent.

The differences here are not seed luck. Across five seeds the standard error of per-hand profit is tiny relative to the effects: the type-based model stays within 3% of the ceiling on every type in both games (and within two standard errors on nearly all), and the continuous model’s Leduc shortfall and its Nash self-leak are many standard errors wide.¹⁰¹

7.8 Adapting to Change — non-stationary opponents

Everything so far assumed a fixed opponent. Real opponents drift and adapt, and a model that learned patiently for ten thousand hands is worse than useless the moment its subject changes style — it is now *confidently* describing a person who no longer exists.

The adaptive agent runs the loop of Section 2: **observe** hands, **update** the model, periodically rebuild the hero strategy as a best response to the current estimate, and **act**. To handle change, it adds a lightweight monitor on the opponent’s aggression (Bayesian online change-point detection¹⁰²) that, on firing, **resets** the model and drops back to safe play while it re-learns.

¹⁰¹Ganzfried, S. & Sandholm, T. (2015). “Safe Opponent Exploitation.” *ACM Transactions on Economics and Computation* 3(2), DOI 10.1145/2716322 — the paper that characterizes when “exploit but never lose to the baseline” is possible; the safety half of the dial and the anchor for Chapter 8.

¹⁰²Adams, R. P. & MacKay, D. J. C. (2007). “Bayesian Online Changepoint Detection.” *arXiv:0710.3742*.

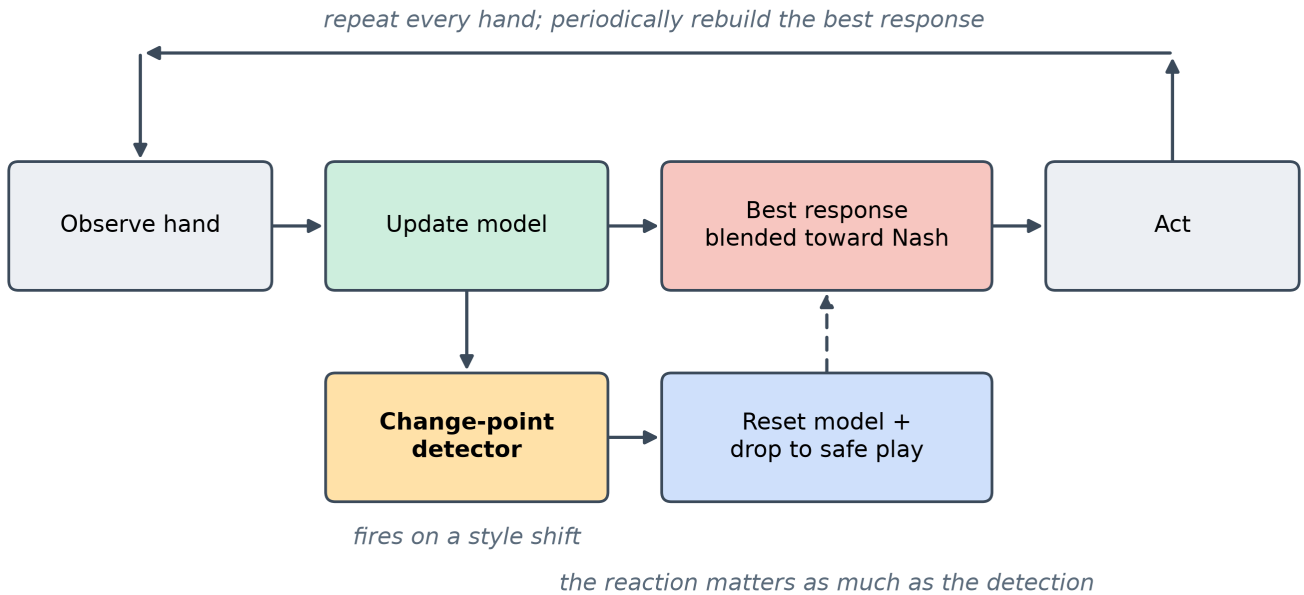


Figure 37: The adaptive exploitation loop: observe, update the model, best-respond (blended toward Nash), act; a change-point detector can reset the model when the opponent’s style shifts.

We switched the opponent’s style at the midpoint of a 20,000-hand match and compared a **static** model (never forgets) against one with **change-point forgetting**. The result is **scenario-dependent**:

Table 19: Static against change-point forgetting, after a mid-match style switch.

Game	Style switch (at the midpoint)	static (after switch)	change-point (after switch)
Kuhn	rock -> maniac	-0.106 ± 0.007	+0.211 ± 0.008
Leduc	rock -> maniac	+1.834 ± 0.054	+0.552 ± 0.017

- **On Kuhn, forgetting wins.** The strategy learned against a rock is bluff-heavy; unleashed on a maniac who calls everything, it *actively loses*. Detecting the switch and re-learning recovers to a healthy profit.
- **On Leduc, forgetting loses.** The maniac there leaks over two chips a hand, so a continuously-adapting model exploits it handsomely without any reset — while the detector, too eager, fires dozens of **false alarms** during the stable stretches (around sixty resets against a single true change), each one throwing away hard-won data and dropping to safe play.

The lesson is that **the reaction to a detected change matters as much as the detection**. A trigger-happy detector paired with a full reset-to-safe can cost more than staleness when the new opponent is exploitable enough that staleness is cheap. Gentler responses — partial forgetting instead of a hard reset, a less nervous detector — are the clear next step, and they connect directly to Chapter 8’s confidence-

scaled exploitation. (Values are means over five seeds $\pm$ standard error; every seed agrees on the direction of the effect.)

7.9 Connections and Forward Pointers

What this chapter establishes. A good opponent model is *necessary but not sufficient* for profitable, safe adaptation. When the model class fits the opponent, best response reaches the exact extractable ceiling. But under partial observability and limited data, an underfit model both leaves money on the table and, against an opponent who cannot be exploited, best-responds to a phantom and *loses*. Forgetting a stale model helps only when staleness is actively harmful. One principle recurs across all of it: **exploitation must be scaled to how well-earned the read is.**

Forward to Chapter 8 and the thesis. This chapter built the **sensor**; Chapter 8 builds the **actuator** — the mechanism that turns a model into *safe* exploitation, bounding how far you deviate from equilibrium by how much your read has earned. The continuous model’s self-inflicted leak against Nash (Section 7) is the empirical case for that mechanism; the confident-but-wrong sweep (Section 6) sets its budget; the consistency theory (Section 5) is the principled backbone the framework extends.¹⁰³

¹⁰³Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Ch. 3 (normal-form games); Ch. 4 (computing solution concepts of normal-form games, §4.1 — linear programming for zero-sum games); Ch. 5 (extensive-form games; §5.2 — imperfect-information games and the sequence form, the machinery underneath every LP in Chapter 8); Ch. 7 “Learning and Teaching”, the learning-in-repeated-games framing, including the tension that your actions both *exploit* and *teach* the opponent. Free: <http://www.masfoundations.org/download.html>

8 Chapter 8 — Safe Exploitation in Imperfect-Information Games

This chapter covers *safe* opponent exploitation: the problem, the mathematics, the family of methods, and controlled experiments on Kuhn Poker and Leduc Hold'em. All numbers were **measured** on reproducible runs and bounded, wherever possible, by *exact* analytical references. Where a run contradicted what theory led me to expect, I say so and reconcile it.

Where this sits in the thesis. Chapter 7 built a **sensor**: a model that estimates a specific opponent's strategy. Chapter 8 builds the **actuator**: the mechanism that turns a model into profit **without becoming exploitable**. It is the second half of the Behavioral Adaptation Framework (Contribution #1), and pinning down *exactly where its guarantees depend on the two-player zero-sum assumption* is the launch point for the multi-agent extension (Contribution #2).

8.1 Why Safe Exploitation — the other half of the dial

A **Nash equilibrium** strategy is built never to lose in the long run: in a two-player zero-sum game it has a fixed *value* v^* , and no opponent can push you below it. Opponent modeling (Chapter 7) lets you *deviate* from that safe strategy to punish a specific opponent's mistakes. The obvious way to cash a model in is to play the **best response** to it. The trouble is that a best response is usually **wildly exploitable itself**: to punish “you always fold to a bet” it stops bluffing certain hands entirely, and a smarter opponent — or the same opponent, done pretending — walks straight through the hole it opened.

Picture a **dial**. All the way to *safety* is Nash: unbeatable, but it never punishes a weak opponent. All the way to *exploitation* is a hard best response to your current read: maximum profit **if** the read is right, but a gaping hole if you are wrong, your sample was small, or you were being *sandbagged* (fed a weak style to bait a big deviation). **Safe exploitation** bolts a *governor* onto that dial: lean toward exploitation as far as you like, but never past the point where a worst-case adversary could drag you below your safe baseline.

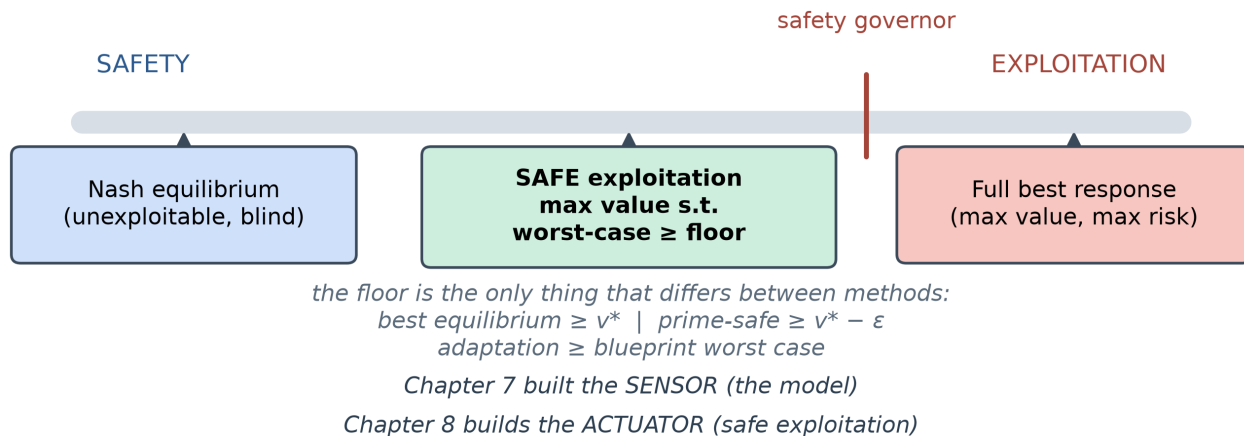


Figure 38: The safety–exploitation dial with a governor: Nash is unexploitable but blind, the full best response is maximally risky; safe exploitation operates in between, and methods differ only in where the floor sits.

This is not paranoia: the full best response to the tight “Rock” style on Kuhn earns **+0.167 per hand**, but its **worst-case value is -0.5** — an adversary who best-responds back can take half a chip a hand off it, while Nash’s worst case is the game value itself. Safe exploitation takes as much of that upside as the opponent’s mistakes allow while refusing the -0.5 downside.

What “safe” means, informally. Your *expected value over the match* must never fall below a chosen floor, **no matter what the opponent does**. The design question of the chapter is *which floor*, and how to extract as much as possible above it.¹⁰⁴

8.2 Exploitation as Constrained Optimization

The single most useful idea in this chapter is that **every safe-exploitation method is the same optimization problem** with one part swapped out:

maximize the hero’s expected value against the opponent model, **subject to** a safety floor on the hero’s worst-case value.

To make that computable, represent the hero’s strategy in the **sequence form** of Section 7.5: the **realization plan** x assigns a weight to each of the hero’s action sequences, subject to the linear **treeplex** constraints. Against a **fixed** opponent policy, the hero’s expected value is then **linear** in x ,

¹⁰⁴Ganzfried, S. & Sandholm, T. (2015). “Safe Opponent Exploitation.” *ACM Transactions on Economics and Computation* 3(2), Article 8. DOI 10.1145/2716322 — the paper that shows that, in a repeated two-player zero-sum game, deviating from equilibrium can be safe exactly when the opponent makes “gifts”, and gives algorithms that risk only what has already been won; the safety half of the dial and the anchor for Chapter 8.

$$\text{EV}(x) = \sum_{z \in Z} [\pi_c(z) \cdot \pi_{-h}(z) \cdot u_h(z)] x_{\text{seq}(z)} = c \cdot x,$$

where Z is the set of terminal states, $\pi_c(z)$ the probability of the chance moves, $\pi_{-h}(z)$ the probability of the opponent's moves and $u_h(z)$ the hero's payoff. So “maximize EV against the model” is a **linear objective** $c \cdot x$ over linear constraints, and the whole method is a **linear program**. A full best response is simply $\max_x c \cdot x$ over the treeplex, which cross-checks the LP against Chapter 7's exact best-response code (they agree to 10^{-6}).

The safety floor — “for *every* opponent σ' , $\text{EV}(x, \sigma') \geq \text{floor}$ ” — hides an inner best response (a minimax), so instead of one giant bilinear program we use **constraint generation** (a double-oracle / cutting-plane loop): solve the LP, call an **exact best response as the worst-case oracle**, and if the worst case is below the floor, add the single linear cut it implies and re-solve. This is finite (there are finitely many pure best responses), transparent, and debuggable.

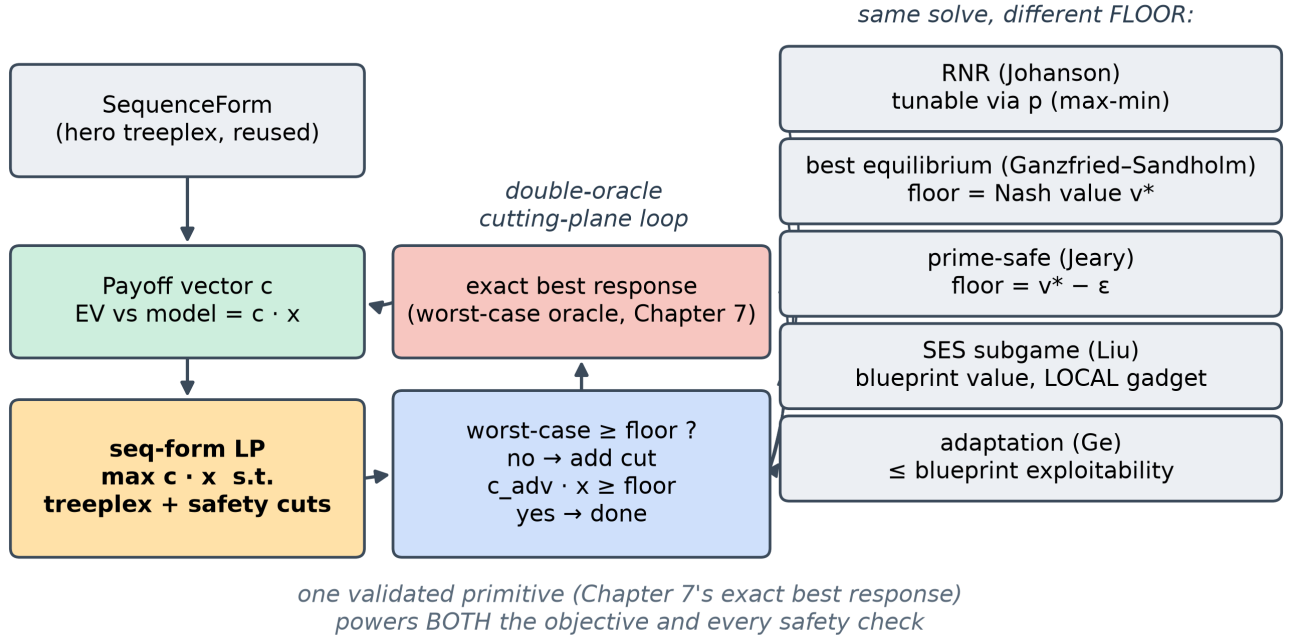


Figure 39: One LP engine, five floors: an exact best response serves as the worst-case oracle, adding cuts until the floor holds; RNR, best equilibrium, prime-safe, SES and adaptation differ only in the floor.

The pay-off is **five methods, one engine**, differing only in the floor, all resting on Chapter 7's *validated* best-response code.¹⁰⁵

¹⁰⁵Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Ch. 3 (normal-form games); Ch. 4 (computing solution concepts of normal-form games; §4.1, linear programming for zero-sum games); Ch. 5 (extensive-form games; §5.2, imperfect-information games and the sequence form, the machinery underneath every LP in Chapter 8); Ch. 7 “Learning and Teaching”, the learning-in-repeated-games framing, including the tension that your actions both *exploit* and *teach* the opponent. Free: <http://www.masfoundations.org/download.html>

8.3 What “Safe” Means — three definitions, in increasing realism

“Safety” sounds absolute, but it is not one thing. Three definitions matter, and the line of research is essentially a *weakening* of the demand to make it achievable in practice.

(a) Ganzfried–Sandholm safety (2015): never earn less than the game value. Their definition is over the repeated game: at least v^* per hand in expectation, whatever the opponent does. Here it is imposed on each hand’s strategy, with an exact Nash equilibrium σ^* as baseline:

$$\text{EV}(x, \sigma') \geq v^* \quad \text{for all opponents } \sigma'.$$

This is the strongest and cleanest notion, and it rests on the **minimax theorem**. Only equilibrium strategies meet this per-hand floor, so the method picks the equilibrium strategy that does best against the model (Ganzfried and Sandholm’s “best equilibrium” baseline; below it is called **best equilibrium**, code id **ganzfried**). The catch is the premise: an exact Nash equilibrium, which is not computable in practice in any large game.

What the full algorithms would add. Their RWYWE (“risk what you’ve won in expectation”) lowers the floor each hand to $v^* - k_t$, where k_t is the *gift* (the opponent’s mistakes) banked in expectation so far, and BEFFE plays best equilibrium until the banked gifts cover full exploitation; both are provably safe over the repeated game; implementing and testing them is future work (RWYWE is the natural two-player baseline for Contribution #2).

(b) Prime-safe / ε -safety (Jeary & Turrini 2023): correct for an imperfect baseline. Every real baseline is an **ε -equilibrium** (from abstraction and finite compute, Chapter 4), so it is itself exploitable by some ε . Prime-safe lowers the floor by exactly the baseline’s own exploitability,

$$\text{floor} = v^* - \varepsilon, \quad \varepsilon = \text{exploitability}(\text{baseline}) \geq 0,$$

i.e. “never earn less than the worst case of the strategy you were going to play anyway.” The ε must be **measured**, not assumed — a point the experiments take literally.

(c) Adaptation safety (Ge et al. 2024): be no more exploitable than your blueprint. The most practical notion. An exploiting strategy is *adaptation-safe* iff

$$\text{exploitability}(x) \leq \text{exploitability}(\text{blueprint}) \iff \text{worst-case}(x) \geq \text{worst-case}(\text{blueprint}).$$

This is weaker than Ganzfried and Sandholm’s absolute floor (by exactly ε), and therefore *achievable* where strict safety is not. Its danger is the mirror image: if the blueprint is *terrible*, almost anything

satisfies it, so adaptation safety is only meaningful with a reasonable baseline (an open requirement I flag rather than resolve).

Table 20: Three safety notions: the floor each guarantees, what it requires, and where it is weak.

Safety notion	Informal floor	Needs	Weakness
Ganzfried–Sandholm (2015)	$\geq v^*$	an <i>exact</i> Nash baseline	an exact Nash equilibrium is not computable in practice in large games
Prime-safe (2023)	$\geq v^* - \varepsilon$	the baseline’s measured exploitability	you must measure ε honestly
Adaptation (2024)	$\leq$ blueprint exploitability	any blueprint	vacuous if the blueprint is bad

8.3.1 Where the two-player zero-sum assumption hides — the thesis attack point

Every floor above rests on one fact: **in a two-player zero-sum game, a Nash strategy secures the value v^* against any opponent** ($\min_{\sigma'} \text{EV}(\sigma^*, \sigma') = v^*$). That is what makes “deviate toward the model but never below the floor” a coherent, enforceable constraint, and it comes straight from the minimax theorem. In an $N > 2$ -**player** game this collapses: a Nash strategy does *not* guarantee a fixed value against arbitrary opponents (no single opponent’s payoff is the negative of yours any more — in a zero-sum game that holds only for the opponents’ payoffs taken together — so the opponents can act in concert against you, and different equilibria pay you differently), so there is no single v^* to anchor to. Equilibria in games with more than two players are “neither unique nor non-exploitable”.¹⁰⁶ N -player safety notions do exist, but they are either very conservative (the maxmin value against a coordinated coalition¹⁰⁷) or a target that cannot be secured in general (equal share, the fair share C/n , is provably not securable against opponents with different fixed strategies¹⁰⁸). **That is the open problem for Contribution #2** — exploiting weak opponents in small N -player games with a bounded loss relative to a baseline strategy, including against colluding opponents. This chapter does not solve it; it makes the failure *precise*.¹⁰⁸¹⁰⁹¹¹⁰

¹⁰⁶Ge, J., Wang, Y., Li, W. & Jin, C. (2025). “Securing Equal Share: A Principled Approach for Learning Multiplayer Symmetric Games.” *ICML*; arXiv:2406.04201.

¹⁰⁷Celli, A. & Gatti, N. (2018). “Computational Results for Extensive-Form Adversarial Team Games.” *AAAI* 32(1). DOI 10.1609/aaai.v32i1.11462.

¹⁰⁸Johanson, M., Zinkevich, M. & Bowling, M. (2007). “Computing Robust Counter-Strategies.” *NIPS 2007*, 721–728 — Restricted Nash Response, the tunable ancestor of the methods above.

¹⁰⁹Jeary, L. & Turrini, P. (2023). “Safe Opponent Exploitation For Epsilon Equilibrium Strategies.” *arXiv:2307.12338*.

¹¹⁰Ge, Z., Xu, Z., Ding, T., Meng, L., An, B., Li, W. & Gao, Y. (2024). “Safe and Robust Subgame Exploitation in Imperfect Information Games.” *ICML*, PMLR 235, 15255–15270.

8.4 Sequence-Form LP and Constraint Generation

The implementation (a **HeroTreeplex** on top of Chapter 7’s sequence-form code, SciPy’s HiGHS solver, and the full constraint-generation loop) is described in the report. Each generated cut is the payoff vector against a *specific* adversary best response, a valid linear lower bound on the true worst case, so adding cuts monotonically tightens the relaxed safety constraint. Only the floor changes between methods: best equilibrium uses v^* , prime-safe $v^* - \varepsilon$, adaptation the blueprint’s worst case, and RNR a max-min in p (§ 8.5); the subgame method (§ 8.7) adds *pins* that hold the hero’s play outside the subgame at the blueprint.

One numerical wrinkle decides safety. An *approximate* Nash’s self-play value can sit a hair **above** the game’s true max-min, so requiring the exact $\mathbf{wc} \geq \mathbf{floor}$ can make the LP infeasible and, on an unlucky cut path, return an *unsafe* strategy. A small feasibility slack on the cuts ($\mathbf{floor} - \mathbf{slack}$, with $\mathbf{slack} \approx 5 \cdot 10^{-4}$, independent of the convergence tolerance) keeps the true max-min strategy feasible.¹¹¹

8.5 Restricted Nash Response and the Bang-Bang Switch

One of the earliest principled methods, **Restricted Nash Response** (Johanson 2007; close to the ε -safe strategies of McCracken and Bowling, 2004), introduces the *tunable knob*: compute the hero’s equilibrium against a **p -restricted** opponent, forced to play the fixed model with probability p and free to play adversarially with probability $1 - p$. Sweeping p from 0 to 1 traces a path from Nash ($p = 0$) to full best response ($p = 1$). Because the free component best-responds to the hero, RNR is a **max-min**:

$$\text{RNR}(p) = \arg \max_x \left[p \cdot \text{EV}(x, \text{model}) + (1 - p) \cdot \min_{\sigma'} \text{EV}(x, \sigma') \right],$$

which we solve with the same cutting-plane machinery plus an auxiliary worst-case variable.

A flag worth keeping. The project’s own step notes originally described RNR as a *naive behavioral blend*, $(1 - p) \cdot \text{Nash} + p \cdot \text{BR}$. That is **not** Johanson’s algorithm; we implemented and labelled *both*. The naive blend traces a smooth line (grey in the next figure): profit and exploitability rise together from Nash to full BR.

The measured surprise — canonical RNR is bang-bang. I predicted a smooth, monotone frontier that dominates the blend everywhere. On Kuhn versus the Rock, canonical RNR instead returns the **same safe strategy** for all $p \in [0, 0.6]$ (EV -0.044 , exploitability ≈ 0) and then **jumps straight to the full best response** at $p \approx 0.7$ (EV $+0.167$, exploitability 0.444), with no intermediate points between the sampled values of p (steps of 0.1).

¹¹¹The constraint-generation / double-oracle idea traces to McMahan, H. B., Gordon, G. J. & Blum, A. (2003). “Planning in the Presence of Cost Functions Controlled by an Adversary.” *ICML* — the general recipe for “optimize against a worst case you discover as you go.”

Table 21: Restricted Nash Response across the mixing parameter: the canonical form against a naive blend.

p	canonical RNR — EV	canonical — exploitability	naive blend — EV	naive — exploitability
0.0	−0.044	~0.000	−0.047	0.000
0.3	−0.044	~0.000	+0.017	0.133
0.6	−0.044	~0.000	+0.081	0.266
0.7	+0.167	0.444	+0.103	0.311
1.0	+0.167	0.444	+0.167	0.444

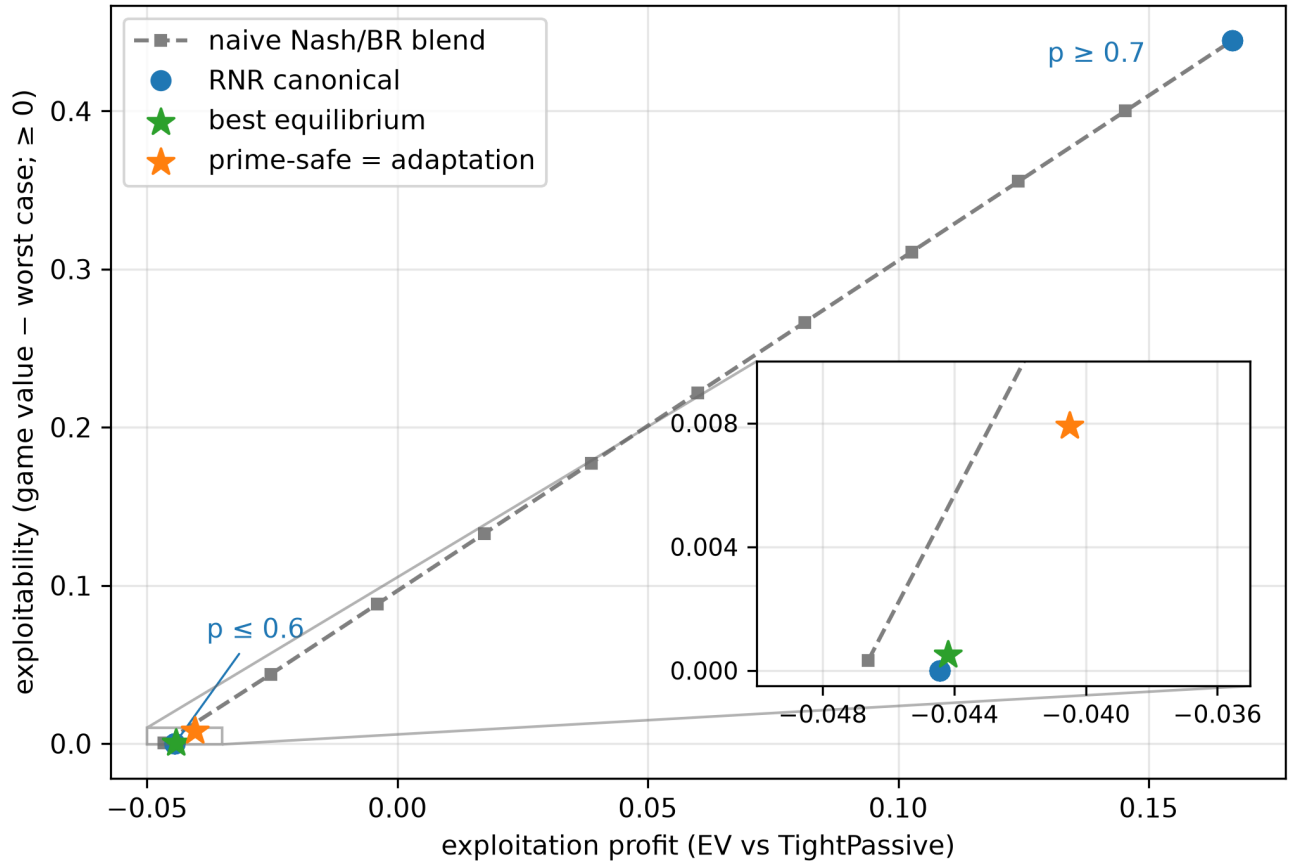


Figure 40: The exploitation–safety frontier on Kuhn: canonical RNR gives only a safe cluster ($p \leq 0.6$) and a full-BR cluster ($p \geq 0.7$); grey is the dominated naive blend; stars mark the LP methods at the safe corner.

Why (I checked before trusting the story). The RNR objective is **linear** in x over a **polytope**, so its optimum is a **vertex** and switches vertices only when p crosses a critical ratio; Kuhn’s tiny polytope has few vertices, so the transition is a single jump. Johanson’s smooth curve is a *large-game* phenomenon

(many vertices), or comes from the *data-biased* variant that makes p per-information-set. The naive blend is **dominated at the safe corner** (at exploitability ≈ 0 the LP methods reach EV -0.044 versus the blend’s -0.047). What is bang-bang is the map from p to a strategy, not the frontier: mixing the two RNR strategies reaches every point on the chord, because EV is linear in the mixture and the worst case is concave. The switch happens at $p \approx 0.68$; any further vertex would be optimal only for p between 0.6 and 0.7, which the sweep did not sample. Against the Rock the chord improves on the naive blend by only ≈ 0.002 , so here choosing *where* to deviate buys almost nothing; Johanson et al. report strongly concave frontiers in a large abstracted hold’em game. What the small game does show is that p is not a dial: the switching threshold *moves with how exploitable the opponent is*, which recurs in § 8.6.¹¹²

8.6 Best Equilibrium on Kuhn — safe, but only a little more profitable (the core result)

The central experiment solves each method against a *perfect* model of each opponent type and scores it on **profit** (EV vs that opponent) and **safety** (worst-case value), with the Nash floor at $v^* = -0.056$ (Kuhn’s first-player value, $\approx -1/18$). All numbers are exact (full-tree), from the scale run.

Table 22: Kuhn: expected value and worst-case value for every method against every opponent (“best eq.” is the best-equilibrium baseline, code id `ganzfried`).

Opponent	metric	nash	full_br	rnr_0.5	best eq.	prime_safe	adaptation
Rock (TightPassive)	EV	-0.047	$+0.167$	-0.044	-0.044	-0.040	-0.040
	worst-case	-0.056	-0.500	-0.056	-0.056	-0.063	-0.063
Maniac (LooseAggr.)	EV	$+0.118$	$+0.333$	$+0.278$	$+0.131$	$+0.151$	$+0.151$
	worst-case	-0.056	-0.167	-0.111	-0.056	-0.063	-0.063
AlwaysBet	EV	$+0.113$	$+0.333$	$+0.333$	$+0.115$	$+0.131$	$+0.131$
	worst-case	-0.056	-0.167	-0.167	-0.056	-0.063	-0.063
AlwaysPass (most exploitable)	EV	$+0.146$	$+0.975$	$+0.975$	$+0.222$	$+0.266$	$+0.266$
	worst-case	-0.056	-0.500	-0.333	-0.056	-0.063	-0.063
Nash (control)	EV	-0.056	-0.055	-0.056	-0.056	-0.055	-0.055
	worst-case	-0.056	-0.333	-0.056	-0.056	-0.063	-0.063

¹¹²Johanson, M. & Bowling, M. (2009). “Data Biased Robust Counter Strategies.” *AISTATS*, PMLR 5, 264–271 — makes the p knob depend on how much data supports each part of the strategy, which is what smooths the frontier in practice.

(`ses_subgame` is not shown: on Kuhn the subgame is the whole game and the method coincides with adaptation.)

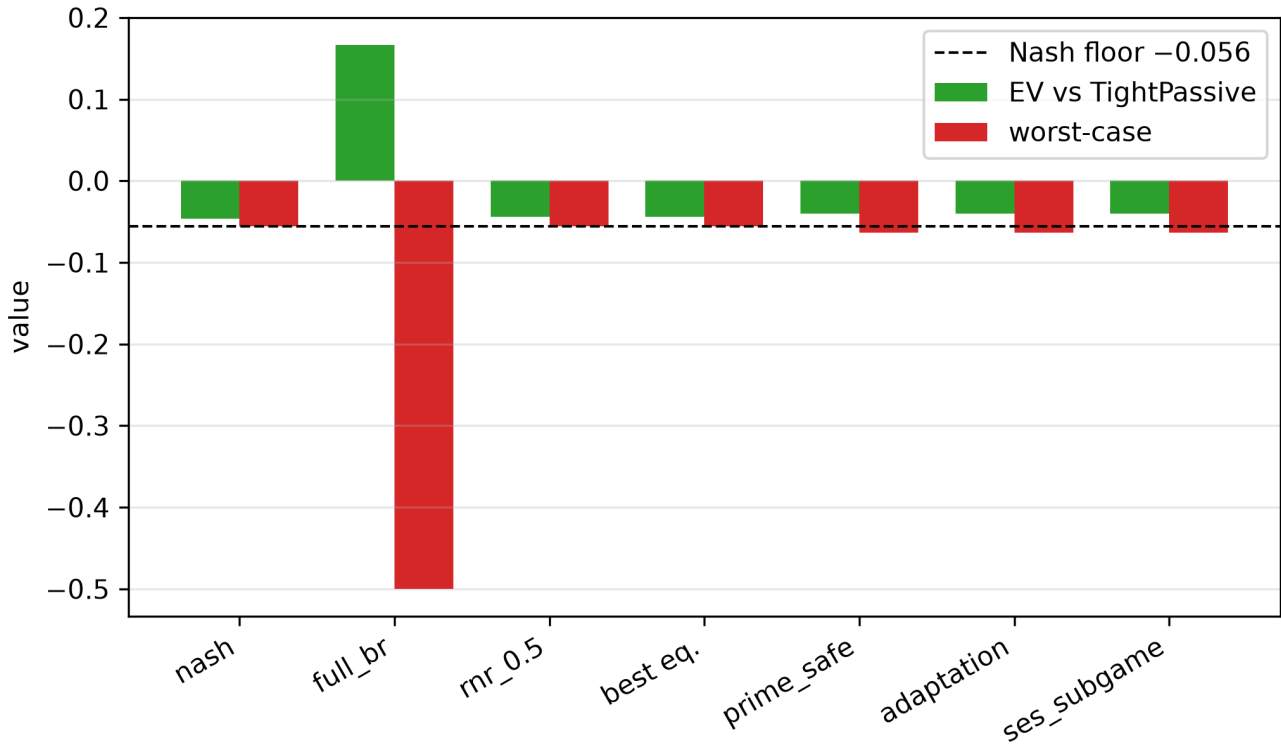


Figure 41: Methods versus the Rock on Kuhn: green is EV, red is worst-case value, dashed is the Nash floor. Only the full best response’s worst case plunges to -0.5 ; every principled method stays near the floor.

Three results carry the chapter.

1. **The full best response is the cautionary tale.** It always wins the most against a fixed model — up to **+0.975** against *AlwaysPass* — but its worst case collapses to -0.5 . It is the only method that is unsafe against *every* opponent (*rnr_0.5* is unsafe against three), because an adversary can always best-respond back to the hole it opens.
2. **Best equilibrium is safe and still profitable — but only slightly.** Against *every* opponent its worst case stays at the Nash floor (safe within 10^{-3}) **and** it beats Nash’s own EV on every exploitable type — most vividly **+0.222 versus Nash’s +0.146** against *AlwaysPass*, and +0.131 versus +0.118 against the Maniac. *You can exploit while provably never dropping below equilibrium value* — but only a little: against the four exploitable types best equilibrium gains 0.002–0.076 per hand over Nash, 1–9 % of what the full best response gains, because a per-hand floor at v^* admits only equilibrium strategies (§ 8.3).
3. **A single global p is not a safety setting.** *rnr_0.5* is safe against the Rock but has *already jumped* to the full best response against the highly exploitable *AlwaysPass* / *AlwaysBet* (identical

EV to `full_br`, worst case $-0.33 / -0.17$, unsafe). This is the switching threshold of § 8.5 moving with opponent exploitability, and the concrete reason best equilibrium, which constrains the *value* rather than a *knob*, is the better primitive.

Prime-safe and adaptation spend a measured ϵ -budget. The run *measured* the early-stopped-CFR baseline’s exploitability at $\epsilon = 0.0074$; their worst case comes out at $-0.063 = v^* - 0.008$ against every opponent, matching the ϵ -adjusted floor, and they earn a little more than best equilibrium ($+0.266$ versus $+0.222$ on `AlwaysPass`). They look “unsafe” in the table only because the flag compares to v^* ; against their *own* floor they are safe by construction. They coincide because the same early-stopped CFR strategy serves as the prime-safe baseline and the adaptation blueprint.¹⁰⁴

8.7 Real-Time Safety — subgame gadgets (SES)

The guarantee of best equilibrium is *global*: the safety constraint is enforced over the **whole** game tree, which a real game cannot re-solve every time the model updates. The **Safe Exploitation Search** idea (Liu et al. 2022) makes safety a **local** property of a single **subgame**: play the blueprint everywhere, but at a chosen subgame re-optimize the hero’s play to exploit the model — with a **gadget** that bounds how much the local deviation can add to exploitability (the bound grows with the exploitation level and the error of the opponent model). The stricter guarantee used here — never more exploitable than the blueprint — is Ge et al.’s adaptation safety, which their OX-Search achieves for subgame re-solving.

The implementation enforces it directly on the treeplex: **pin** every hero sequence *outside* the subgame to its blueprint realization weight, and set the safety floor to the blueprint’s own worst-case value. Because “play the blueprint inside the subgame too” is always feasible and attains exactly that floor, the local exploit is safe by construction. The one requirement is that the subgame be **downward-closed** (once you are in it you stay in it), which holds for natural choices like “Leduc round 2” or “after a King flops.”

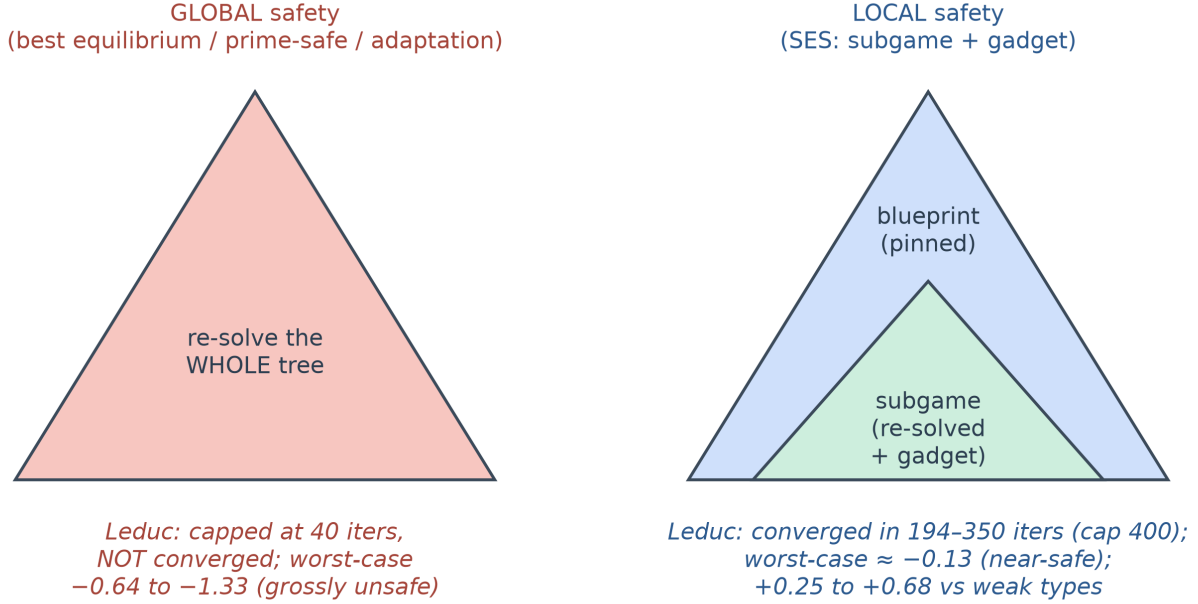


Figure 42: Global versus local safety. Global methods re-solve the whole tree; the subgame method pins play outside a chosen subgame to the blueprint and re-solves only that subgame with a gadget, a far smaller problem.

§ 8.8 shows why “local” matters: on Leduc the subgame solve converged within its budget and the global solves did not converge within theirs.¹¹³¹¹⁴¹¹⁵

8.8 At Scale — Kuhn works; Leduc breaks globally, holds locally

Leduc Hold'em (Section 3.4) is still exactly solvable, but large enough to stress the methods. The run capped the constraint-generation loop at 40 iterations (tolerance 10^{-2} , subgame set to the King-flop) and recorded for each cell whether the solve **converged** or hit the cap. Game value $v^* = -0.086$.

¹¹³Liu, M., Wu, C., Liu, Q., Jing, Y., Yang, J., Tang, P. & Zhang, C. (2022). “Safe Opponent-Exploitation Subgame Refinement.” *NeurIPS* — Safe Exploitation Search (SES) with a gadget; the added exploitability is bounded from above (Theorem 4.1).

¹¹⁴Ge, Z. et al. (2024), *op. cit.* — OX-Search: subgame re-solving with a gadget that keeps the refined strategy no more exploitable than the blueprint (adaptation safety, Theorem 4.3), applicable in nested fashion at each newly reached information set; motivated by the “being taught and exploited” problem; tested on Leduc and Flop Hold'em, two-player.

¹¹⁵Milec, D., Kovařík, V. & Lisý, V. (2025). “Adapting Beyond the Depth Limit: Counter Strategies in Large Imperfect Information Games.” *AAMAS 2025* (extended abstract), 2675–2677; arXiv:2501.10464 — using opponent-model information *past* the search horizon.

Table 23: Leduc: the subgame method against the global solvers.

Opponent	metric	nash	full_br	ses_subgame	best eq.	prime_safe / adaptation
Rock	EV	+0.201	+0.937	+0.247	+0.624	+0.635
	worst-case	−0.089	−1.633	−0.130	−0.838	−0.744
	converged?	✓	✓	✓ (194 it)	✗ capped	✗ capped
Maniac	EV	+0.438	+2.177	+0.682	+1.806	+1.842
	worst-case	−0.089	−1.100	−0.130	−0.638	−0.657
	converged?	✓	✓	✓ (211 it)	✗ capped	✗ capped
CallingStation	EV	+0.559	+1.464	+0.663	+1.342	+1.363
	worst-case	−0.089	−1.000	−0.130	−0.915	−0.686
	converged?	✓	✓	✓ (350 it)	✗ capped	✗ capped
LoosePassive	EV	+0.449	+1.405	+0.559	+1.306	+1.309
	worst-case	−0.089	−4.200	−0.133	−1.239	−1.334
	converged?	✓	✓	✗ (400 it)	✗ capped	✗ capped

The headline finding is negative and empirical: within a 40-iteration cap, the global solve did not converge even on Leduc. I predicted best equilibrium would be safe on Leduc as it is on Kuhn. Instead the **global** solvers (best equilibrium **ganzfried**, **prime_safe**, **adaptation**, and **rn timer**) all hit the cap, leaving worst-case values of **−0.64 to −1.33** — grossly unsafe. The reason, which I confirmed rather than assumed: the loop adds one adversary cut per iteration, and on Leduc the set of relevant pure best responses is large, so 40 cuts nowhere near pin the true worst case and the master LP keeps returning optimistic-but-unsafe strategies.

The positive half is the subgame method. SES **converged** on three of four exploitable opponents (194–350 iterations), because it re-solves only the small King-flop subgame — a far smaller LP with a far smaller adversary set. It extracts real value (**+0.25 to +0.68**, beating Nash) at a worst case of $\approx$ **−0.13**, an order of magnitude closer to safe than the global methods: the *global-versus-local* distinction of § 8.7 as a measured fact. The comparison is not at equal budgets, though: the global solvers were stopped after 40 iterations (about 2.5 s per cell), while SES had a cap of 400 and used 194–400 iterations (30–79 s per cell). So the global loop is slow and unsafe *within this budget*, not necessarily with a larger one. Chapter 14 settles the scaling question: its one-shot dual LP, which replaces the loop, solves the full-Leduc problem in 0.02–0.04 s, so the wall was the loop’s, not the global problem’s.

I keep myself honest on SES, though: its residual exploitability (≈ 0.043) exceeds the 0.01 tolerance, so it too is flagged unsafe, and on **LoosePassive** it ran all 400 iterations without converging. But SES’s own floor is the blueprint’s worst case — the early-stopped CFR baseline (−0.120), not Nash (−0.089) — and the three converged cells sit 0.010 below it, exactly at the tolerance. So SES held its gadget floor, and the residual is mostly the blueprint’s own exploitability; a rerun with a tighter tolerance would show whether

it is *provably* safe.

8.8.1 The teaching attack — and why realized profit is the wrong lens

The last experiment is the deception stress test the safety machinery is meant to survive. A deceptive opponent plays the weak Rock bait for 10 000 hands, then switches to a strong Nash “reveal” for 10 000 more; a Chapter 7 model feeds each solver every 500 hands (5 seeds).

Table 24: The teaching attack: realised profit and safety violations per method.

method	mean/hand (all)	mean/hand (after switch)	safety violations / seed
full_br	+0.051	−0.061	40, 40, 40, 40, 40
best eq. (ganzfried)	−0.048	−0.055	0, 0, 0, 0, 0
adaptation	−0.046	−0.055	40, 40, 40, 40, 40
nash	−0.051	−0.061	0, 0, 0, 0, 0

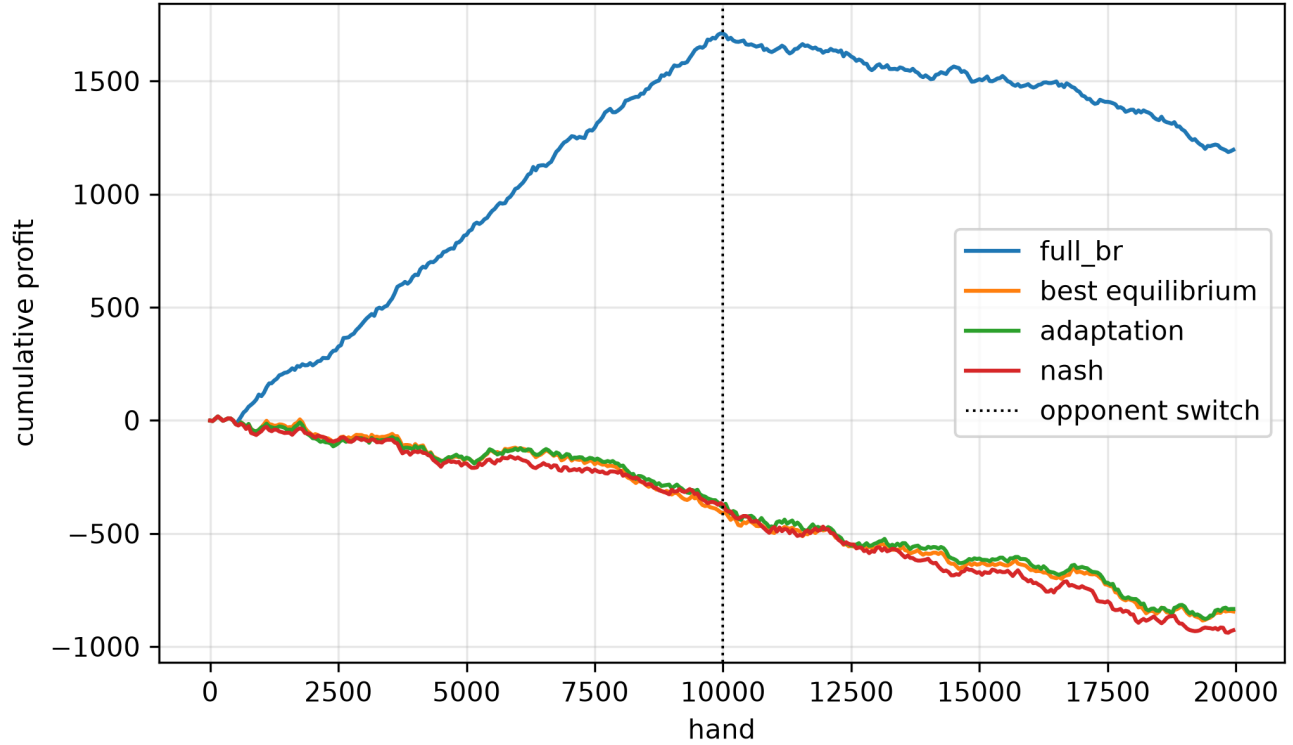


Figure 43: Teaching attack on Kuhn, cumulative profit (seed 0). The full best response (blue) climbs to about +1700 on the bait and ends far ahead; the safe methods refuse the bait and pay the first-player tax throughout.

The naive story — “the teaching attack punishes the greedy exploiter” — **did not hold in realized profit**. The full best response ends *hugely net-positive* (+0.051/hand overall): it banks a windfall during the bait, and a *Nash* revealer claws back barely more than the game value per hand, so the windfall is never repaid within 10 000 hands. What *does* separate safe from unsafe is the **exact worst-case / safety-violation count**: the full best response violated the Nash floor at **40 of 40** refits, best equilibrium at **0 of 40**. A strategy’s worst case is realized by an opponent that *best-responds to it*, and a stationary Nash reveal is not that adversary. The design lesson: **measure safety by the worst case, not by realized profit against a benign opponent**; to punish the greedy exploiter *in profit*, the reveal must be an adaptive counter-exploiter, not a fixed Nash. (Adaptation’s 40 violations are by design: it targets a floor *below* v^* , so it trips the v^* -referenced counter.)

8.9 Connections and Forward Pointers

What this chapter establishes. Safe exploitation is one idea — *maximize value against the model subject to a safety floor*. On a fully solvable game best equilibrium is safe against every opponent while earning slightly more than Nash against every exploitable one, where the naive best response earns more but is ruinously exploitable; prime-safe and adaptation extend the guarantee to imperfect baselines by spending a *measured* ε -budget. The most valuable result is negative: on Leduc the *global* solve did not converge within a 40-iteration cap while the *local* subgame method (cap 400) did, if under unequal budgets.

Backward connections. Chapter 7’s sensor supplies this chapter’s *objective*, its exact best response is the *worst-case oracle*, and the sequence form is the *variable* the LP optimizes. The continuous model’s self-inflicted leak against Nash on Leduc (Chapter 7, § 7.7) is what the safety floor is designed to prevent.

Forward to the thesis. The Leduc non-convergence argues for two scalable paths: an **exact one-shot dual LP** for the worst-case constraint, or **local / subgame** safety (SES, OX-Search) in real time; Chapter 14 took the first (full Leduc in 0.02–0.04 s). RWYWE (§ 8.3), the stronger two-player baseline for Contribution #2, needs only the existing LP with a floor that moves with the banked gifts; implementing it is future work. Either way, the safety floor is what turns Chapter 7’s fragile sensor into a deployable adaptive agent.

9 Chapter 9 — Multi-Agent Reinforcement Learning: Coordination, Competition, and Communication

This chapter covers multi-agent reinforcement learning (MARL): the problem that makes it different, the mathematics that frames it, the methods that attack it, and controlled experiments on small, exactly-solvable games. **All experimental numbers were measured** on reproducible runs and, wherever possible, are bounded by *exact* references (Nash equilibria, exact best-response values) rather than by other simulations. Where a run contradicted my expectation, I keep the expectation and reconcile it with what happened — those gaps are the most instructive parts.

Where this sits in the thesis. Chapters 2–8 worked almost entirely inside **two-player zero-sum** games (Pluribus in Chapter 6 being the exception that gave up the guarantees): there is a value v^* , a Nash strategy secures it against *any* opponent, and CFR provably converges to it. Chapter 9 is the pivot into the **multi-agent** world, where those three comforts weaken or vanish. It carries three thesis hooks. **LOLA** (Foerster et al., 2018) — differentiating through an opponent’s *learning step* — can be read as *dynamic* opponent modeling, the moving-target complement to Chapter 7’s static read (Contribution #1). **PSRO** — a game played over a *population of policies* — is both a candidate framework for safe exploitation where there is no minimax theorem (Contribution #2) and a general multi-agent *evaluation methodology* (Contribution #3). The missing minimax anchor for $N > 2$, where every two-player guarantee breaks, is named here and left open for the thesis to attack.

9.1 Why multi-agent RL is a different problem

A single agent (Chapter 1) learns against a **fixed** world: a stationary MDP, whose fixed optimal value Q^* it can converge toward. Chapters 2–8 instead computed **equilibria** — strategies optimal against a *perfectly rational* opponent who has already finished reasoning.

Multi-agent RL sits between these two and is harder than both. Several agents **learn at the same time in a shared world**, so from any one agent’s seat the “environment” — which now *includes the other agents* — **keeps changing** as everyone updates. This is **non-stationarity**: agent i ’s effective transitions and rewards depend on the others’ policies π_{-i} , which move every update, so the target Q^* it chases is itself in motion. It also creates two problems a lone agent never has: **coordination** (how do cooperating agents learn to act together without being told how?) and **credit assignment** (when the team succeeds, whose actions mattered?).

A picture to hold onto: **learning to dance with a partner who is also learning to dance**. If your partner’s steps were fixed, you could memorize a routine that fits them — that is single-agent RL. If your partner were a flawless professional, you could study their known style and prepare the perfect counter — that is equilibrium computation. But when *both* of you are improving in real time, every adjustment you make changes what they should do, and vice-versa; you step on each other’s toes, over-correct, and oscillate — until you either lock into a shared rhythm (a coordinated equilibrium) or cycle forever (as

in Matching Pennies, Section 9.4). The entire field is the art of *stacking the deck so the synchronization happens reliably*.¹¹⁶

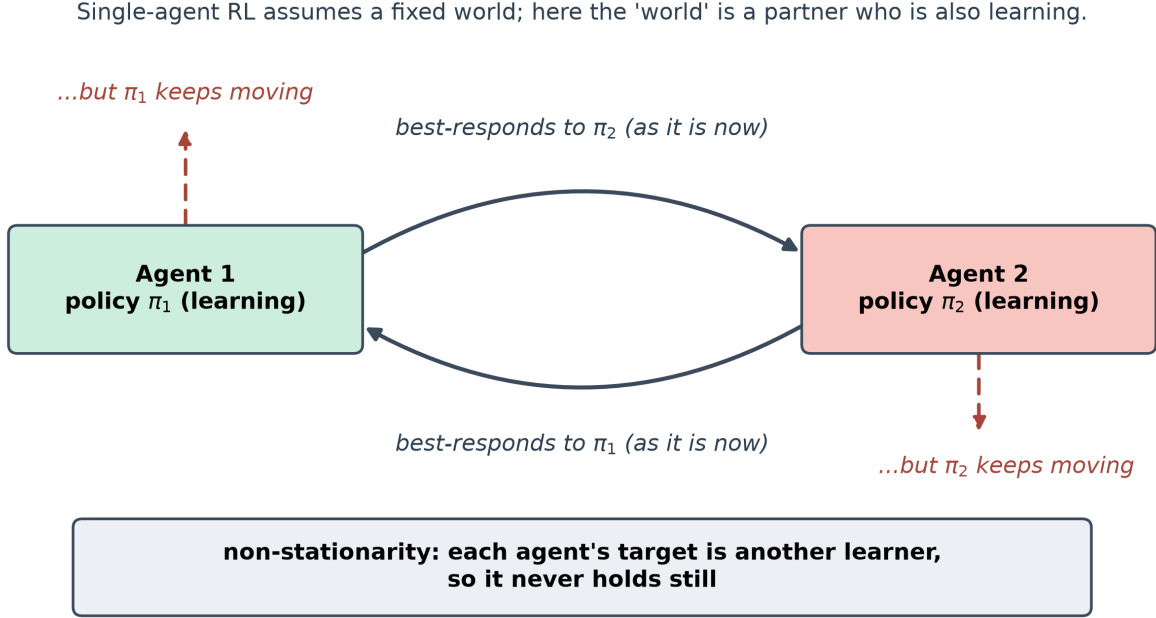


Figure 44: Non-stationarity as two partners learning to dance at once: each optimizes against the other’s current policy, which is itself moving. Every method in this chapter makes that moving target hold still long enough to learn against.

9.2 Markov games — the bridge from Chapters 2–8 to MARL

Chapters 2–8 reasoned about **extensive-form games (EFGs)**: a game tree, **information sets**, and counterfactual values feeding CFR. MARL’s standard formalism is the **Markov (stochastic) game**, and stating the connection keeps the switch from feeling like starting over. A **Markov game** is the tuple

$$(\mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, P, \{R_i\}_{i=1}^N, \{\Omega_i\}, O, \gamma),$$

with states $s \in \mathcal{S}$, one action set $\mathcal{A}_i$ per agent, a transition law $P(s' \mid s, a_1, \dots, a_N)$ driven by the **joint** action, per-agent rewards $R_i(s, a_1, \dots, a_N)$, and (in the partially observed case) observations $o_i = O_i(s)$. Each agent has a policy $\pi_i(a_i \mid o_i)$; together they form a **joint policy** $\pi = (\pi_1, \dots, \pi_N)$. The special cases are exactly the previous chapters:

- $N = 1$ recovers an ordinary **MDP** (Chapter 1).

¹¹⁶Zhang, K., Yang, Z. & Başar, T. (2021). “Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms.” *Handbook of RL and Control* — §1–2 for non-stationarity and the taxonomy used here.

- $N = 2$ and a single state give a **matrix game** (zero-sum when $R_1 = -R_2$; this chapter’s testbed, Section 9.4).
- Sequential, imperfect-information, zero-sum gives an **EFG** (Chapters 2–8): a Markov game whose “state” is a *history* and whose partial observability is precisely an **information set**.

What survives the bridge is the *vocabulary*: the EFG’s behavioral strategy at an information set is exactly the Markov-game policy $\pi_i(a_i \mid o_i)$, the information set becomes the observation o_i , and the counterfactual value of a history becomes a centralized critic’s estimate at a state. What does **not** survive is the *guarantees*. CFR’s counterfactual decomposition needs a game tree with perfect recall, which general Markov games (loops, simultaneous moves) do not provide; and with $N > 2$ players CFR loses its guarantee of converging to a Nash equilibrium even on such a tree (Abou Risk & Szafron, 2010),¹¹⁷ so MARL falls back to gradient/value learning whose convergence is not guaranteed (hence the cycling of Section 9.4). Most consequentially, the **minimax value anchor** disappears: in two-player zero-sum a Nash strategy guarantees v^* against any opponent — the fact that made Chapter 8’s safe exploitation coherent — and for $N > 2$ there is no such single value. That missing anchor is the precise gap Contribution #2 inherits.¹¹⁸

9.3 The family of methods

Every method here is a different structural answer to “the other agents are learning while I learn.”¹¹⁹ MADDPG and MAPPO are the two variants of centralized training with decentralized execution (CTDE) tested here; value-factorization variants such as QMIX¹²⁰ are not used.

Table 25: The MARL method families: what each does, when to reach for it, and its main weakness.

Approach	What it does	Reach for it when	Main weakness
Independent Learning (IL)	each agent runs its own single-agent RL, treating others as environment	a quick baseline; near-stationary settings	non-stationarity $\rightarrow$ cycling, coordination failure

¹¹⁷Abou Risk, N. & Szafron, D. (2010). “Using Counterfactual Regret Minimization to Create Competitive Multiplayer Poker Agents.” *AAMAS*, 159–166 — CFR’s convergence guarantee covers two-player zero-sum games with perfect recall; with more players it is lost.

¹¹⁸Littman, M. L. (1994). “Markov Games as a Framework for Multi-Agent Reinforcement Learning.” *ICML* — the paper that introduced this framing and the minimax-Q algorithm.

¹¹⁹Albrecht, S. V., Christianos, F. & Schäfer, L. (2024). *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches* (MIT Press), Ch. 5 and 9 — a current textbook treatment of these challenges and methods.

¹²⁰Rashid, T., Samvelyan, M., Schroeder de Witt, C., Farquhar, G., Foerster, J. & Whiteson, S. (2018). “QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning.” *ICML*. arXiv:1803.11485.

Approach	What it does	Reach for it when	Main weakness
MADDPG (CTDE)	each agent’s critic sees all agents’ obs+actions at training; each actor sees only its own obs at execution	mixed cooperative-competitive tasks; the canonical CTDE template	critic input grows with N ; the actor update is fiddly
MAPPO (CTDE)	plain PPO with a centralized value $V(\text{global state})$ and shared parameters	cooperative MARL where you want a simple, strong baseline	on-policy sample cost; “simple” but tuning-sensitive
PSRO	maintain a population of policies; solve a meta-Nash over it; train a best response to that mixture; add it; repeat	competitive/general games; when you want a game-theoretic convergence target	a full best response per round; an approximate oracle weakens the guarantee
LOLA	optimize assuming the opponent takes one learning step ; differentiate <i>through</i> their update	2-player differentiable games where naive learning fails (IPD)	assumes you know and can differentiate the opponent’s update
CommNet	agents broadcast a differentiable message ; each receives the mean of the others’ and feeds it into its policy	cooperative tasks with partial observability	mean pooling discards <i>who</i> said what

Two axes cut across the table. **What is centralized, and when?** Nothing for IL; the critic at training only for CTDE; a whole meta-game solve between rounds for PSRO; information at execution for CommNet. **Is the opponent static or a learner?** Everything except LOLA treats the opponent’s strategy as fixed while you respond; LOLA anticipates the opponent’s *next update*.

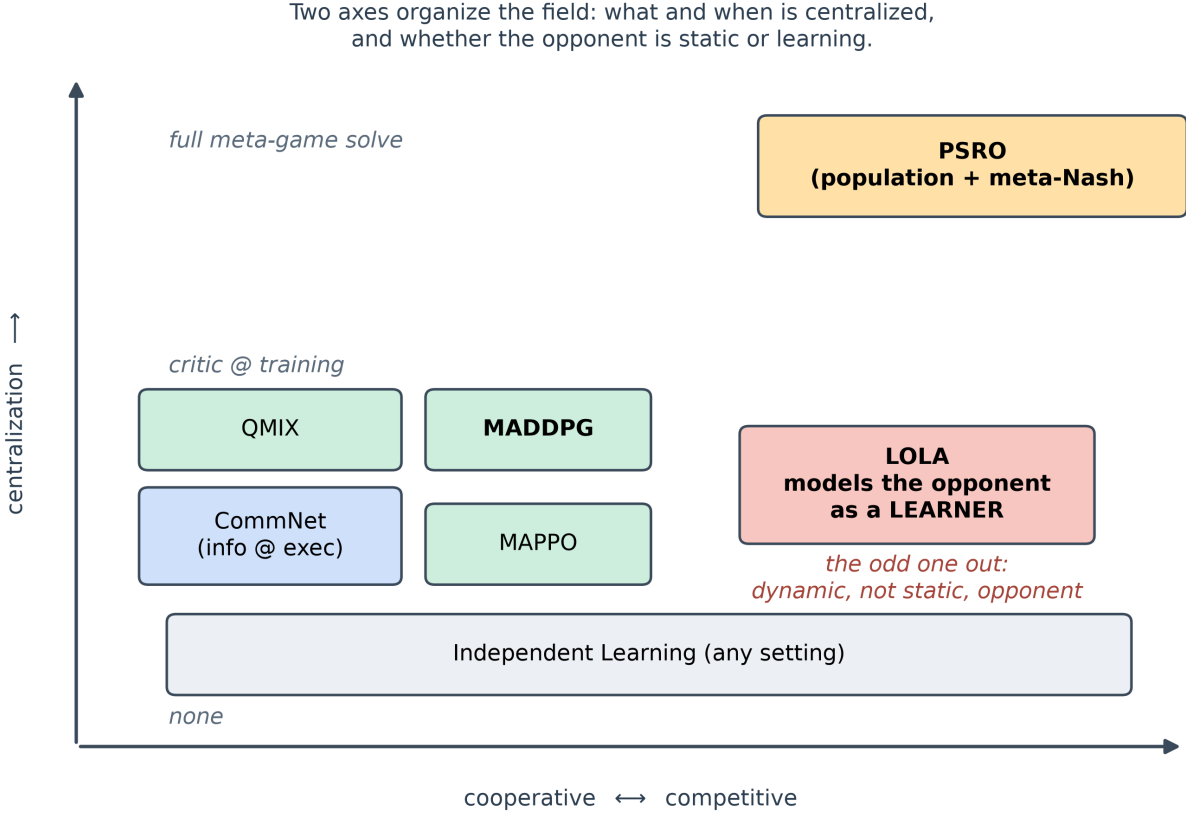


Figure 45: The method family on two axes: competitive vs cooperative target setting, and how much is centralized and when. LOLA is the outlier that models the opponent as a learner rather than a fixed strategy.

9.4 Independent learning and its failure modes

The cleanest way to see *why* the rest of the field exists is to run the control that fails. Independent learning drops each agent into its own gradient loop and pretends the others are part of the floor. On four canonical 2×2 matrix games — each a different qualitative case — it produces four different fates.

The games and their analytic Nash equilibria (ground truth): **Prisoner’s Dilemma** (Defect strictly dominates, so the unique Nash is mutual Defect); **Matching Pennies** (zero-sum; unique fully mixed Nash $(\frac{1}{2}, \frac{1}{2})$, value 0); **Stag Hunt** (two pure Nash — the payoff-dominant (Stag, Stag) and the risk-dominant (Hare, Hare) — plus a mixed one); **Battle of the Sexes** (two pure Nash that disagree on which to pick, plus a mixed one). Two independent gradient learners with exact gradients (clean, noise-free dynamics) give, measured:

Table 26: Independent learners on four matrix games, against the analytic Nash equilibrium.

Game	Measured outcome	NashConv	Matches analytic Nash?
Prisoner’s Dilemma	$x \rightarrow [0.001, 0.999]$ (Defect)	0.0013	yes — the dominant-strategy equilibrium
Stag Hunt	all seeds $\rightarrow$ (Hare, Hare)	0.0013	yes — the <i>risk-dominant</i> pure Nash
Battle of the Sexes	all seeds $\rightarrow$ one pure Nash	0.0013	yes — a pure Nash
Matching Pennies	does not converge (last iterate drifts to the boundary)	1.4–1.8	no — it never settles

Three of the four converge to a genuine Nash, and the interesting details are in the “how.” In Stag Hunt the learners reliably pick the **risk-dominant** equilibrium (Hare) even though Stag pays more — a unilateral move toward Stag is punished, so gradient dynamics slide to the safe corner. In Battle of the Sexes every seed lands on the *same* pure equilibrium: coordination is solved, but not in a seed-diverse way. These are the coordination and equilibrium-selection problems made concrete.

Matching Pennies is the headline: it **never converges**, which is exactly the point.¹²¹

Reconciliation (kept prediction $\rightarrow$ what actually happened). I predicted Matching Pennies would trace a clean orbit at roughly constant radius around $(\frac{1}{2}, \frac{1}{2})$. Two learners, run two ways, both failed to converge — but neither orbited cleanly. The projected-gradient (IGA) learner of the exploration scripts *spirals outward* (distance-to-Nash grew across time-windows from 0.30 to 0.48); the softmax-logit learner of the implementation *drifts to the corners* (final profiles like $x = [0.96, 0.04]$, $y = [0.03, 0.97]$, $\text{NashConv} \approx 1.8$). The closed orbit is the right picture only for the continuous-time dynamics (Singh et al., 2000); with a fixed step of 0.1 each update lands slightly outside the curve, so the trajectory spirals out to the boundary. **The lesson is unchanged and arguably sharper:** under naive simultaneous learning the *last iterate* does not converge in a game with only a mixed equilibrium — what converges is the *time-average* (Section 9.6).

¹²¹Singh, S., Kearns, M. & Mansour, Y. (2000). “Nash Convergence of Gradient Dynamics in General-Sum Games.” *UAI* — the analysis of why gradient ascent cycles rather than converges in games like Matching Pennies.

Independent learners in 2x2 games (strategy-space trajectories)

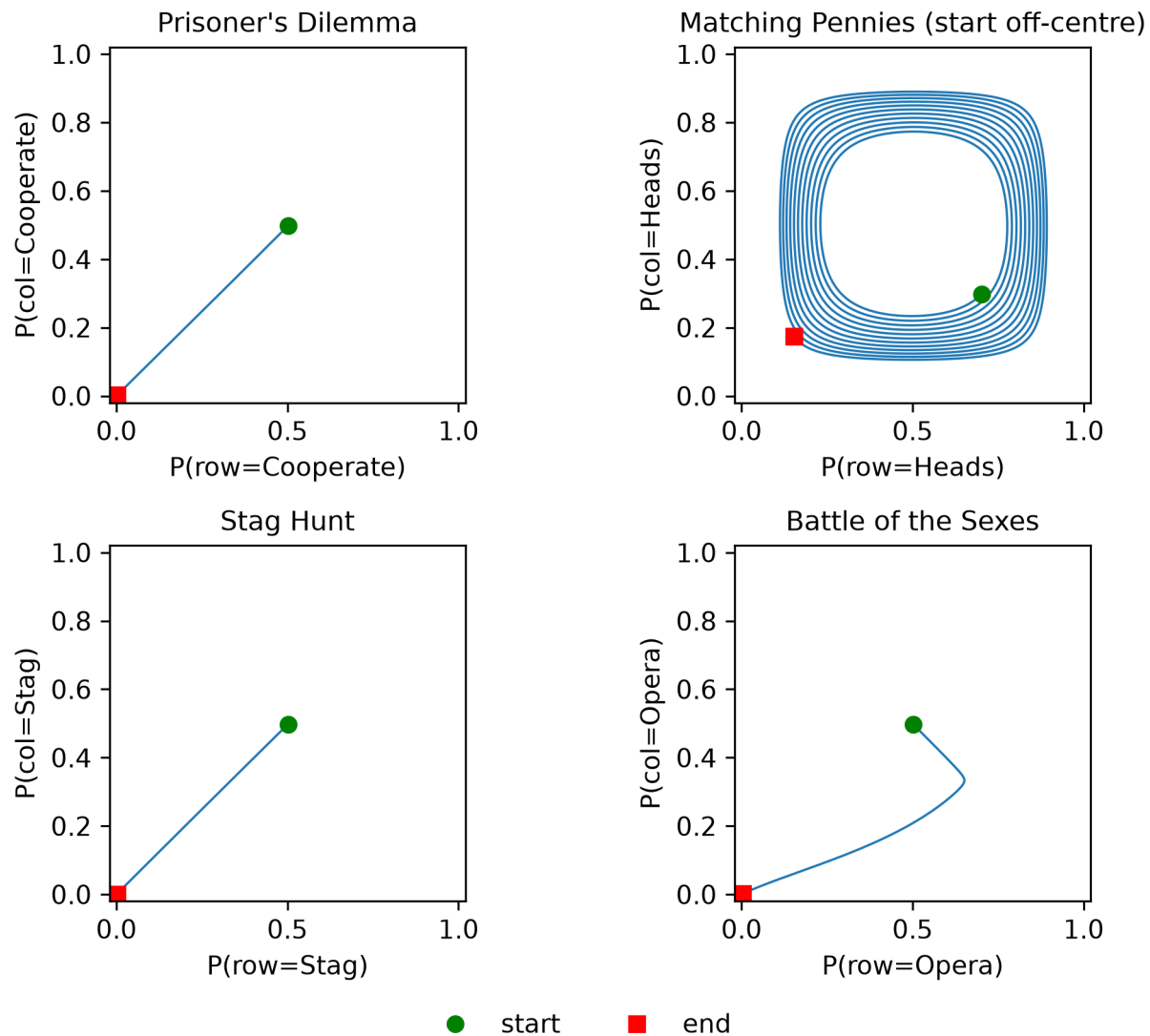


Figure 46: Independent learners on the four matrix games: Prisoner's Dilemma collapses to mutual defection, Stag Hunt and Battle of the Sexes settle on a pure equilibrium, and Matching Pennies drifts away from the mixed Nash.

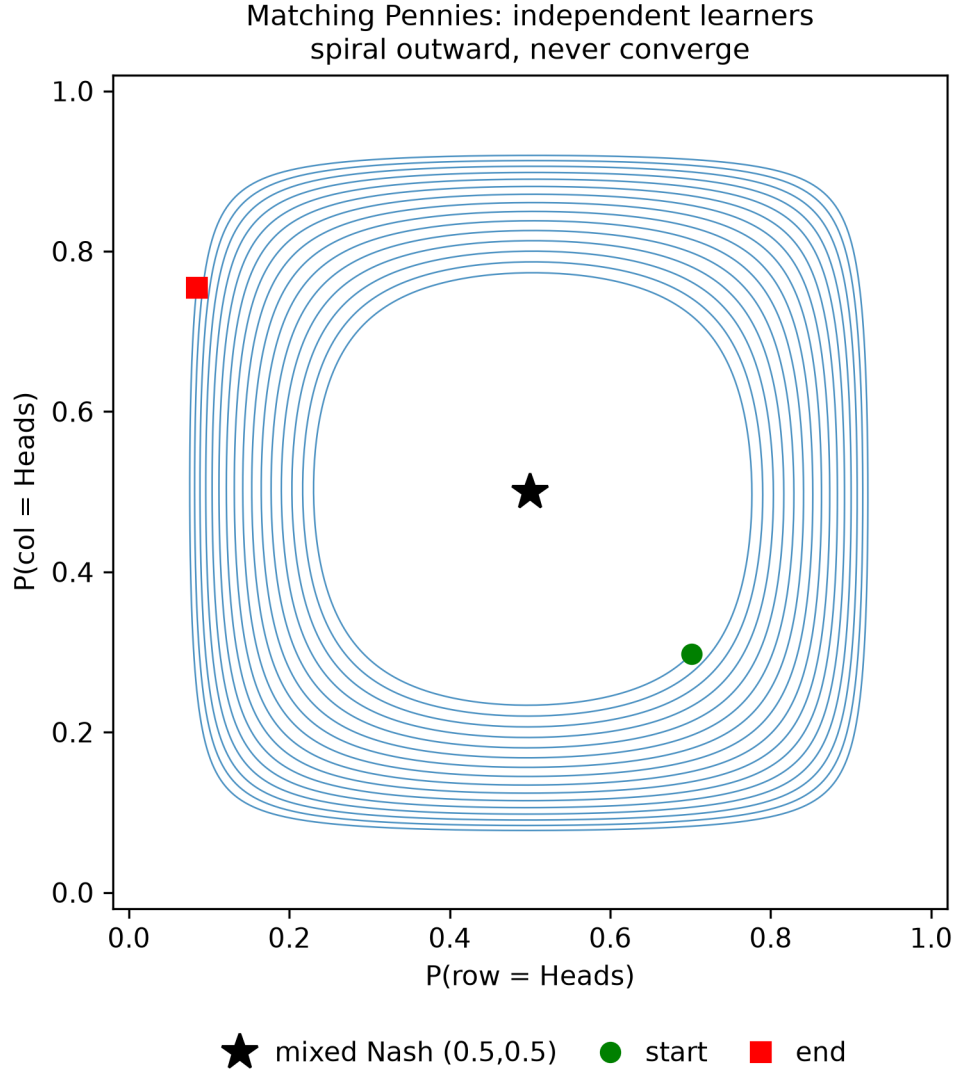


Figure 47: Matching Pennies: the distance to the mixed Nash grows across successive time-windows (0.30 to 0.48) as the trajectory spirals out to the boundary; the last iterate never converges.

9.5 Centralized training, decentralized execution (CTDE)

The first structural fix keeps execution realistic — each agent still acts on its own observation — while giving the *learner* a privileged view during training. In **MADDPG** (Lowe et al., 2017) each agent’s **actor** $\pi_i(a_i \mid o_i)$ sees only its own observation, but a **centralized critic** $Q_i(s, a_1, \dots, a_N)$ sees the global state and *every* agent’s action. A per-agent critic watching only o_i faces a non-stationary, partially-observed world, so its value target is noisy; a critic conditioned on everything sees a (near-)deterministic target and is a far lower-variance teacher. **MAPPO** (Yu et al., 2022) is the minimal version — plain PPO with one centralized value $V(\text{global state})$ — and a strong baseline, often competitive with more elaborate methods.

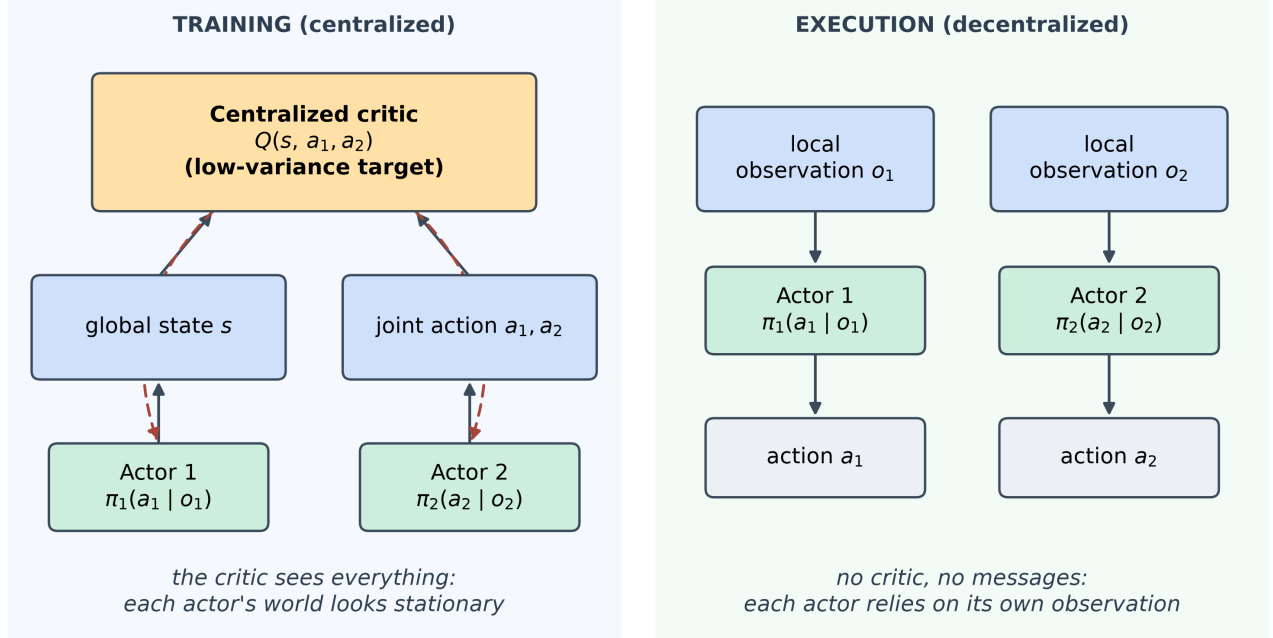


Figure 48: CTDE: at training a centralized critic sees the global state and joint action; at execution each actor acts on its own observation, with no critic and no messages.

Does the centralized critic actually have lower variance? Measured on a one-step cooperative “referential” task (a speaker sees a target, a listener does not, both must name it), and trained to convergence:

Table 27: Centralised against independent critics: the final value-loss residual.

Critic	Final residual (value loss)
centralized $Q(s, \text{joint } a)$	3.2×10^{-11}
independent $Q_i(o_i)$	0.077

The centralized critic drives its residual to essentially zero — the reward is a deterministic function of its inputs — while the independent critic cannot see the target and is stuck predicting the base rate. So the centralized critic fits its value target far more precisely. That alone does not mean lower policy-gradient variance: with converged critics a centralized critic can even increase it (Lyu et al., 2021).¹²² And a better-fitted critic is **not** the same as solving coordination, which is where a prediction broke.

¹²²Lyu, X., Xiao, Y., Daley, B. & Amato, C. (2021). “Contrasting Centralized and Decentralized Critics in Multi-Agent Reinforcement Learning.” *AAMAS*, 844–852. arXiv:2102.04402 — with converged on-policy critics, a centralized critic gives the decentralized actors higher-variance updates. Journal version: Lyu, X., Baisero, A., Xiao, Y., Daley, B. & Amato, C. (2023). *JAIR* 77, 295–354. DOI 10.1613/jair.1.14386.

Reconciliation (kept prediction → what actually happened). I predicted that on the Claus–Boutilier **climbing game**¹²³ — a stateless cooperative matrix game whose optimum (11) is flanked by −30 miscoordination penalties, with a “safe” attractor at 5 — the centralized critic would escape the trap and reach the optimum, beating independent learners. It did not. Measured greedy rewards: **independent learners 7, MADDPG 5, MAPPO 7** (optimum 11, safe 5). No method reached the optimum, and MADDPG actually *underperformed* both IL and MAPPO. (The discrete-action MADDPG variant used here replaces the deterministic policy gradient with a COMA-style counterfactual baseline (Foerster et al., 2018),¹²⁴ so the result speaks for that variant, not for MADDPG as published, and its actor update is the piece to scrutinize next.) The honest reading: a more precise critic is **not sufficient** to overcome relative over-generalization¹²⁵ plus the hard-exploration risk of the −30 penalties — the agents will not try the risky joint action long enough to discover the 11. **CTDE buys a better critic, not automatic coordination.**

A methodological note to carry forward: both CTDE effects above, and the communication benefit (Section 9.7), were **invisible at the fast “smoke” configuration** and appeared only when trained at the larger “scale” configuration. The smoke config proves the code runs; the phenomena need training to convergence.¹²⁶

¹²³Claus, C. & Boutilier, C. (1998). “The Dynamics of Reinforcement Learning in Cooperative Multiagent Systems.” *AAAI*, 746–752.

¹²⁴Foerster, J., Farquhar, G., Afouras, T., Nardelli, N. & Whiteson, S. (2018). “Counterfactual Multi-Agent Policy Gradients.” *AAAI* 32(1). DOI 10.1609/aaai.v32i1.11794 (COMA).

¹²⁵Matignon, L., Laurent, G. J. & Le Fort-Piat, N. (2012). “Independent reinforcement learners in cooperative Markov games: a survey regarding coordination problems.” *Knowledge Engineering Review* 27(1), 1–31. DOI 10.1017/S0269888912000057.

¹²⁶Lowe, R. et al. (2017). “Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments.” *NeurIPS* (MADDPG); and Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A. & Wu, Y. (2022). “The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games.” *NeurIPS Datasets and Benchmarks Track*. arXiv:2103.01955 (MAPPO).

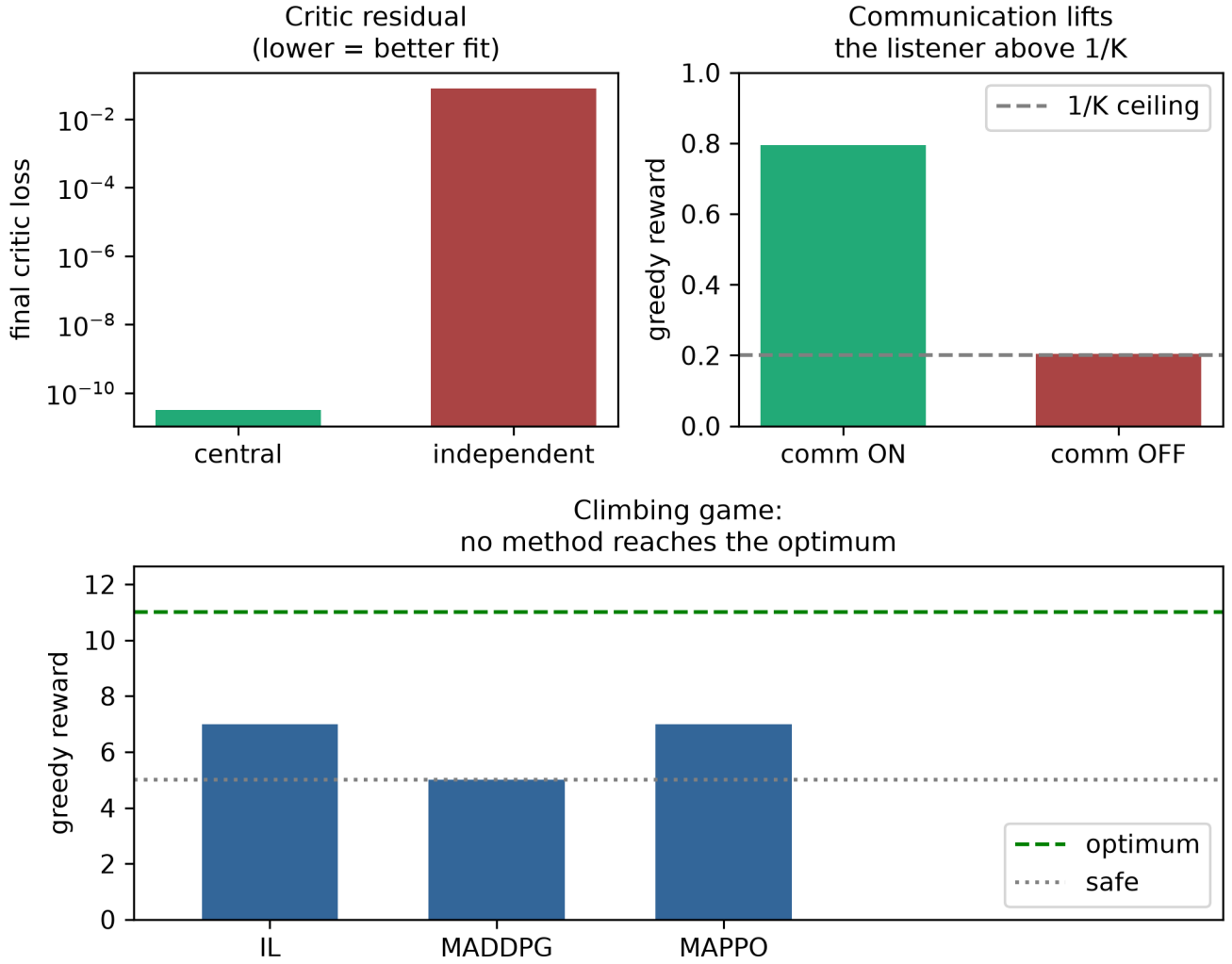


Figure 49: Cooperative CTDE and communication results (scale config): the centralized critic’s residual is orders of magnitude lower; communication lifts the listener far above $1/K$; on the climbing game no method reaches the optimum (11), and MADDPG (5) trails IL and MAPPO (7).

9.6 PSRO — game theory over a population of policies

The second structural fix is the through-line of the chapter and the direct descendant of Chapter 2’s iterated best response. **PSRO** (Policy-Space Response Oracles; Lanctot et al., 2017) treats whole policies as the atoms of a higher game. It maintains a **population** of policies per player, builds the empirical **meta-game** payoff matrix between the populations, solves a **meta-Nash** over it, trains a **best response** (the “oracle”) to the opponent’s meta-Nash mixture, and adds that response to the population — repeat. It unifies self-play, fictitious play, and the double-oracle method under one framework, and its progress metric is *exploitability* — the same NashConv used throughout Chapters 2–8.

PSRO = Chapter 2's iterated best response, lifted from actions to whole policies.

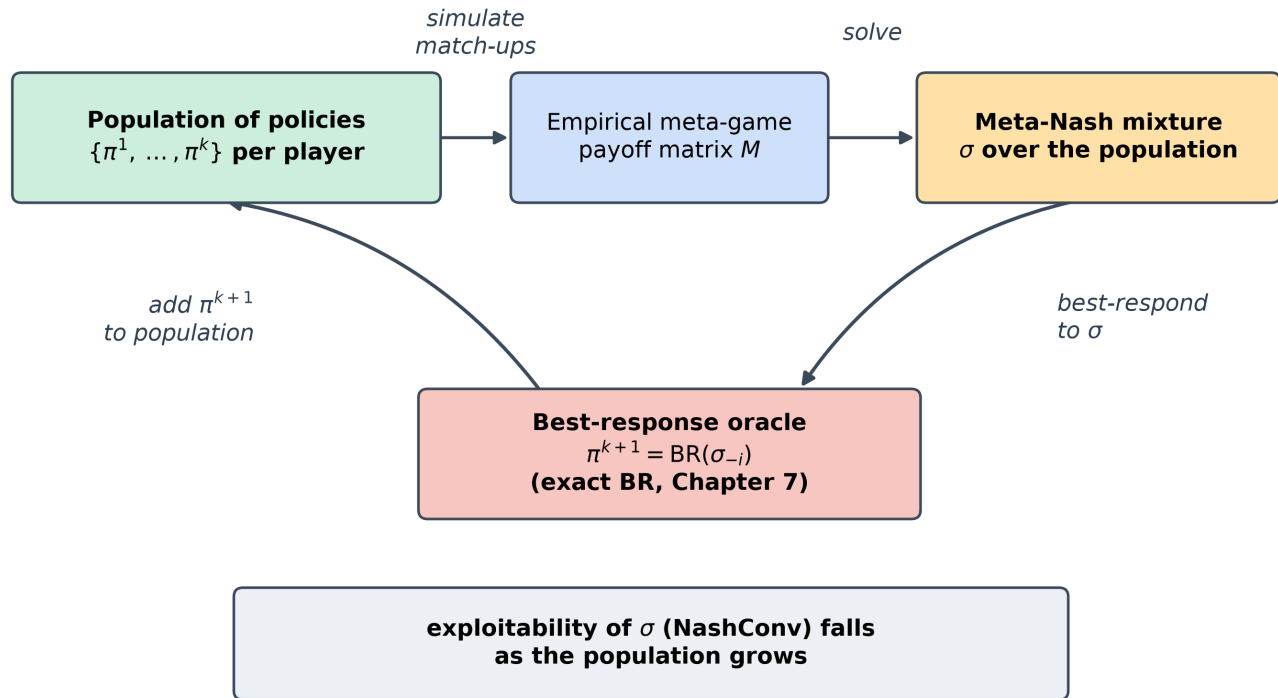


Figure 50: The PSRO double-oracle loop: build the meta-game, solve its meta-Nash, add a best response to the opponent’s mixture. Here the oracle is Chapter 7’s exact best response, so convergence uses the same exploitability yardstick.

The implementation is exact rather than approximate: the oracle is Chapter 7’s **exact best response** on Kuhn and Leduc, applied to the opponent’s meta-Nash mixture collapsed (by Kuhn’s theorem¹²⁷) to a single behavioral policy.

Why a population and not just the last self-play policy? Because self-play converges in the *average*, not the last iterate. Measured on Kuhn with fictitious-play self-play: the **average**-iterate exploitability fell from 0.24 to 0.031 over 200 iterations, while the **last**-iterate exploitability kept oscillating between 0.33 and 0.83. PSRO’s meta-mixture is the population-level version of the averaging that CFR and fictitious play rely on.

PSRO convergence, measured (exploitability = NashConv of the meta-Nash mixture in the full game):

¹²⁷Kuhn, H. W. (1953). “Extensive Games and the Problem of Information.” *Contributions to the Theory of Games II*, 193–216. DOI 10.1515/9781400881970-012.

Table 28: PSRO exploitability trajectory, per game family.

Game	Exploitability trajectory	Verdict
Kuhn Poker	$0.917 \rightarrow \sim 2 \times 10^{-16}$ by round 6	converges to machine zero
matrix (Matching Pennies)	$2.0 \rightarrow 0$ by round 2	converges
Rock–Paper–Scissors (exploration)	$2.0 \rightarrow 0.017$; population $\rightarrow$ $\{R,P,S\}$, mixture $\rightarrow$ uniform	converges
Leduc Hold'em	$4.75 \rightarrow 2.16$ over 20 rounds	decreases, but far above target
Goofspiel ($K = 3$, smoke config)	$1.33 \rightarrow 0$	converges
Goofspiel ($K = 4$)	oscillates $1.24 \leftrightarrow 2.0$ (8 rounds)	does not settle

On the small games the population quickly spans the strategies needed and exploitability collapses. Two results did not go as predicted.¹²⁸

Reconciliation 1 (Leduc). I predicted PSRO would drive Leduc exploitability below 0.5 within 20 iterations. Measured, it fell from 4.75 to 2.16 — after strong oscillation over the first eight rounds (up to 6.83) a clear decline follows, but it stays nowhere near 0.5. This is **genuine slow convergence**, not a bug: Leduc’s tree is far larger than Kuhn’s, and a population of 20 *pure* best responses is too small to closely approximate its mixed Nash. The lesson — exploitability decreases as the population grows — holds; the *rate* is the scaling wall, and it rhymes with Chapter 8’s global-vs-local scaling finding.

Reconciliation 2 (Goofspiel $K = 4$). I predicted non-increasing exploitability. At $K = 3$ (smoke config) it converged to 0 cleanly; at $K = 4$ it oscillates between ~ 1.2 and 2.0 and does not settle. This is the one result I cannot yet explain, and per the workflow I am **documenting it, not fixing it**. Suspects: the Goofspiel PSRO driver never de-duplicates best-response policies, and a pure-strategy population is likely too weak for the larger game’s mixed meta-Nash. Flagged as an open code item, not a validated result.

¹²⁸Lanctot, M. et al. (2017). “A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning.” *NeurIPS* (PSRO). Related: McMahan, H. B., Gordon, G. & Blum, A. (2003). “Planning in the Presence of Cost Functions Controlled by an Adversary.” *ICML* (the double-oracle method PSRO generalizes); Tuyls, K. et al. (2020). “Bounds and dynamics for empirical game theoretic analysis.” *Autonomous Agents and Multi-Agent Systems* 34(1), art. 7. DOI 10.1007/s10458-019-09432-y (EGTA).

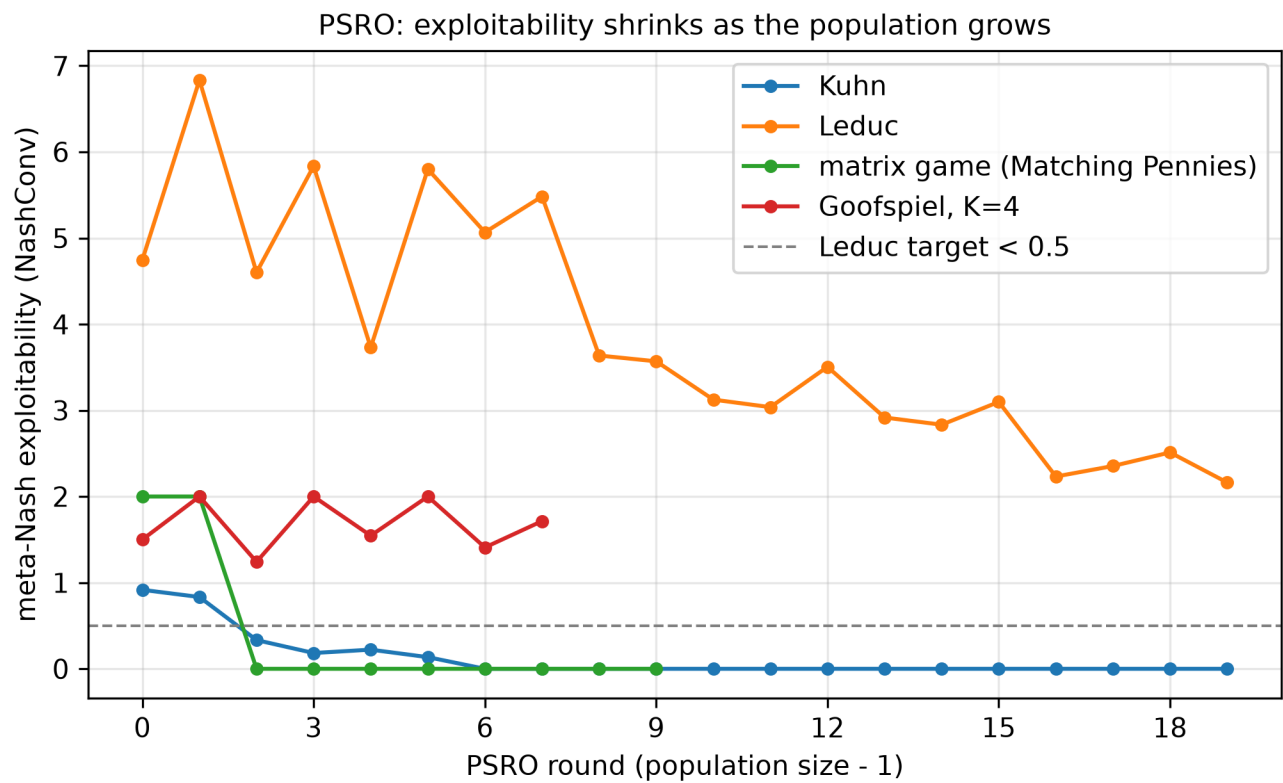


Figure 51: PSRO exploitability vs population size (scale config): Kuhn and the matrix game collapse to near zero; Leduc oscillates, then declines but stays well above 0.5; Goofspiel K=4 oscillates (a flagged anomaly).

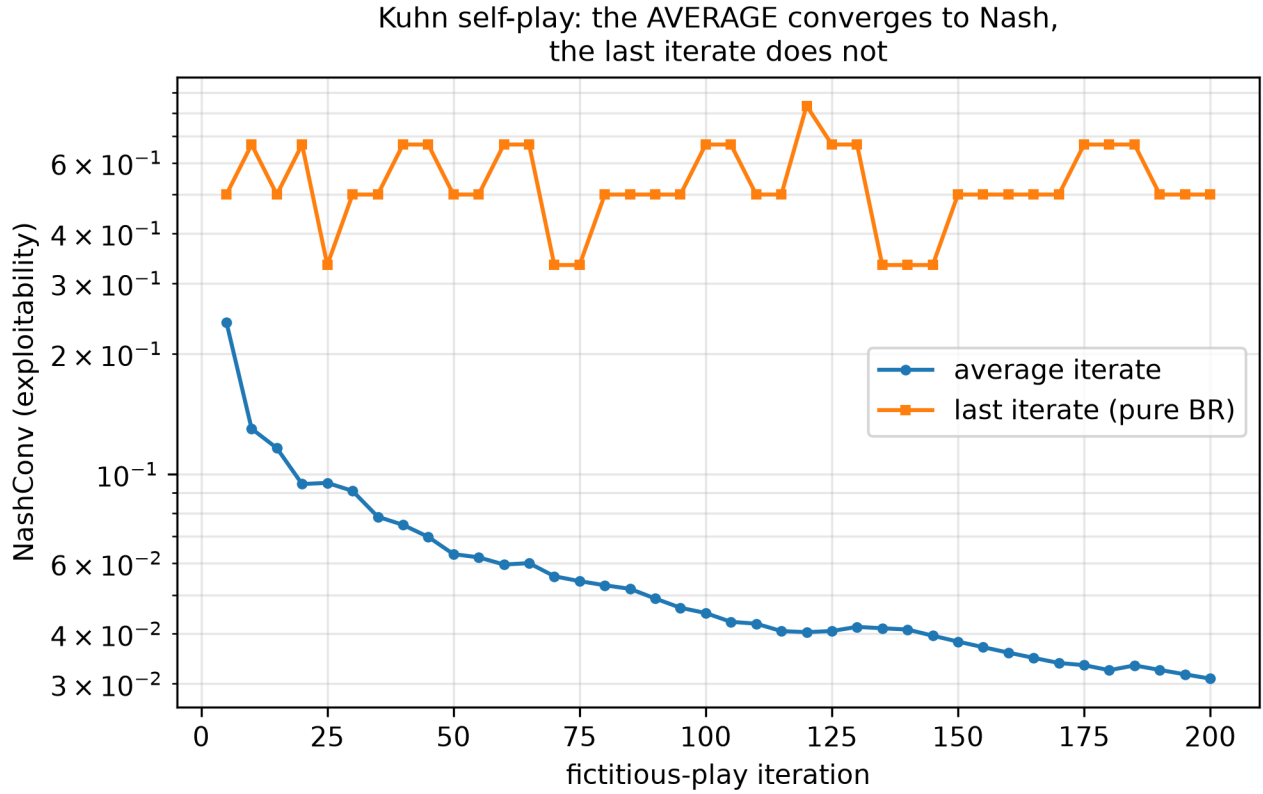


Figure 52: Self-play on Kuhn: the average-iterate exploitability falls toward zero while the last iterate keeps oscillating, which is why self-play and PSRO rely on averaging.

9.7 Learned communication (CommNet)

CTDE centralizes information at *training*; communication centralizes it at *execution*, through a channel the agents **learn**. In **CommNet** (Sukhbaatar et al., 2016) each agent encodes its observation, emits a **message**, and receives the **mean** of the others’ messages as extra policy input. Trained end-to-end, the protocol emerges: on the referential task, the speaker (who alone sees the target) must learn to encode it and the listener to decode it.

The test is sharp because without a channel the listener is capped at pure guessing, $1/K$. Measured (scale config, $K = 5$, so the guessing ceiling is 0.2):

Table 29: Learned communication: team reward with the channel on and off.

Channel	Greedy team reward
communication ON	0.795
communication OFF	0.204

With the channel the listener climbs well above the $1/K$ ceiling; without it, it sits exactly at the ceiling. Communication is doing real work — and the Section 9.5 note applies: at the smoke configuration both numbers were 0.24, and the benefit appeared only at scale.¹²⁹

9.8 LOLA — modeling the opponent as a learner

Every method so far treats the opponent’s strategy as fixed while you respond. **LOLA** (Learning with Opponent-Learning Awareness; Foerster et al., 2018) is the exception, and the one most directly connected to the thesis. Each agent optimizes against the policy the opponent will hold *after one learning step*, and differentiates *through* that step. The extra term is a mixed second derivative — how the opponent’s update depends on *my* parameters — and it is what turns self-interested agents cooperative.

The classic demonstration is the memory-1 **Iterated Prisoner’s Dilemma** (five cooperation probabilities per agent; closed-form discounted return). Naive gradient learners, each maximizing its own return against the other’s current policy, converge to **mutual defection**; LOLA learners, each accounting for the other’s upcoming update, reach **mutual cooperation**. Measured (per-step discounted return; full cooperation ≈ 3 , mutual defection ≈ 1):

Table 30: LOLA against naive learners on the iterated Prisoner’s Dilemma.

Learners	Return
naive vs naive	1.04
LOLA vs LOLA	2.82

Cooperation emerges where naive learning defects (my prediction of ≈ 3 was slightly high; 2.82 is near-cooperation). A built-in check confirms the mechanism: with the look-ahead learning rate set to zero, LOLA’s gradient reduces exactly to the naive gradient, so the cooperation comes from the second-order look-ahead term.

Conceptually this is **dynamic** opponent modeling: Chapter 7 inferred an opponent’s *current* strategy; LOLA anticipates their *learning trajectory*. Combining the two — a static read that seeds a dynamic look-ahead — is a candidate for Contribution #1, not a solved thing.¹³⁰

¹²⁹Sukhbaatar, S., Szlam, A. & Fergus, R. (2016). “Learning Multiagent Communication with Backpropagation.” *NeurIPS* (CommNet).

¹³⁰Foerster, J., Chen, R. Y., Al-Shedivat, M., Whiteson, S., Abbeel, P. & Mordatch, I. (2018). “Learning with Opponent-Learning Awareness.” *AAMAS*, 122–130. arXiv:1709.04326 (LOLA).

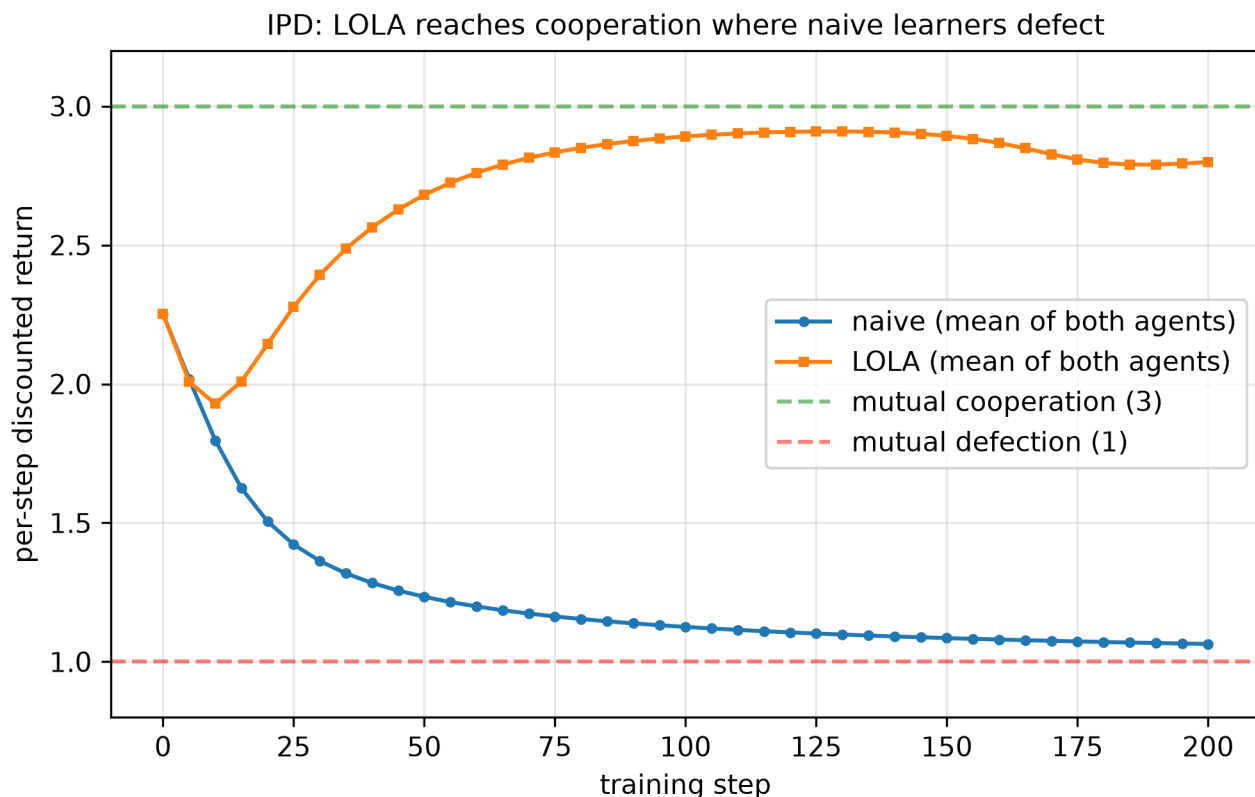


Figure 53: LOLA vs naive learners on the Iterated Prisoner’s Dilemma: naive return collapses toward mutual defection (~ 1), while LOLA climbs toward mutual cooperation (~ 2.8).

9.9 Honest notes, limitations, and where this hands off

What did not go as predicted. (1) Matching Pennies at a fixed step spirals to the boundary rather than orbiting. (2) PSRO on Leduc stalls at ~ 2.16 after 20 rounds, not < 0.5 — the pure-population scaling wall. (3) Goofspiel $K = 4$ oscillates — an unexplained code anomaly, documented, not fixed. (4) On the climbing game no method reached the optimum and MADDPG trailed IL. The two code items (Goofspiel $K = 4$; MADDPG’s counterfactual baseline) must be investigated before those pieces are reused, and the neural claims rest on the scale numbers, not the smoke ones.

Trust. Every equilibrium target is *exact* (analytic Nash for the matrix games; Chapter 7’s exact best response and NashConv for Kuhn/Leduc/Goofspiel), so the game-theoretic results are bounded by ground truth. The neural results are qualitative inequalities (central $<$ independent; comm ON $>$ comm OFF), seed- and version-sensitive by construction. The figures are generated from the committed JSON results (see `../figures/README.md`).

Backward and forward connections. PSRO is Chapter 2’s iterated best response lifted to a population and reuses Chapter 7’s exact best-response engine; the Leduc wall echoes Chapter 8. Forward, the three hooks are concrete: LOLA as dynamic opponent modeling (Contribution #1), PSRO’s meta-game as

an evaluation methodology (Contribution #3), and, above all, the **missing $N > 2$ minimax anchor** (Contribution #2).

9.10 Key takeaways for the thesis synthesis

- **Non-stationarity is *the* problem**, and it is structural, not a compute-budget issue — in Matching Pennies the last iterate does not converge no matter how long you train.
- **CTDE buys a better critic, not coordination**: residual 3.2×10^{-11} vs 0.077, yet the climbing game stayed unsolved.
- **PSRO is the game-theory $\leftrightarrow$ MARL bridge**: Kuhn to machine-zero exploitability, RPS to uniform; Leduc declines but hits the scaling wall that motivates the thesis’s scalable methods.
- **Self-play/PSRO succeed in the average/population, not the last iterate** (Kuhn average NashConv $0.24 \rightarrow 0.031$ while the last iterate oscillates) — the reason a population exists.
- **LOLA reframes the opponent as a learner** and turns IPD defection into cooperation ($1.04 \rightarrow 2.82$) — the dynamic complement to Chapter 7’s static opponent model (Contribution #1).
- **The $N > 2$ minimax gap is the open door to the thesis** (Contribution #2): the vocabulary of Chapters 2–8 crosses into MARL, but the safety anchor does not.

10 Chapter 10 — Population-Based Training and Evolutionary Game Theory

This is a ground-up chapter on training and evaluating **populations** of agents: the evolutionary mathematics that says whether a population can settle at all, the transitive/cyclic structure that decides whether self-training converges or spins, and a small AlphaStar-style **league** built on a solvable poker game. **All experimental numbers reported here were measured** on reproducible runs and, wherever possible, are bounded by *exact* references (analytic ESS/Nash for the matrix games; Chapter 7’s exact best response for Leduc). Where a run contradicted my expectation, I keep the expectation and reconcile it with what happened.

Where this sits in the thesis. Chapters 2-8 solved *one* fixed game; Chapter 9 added a *second* learner; Chapter 10 goes to a *whole population*. Three thesis hooks: **main exploiters** are automated opponent modelers (Contribution #1); the **AlphaStar league** uses exploiters to make its main agents robust — close to population-level safe exploitation, but its robustness is supported only empirically, with no formal guarantee¹³¹ — exactly where Contribution #2 lives; and **EGTA / meta-Nash** is the population evaluation methodology (Contribution #3).

10.1 Why populations — and why structure decides everything

Self-play has a famous failure mode: a player that only ever trains against its current self can go around in circles. If the game is **rock-paper-scissors**, “get better” has no meaning — Rock beats Scissors, loses to Paper, and improving against one opponent just makes you worse against another. The single most important idea in this chapter is that games have two kinds of structure mixed together: a **transitive** part (a genuine skill ladder — there *is* a better player) and a **cyclic** part (a wheel of counters — there is only *what beats what*). Whether a self-training population converges to something good or spins forever is decided by *which part dominates*, not by the learning rate.

A picture to hold onto: a **dojo**. The **main** students are what you are trying to produce. You keep a room full of **sparring partners** whose entire job is to find and punish a specific weakness in the current students (exploiters). And you keep a **museum of past champions** — frozen snapshots — so nobody wins by forgetting how to beat an old style. Training is then a loop: students spar, the strong ones get copied and slightly mutated (population-based training), and periodically a copy of everyone is frozen into the museum. The league is that dojo, made mathematical.¹³¹

10.2 The family of methods

¹³¹Vinyals, O. et al. (2019). “Grandmaster level in StarCraft II using multi-agent reinforcement learning.” *Nature* 575(7782), 350–354. DOI 10.1038/s41586-019-1724-z (AlphaStar; the league and PFSP).

Table 31: The population-based method families: what each does, when to reach for it, and its main weakness.

Approach	What it does	Reach for it when	Main weakness
Self-play	train the latest policy against itself	transitive games; a quick baseline	cycles in cyclic games; last iterate does not converge
PSRO	population + meta-Nash + best-response oracle; add the BR, repeat	want a game-theoretic target	a full best response per round; the <i>mixture</i> is not the answer (see §10.7)
PBT	copy the fitter agents (exploit) + perturb hyper-parameters (explore)	online hyper-parameter search	can collapse diversity or regress (see §10.5)
AlphaStar league	PBT with main / main-exploiter / league-exploiter types + freezing + PFSP	non-transitive games where self-play cycles	much compute; heuristic, no safety guarantee

Two diagnostics cut across them: **replicator dynamics** (§10.3) say where a population can *rest*, and the **spinning-top decomposition** (§10.4) measures how transitive vs cyclic it is — together they predict, *before* you spend the compute, whether the league in §10.5 will settle or spin. The line runs PBT¹³² and PSRO¹³³ (2017) → AlphaStar’s league, Grandmaster in StarCraft II, above 99.8% of ranked human players (2019)¹³¹ → the spinning top (2020);¹³⁴ such populations are evaluated with EGTA.^{135 136}

10.3 Replicator dynamics — where a population can rest

The **replicator equation** is the simplest model of selection: a strategy’s share grows in proportion to how much it beats the *population average*,

¹³²Jaderberg, M., Dalibard, V., Osindero, S., Czarnecki, W. M. et al. (2017). “Population Based Training of Neural Networks.” *arXiv:1711.09846*.

¹³³Lanctot, M. et al. (2017). “A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning.” *NeurIPS* (PSRO). See also: McMahan, H. B., Gordon, G. & Blum, A. (2003). “Planning in the Presence of Cost Functions Controlled by an Adversary.” *ICML*, 536–543 (the double-oracle method PSRO generalizes).

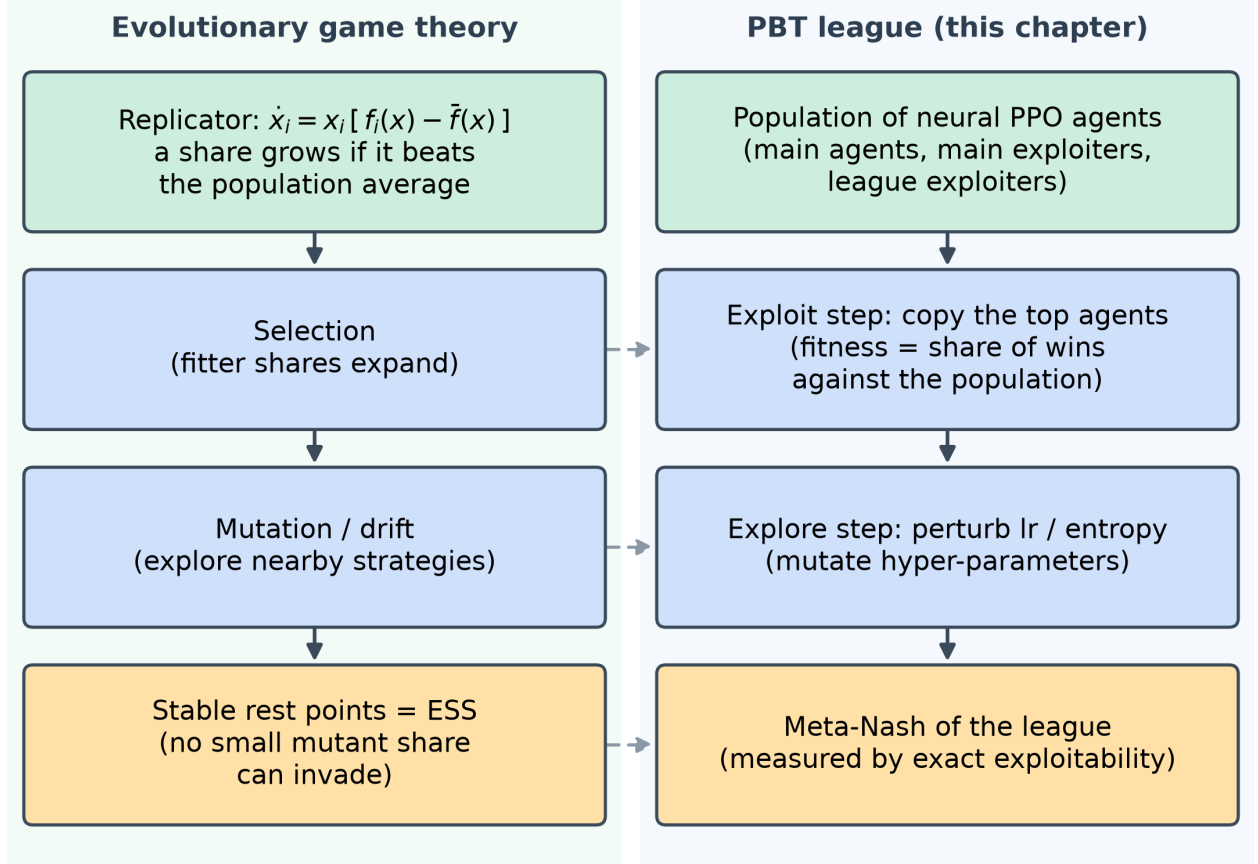
¹³⁴Czarnecki, W. M., Gidel, G., Tracey, B., Tuyls, K., Omidshafiei, S., Balduzzi, D. & Jaderberg, M. (2020). “Real World Games Look Like Spinning Tops.” *NeurIPS*; arXiv:2004.09468 — the spinning-top geometry and why training needs populations.

¹³⁵Wellman, M. P. (2006). “Methods for Empirical Game-Theoretic Analysis.” *AAAI*, 1552–1555.

¹³⁶Tuyls, K., Pérolat, J., Lanctot, M., Hughes, E., Everett, R., Leibo, J. Z., Szepesvári, C. & Graepel, T. (2020). “Bounds and dynamics for empirical game theoretic analysis.” *Autonomous Agents and Multi-Agent Systems* 34(1), 7. DOI 10.1007/s10458-019-09432-y.

$$\dot{x}_i = x_i [f_i(x) - \bar{f}(x)], \quad \bar{f}(x) = \sum_j x_j f_j(x).$$

Every Nash equilibrium is a rest point, but not conversely (every pure strategy is one too); stable rest points are Nash equilibria, and every **evolutionarily stable strategy (ESS)** — one that, adopted by the whole population, cannot be invaded by a small mutant share — is asymptotically stable.¹³⁷ A PBT league does the same discretely: copy the fitter agents (selection) and perturb them (mutation).



dashed = "discrete analog of": the league is replicator dynamics with learned agents instead of fixed strategy shares

Figure 54: Evolutionary game theory as the continuous idealization of a PBT league: replicator selection ↔ copying the fitter agents, mutation ↔ perturbing hyper-parameters, stable rest points (ESS) ↔ the meta-Nash mixture the league seeks.

On four canonical symmetric games (deterministic dynamics), the measured results match theory exactly:

¹³⁷Hofbauer, J. & Sigmund, K. (2003). "Evolutionary game dynamics." *Bulletin of the American Mathematical Society* 40(4), 479–519 — the "folk theorem" of replicator dynamics (§2.3), asymptotic stability of an ESS (§2.6), and the closed orbits of rock-paper-scissors (Theorem 2).

Table 32: Replicator dynamics on four symmetric games, against the analytic reference.

Game	Analytic reference	Measured final	Converged?
Prisoner’s Dilemma	Defect dominates; ESS $(0, 1)$	$[0.0, 1.0]$	yes
Hawk-Dove	interior ESS $p(\text{Hawk}) = V/C = 0.5$	$[0.5, 0.5]$ (orbit radius 0.0)	yes
Rock-Paper-Scissors	Nash = uniform; no ESS	orbits centre, radius 0.095	no
Stag Hunt	two pure ESS (basin-dependent)	$[0.8, 0.2] \rightarrow [1, 0];$ $[0.2, 0.8] \rightarrow [0, 1]$	yes

Rock-Paper-Scissors never converges — its interior fixed point is a *centre*, so the population orbits it forever (closed in continuous time; explicit Euler turns them into a slowly widening spiral). That last row is the whole motivation for the rest of the chapter: a cyclic game has no stable population.¹³⁸

¹³⁸Hofbauer, J. & Sigmund, K. (1998). *Evolutionary Games and Population Dynamics* (Cambridge) — replicator dynamics, ESS, and the RPS centre; see also the Bloembergen, Tuyls, Hennes & Kaisers survey (JAIR 53, 659–697, 2015) connecting replicator dynamics to multi-agent learning.

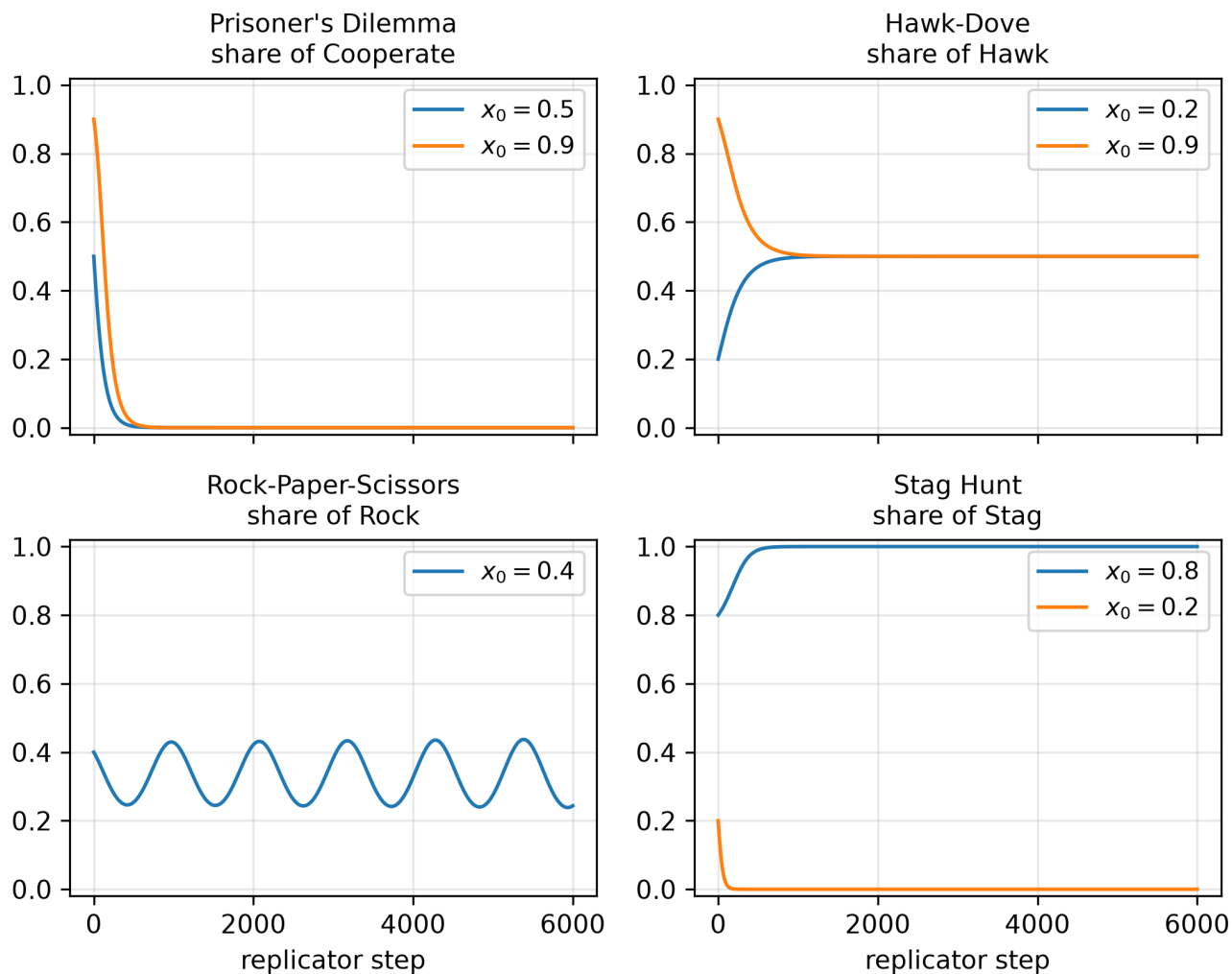


Figure 55: Replicator trajectories (share of the first strategy): Prisoner’s Dilemma → all-Defect, Hawk-Dove → the 0.5 ESS, Rock-Paper-Scissors → an orbit that never converges, Stag Hunt → all-Stag or all-Hare by start.

10.4 The spinning top — transitive vs cyclic structure

If cyclic games are the problem, we need to *measure* how cyclic a game is. The **spinning-top decomposition** splits a payoff matrix into a **transitive** component (a skill ladder, captured by per-strategy ratings) and a **cyclic** remainder (the rock-paper-scissors part).¹³⁹ The name is the shape Czarnecki et al. found in real games: widest (most cyclic) at intermediate strength, narrow towards the extremes of skill.¹³⁴

¹³⁹Balduzzi, D., Tuyls, K., Pérolat, J. & Graepel, T. (2018). “Re-evaluating Evaluation.” *NeurIPS*; arXiv:1806.02643 — the Hodge decomposition of an antisymmetric payoff matrix into transitive and cyclic components; Nash averaging.

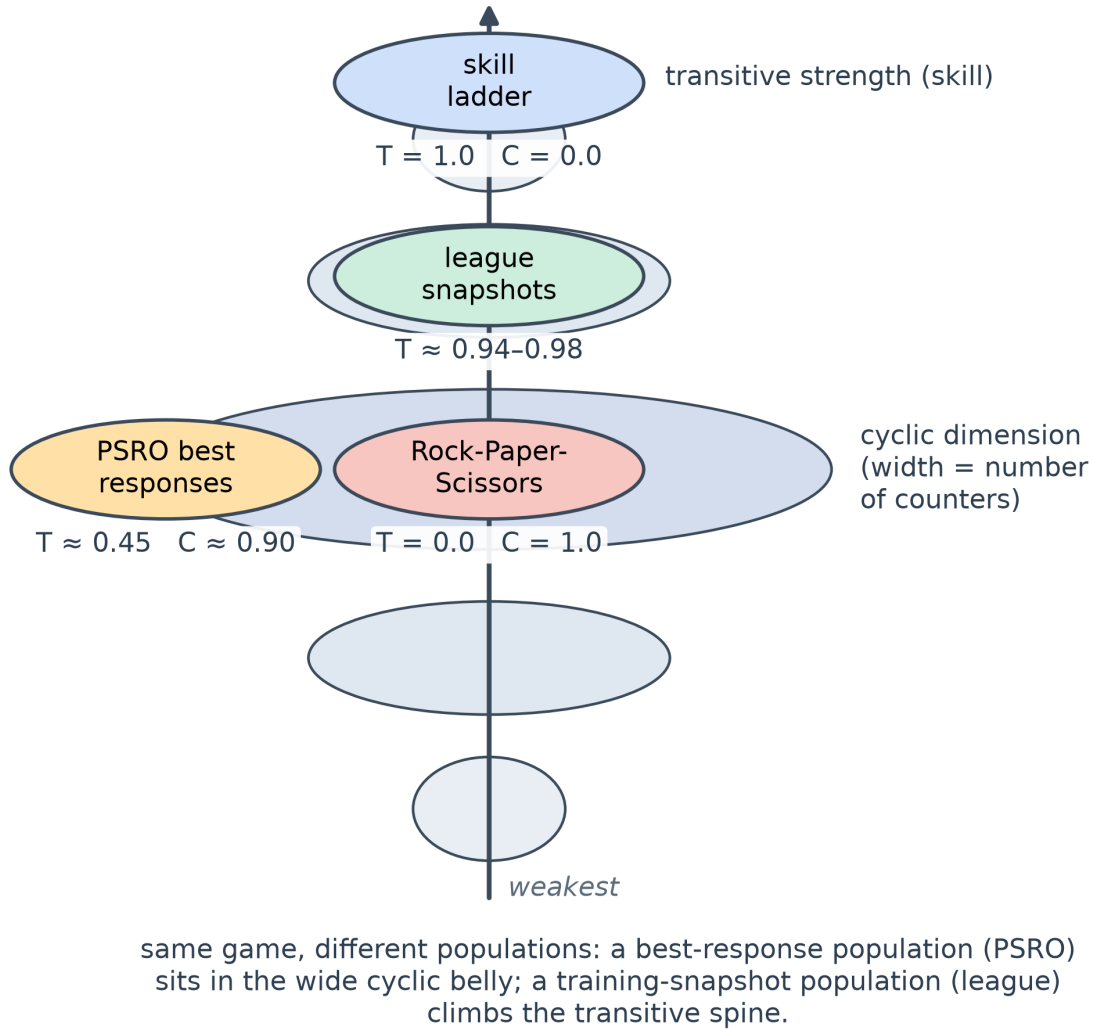


Figure 56: The spinning top (schematic): transitive skill on the vertical axis, cyclic width across. RPS sits in the widest belly ($T = 0.0$), a skill ladder on the spine ($T = 1.0$); PSRO-Leduc ~ 0.45 , league snapshots $\sim 0.94-0.98$.

The implementation uses the **combinatorial-Hodge** (ratings-difference) method,¹³⁹ not the SVD rank-1 decomposition the original plan suggested, which wrongly reports Rock-Paper-Scissors — and a pure skill ladder — as ≈ 0.707 transitive (the singular values of an antisymmetric matrix come in equal pairs). Hodge gives RPS 0.0 and the ladder 1.0. The two *real* populations are the headline — and they disagree, on the same game.¹⁴⁰

Reconciliation (kept prediction $\rightarrow$ what actually happened). I framed poker as a skill ladder and expected Leduc’s meta-game to be mostly transitive. Measured, it depends entirely on *which population you decompose*. A population of **best responses** (PSRO) is

¹⁴⁰Balduzzi, D. et al. (2019). “Open-ended Learning in Symmetric Zero-sum Games.” *ICML* — the transitive/cyclic decomposition of functional-form games; PSRO_{rN}.

mostly **cyclic** (≈ 0.45 transitive, cyclic 0.89-0.91, 27 three-cycles): the best response beats the current mixture, a newer best response beats *that*, and so on — Czarnecki et al.’s spinning top in action. A population of **training-trajectory snapshots** (the league) is mostly **transitive** (≈ 0.94 -0.98), because later snapshots are usually stronger than earlier ones. Neither is a bug; the transitive/cyclic ratio is a property of the *population*, and choosing how you build the population is choosing whether you see a wheel or a ladder.

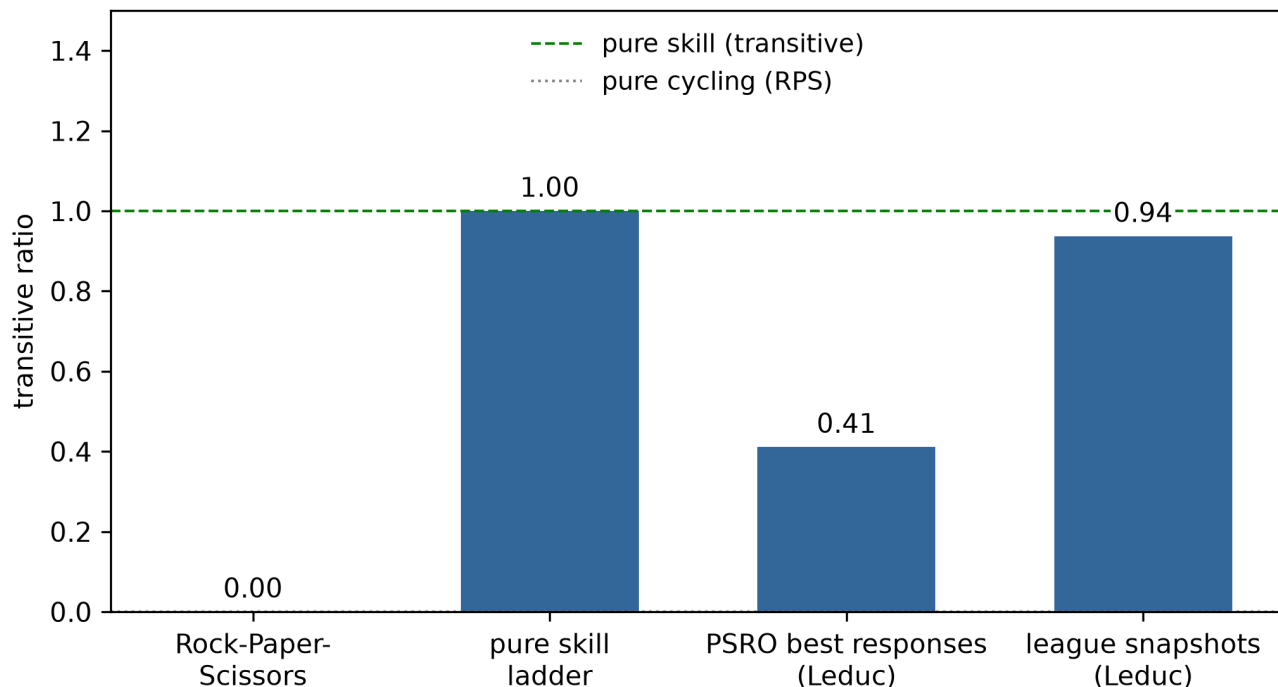


Figure 57: Transitive ratio of four populations: RPS 0.0, a pure skill ladder 1.0, the PSRO-Leduc best-response meta-game ~ 0.41 -0.46 (mostly cyclic), the league snapshot meta-game ~ 0.94 -0.98 (mostly transitive).

10.5 The AlphaStar-style PBT league

With the diagnostic in hand, we build the dojo on **Leduc Hold'em**: neural PPO agents of three types — **main** agents (the product; they play everyone via PFSP self-play), **main exploiters** (hunt weaknesses in the *current* mains) and **league exploiters** (hunt weaknesses in the *frozen history*). PBT copies the top agents and perturbs their learning rate / entropy; frozen snapshots periodically join the museum; **PFSP** (prioritized fictitious self-play) samples opponents more often the harder they are to beat. Every network is extracted to a **tabular** policy so Chapter 7’s **exact** best response measures its exploitability — the NashConv used since Chapter 2 (the sum of the two best-response gains; OpenSpiel’s exploitability is half of it).

PFSP matchmaking: each opponent is sampled with a probability that rises with how hard it is to beat (training focuses where the agent is losing).

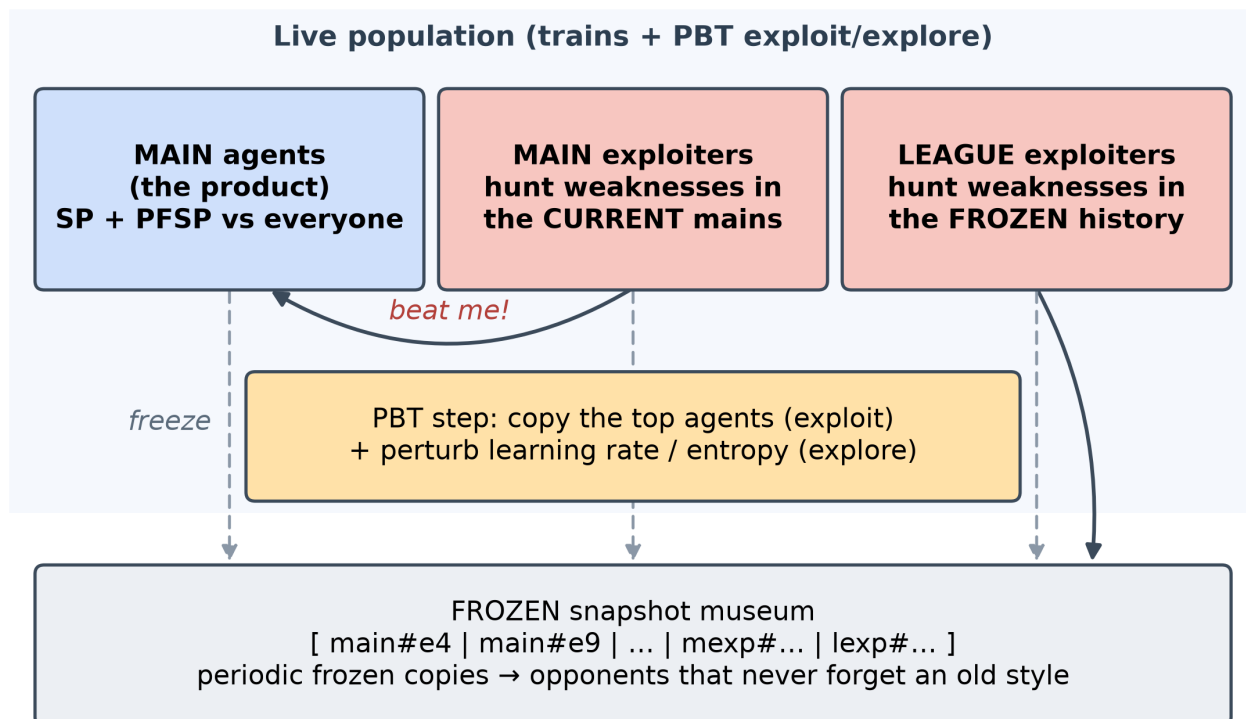


Figure 58: The league: main agents, main exploiters (target the current mains) and league exploiters (target the frozen history), with periodic freezing into a snapshot museum and PFSP matchmaking toward opponents the agent loses to.

Does it improve? Measured, over two configs (smoke: 7 agents, 15 epochs; scale: 8 agents, 120 epochs, 48 frozen snapshots):

Table 33: League training metrics at both scales.

Metric	Smoke	Scale
min-main exploitability	4.67 → 3.04 (ends at min)	4.73 → min ≈ 1.21 → 2.05
meta-Nash exploitability	4.73 → 3.04	5.01 → min ≈ 1.32 (plateau 1.60) → 2.96
final Elo (live agents)	1176-1211	1198-1210

(The 2.96 is the meta-Nash of the population at the start of epoch 119, before its last training step; the 3.418 in §10.7 is the final 56-agent population.)

Reconciliation (kept prediction → what actually happened). I predicted a *monotone*

decrease in exploitability. Smoke’s 15 epochs oblige; scale’s 120 do not: exploitability falls steeply within ~ 15 epochs, is lowest around epochs 20–40 (meta-Nash ≈ 1.32 at epoch 21; best snapshot from epoch 29; the lowest min-main value, ≈ 1.21 , at epoch 66), holds a ≈ 1.60 meta-Nash plateau, and then **regresses** to ≈ 2.05 / ≈ 2.96 by epoch 119. The best agents are the *frozen snapshots* from the first half; the *live* mains get worse late (churn / partial forgetting). Self-play, which has no exploiters, also degrades after epoch 100, so exploiter pressure is not the only explanation. The remedy (untested here) is best-snapshot retention / population regularization. As in Chapter 9: **scale reveals what smoke hides**.

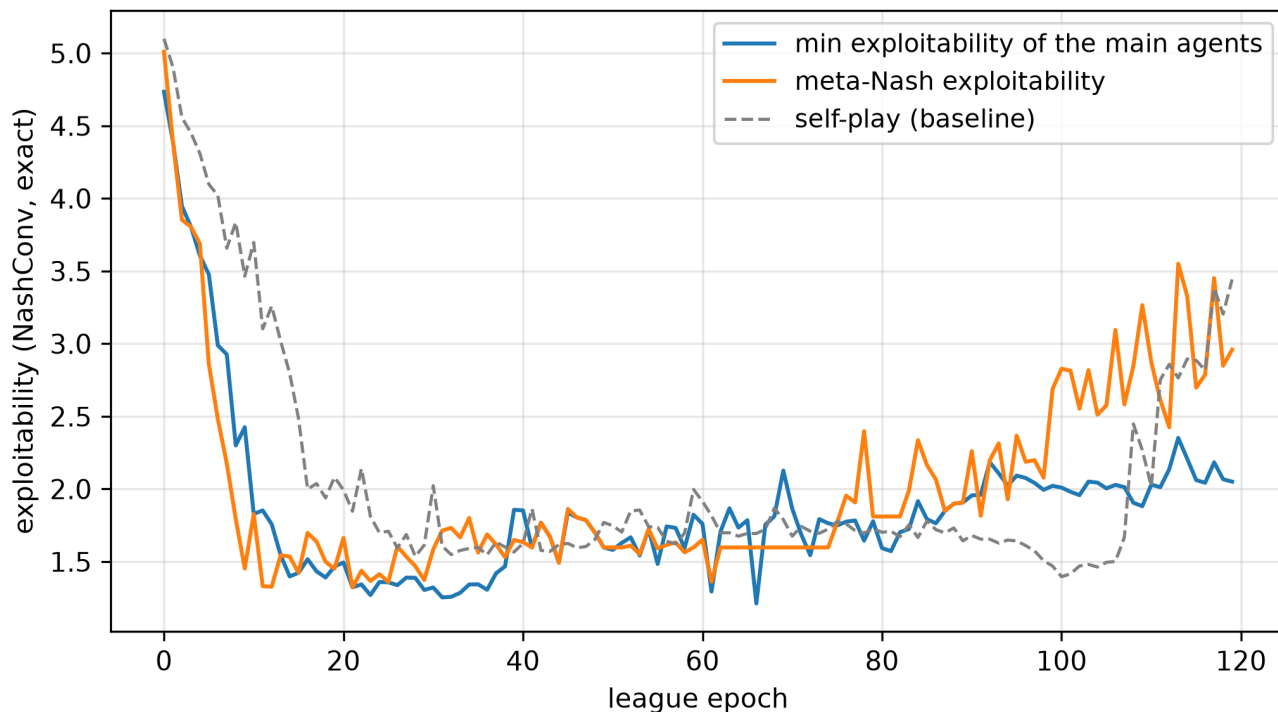


Figure 59: League exploitability (NashConv) over 120 epochs (scale): min-main and meta-Nash fall steeply within ~ 15 epochs, are lowest around epochs 20–40 and rise late. Dashed grey: self-play, which also degrades after epoch 100.

10.6 Diversity — is the population actually diverse?

A league is only as good as the *variety* of strategies it holds. Measured by the **effective population size** (participation ratio of the meta-Nash weights), **behavioral clustering** and **exploit coverage**, the population is only weakly diverse: the participation ratio rises from 1.0 (smoke) to 1.9 (scale), but behavioral clustering finds a **single cluster** at both scales — even at scale, where the maximum pairwise behavioral distance (0.48) exceeds the clustering threshold (0.30), single-linkage merges the chain. Diversity here is *weight-level*, not *behavior-level* (the agents play near-identically). The mechanism is clearest on a mini-PBT over matrix games:

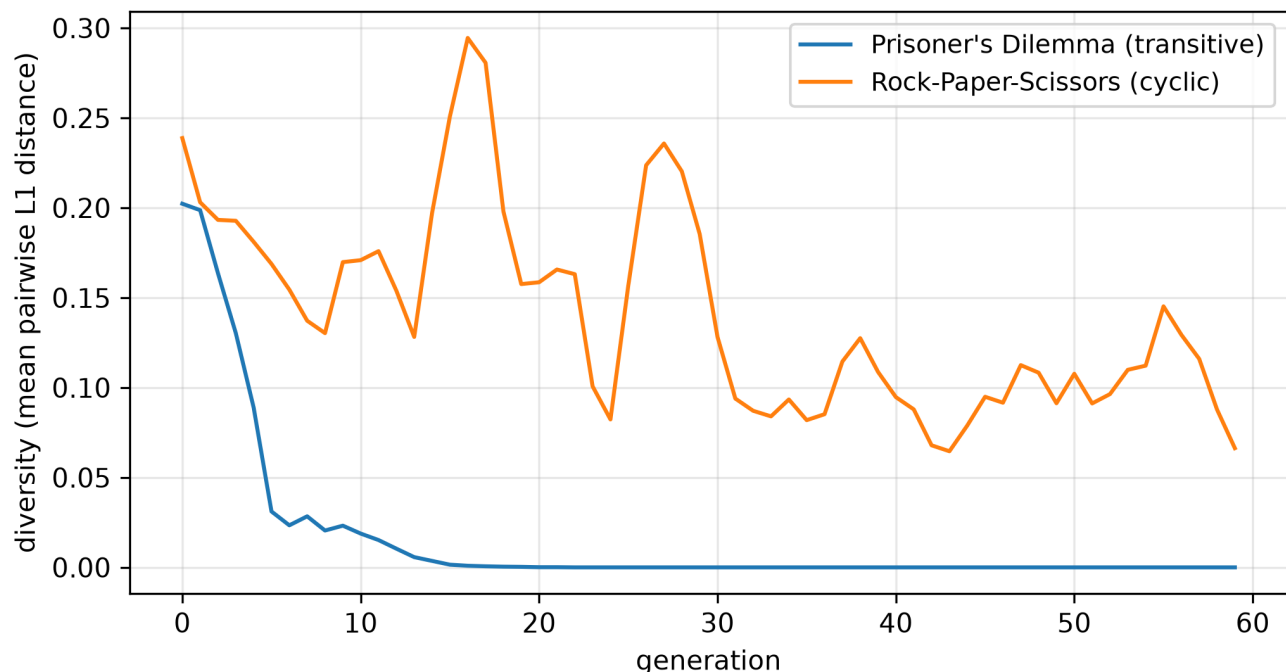


Figure 60: Mini-PBT diversity over generations: on the transitive Prisoner’s Dilemma the population collapses to one strategy (diversity $\rightarrow 0$); on cyclic Rock-Paper-Scissors it churns indefinitely (0.07-0.29).

Transitive games *kill* diversity (there is one best, everyone converges to it); cyclic games *force* it. The Leduc league sits near the transitive end, which is exactly why its diversity is thin — and why the diversity benefits reported for AlphaStar, whose league created almost 900 distinct players, do not materialize at this scale.¹³¹

10.7 EGTA — evaluating the population, and a surprise

The last tool is **empirical game-theoretic analysis (EGTA)**: treat whole policies as the “strategies” of a higher game, play every pair to fill an empirical payoff matrix, solve its **meta-Nash** mixture, and score it — the population lift of exploitability.

EGTA = Nash of a game whose "strategies" are whole policies — the population lift of exploitability.

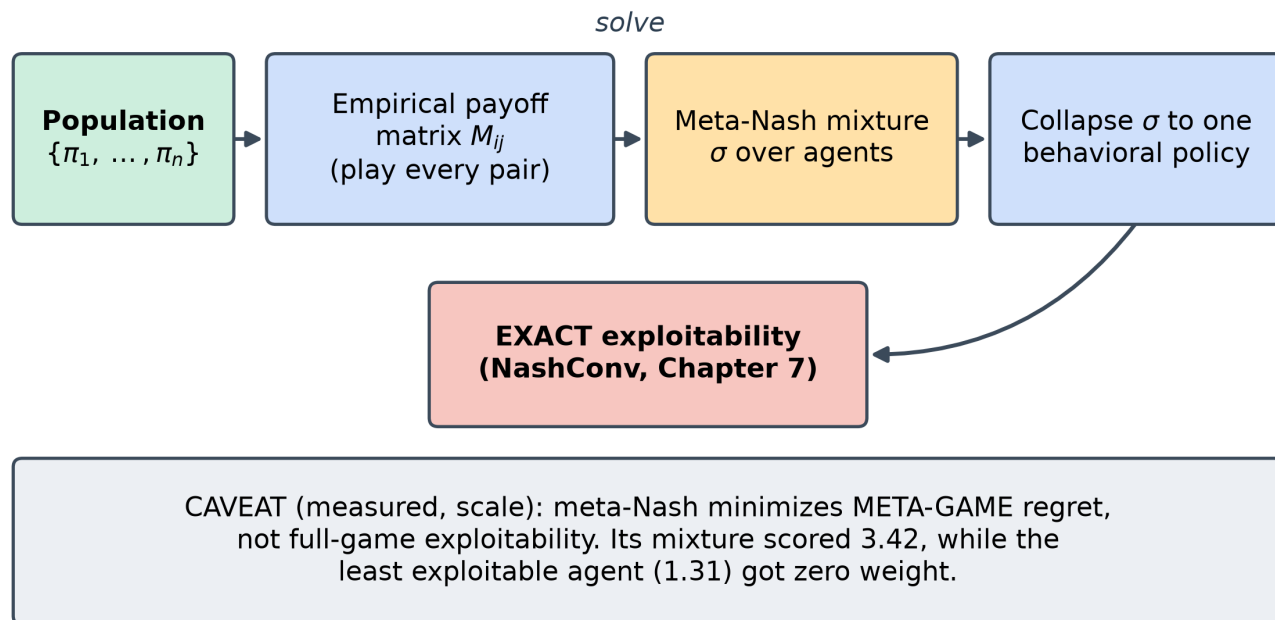


Figure 61: The EGTA pipeline: play every pair of agents, solve the meta-Nash of the empirical payoff matrix, collapse the mixture to one behavioral policy, and measure its exact full-game exploitability.

The original plan expected the league’s meta-Nash to be *less* exploitable than any single member. Measured:

Table 34: Meta-Nash exploitability against the best individual agent, per configuration.

Config	meta-Nash exploitability	best individual	meta-Nash $\leq$ best?
Smoke	2.665	2.665	yes (all weight on the best agent)
Scale	3.418	1.305	no

Reconciliation (kept prediction → what actually happened). Smoke confirmed the expectation trivially — the meta-Nash put *all* weight on the single best agent, so meta = best = 2.665. At scale the meta-Nash spreads weight (0.645 on one agent, smaller weights on two others), and the collapsed behavioral mixture scores 3.418 — **worse** than the best single snapshot (1.305). Not a mixing bug: the identical code path gave meta = best in smoke, and the mixture is *less* exploitable than each of its three components (3.558, 3.932, 4.262). The cause is the objective: the meta-Nash minimizes **meta-game regret** (doing well *against*

the population), not **full-game exploitability**, so the least exploitable agents received zero weight. A league should *ship* a selected, best-response-robust member — not the meta-Nash mixture.

How does the league compare to the alternatives on Leduc exploitability?

Table 35: Leduc exploitability (NashConv) by method; for self-play both the final agent and the best iterate are shown.

Method (scale)	Exploitability (NashConv)
CFR-Nash (the floor)	0.0099
League — best individual (best of the run)	1.305
Self-play — best iterate (epoch 100)	1.396
PSRO (exact best-response oracle)	2.163
League — meta-Nash mixture	3.418
Self-play — final agent	3.683

The league’s **best individual** is the best learned result, but its lead needs a careful reading: self-play’s best iterate is 1.396 before it degrades to 3.683, and PSRO reaches 2.163 — one run each, all far above the CFR-Nash floor. The league’s **mixture** (3.418) beats only the final self-play agent. Ship a selected member, not the mixture.¹³³

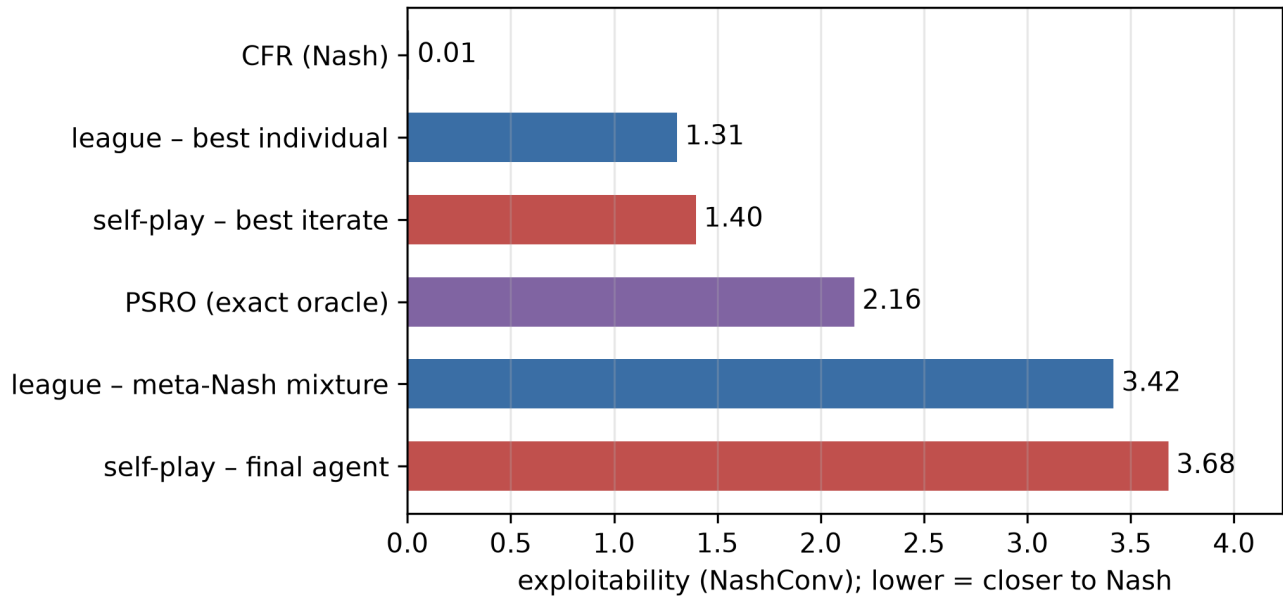


Figure 62: Leduc exploitability (NashConv) by method (scale): CFR-Nash ~0.01; the league’s best individual (1.31) is comparable to self-play’s best iterate (1.40) and below PSRO (2.16), the league mixture (3.42) and final self-play (3.68).

10.8 Honest notes, limitations, and where this hands off

What held up. Replicator dynamics reproduce every analytic outcome of the matrix games;¹³⁷ the Hodge decomposition labels the pure cases correctly; the league produces an individual agent at 1.305; and EGTA gives a working population-level exploitability.

What did not, and why it matters. Three honest caveats travel forward. (1) The league’s exploitability **regresses late** at scale ($4.73 \rightarrow \approx 1.21 \rightarrow \approx 2.05$); the best agents are frozen snapshots. (2) The **meta-Nash mixture is more exploitable than the population’s best member** at scale (3.42 vs 1.31), because it gives that member no weight — choosing by meta-Nash does not guarantee robustness. (3) Leduc’s meta-game is **cyclic as a best-response population** but **transitive as a snapshot population**. And a methodological point: the late regression and the mixture’s weakness are both invisible at the fast smoke config.

Trust. Every evolutionary target is *analytic*, and every league exploitability is Chapter 7’s *exact* Nash-Conv on tabular-extracted policies. The neural results are a single PBT run per config: the *directions* (early improvement; best snapshot < PSRO < final self-play agent; late regression; meta-Nash mixture > the population’s best member) are the trustworthy claims, not the third-decimal magnitudes.

Backward and forward connections. Backward: the league is Chapter 9’s PSRO made asynchronous with neural oracles, and it reuses Chapter 7’s exact best response wholesale; the “meta-Nash of a population” is Chapter 2’s Nash lifted one level. Forward: main exploiters are automated opponent modelers (Contribution #1); EGTA/meta-Nash is the evaluation methodology (Contribution #3); and the league’s missing guarantee — it can regress and its mixture can be exploitable — is the population form of the **missing $N > 2$ safety anchor** (Contribution #2). The transitive/cyclic diagnostic predicts Chapter 11’s FFA coalition games will be strongly cyclic, so naive PBT there will cycle.

10.9 Key takeaways for the thesis synthesis

- **Structure decides convergence.** The transitive/cyclic (spinning-top) ratio is a *pre-training diagnostic*: transitive games settle (and kill diversity), cyclic games spin (and force diversity). Measured: RPS 0.0 transitive, skill ladder 1.0, PSRO-Leduc ≈ 0.45 (cyclic), league snapshots ≈ 0.94 -0.98 (transitive).
- **The league produces strong individuals but carries no guarantee.** Best snapshot 1.305 beats PSRO 2.163 and matches self-play’s best iterate (1.396; its final agent is 3.683) — one run each — yet exploitability *regresses late* ($\rightarrow 2.05$) and the meta-Nash *mixture* (3.42) is worse than the population’s best member (1.31): meta-Nash optimizes meta-game regret, not full-game exploitability, so score the collapsed mixture directly and ship a selected member.
- **Exact evaluation is the anchor.** Grading tabular-extracted neural policies with Chapter 7’s exact best response ground-truths every population claim.

- **The population-safety gap is the open door (Contribution #2):** the AlphaStar exploiter mechanism is the population analog of Chapter 8’s safe exploitation, but it is heuristic — its robustness is shown only empirically,¹³¹ and in this chapter’s small league it guaranteed neither monotone improvement nor a non-exploitable mixture.

11 Chapter 11 — Coalition Formation and Detection

This chapter is about the thing that only exists once a game has a **third player**: temporary **alliances** that form, get exploited, and get betrayed. It uses **So Long Sucker (SLS)** — the four-player game of Hausner, Nash, Shapley & Shubik, published in 1964, whose outcome depends almost entirely on bargaining between the players¹⁴¹ — and extends the only published RL treatment of it¹⁴², whose agents are coalition-blind, with a coalition detector and a Shapley-based reward. At $N \geq 3$ **Nash and exploitability stop being tractable and stop being meaningful**, so “did it work?” can no longer be a single number. Numbers are measured and, where possible, bounded by *exact* references (textbook Shapley/core values; an exact 2-player minimax solver for the SLS endgame); where a run contradicted my expectation, including a real engine bug, the expectation is kept and reconciled.

Where this sits in the thesis. Chapter 11 removes the *exact* 2-player best-response oracle that Chapters 2-10 leaned on. The **coalition detector** lifts opponent modeling from “what kind of player is this?” to “who is allied with whom?” (Contribution #1). The **safe baseline loses its Nash anchor** — with no minimax value and an empty core there is no stable allocation for safety to rest on; a behavioral anchor in the spirit of piKL is a candidate without a guarantee — the gap this chapter frames but does not close (Contribution #2). And the **EGTA meta-game + Shapley credit** stand in for exploitability, which with more than two players no longer measures what an agent is guaranteed, least of all against a coalition (Contribution #3).

11.1 Why the third player changes everything

With two players, a zero-sum game has a *value*: there is a minimax-optimal strategy, and “how far from optimal are you?” (exploitability) is a single, meaningful number that anchored every chapter since Chapter 2. Add a third player and two players can **gang up** on the third; the interesting question becomes “who allies with whom, for how long, and who betrays first?” Nash equilibrium in a 4-player free-for-all is both **intractable** to compute and **without guarantees**: finding one is at least as hard as in general two-player games¹⁴³, equilibria chosen independently need not form an equilibrium¹⁴⁴, and it guards only against unilateral deviations, not against the coalitions that actually decide the game¹⁴⁵.

A picture to hold onto: a **dinner party** with four guests and one house. Everyone pairs up to get things done, but the smartest move is to have quietly betrayed your partner one turn *before* they betray you. So Long Sucker makes this literal: you hold coloured chips, you place them into other players’ piles (a handshake) or capture their piles (the knife), and you are eliminated when you run out. In this chapter’s

¹⁴¹Hausner, M., Nash, J., Shapley, L. & Shubik, M. (1964). “So Long Sucker - A Four-Person Game.” In M. Shubik (Ed.), *Game Theory and Related Approaches to Social Behavior*, 359–361. New York: Wiley.

¹⁴²Sharan, M. & Adak, C. (2024). “Reinforcing Competitive Multi-Agents for Playing ‘So Long Sucker.’” *arXiv:2411.11057*.

¹⁴³Daskalakis, C., Goldberg, P. W. & Papadimitriou, C. H. (2009). “The Complexity of Computing a Nash Equilibrium.” *SIAM Journal on Computing*, 39(1), 195–259.

¹⁴⁴Brown, N. & Sandholm, T. (2019). “Superhuman AI for multiplayer poker.” *Science*, 365(6456), 885–890.

¹⁴⁵Bernheim, B. D., Peleg, B. & Whinston, M. D. (1987). “Coalition-Proof Nash Equilibria I. Concepts.” *Journal of Economic Theory* 42(1), 1–12. DOI 10.1016/0022-0531(87)90099-8.

engine the alliance is **encoded entirely in the moves**: the free, unenforceable negotiation of the real game is not modelled, which is why a detector can read the coalition straight off the move stream.

The methodological consequence is the most important idea in the chapter: **at $N \geq 3$ you trade exact evaluation for empirical evaluation**. No exploitability; instead win-rate, a coalition score from the detector, and the cyclic ratio of an empirical meta-game — all anchored by the one subgame that *is* exactly solvable, the 2-player endgame.

11.2 The coalition detector — reading alliances off the moves

Instead of inferring a hidden *type* or *hand* (Chapter 7), the detector infers a hidden *social structure*. It watches the move log and accumulates two matrices: **help** (player i placed a chip into player j 's pile) and **harm** (player i captured player j 's pile). Their difference is **net support**; the pair with the highest mutual net support is the strongest coalition.

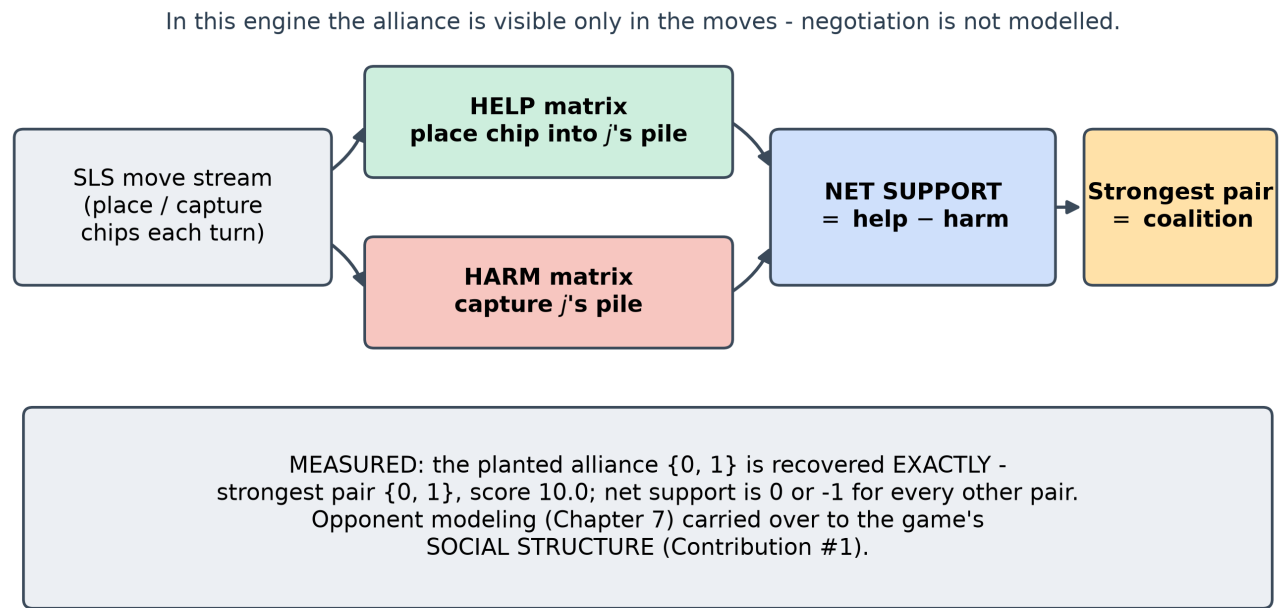


Figure 63: The coalition detector: chip placements count as HELP, captures as HARM; differenced into net support, a reciprocal alliance shows up as a strong mutual edge (Contribution #1).

Scripted with two players who systematically help each other and harm the rest, and told nothing, the detector recovers the planted $\{0, 1\}$ alliance **exactly**, with a well-separated score (10.0 on the allied pair, 0 or -1 everywhere else). This is a sanity check against one scripted, overt pair; false positives on non-colluding play and covert colluders were not tested.

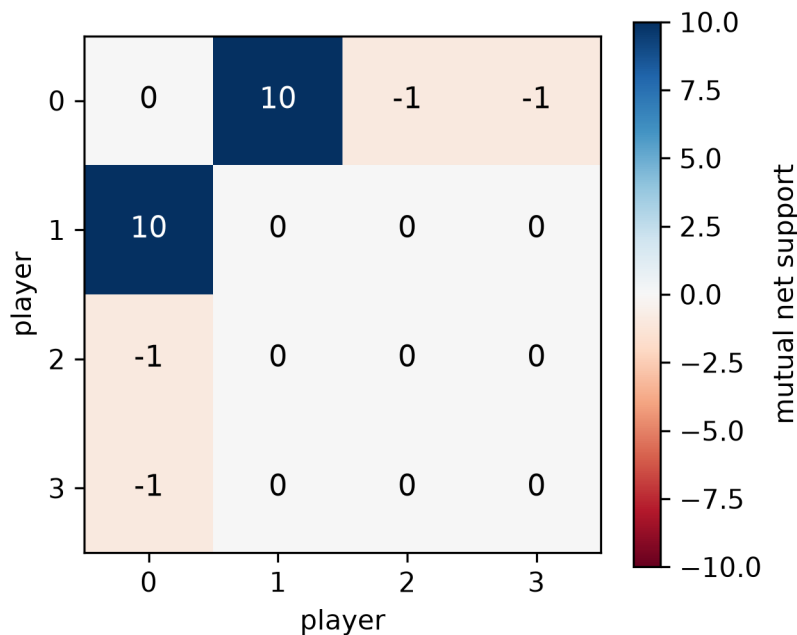


Figure 64: Coalition-score matrix inferred purely from chip placement: the planted alliance between players 0 and 1 scores 10; the other pairs are neutral (0) or hostile (-1).

11.3 Shapley credit for a competitive game

To *learn* to form coalitions, an agent needs a reward that says “you contributed to a coalition.” The classical tool is the **Shapley value**: the unique split of a coalition’s worth that satisfies Shapley’s axioms (efficiency, symmetry, null player, additivity), averaging each member’s marginal contribution over all join orders. Two textbook toys pin down what “fair” and “stable” mean, and both reproduce exactly:

Table 36: Shapley value and core stability on two cooperative-game toys.

Cooperative-game toy	Shapley value	Core (stability)
Glove game	$(2/3, 1/6, 1/6)$	non-empty (a stable split exists)
3-player majority	$(1/3, 1/3, 1/3)$	empty (no stable split)

The **empty core** of the majority game is the conceptual heart of the chapter: *any* allocation can be overturned by some coalition, so cooperation is *inherently unstable*. SLS is constant-sum (only one player wins), and every essential constant-sum game has an empty core¹⁴⁶; so the SLS coalition game, in which a coalition’s value is the win probability it can guarantee, has no stable allocation either (it is essential

¹⁴⁶Ferguson, T. S. *Game Theory* (lecture notes, UCLA), Part IV “Games in Coalitional Form”, § 2.3, Theorem 1.

because no player can guarantee a win alone). Together with the absence of a minimax value beyond two players, this is why N-player “safe” play cannot be anchored to a stable equilibrium (Contribution #2).

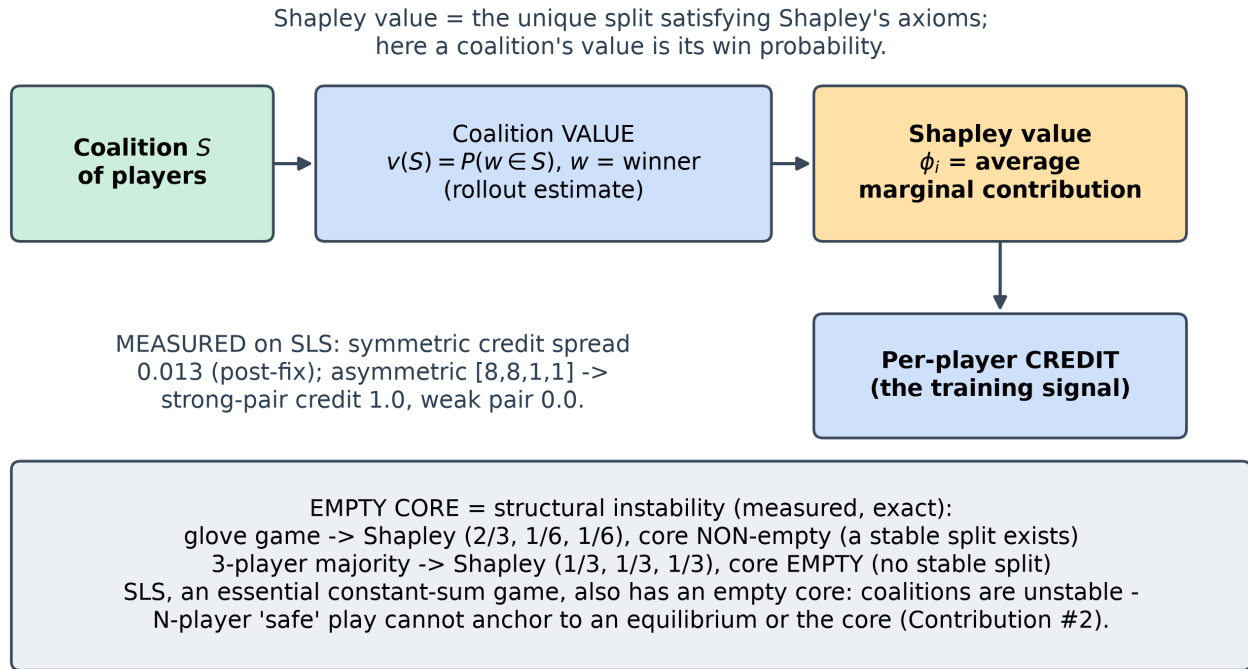


Figure 65: Shapley credit in a competitive game: coalition value becomes the rollout-estimated probability that a member wins. The majority game’s empty core is the structural signature of SLS coalitions: they will break.

SLS has no shared pot to split, so we **redefine the coalition value** as *the probability that a member of the coalition wins*, estimated by Monte-Carlo rollouts; each player’s credit is the Shapley value of that win-probability function. On SLS positions, a genuinely symmetric position gives near-equal credit (spread 0.013), and an asymmetric [8, 8, 1, 1] position hands *all* credit to the strong pair (coalition value 1.0). The symmetric result is reported **after** a bug fix.¹⁴⁷

¹⁴⁷Shapley, L. S. (1953). “A value for n-person games.” In *Contributions to the Theory of Games II* (Annals of Mathematics Studies 28), 307–318. Princeton University Press; Chalkiadakis, G., Elkind, E. & Wooldridge, M. (2011). *Computational Aspects of Cooperative Game Theory*. Morgan & Claypool (core, Shapley value, nucleolus); Wang, J., Zhang, Y., Kim, T.-K. & Gu, Y. (2020). “Shapley Q-Value: A Local Reward Approach to Solve Global Reward Games.” *AAAI* 34(5), 7285–7292 (Shapley-based credit assignment in MARL).

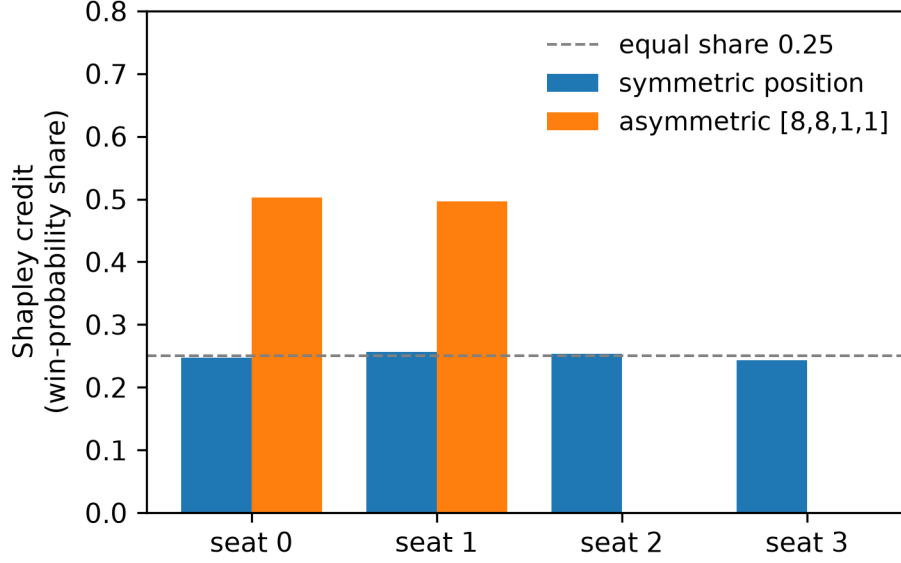


Figure 66: Shapley credit on two SLS positions: near-equal in the symmetric position (spread 0.013 after the fix), fully concentrated on the strong pair in [8, 8, 1, 1].

Reconciliation (kept prediction -> what actually happened). I predicted a symmetric position would give a symmetric credit spread (< 0.15). The first run gave 0.54 — a red FAIL — with Player 0 winning $\sim 2\times$ its fair share across three independent scripts. The mechanism: **$\sim 99.5\%$ of random games in this engine end in a deadlock** (all live hands empty — its simplified rules skip a player with an empty hand), so the winner is decided by a most-chips **tie-break** whose lowest-index rule quietly handed seat 0 its edge. An **unbiased random tie-break** fixed it: symmetric spread $0.54 \rightarrow 0.013$, all-random winners now uniform. It also revealed that an impressive ~ 0.87 hero win-rate had been the *same* artifact (the hero always sat in seat 0); the fair number is ~ 0.41 . In a game that almost always ends in a near-tie, the tie-break rule is the most load-bearing line in the engine.

11.4 Coalition-aware MAPPO — and when the coalition score actually rises

Each SLS seat is a masked, episodic PPO agent learning by self-play. The reward is a **blend**:

$$r = \alpha r_{\text{sparse}} + (1 - \alpha) (\text{Shapley coalition credit}),$$

where r_{sparse} is the sparse winner-takes-all reward and the second term is the centered Shapley credit from Section 11.3, computed either cheaply from critic values (**proxy**) or by rollouts (**counterfactual** in the code). The weight α dials between “just win” ($\alpha = 1$) and “form coalitions” ($\alpha = 0$).

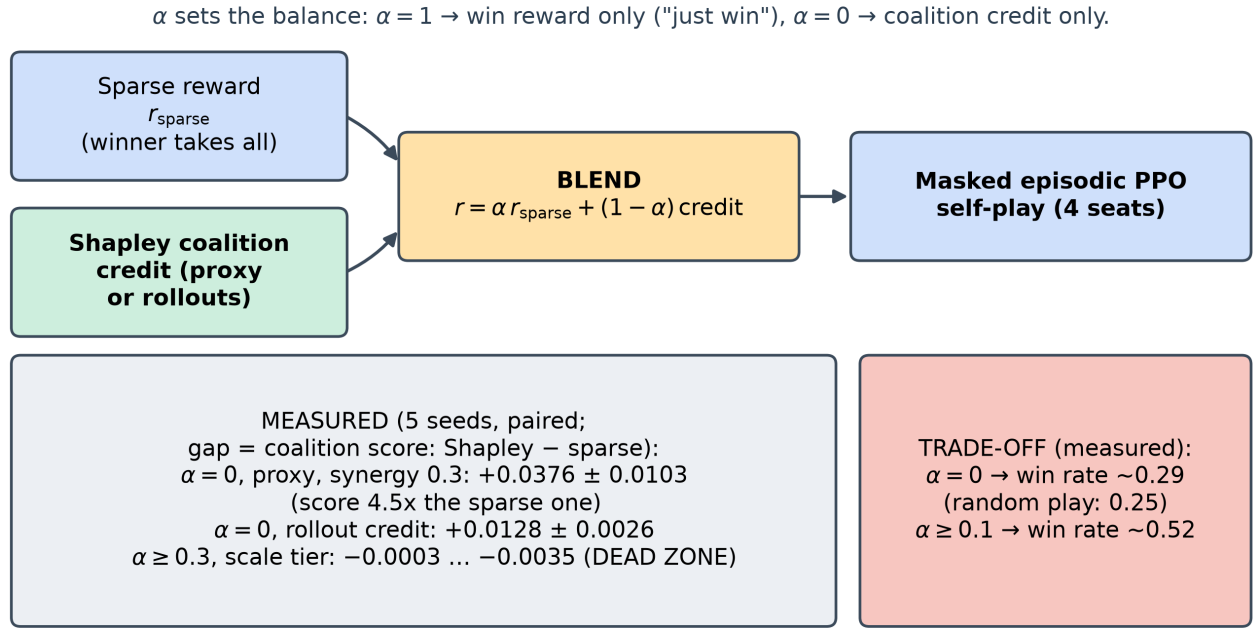


Figure 67: Coalition-aware MAPPO reward blend: the sparse win signal is mixed with Shapley coalition credit by weight alpha (1 = sparse only, 0 = credit only).

Do coalition-aware agents form coalitions more than sparse agents? Answering required a **5-seed paired sweep** over $\alpha \times$ credit-mode $\times$ synergy, because a single config misled me badly. There are two tiers: smoke (5 chips, 400 training games) and scale (7 chips, 1500 games). Paired gap = coalition score of Shapley agents minus sparse agents; ** = gap above two standard errors, which with 5 seeds corresponds to roughly $p < 0.12$:

Table 37: Paired coalition-score gap of Shapley credit over the sparse baseline, per regime.

Regime	Paired gap (Shapley - sparse)
scale, proxy, $\alpha = 0$, synergy 0.3	$+0.0376 \pm 0.0103$ (Shapley agents' score ~ 4.5 x the sparse baseline's)
scale, rollout credit, $\alpha = 0$	$+0.0128 \pm 0.0026$
scale, $\alpha \geq 0.3$	$-0.0003 \dots -0.0035$ (all negative; smoke: $-0.003 \dots +0.001$, none significant)
smoke, proxy, $\alpha = 0$, synergy 0.1	$+0.0024 \pm 0.0008$ (tiny)

Reconciliation (kept prediction $\rightarrow$ what actually happened). My single-config runs used the default $\alpha = 0.3$ and showed the coalition signal collapse at scale, which I first read as “the proxy credit is too weak once training is longer.” The sweep overturned that: α is the dominant knob, and 0.3 is a dead zone. The coalition score rises significantly *only*

at low α (at $\alpha \approx 0$ the Shapley agents exceed sparse by +0.038), while *every* scale-tier $\alpha \geq 0.3$ cell is negative — the sparse term drowns the coalition signal. The effect is also **larger in the scale tier** (game size and training length are confounded there), and the **cheap proxy beats the expensive rollout credit**. The fix is “weight the coalition credit heavily,” not “compute a truer credit”; whether the effect reflects coalition formation is not yet tested.

Mechanism caveat. In `shapley.py` the rollout value is $v(S) = \sum_{i \in S} P(i \text{ wins})$ — an additive game, in which each player’s Shapley value is simply its own win probability (and in training it is computed once per batch, from the initial state). The centered proxy credit is proportional to the agent’s own critic estimate minus the table mean; synergy only rescales it. Neither signal encodes who helped whom, and the coalition score is the mean *absolute* mutual net support, so it also rises with mutual hostility. The low- α effect therefore shows that replacing the win reward with a dense signal changes how the agents interact; whether coalitions form that way needs a control with an equally dense reward that carries no coalition information.

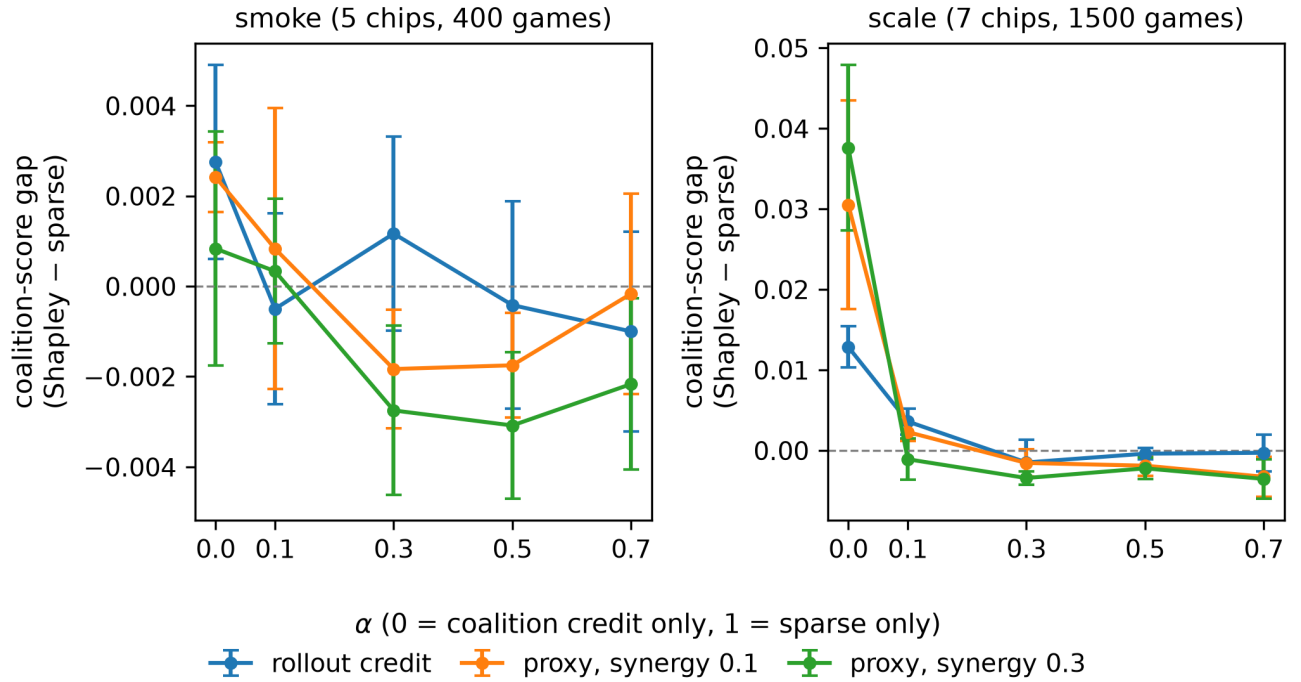


Figure 68: Paired coalition-score gap across the alpha x credit x synergy sweep (5 seeds, ± 1 SE): significant only at low alpha (peak +0.038, proxy, alpha = 0); negative at scale for every alpha ≥ 0.3 .

There is no free lunch: pure coalition credit ($\alpha = 0$) drops win-rate to ~ 0.29 (near the 0.25 random floor), while $\alpha \geq 0.1$ keeps it ~ 0.52 . Coalition-*forming* is the primary target and winning is secondary, as the chapter’s plan framed it — now quantified.¹⁴⁸

¹⁴⁸The MAPPO implementation with a centralized critic is from Chapter 9. piKL (regularizing search toward a human-like policy) was introduced by Jacob, A. P. et al. (2022). “Modeling Strong and Human-Like Gameplay with KL-Regularized Search.” *ICML*, arXiv:2112.07544, and applied to Diplomacy by Bakhtin, A. et al. (2023). “Mastering the Game of No-Press Diplomacy via Human-Regularized Reinforcement Learning and Planning.” *ICLR*, arXiv:2210.05492. The anchor there

11.5 EGTA and the spinning top — is SLS a wheel or a ladder?

The final tool evaluates the *population* with **empirical game-theoretic analysis (EGTA)**, as in Section 10.7. To reuse Chapter 9’s meta-Nash solver and Chapter 10’s **spinning-top** (Hodge) decomposition — both 2-player tools — the 4-player payoff *tensor* is **projected** to a pairwise matchup matrix, then split into a **transitive** (skill-ladder) component and a **cyclic** (rock-paper-scissors) component.

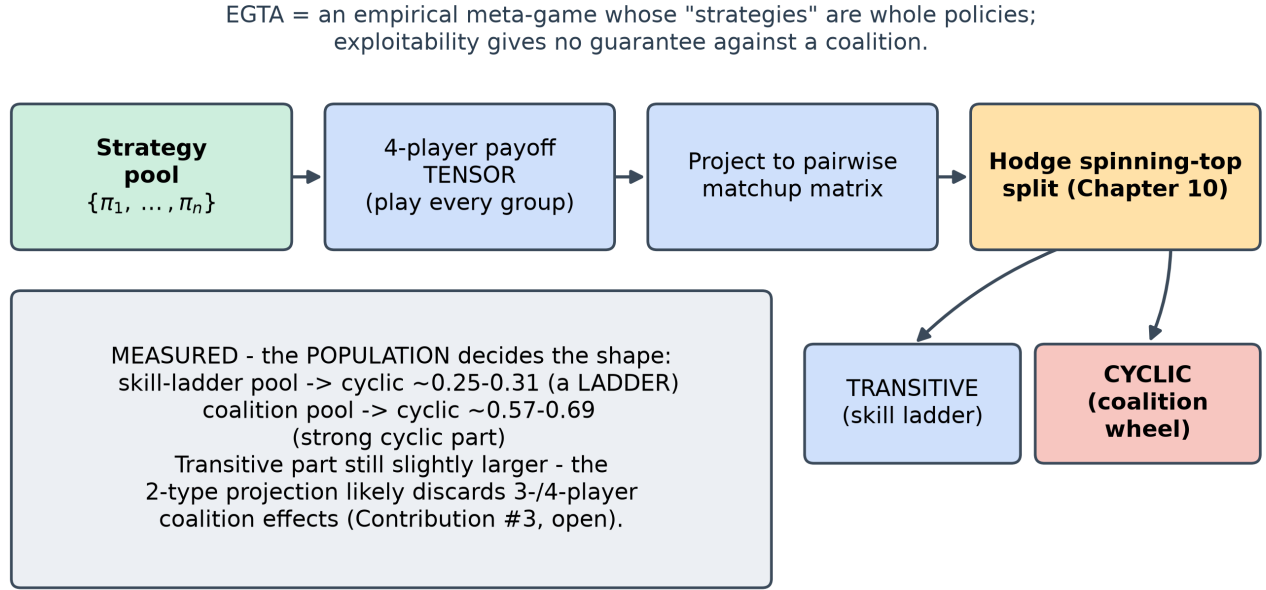


Figure 69: EGTA + spinning top for SLS: the 4-player payoff tensor is projected to a pairwise matrix and split into transitive and cyclic parts; the projection may underestimate the cyclic ratio.

Chapter 10 predicted FFA coalition games would be strongly cyclic. Measured, it depends entirely on **which population you decompose**.¹⁴⁹

Table 38: Cyclic ratio and structure of two SLS populations.

Population	Cyclic ratio	Structure
Skill-ladder pool	0.25-0.31	transitive-dominant (a ladder)
Coalition pool (ally/betray strategies)	~ 0.57-0.69	strong cyclic component; transitive still slightly larger

imitates human play and carries no safety guarantee; using it as the anchor for N-player safe exploitation is this thesis’s proposal.

¹⁴⁹Balduzzi, D. et al. (2019). “Open-ended Learning in Symmetric Zero-sum Games.” *ICML*, PMLR 97, 434–443 — the transitive/cyclic decomposition; Czarnecki, W. M. et al. (2020). “Real World Games Look Like Spinning Tops.” *NeurIPS*, arXiv:2004.09468 — the spinning-top geometry.

Reconciliation (kept prediction -> what actually happened). I expected a large cyclic component and, at first, saw a near-perfect skill ladder (cyclic ~ 0.07). That was partly the seat-0 bug (Section 11.3) and partly **pool composition**: the default baseline pool *is* a skill ladder. A coalition pool (fixed-ally + betrayer strategies) pushes the cyclic ratio to ~ 0.57 - 0.69 , confirming the *direction* of the prediction but staying **short of strict dominance** (cyclic² just under 0.5). The prime suspect is that the **2-type projection discards 3-/4-player coalition effects**; a tensor-native decomposition is the open question.

11.6 Honest notes, limitations, and where this hands off

The exact anchors hold: the engine matches the 2-player minimax endgame with **zero** mismatches, and the Shapley code reproduces the glove and majority toys, *including their core*, to four decimals.

Two corrections travel forward. (1) The “symmetric” game was not symmetric — a most-chips **tie-break bug** handed seat 0 $\sim 2\times$ its fair share and inflated the hero win-rate from a true ~ 0.41 to a false ~ 0.87 ; fixed, but it proves the engine’s *rules*, not its solver, are where the risk lives. (2) “Coalitions don’t emerge at scale” was a **mis-set blend weight** ($\alpha = 0.3$ is a dead zone), not a failure of the method. As in Chapters 9-10, **a single config hides what a seeded sweep reveals**.

Trust. The exact targets are deterministic; the training claims rest on a 5-seed paired sweep with error bars, so the *directions* are the trustworthy claims, not third-decimal magnitudes. The standing caveat is **engine fidelity**: the engine is certified against its own ruleset, not against the De Carufel & Jerade formalization¹⁵⁰, and the tie-break bug shows that reconciliation is not cosmetic.

Connections. Backward: Chapter 7’s opponent model is lifted to social structure; Chapters 9-10’s meta-Nash and spinning-top tools are reused on the projected meta-game; Chapter 8’s “safe = bounded deviation from Nash” loses its footing. Forward: Chapter 12 treats language models as strategic agents but not negotiation; N-player safety (Contribution #2) and EGTA on the payoff tensor as a multi-agent generalization of exploitability (Contribution #3) remain tasks for the thesis.

11.7 Key takeaways for the thesis synthesis

- **At $N \geq 3$, exact evaluation is gone.** Evaluation becomes empirical (win-rate + coalition score + EGTA cyclic ratio), anchored by the 2-player minimax endgame (0 mismatches).
- **A dense credit raises the coalition score, which is not yet evidence of learned coalitions.** At $\alpha \approx 0$ the gap is $+0.038$ (5 seeds, scale tier); at $\alpha \geq 0.3$ it vanishes. The credit as implemented does not encode who helped whom; a coalition-free dense-reward control is needed (Section 11.4).
- **Weighting the credit costs winning.** $\alpha = 0$ drops win-rate to ~ 0.29 , near the 0.25 random level; $\alpha \geq 0.1$ restores ~ 0.52 .

¹⁵⁰De Carufel, J.-L. & Jerade, M. R. (2024). “So Long Sucker: Endgame Analysis.” *arXiv:2403.17302*.

- **Empty core $\Rightarrow$ structural instability.** No stable allocation exists, so coalitions *will* break and N-player “safe” play needs an anchor other than Nash or the core (Contribution #2).
- **Which population you decompose decides ladder-vs-wheel** (the Chapter 10 lesson): a skill-ladder pool is ~ 0.3 cyclic, a coalition pool ~ 0.57 - 0.69 , with the transitive part still slightly larger.
- **Engine rule fidelity outranks solver code.** The seat bias and the sub-dominant cyclic ratio trace to the engine’s simplifications; the Shapley/EGTA math passed its textbook checks exactly.

12 Chapter 12 — Sequence Models and LLM Agents in Strategic Settings

12.1 Why reframe reinforcement learning as sequence prediction

Classical RL optimises: estimate a value function, back up rewards, improve the policy. The Decision Transformer (Chen et al., 2021) proposes something different — treat the whole problem as **supervised sequence modelling**. Feed a GPT-style model a trajectory of triples

$$(\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \dots)$$

where $\hat{R}_t = \sum_{t' \geq t} r_{t'}$ is the *return-to-go*, and train it to predict a_t from everything up to s_t . There is no Bellman backup and no policy-improvement step. At inference you simply **condition**: ask for a high return-to-go and the model produces the actions that historically preceded such returns.

The appeal for this dissertation is that it works entirely **offline**, on a fixed dataset — the regime of recorded game logs, where self-play is unavailable.¹⁵¹

12.2 The luck-versus-skill trap

The reframing hides an assumption: that the return-to-go is something the agent *earned*. In a stochastic environment it is not. Paster et al. (2022) show that return-conditioned policies systematically chase luck; the smallest example makes it obvious.

Consider a one-step bandit. Action A pays 0.5 deterministically. Action B pays 1.0 with probability 0.4 and 0 otherwise, so $\mathbb{E}[B] = 0.4 < 0.5$. Action A is EV-optimal. But the *only* way a trajectory ever achieves a return of 1.0 is to take B **and get lucky**. Condition on “return = 1.0” and you recover exclusively B-trajectories:

$$P(a = B \mid \hat{R} = 1.0) = 1.00 \quad (\text{measured in own simulation})$$

The return-conditioned learner confidently selects the action with the *worse* expected value. This is not a bug in the model; it is what conditioning on an outcome means when the outcome is partly noise.¹⁵²

12.3 ARDT — conditioning on what you can guarantee

The fix is to condition not on the return that *happened* but on the return the protagonist can **guarantee against a worst-case opponent** — the minimax return-to-go. ARDT (Tang et al., 2024) estimates it with **expectile regression**, which minimises the asymmetric squared loss

¹⁵¹Chen et al., *Decision Transformer: Reinforcement Learning via Sequence Modeling*, NeurIPS 2021 (arXiv:2106.01345).

¹⁵²Paster, McIlraith & Ba, *You Can’t Count on Luck: Why Decision Transformers and RvS Fail in Stochastic Environments*, NeurIPS 2022 (arXiv:2205.15967).

$$L_{\text{ER}}^\alpha(u) = \mathbb{E}_u[|\alpha - \mathbf{1}(u > 0)| \cdot u^2]$$

whose limits give the operators you need: $\alpha \rightarrow 0$ recovers the **minimum** and $\alpha \rightarrow 1$ the **maximum**. The trajectories are relabelled with the estimated minimax return and a standard DT is trained on the relabelled data.

Two details of the published method, both confirmed in the source, matter. The pessimistic side is **low** α (the chapter’s original plan wrongly called $\tau = 0.9$ pessimistic; the paper runs $\alpha = 0.01$). And the relabel target is a state-**action** value, $\tilde{R}_t = \tilde{Q}_\nu(s_t, a_t)$, produced by two *coupled* estimators alternately fitted with α and $1 - \alpha$ losses. A state-only value $V(s)$ cannot distinguish “this state is bad” from “*this action* in this state is bad”, which is exactly the discrimination the method relies on.

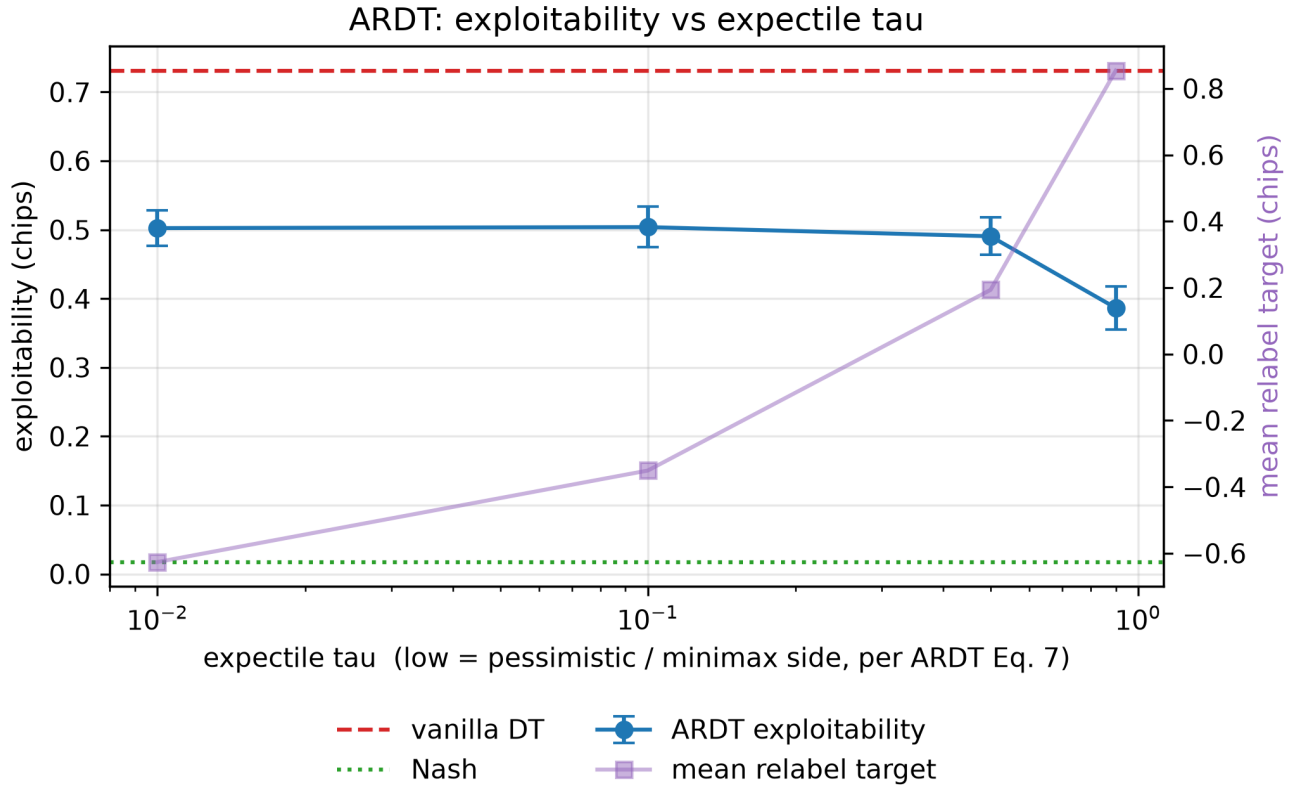


Figure 70: ARDT exploitability against τ (second axis: mean relabel target); three-seed means (vanilla DT 0.731, equilibrium 0.017 chips).

Measured on Kuhn with a deliberately simplified state-only proxy, the relabel target moves monotonically with τ as theory requires — but exploitability is *lowest* on the optimistic side. The most likely cause is the missing action argument; a $\tilde{Q}(s, a)$ variant has not been run, so this neither refutes nor confirms ARDT.¹⁵³

¹⁵³Tang, X., Marques, A., Kamalaruban, P. & Bogunovic, I., *Adversarially Robust Decision Transformer*, NeurIPS 2024

12.4 What return conditioning actually does in poker

Kuhn Poker’s equilibrium and exploitability are exactly computable (see Chapter 2),¹⁵⁴ so the question can be settled rather than argued. Conditioning **changes** the policy substantially but does not **steer** it: exploitability is flat across the range of real target returns, except for a sharp spike (to ≈ 1.9 chips) at the modal payoff, which in Kuhn is the payoff of folding. On the same data plain behavioural cloning reaches 0.055 chips of exploitability — close to Nash (0.016) — against 0.799 for the DT: return conditioning actively destroys information (the DT value varies between retrains, 0.67–0.80).

The natural explanation is that in a four-payoff game the *magnitude* of the return encodes which betting line was played, while its *sign* encodes who held the better card. Conditioning therefore selects the shape of the hand rather than the quality of the play.

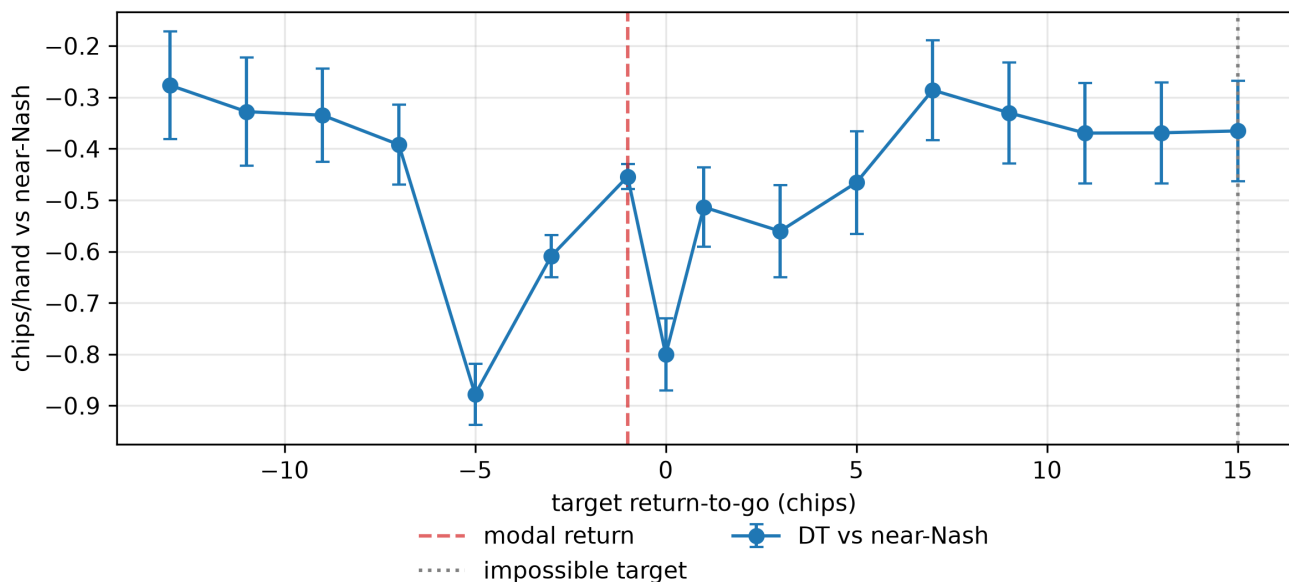


Figure 71: Decision Transformer on Leduc Hold’em (chips per hand vs a near-Nash opponent) against the conditioned target return, $\pm$ SE (Pearson $r = +0.062$). Dashed: most common return (-1); dotted: impossible target ($+15$).

Leduc Hold’em tests that explanation with fifteen payoff values instead of four, two betting streets and a board card. **The notch at the modal return disappears as the explanation predicts — but steering still does not appear** (Pearson $r = +0.062$); two unexplained dips remain, at 0 (the second most common return) and at -5 . The payoff-alphabet account thus explains the collapse but not the failure, which is more fundamental: in a zero-sum imperfect-information game the realised return is dominated by the opponent’s private card and actions, which the protagonist does not control.

(arXiv:2407.18414) — Section 3 and Algorithm 1.

¹⁵⁴Kuhn, H. W. (1950). “A Simplified Two-Person Poker.” In H. W. Kuhn & A. W. Tucker (Eds.), *Contributions to the Theory of Games*, Vol. I (Annals of Mathematics Studies 24), pp. 97–103. Princeton University Press. DOI 10.1515/9781400881727-010.

This is the chapter’s main conclusion: on recorded logs, a return-conditioned Decision Transformer is the wrong instrument, and the value of ARDT’s relabeling is that it replaces a conditioning target the agent does not control with one that depends mainly on its own actions.

12.5 Language models as strategic agents

The second paradigm skips training altogether: describe the rules in English and let a language model play. Three open-weight models were run locally (the quantisation of the weights was not recorded): gpt-oss-20b, Qwen2.5-7B-Instruct and OpenThinker3-7B (a reasoning fine-tune of Qwen2.5-7B-Instruct). Results are for Qwen2.5-7B-Instruct unless stated otherwise; all are own measurements. Similar experiments on Kuhn, Leduc and Limit Hold’em find larger models also far from equilibrium, their actions often diverging from their stated reasoning¹⁵⁵; prompting explicit opponent reasoning lets GPT-4 adapt.¹⁵⁶ Against the same exact ruler, LLMs are **honestly exploitable** — but the interesting result is *where* they fail.

They get hand **ranking** right and mixing **frequencies** wrong. Every real model tested value-bets the King with probability 1.00 where the CFR equilibrium mixes at 0.68, and none bluffs the Jack under a plain prompt in 24 samples per information set. (Kuhn’s first-player equilibria form a one-parameter family — bluff the Jack with $\alpha \in [0, 1/3]$, bet the King with 3α ¹⁵⁴ — so each behaviour alone is an equilibrium action; only the combination is not.) A 7B model matches gpt-oss-20b (a mixture-of-experts model with 3.6B of 21B parameters active per token), so three models on one toy game say nothing general about scale.

¹⁵⁵Lin, M., Dai, E., Liu, H. et al., *How Far Are LLMs from Professional Poker Players? Revisiting Game-Theoretic Reasoning with Agentic Tool Use*, ICLR 2026 (arXiv:2602.00528).

¹⁵⁶Guo, J., Yang, B., Yoo, P., Lin, B. Y., Iwasawa, Y. & Matsuo, Y., *Suspicion-Agent: Playing Imperfect Information Games with Theory of Mind Aware GPT-4*, 2023 (arXiv:2309.17277).

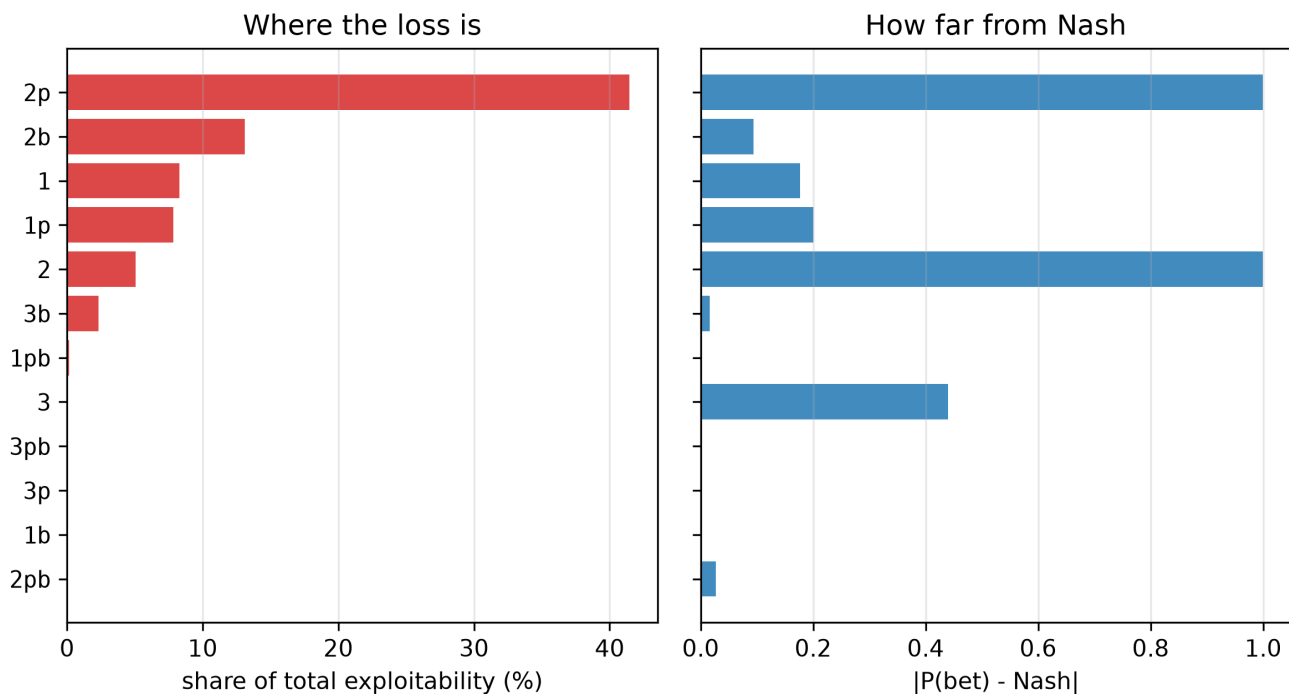


Figure 72: Each information set’s share of Qwen2.5-7B-Instruct’s total Kuhn exploitability (0.357 chips) beside its deviation from Nash (1/2/3 = J/Q/K; p = pass, b = bet). The orderings barely correspond.

Decomposing the loss per decision overturns the obvious reading. The King “failure” — the largest visible deviation from equilibrium — costs **0.1%** of the total loss, because over-betting a hand that is never behind is nearly free. A single Queen decision costs **41.4%**. Deviation magnitude and cost are almost uncorrelated, which is the concrete argument for reporting per-decision diagnostics rather than a single exploitability number.

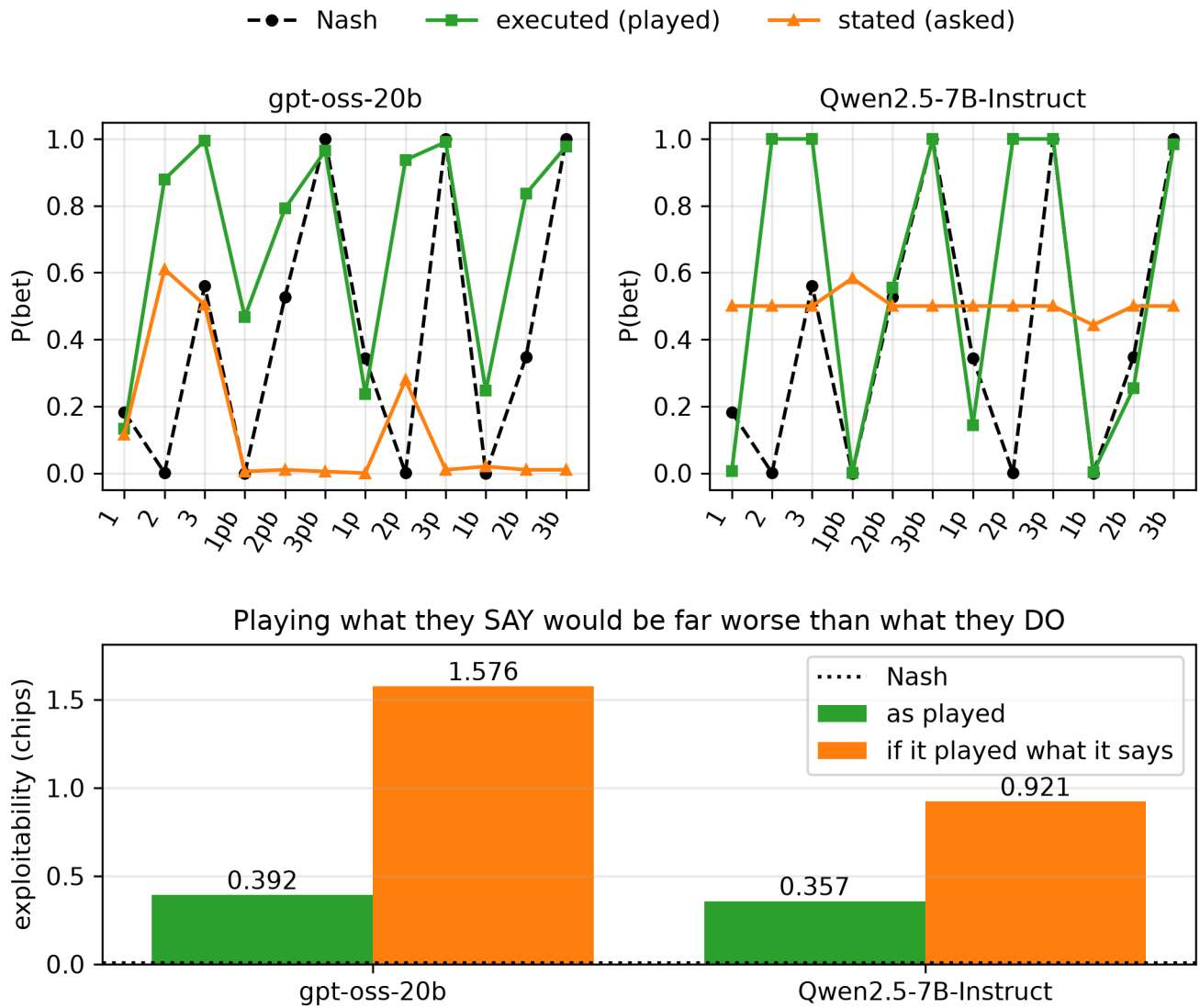


Figure 73: Stated and played betting frequencies of gpt-oss-20b and Qwen2.5-7B-Instruct per information set vs equilibrium (top; mean absolute deviation, stated vs played: 0.43 vs 0.33, 0.35 vs 0.25), and exploitability of both strategies (bottom).

Asking a model what it intends is *worse* than measuring what it does. Scored as strategies, the frequencies these models **state** are 2.6× and 4.0× more exploitable than the strategies they actually **play**. One model answers “50%” almost everywhere, the other “near 0%”, even with the strongest hand. Behavioural probing is more accurate than introspection, not merely more convenient.

12.6 Exploitation, adaptation, and the limits of the toy game

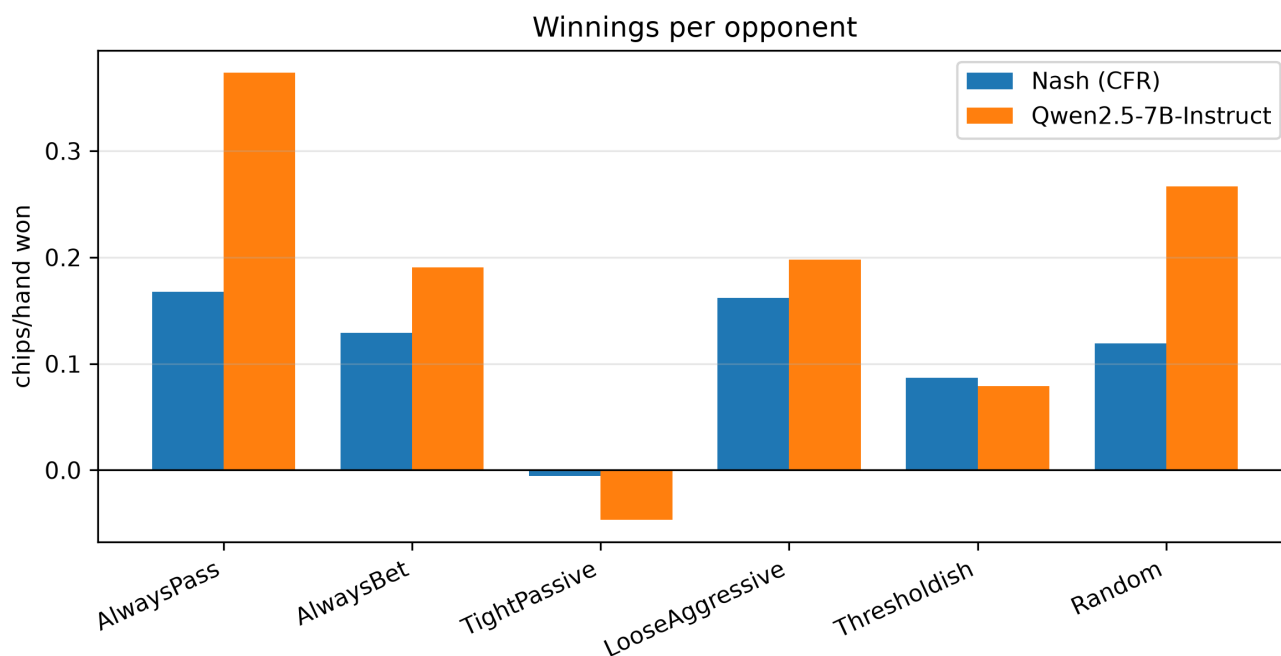


Figure 74: Winnings per zoo opponent: Qwen2.5-7B-Instruct averages 0.177 chips per hand (equilibrium 0.110) at exploitability 0.357 (equilibrium 0.006).

Exploitability measures how a perfect adversary punishes you; it says nothing about how well you punish weakness. Against a zoo of deliberately exploitable archetypes, the language model **wins 0.177 chips per hand against 0.110 for equilibrium play (61% more), while its exploitability is 0.357 chips against 0.006** — and nearly 90% of the advantage comes from the passive and random opponents. Against the two most competent archetypes it does *worse* than equilibrium. That is the safe-exploitation trade-off of the thesis’s second contribution, observed rather than assumed.

Given the last 20 hands of the session in context, the model does **not** adapt: against a trivially passive opponent it captures 83% of the available exploitation in the first half and no more in the second (0.824 vs 0.827); the other two opponents are underpowered. What looks like opponent modelling is a fixed loose-aggressive prior. Told the opponent’s type in the prompt, however, the models shift their bluffing by up to 0.92 — what is missing is inference from history, not responsiveness to a description.

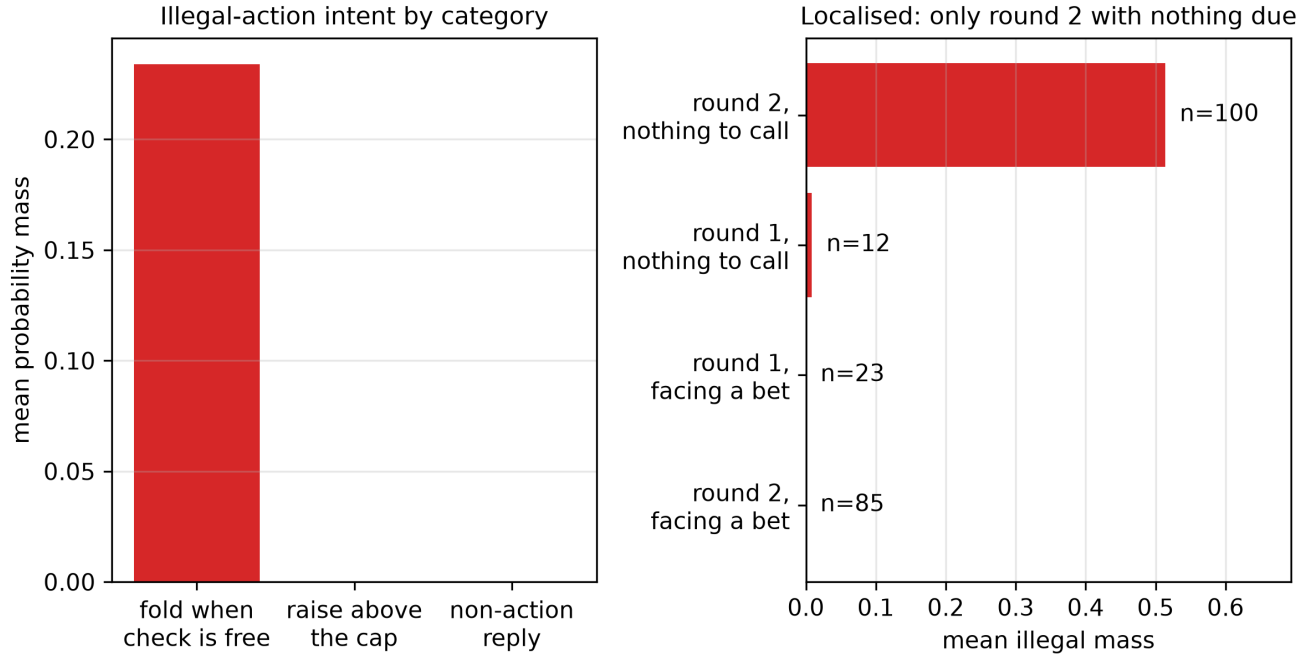


Figure 75: Illegal-action intent on Leduc Hold'em (Qwen2.5-7B-Instruct) by category and situation (on average 0.234 of the probability mass over 220 information sets): almost all is folding when a check is free, in round two.

Finally, the toy game flatters these models. One street and one board card later, the LLM is statistically indistinguishable from the Decision Transformer, loses on average to weak opponents (winning only against the two passive types), and wants to take an illegal action with roughly a quarter of its probability mass. That failure is a *single* misconception rather than diffuse confusion — it wants to **fold when checking is free**, almost exclusively on weak unpaired hands against a high board, and never tries to raise past the betting cap. Any optimism drawn from the toy game is a property of the toy game.

12.7 Key takeaways for the thesis synthesis

- **Conditioning on a quantity the agent only partly controls does not steer it.** Observed on two toy games and consistent with Paster et al.'s Theorem 2.1; hence ARDT's relabeling, not the Decision Transformer itself, is the more promising route for recorded poker logs — provided chance events (the deal) are handled, where ARDT's minimax target is no longer exact.¹⁵⁷ It most likely needs the published state-action value $\tilde{Q}(s, a)$, not $V(s)$ — still to be tested.
- **A scalar score does not diagnose, and self-report is worse than behaviour:** one decision carried 41.4% of the loss, the most conspicuous deviation 0.1%; stated strategy was worse than played strategy for every model tested.

¹⁵⁷Tang, X., Cheng, Z. & Kumar, S., *Robust Adversarial Reinforcement Learning in Stochastic Games via Sequence Modeling*, Reliable ML Workshop @ NeurIPS 2025 (arXiv:2510.11877) — proposes CART; notes that ARDT's minimax target is exact only for deterministic transitions.

- **The model did not infer its opponent from in-context history** (one 7B model, a 20-hand window) but responded when told the opponent’s type: the opponent model must be built explicitly and its output passed to the policy (the first thesis contribution).
- **Measurement protocol is part of the result.** At greedy decoding a language model plays a pure strategy and every frequency degenerates to 0 or 1; four findings in this chapter were measurement artefacts, each caught by a cheap check against an exactly computable quantity.

13 Chapter 13 — Behavioral Analysis of Real Hand Histories

Chapters 2–12 studied games that could be solved or simulated, where every strategy was known and every result could be checked against an exact reference. This chapter turns to what real people do. It takes two million online poker hands and asks what can be learned about the players from the record alone, down to whether two accounts are helping each other and whether an account is a bot. **All numbers reported here were measured** on reproducible runs, with three seeds wherever training or sampling is involved. Where a run contradicted what the plan predicted, the prediction is kept and reconciled with what happened.

The data. All data used here are public. The PHH dataset¹⁵⁸ contains 21.6 million anonymized no-limit hold'em hands from July 2009, collected from six online rooms by the hand-history vendor HandHQ. Its iPoker part includes **2,032,655 hands at \$0.50/\$1 blinds**, and these are used here. The second source is the **10,000 released hands of Pluribus**¹⁵⁹, each played by the program and five professionals. It is the one public sample in which a bot is known, which makes it the ground truth for bot detection.

Where this sits in the thesis. The chapter is the practical core of Contribution #1, the behavioral adaptation framework, on its data side; the adaptive agent itself is outside its scope. Real-time opponent modeling with a conservative fallback already works in two-player games, including small poker games¹⁶⁰¹⁶¹, and N-player opponent modeling exists without any safety criterion¹⁶². So what C1 still needs is the N-player, within-match, safety-coupled version. It also covers the fair-play task of the individual plan, detecting collusion and bots. In the literature, collusion detection is scored offline, as a detection task¹⁶³¹⁶⁴. No counter-example was found to the statement that none of these methods couples detection to what the detecting agent does next. This chapter also stays on the offline side of that line.

¹⁵⁸Kim, J. (2024). “Recording and Describing Poker Hands.” *IEEE Conference on Games (CoG 2024)*, 1–8. DOI 10.1109/CoG60054.2024.10645611; arXiv:2312.11753 — the PHH format. The dataset is github.com/uoftcprg/phh-dataset (MIT licence; HandHQ subset of 21,605,687 no-limit hands, July 2009, six rooms, 5,996,345 of them on the iPoker Network).

¹⁵⁹Brown, N. & Sandholm, T. (2019). “Superhuman AI for multiplayer poker.” *Science* 365(6456), 885–890. DOI 10.1126/science.aay2400 — 10,000 hands against five of 13 professionals, 100 big blinds per hand; on style: limping is “suboptimal for any player except the ‘small blind’ player”, and Pluribus donk-bets “far more often than professional humans do”.

¹⁶⁰Fu, H., Tian, Y., Yu, H., Liu, W., Wu, S., Xiong, J., Wen, Y., Li, K., Xing, J., Fu, Q. & Yang, W. (2022). “Greedy when Sure and Conservative when Uncertain about the Opponents.” *ICML*, PMLR 162, 6829–6848.

¹⁶¹Caen, A., Winands, M. H. M. & Soemers, D. J. N. J. (2026). “StratFormer: Adaptive Opponent Modeling and Exploitation in Imperfect-Information Games.” *Computers and Games 2026* (accepted); arXiv:2604.25796. See also Murgoci, V., Spaan, M. & Oren, Y. (2026). “AlphaExploit: Going beyond the Nash equilibrium in poker by learning to exploit suboptimal play.” arXiv:2605.09150 — both two-player.

¹⁶²Ganzfried, S., Wang, K. A. & Chiswick, M. (2024). “Opponent Modeling in Multiplayer Imperfect-Information Games.” *Proc. 6th Int. Conf. Distributed Artificial Intelligence (DAI '24)*, 39–45. DOI 10.1145/3719545.3721108.

¹⁶³Mazrooei, P., Archibald, C. & Bowling, M. (2013). “Automating Collusion Detection in Sequential Games.” *AAAI* 27(1), 675–682. DOI 10.1609/aaai.v27i1.8674 — three-player limit hold'em, synthetic colluders. See also Greige, L. et al. (2022). “Collusion Detection in Team-Based Multiplayer Games.” arXiv:2203.05121 (social-graph features on real data, not poker).

¹⁶⁴Bonjour, T., Aggarwal, V. & Bhargava, B. (2022). “Information Theoretic Approach to Detect Collusion in Multi-Agent Games.” *UAI*, PMLR 180, 223–232.

13.1 From solving games to reading logs

Picture a casino security room in which the cameras are off and only the dealer’s logbook is left. Each line says something like “seat 3 raised to 4, seat 5 folded”. A floor manager who reads enough pages learns each regular’s habits and notices two players who never raise each other. They also notice the account that plays three tables all day with the same rhythm. The pipeline in this chapter is that floor manager written down, with one addition the floor manager rarely has: a way to check how often each suspicion is right.

The logbook has limits. In the iPoker records, stacks are written as infinite, so an all-in is invisible. The winnings field is zero in every hand, so results have to be recomputed from the actions. Table names are identical, and hole cards are known only when shown, which leaves 86 % of hands with no known cards. Every statistic in this chapter is therefore defined on what the record contains, and the chapter says so where it matters. This is also the reason the chapter starts with a parser rather than a model. A mistake in parsing does not fail loudly; it quietly corrupts every number downstream.

13.2 The data and the parser

The parser replays each hand with its own no-limit engine. A second, independent validator is pokerkit, a rules engine by the author of the PHH format¹⁶⁵. The parsed hands become four data products, and each analysis reads one of them (the pipeline figure below).

¹⁶⁵Kim, J. (2025). “PokerKit: A Comprehensive Python Library for Fine-Grained Multivariant Poker Game Simulations.” *IEEE Transactions on Games* 17(1), 32–39. DOI 10.1109/TG.2023.3325637.

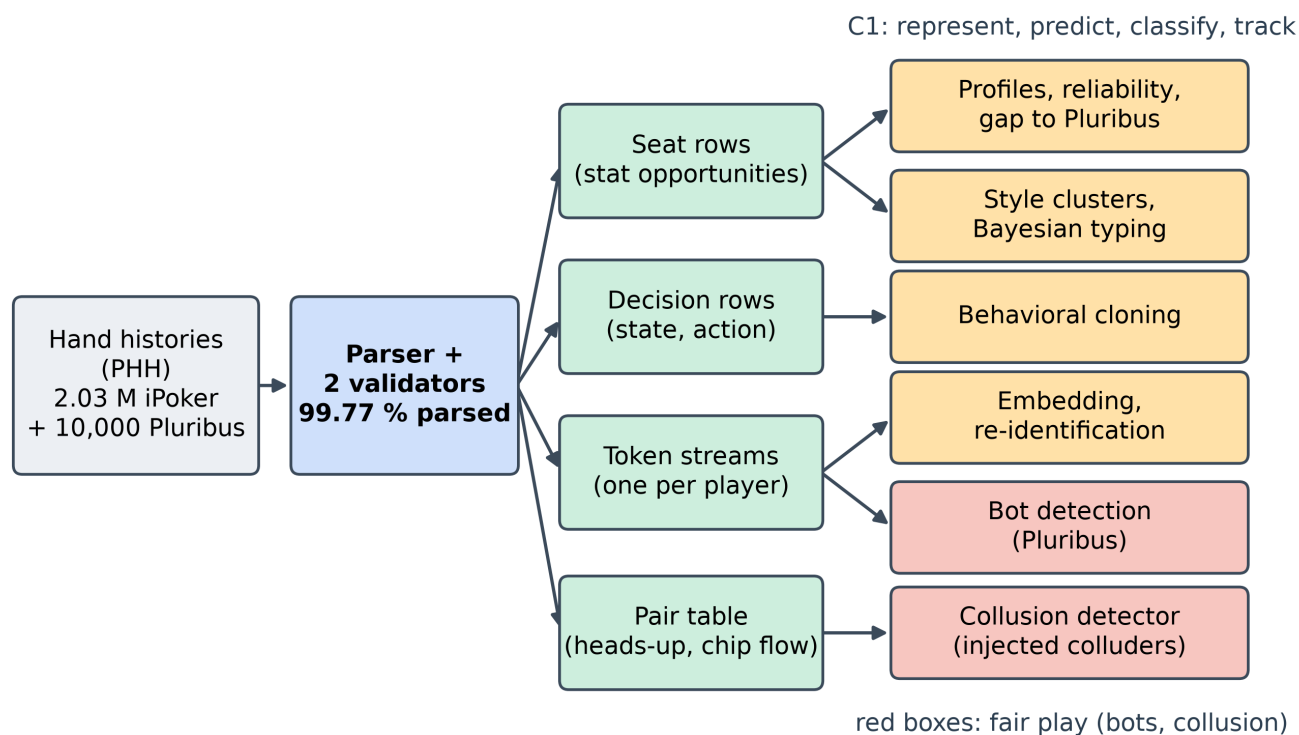


Figure 76: The chapter’s pipeline: PHH hand histories are replayed by the parser and checked by two validators; the parsed hands become four data products, each feeding one or two analyses. Red boxes are the fair-play analyses.

Three conventions of the format came to light only because two independent replays disagreed: reversed heads-up blinds, negative blinds that are *posts* rather than refunds, and all-ins that show up as a street dealt with no betting on it (4.3 % of hands). With these handled, the parser replays 99.77 % of the iPoker hands and every Pluribus hand.

13.3 Describing a player: statistics and how many hands they need

Poker players describe each other with a handful of tracker statistics: how often a player puts money in voluntarily before the flop (VPIP), how often they raise before the flop (PFR), how often they re-raise a single raise (3-bet), how often the pre-flop raiser bets the flop (c-bet), how often a player who sees the flop goes to showdown (WTSD). Each of these is a count over its own *opportunities*. A 3-bet can only happen when a player faces exactly one raise, so the 3-bet rate is divided by those situations, not by all hands.

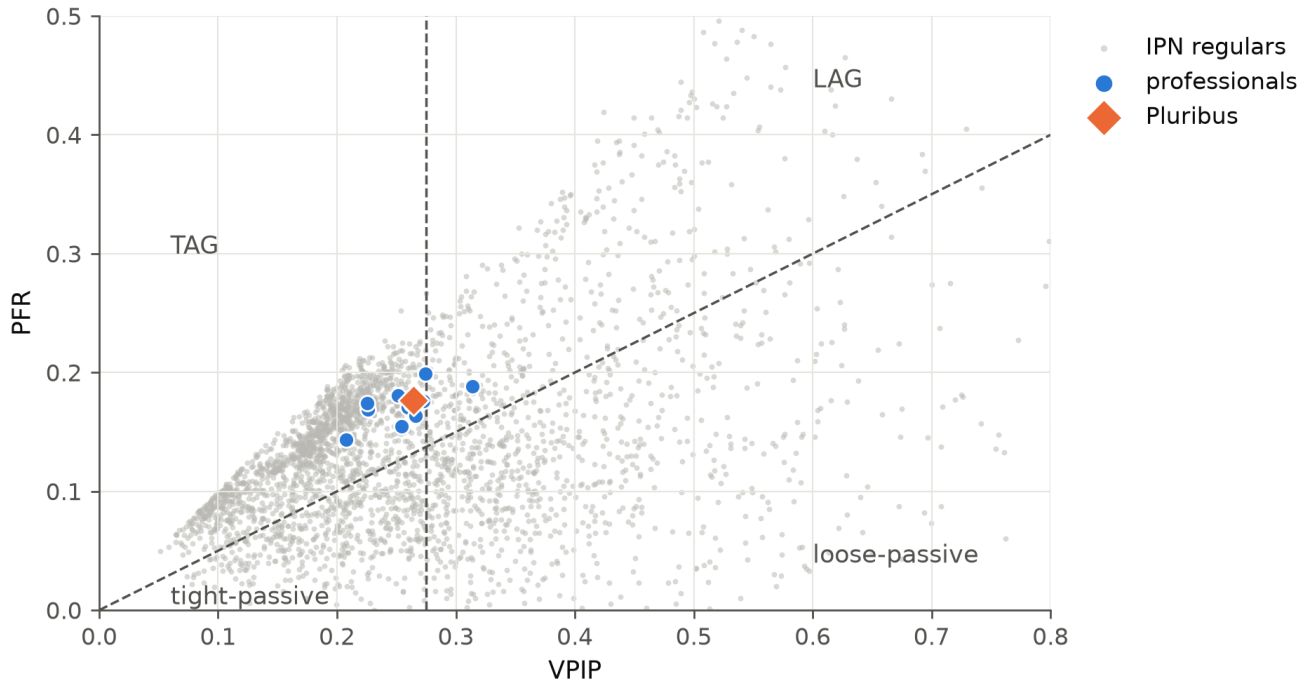


Figure 77: The VPIP $\times$ PFR map for the 2,906 iPoker regulars (grey), the 13 professionals of the Pluribus games (blue) and Pluribus (orange). Dashed lines: loose at VPIP ≥ 27.5 %, aggressive when PFR $\geq$ VPIP / 2.

The 2,906 players with at least 500 hands (the plan's threshold) hold 84.6 % of all seats. On the classic map, 1,323 are tight-aggressive, 730 loose-passive, 462 tight-passive and 391 loose-aggressive.

How many hands does each number need? The honest way to answer is to measure it. The test takes two disjoint random samples of m hands from every player who has at least $2m$, computes a statistic on both, and correlates the two across players. A statistic that describes the player gives the same value on both samples; one that mostly describes the cards does not.

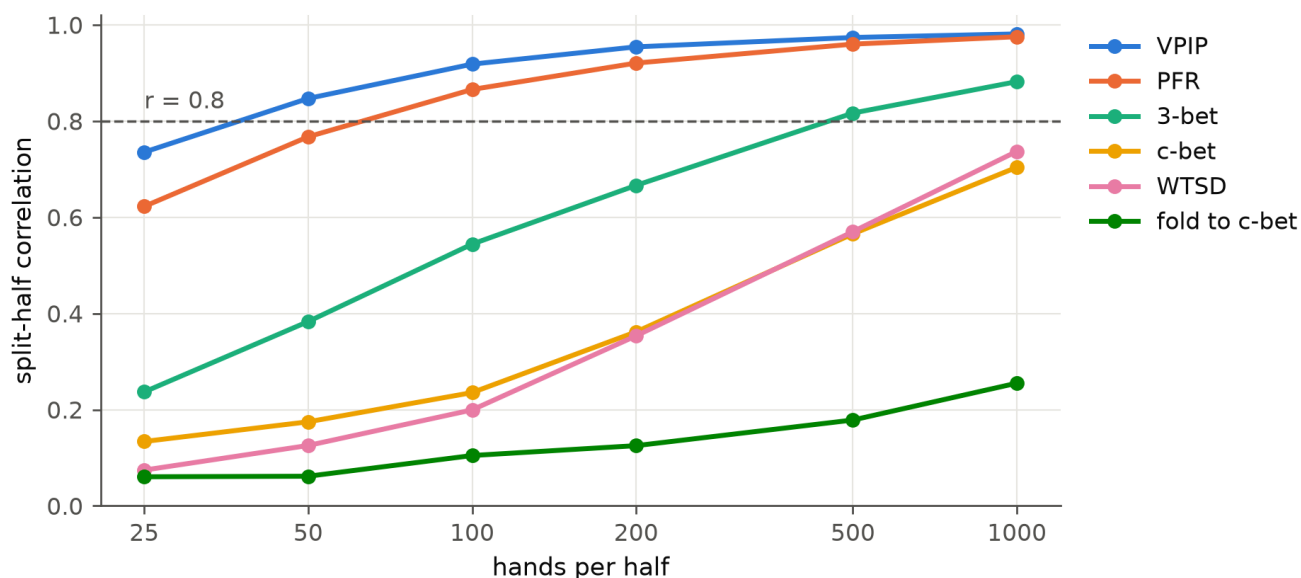


Figure 78: Split-half correlation of six statistics across the iPoker regulars against hands per half (3 seeds). VPIP and PFR pass $r = 0.8$ below 100 hands, 3-bet near 500; the post-flop statistics stay below 0.8 at 1,000.

VPIP reaches a correlation of 0.92 with 100 hands per half, and PFR 0.87. 3-bet needs about 500 hands to reach 0.82. The post-flop statistics are much slower: c-bet and showdown frequency are at 0.57 after 500 hands and about 0.7 after 1,000, and fold-to-c-bet is at 0.25 after 1,000. The plan’s rule of thumb, 500 hands for stable statistics, is right for the pre-flop numbers and wrong for the post-flop ones. The chapter therefore keeps 500 hands as the inclusion threshold and adds a minimum number of *opportunities* per statistic. Win-at-showdown is dropped altogether, because cards are known for only a median 41 % of a regular’s showdowns.

13.4 How far real players are from a strong reference

The plan asks for the gap between real play and equilibrium play, the “exploitation opportunity” that Contribution #2 would act on. For six-player no-limit hold’em no computed equilibrium is available. The defensible reference is the strongest six-player strategy with public hands: Pluribus’s own frequencies in its 10,000 six-handed hands. Pluribus is not an equilibrium, and a frequency gap is not an expected value. It says where an exploiter would look, not how much it would win.

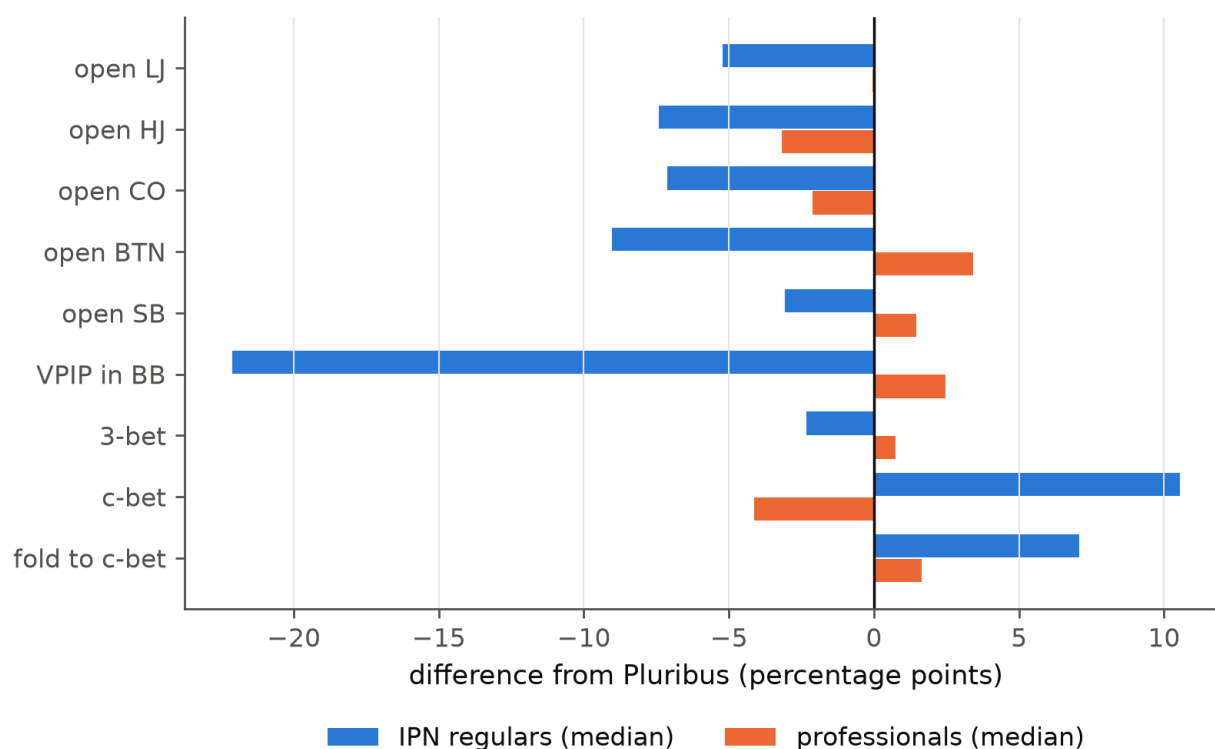


Figure 79: Median difference from Pluribus’s six-handed frequencies (percentage points) for the 1,046 iPoker regulars with at least 500 six-handed hands and the 13 professionals. The regulars defend the big blind far less and c-bet far more.

The two populations separate cleanly. Averaged over eleven pre-flop and flop frequencies, the median iPoker regular differs from Pluribus by 9.3 percentage points; the median professional by 3.9. Only 0.19 % of the 1,046 regulars with enough six-handed hands are closer to Pluribus than the median professional. The 2009 regulars defend the big blind far less: their median VPIP in the big blind is 22 points below Pluribus’s. After the flop they c-bet 10.5 points more often, and they fold to c-bets 7.1 points more often. The two flop gaps pull in opposite directions: these players bet the flop too often and give up on it too often. In the language of Chapter 8, these are the frequencies a safe exploiter would deviate against.

13.5 Predicting decisions: behavioral cloning and the baseline trap

Behavioral cloning is the simplest model of behavior: predict the action a player took from the situation they faced. The situation here is the public state — street, position, table size, pot, price, the betting so far and the board. Hole cards are left out on the iPoker data, because no card is known in 86 % of hands and, where known, selected by having been shown. The actions are six classes: fold, check, call, and a small, medium or large bet or raise relative to the pot. The model is trained on days 1–18 of the month and tested on days 19–25, so the test is the next week of the same people.

The plan set the target at 55 % accuracy, “with a random baseline of 14 %”. The model reaches 72.0 %,

but that number says little. More than half of all decisions are folds, and a model that always predicts the most common legal action scores 70.2 %. The informative quantity is the gain over that baseline. A frequency table over the coarse situation adds 0.4 points, the neural network 1.4 points, and giving the network the acting player’s style vector — twelve statistics computed from the training days only — adds another 0.4 points. The same style vector lowers the log-loss clearly, from 0.800 to 0.751 nats (3 seeds, standard error 0.0002). This is the quantitative form of a C1 claim: representing *who* acts improves the prediction of what they will do, and it improves the calibration of the prediction more than its top-choice accuracy.

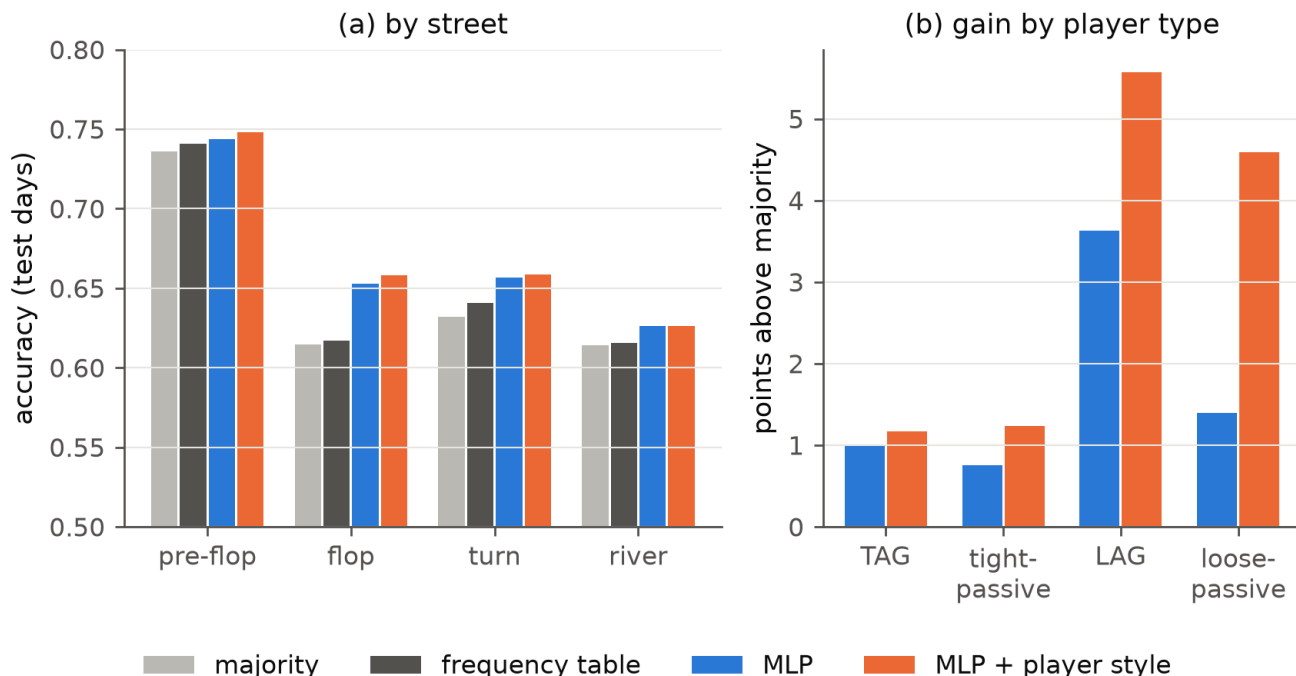


Figure 80: Behavioral cloning on the iPoker test days: (a) accuracy by street for the majority baseline, the frequency table, the MLP and the MLP with style vector (3 seeds); (b) accuracy above the majority baseline by player type.

Reconciliation (kept prediction → what actually happened). The plan predicted that pre-flop decisions would be the most predictable, the button the most predictable position, and tight-aggressive players more predictable than loose-aggressive ones, and those more than weak loose players. Pre-flop is indeed the most predictable street (0.748 against 0.626–0.659 post-flop), but the majority baseline shows the same order. The button is the *least* predictable position (0.639) and early position the most (0.813). Raw accuracy does order the types as predicted (tight-aggressive 0.786, loose-aggressive 0.650, loose-passive 0.642). Measured *above the baseline*, the order reverses: +1.2 points for tight-aggressive, +4.6 for loose-passive and +5.6 for loose-aggressive. I suspected a leak first and checked that the style vector uses only training days and that the test week is disjoint; it does, and it is. The predictions confused “easy to predict” with “folds a lot”. The lesson survives in a sharper form: the style vector

adds most for the players furthest from the population (+3.2 points for loose-passive players, +0.2 for tight-aggressive ones).

On the Pluribus table every hole card is known, and the same network reaches 81.8 % (84.5 % with the style vector, majority 69.7 %), with 92.6 % before the flop. That is the size of the part of behavior that the iPoker data hides. Kumar et al.¹⁶⁶ argue that on noisy data with sparse rewards, offline reinforcement learning can beat cloning. Poker logs are exactly that kind of data, so cloning here is used only as a description of behavior, not as a policy to play.

13.6 A behavioral fingerprint: player2vec for poker

Twelve statistics are a hand-made summary. The player2vec approach¹⁶⁷ replaces them with a learned one. Each in-game event becomes a token, and a Transformer¹⁶⁸ is trained to predict masked tokens from their context, as in BERT¹⁶⁹. The original paper applied this to a casual mobile game and read its clusters qualitatively; it did not test what the embedding captures.

For poker, each hand a player sits in becomes a short sentence: position and table size, one token per decision, and a token for how the hand ended. A 4-layer encoder is trained on each player’s stream from days 1–18 only.

What should a player embedding capture? At least the player. That gives a test that needs no labels. Each player is embedded twice, once from N hands of the training days and once from N different hands of the last week, which the model never saw. Then, for every player’s last-week vector, all players are ranked by similarity. The question is whether the player’s own first vector comes top. The statistics vector gets the same test.

¹⁶⁶Kumar, A., Hong, J., Singh, A. & Levine, S. (2022). “When Should We Prefer Offline Reinforcement Learning Over Behavioral Cloning?” *ICLR 2022*; arXiv:2204.05618.

¹⁶⁷Wang, T., Honari-Jahromi, M., Katsarou, S., Mikheeva, O., Panagiotakopoulos, T., Asadi, S. & Smirnov, O. (2024). “player2vec: A Language Modeling Approach to Understand Player Behavior in Games.” arXiv:2404.04234 (preprint).

¹⁶⁸Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. & Polosukhin, I. (2017). “Attention Is All You Need.” *NeurIPS 30*; arXiv:1706.03762.

¹⁶⁹Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” *NAACL-HLT*, 4171–4186. DOI 10.18653/v1/N19-1423.

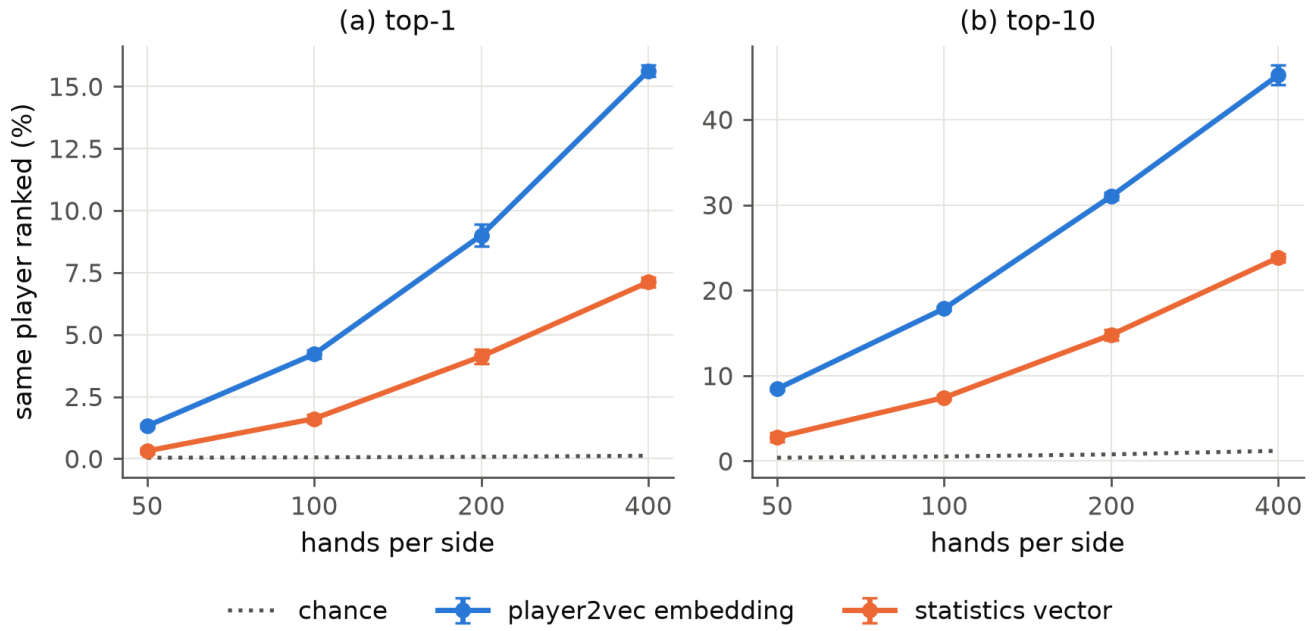


Figure 81: Re-identification on unseen days: share of players whose own later hands rank first (a) or in the top ten (b), for player2vec and the statistics vector, against hands per side (3 seeds, ± 1 SE).

The embedding recognizes a player at least twice as often as the statistics at every sample size. With 400 hands on each side it ranks the right player first among 834 for 15.6 % of them (± 0.2 , 3 seeds) and in the top ten for 45.2 %, against 7.1 % and 23.8 % for the statistics; chance is 0.12 %. With 100 hands the figures are 4.2 % against 1.6 %. Re-identification is also the multi-accounting question, whether one person can be recognized across accounts. The answer here is “sometimes, from a few hundred hands”: enough to rank candidates for a human to review, and far from proof.

The embedding also carries the classic axes, although it never sees a VPIP number. A fifteen-nearest-neighbor classifier recovers a regular’s VPIP $\times$ PFR quadrant from the embedding with 90 % accuracy, slightly more than from the statistics vector that contains VPIP and PFR (88 %). Cosine similarity is 0.87 within a quadrant and 0.75 between quadrants, so the plan’s check “intra-cluster similarity greater than inter-cluster similarity” holds.

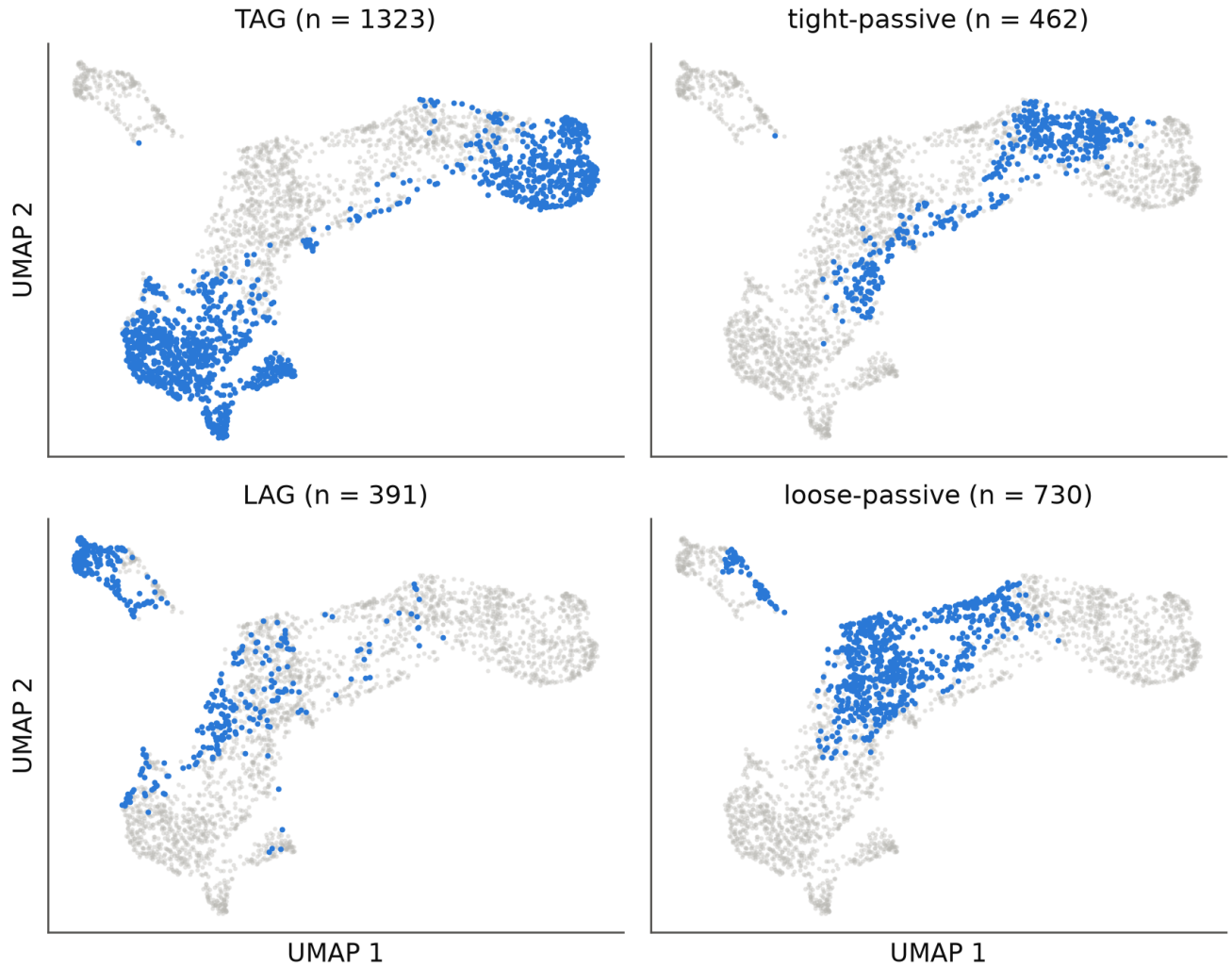


Figure 82: UMAP projection of the player2vec embeddings of the 2,906 iPoker regulars (seed 0), one panel per VPIP $\times$ PFR quadrant, in blue over all players in grey.

13.7 Types, stability and online inference

The plan expected four k-means clusters matching the four archetypes: tight-aggressive, loose-aggressive, tight-passive (“nit”) and loose-passive (“fish”). The data does not have four natural clusters. The silhouette¹⁷⁰, a measure of how well separated clusters are, is 0.16 for two or three clusters and 0.13 for four, low values throughout. k-means with four clusters is stable across seeds (adjusted Rand index¹⁷¹ 0.94 and 0.97), but what it finds is two tight-aggressive groups, one loose-aggressive and one loose-passive group, with no tight-passive cluster. Playing styles form a continuum with a dense tight-aggressive core, and “four archetypes” is a convention for describing it.

¹⁷⁰Rousseeuw, P. J. (1987). “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis.” *Journal of Computational and Applied Mathematics* 20, 53–65. DOI 10.1016/0377-0427(87)90125-7.

¹⁷¹Hubert, L. & Arabie, P. (1985). “Comparing partitions.” *Journal of Classification* 2(1), 193–218. DOI 10.1007/BF01908075.

Do players stay in their cluster? The plan’s test splits each regular’s hands into the first and second half of the month and asks whether both halves land in the same cluster; the target is more than 70 %. The measured figure is 73.6 % (± 0.2 , 3 seeds), so the target is met. A control shows how little that says: splitting the same hands *randomly*, which removes any change over time, gives 78.2 %. Most switching is therefore sampling noise at cluster borders, and genuine change over the month accounts for about five points.

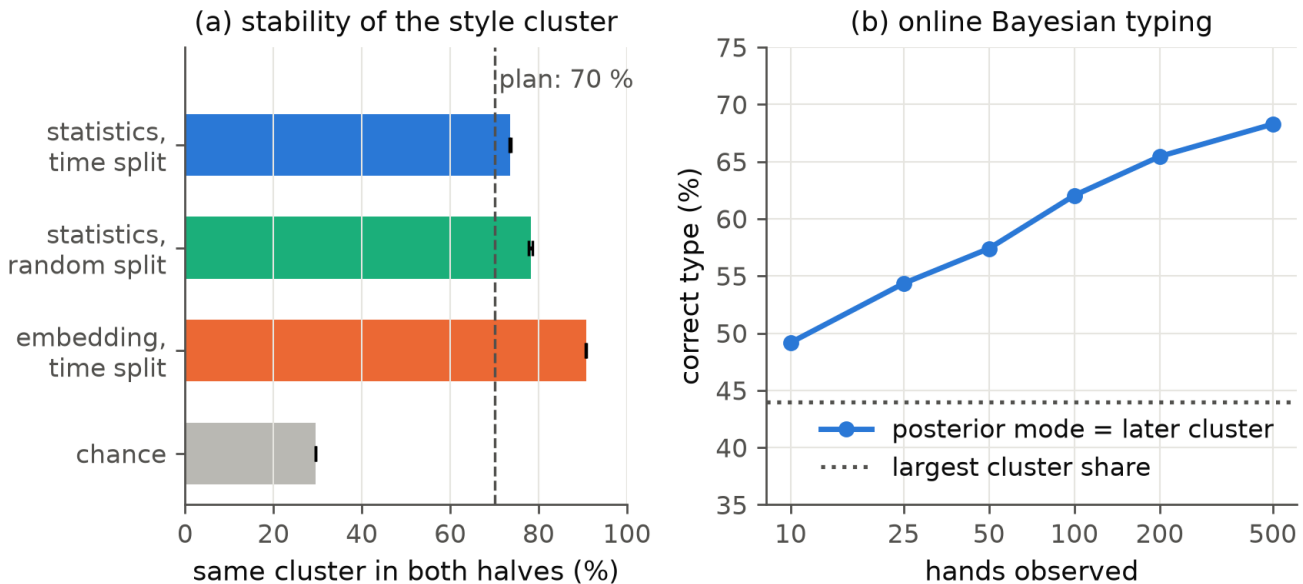


Figure 83: (a) Share of regulars in the same style cluster in both halves of their hands, chronological and random halves, with chance; (b) online Bayesian typing: share whose posterior mode after n hands matches their later cluster.

Clusters are still useful as the *prior* of an online model, which closes a loop the plan left open: data $\rightarrow$ clusters $\rightarrow$ prior $\rightarrow$ Bayesian update. This is the Bayesian opponent modeling of Chapter 7 and of *Bayes' Bluff*¹⁷², applied to real players. The four clusters become four types, each with its own rates for per-hand events. The posterior over a player’s type is then updated hand by hand. The truth is the cluster of the same player’s *later* hands, so the test does not reward memorizing the data it predicts. Over the 1,702 players with at least 1,000 hands, the posterior’s best guess is right for 49 % after 10 hands, 62 % after 100 and 68 % after 500; always guessing the largest cluster scores 44 %. Two thirds of the players reach a posterior of at least 0.9 on their later type within 500 hands, after a median 85 hands. The ceiling near 70 % is the temporal stability above: a model cannot predict a later cluster more reliably than players stay in it.

¹⁷²Southey, F., Bowling, M., Larson, B., Piccione, C., Burch, N., Billings, D. & Rayner, C. (2005). “Bayes’ Bluff: Opponent Modelling in Poker.” *UAI 2005*, 550–558; arXiv:1207.1411.

13.8 Collusion: detection on injected colluders in real sessions

Collusion in online poker takes three classic forms: colluders share information about their cards, they *soft-play* each other by not betting against a partner, or they *chip-dump* by losing on purpose to move money. No labelled collusion data is public. The main poker detectors were validated on synthetic colluders¹⁶³¹⁶⁴. Work on synthetic test benches argues that synthetic data should come first and be verified on real cases later¹⁷³.

This chapter keeps the ground truth synthetic but puts the colluders into *real* sessions. For each seed, 40 pairs of real players are chosen who already share at least 300 hands and 30 heads-up confrontations, so that sitting together cannot give them away. In a share q of their heads-up confrontations the rest of the hand is rewritten: for soft play both check or call to the end; for chip dumping one bets the pot, the other raises to three times that, and the first folds. Every rewritten hand was accepted by both the parser and pokerkit, so the injected hands are legal poker, not noise.

The detector carries over Chapter 11's help/harm idea: compare what a player does *to the partner* with what they do to everyone else. Harm withheld is soft play: a player's aggression against the partner, standardized against the same player's aggression when heads-up against anyone else. Help given is dumping: the chip flow between the two in their heads-up hands, standardized by its own spread. Chapter 11's score with raw event counts was kept as a baseline. A first run flagged pairs with soft-play scores above 100, all from heads-up *tables*, where the two players meet in every hand: it had compared a heads-up table with being left heads-up at a six-player table, which is a different game. The headline run compares like with like and uses multiway tables only.

¹⁷³Smed, J., Knuutila, T. & Hakonen, H. (2007). "Towards Swift and Accurate Collusion Detection." *GAME-ON 2007*, 103–107.

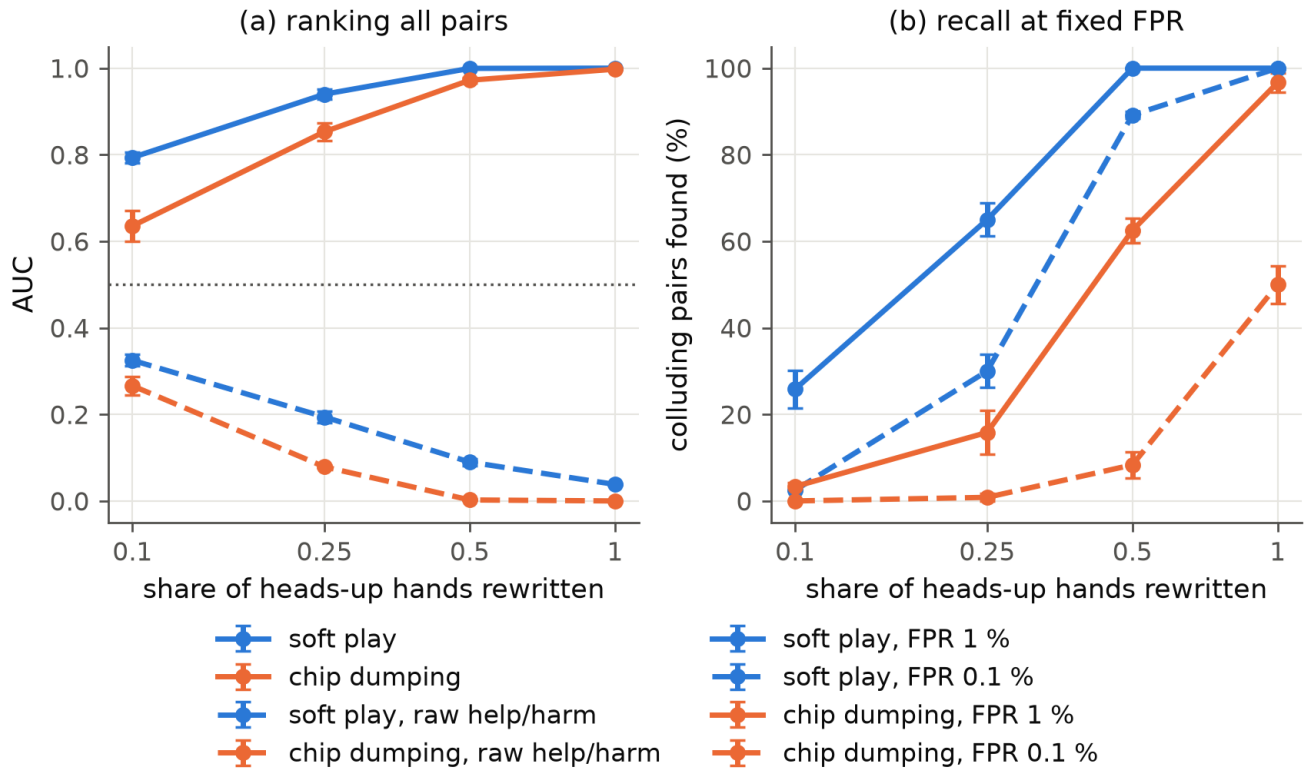


Figure 84: Collusion injected into real iPoker sessions (multiway tables, 3 seeds, ± 1 SE): (a) AUC of the union detector and of Chapter 11’s raw help/harm score against the share rewritten; (b) pairs found at 1 % and 0.1 % false positives.

Against the 2,802 real pairs of the same stratum, which are presumed clean, soft play is the easier pattern. If a quarter of the confrontations are soft, the detector ranks the colluding pair above a random real pair 94 % of the time ($\text{AUC } 0.940 \pm 0.011$), and finds 65 % of the pairs at a 1 % false-positive rate. From half the confrontations upward it finds all of them. Dumping needs more: 62 % of pairs are found at 1 % false positives when half the confrontations are dumps, and 97 % when all are. Light collusion, one confrontation in ten, is mostly invisible (26 % and 3 % found).

Reconciliation (kept prediction $\rightarrow$ what actually happened). The plan set the target at “detects 100 % of injected synthetic collusion at the chosen threshold, false positive rate on known-clean data < 5 %”. Measured: met for soft play once half the confrontations are soft (false-positive rate 0.1 % at 90 % recall), and for dumping only when every confrontation is a dump (0.4 %). Chapter 11’s help/harm score does not transfer with raw counts: its AUC is 0.04–0.33 in every setting, worse than chance. I suspected an inverted sign first; the code is right, and the mechanism is in the data. Soft play removes the partner’s folds as well as the bets, so help falls with harm, and dumping *adds* bets, which counts as harm. The idea behind the detector survives; the raw counts do not. It works once each player is compared with their own behavior, and once like is compared with like.

The highest scores among real pairs, 7.29, 4.71 and 4.16, lie above the null’s 99.9th percentile. They are candidates for a human to review, not findings; this is 2009 data with no labels, and one pair in a thousand lies above that percentile by construction.

13.9 Can the pipeline find a bot?

A bot in a human game is the other fair-play concern. The Pluribus hands allow a direct test: cut every player’s hands into blocks of 250, give the pipeline no labels, and see whether its anomaly scores single out Pluribus’s blocks. The Math Flag of the plan predicted that a bot would be *unusually consistent*, so its masked-token loss would be low.

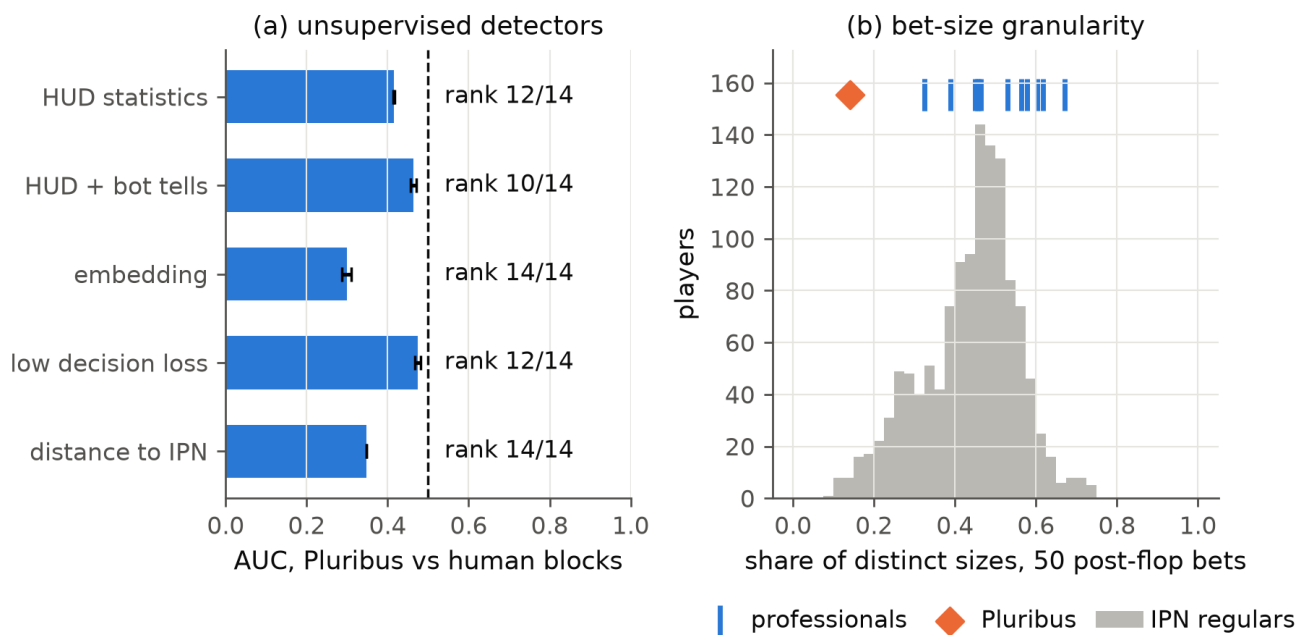


Figure 85: (a) Five unsupervised detectors on 250-hand blocks of the Pluribus games: AUC of Pluribus’s blocks against the professionals’ and Pluribus’s rank among 14; (b) share of distinct bet sizes among 50 post-flop bets per player.

None of the five unsupervised detectors finds Pluribus. Isolation Forest¹⁷⁴ on three feature sets, low decision loss and distance from the iPoker population score AUCs of 0.298–0.475, and Pluribus ranks 10th to 14th of 14 players. Every AUC is *below* one half: Pluribus is more typical of these games than the average professional. Its decision loss, 0.691, sits inside the human range (0.529–0.829). A bot that plays a fixed mixed strategy is predictable in its *distribution* of actions, not in its actions. Low loss would flag a deterministic script, not an equilibrium-style bot.

Looking with the label shows two things that do separate it. First, Pluribus bets from a small menu

¹⁷⁴Liu, F. T., Ting, K. M. & Zhou, Z.-H. (2008). “Isolation Forest.” *IEEE International Conference on Data Mining (ICDM)*, 413–422. DOI 10.1109/ICDM.2008.17.

of sizes. Among 50 of its post-flop bets, only 14 % of the sizes are distinct, against 32–67 % for the professionals, and 16 of 1,275 iPoker regulars (1.3 %) are as low. Second, it randomizes given its cards. Before the flop, with position, situation and exact starting hand fixed, its choice of action has an entropy of 0.143 nats, the highest of the nine players with enough data (humans 0.059–0.107). The size menu and the randomization were found by looking at one bot with its label known. They are hypotheses to be tested on data with more known bots, not validated detectors, and a bot designer who knows them could remove both.

13.10 Honest notes, limitations, and where this hands off

What did not, and why it matters. Four of the plan’s expectations failed on real data. Accuracy targets set against a random baseline are meaningless when most decisions are folds. The four textbook archetypes are not natural clusters. Chapter 11’s raw help/harm score points the wrong way on poker. And a strong bot is not an outlier. Each failure was checked for a bug before it was accepted, and each has a mechanism the chapter can name.

Limitations. (1) The data cover one month of one network in 2009: players, stakes and software have changed since, and every threshold must be re-measured before it is applied to current play. (2) The collusion is synthetic and covers two scripted patterns. Information sharing, the most common form, is not modeled and would not show in these signals. (3) The optional Decision Transformer on real data was not run. The iPoker outcomes are incomplete, and for the Pluribus games there is no simulator to evaluate a return-conditioned policy. Only the luck-driven leakage that Paster et al.¹⁷⁵ warn about could have been measured.

Connections to other chapters. Chapter 14 turns to evaluation methodology (C3), where collusion has so far been measured apart from play. The thesis’s adaptive agent would use the online typing and the collusion signals inside its decisions (C1, C2), which is the coupling this chapter does not attempt.

13.11 Key takeaways for the thesis synthesis

- **Parse before you model.** Three data conventions each corrupted results until two independent replays were compared.
- **Every number needs its baseline and its sample size.** 72 % action accuracy is 1.8 points over “the most common action”; 73.6 % style stability is 4.6 points *under* a random split; VIP is reliable after 100 hands and fold-to-c-bet not after 1,000.
- **Representing the player helps (C1).** The player’s style vector lowers the prediction loss from 0.800 to 0.751 nats. A self-supervised embedding recognizes a player on unseen days 15.6 % of the time among 834 (7.1 % for the statistics, 0.12 % by chance).

¹⁷⁵Paster, K., McIlraith, S. & Ba, J. (2022). “You Can’t Count on Luck: Why Decision Transformers and RvS Fail in Stochastic Environments.” *NeurIPS* 35, 38966–38979.

- **Styles are a continuum, types are still a prior.** There are no four natural clusters, but an online Bayesian model over four learned types reaches a confident type within a median 85 hands.
- **Collusion detection works where each player is compared with themselves.** Strong soft play and dumping are found at 1 % false positives; light collusion is not. Raw help/harm counts fail on poker.
- **A strong bot is not an outlier.** Unsupervised detectors rank Pluribus 10th–14th of 14. Its bet sizes and its randomization separate it, and these are hypotheses still to be tested on other data.

14 Chapter 14 — Evaluation Frameworks and Exploitability Metrics

This chapter is about how to evaluate game-playing agents that **adapt**: agents that watch their opponents and change their play. It builds the standard toolkit — exact exploitability, population rankings and variance reduction — validates it, and uses it to measure what it misses. **All experimental numbers were measured** on three small poker games (two-player Kuhn and Leduc, three-player Kuhn) and Chapter 11’s four-player So Long Sucker, *exactly* wherever the game allows; where a run contradicted my expectation, both are kept.

Where this sits in the thesis. This chapter is Contribution 3 (evaluation methodology). The September 2026 literature check narrowed that contribution: combining exploitability, population ranking and confidence is not new on its own — a benchmark for repeated rock–paper–scissors already scores return and exploitability together, for one game¹⁷⁶. What no protocol found reports together is (a) the **gain** against a sub-optimal population, (b) the **speed** of adaptation and the **recovery** after an opponent switches, and (c) the **worst case**, or with more than two players a coalition-aware substitute — each with confidence bounds, applied unchanged across several imperfect-information games. The chapter builds such a protocol and shows, on concrete agent zoos, where the existing measures break. It is also the yardstick for Chapter 7’s adaptive agents (Contribution 1) and for the question Chapter 8 left open — what “safe” can mean with three players (Contribution 2).

14.1 Why evaluating an adaptive agent is hard

There are two ways to grade a game-playing agent. The first asks *how badly can it be beaten?* In a two-player zero-sum game the answer is **exploitability**: how far a best-responding opponent can push the agent below the game value. The second asks *how well does it do against the opponents it will meet?*, answered by head-to-head win rates and ratings from Elo¹⁷⁷ to Nash averaging¹⁷⁸, α -Rank¹⁷⁹ and voting-based evaluation¹⁸⁰. The two are known to disagree: bots indistinguishable head-to-head were about 1,300 mbb/game apart in exploitability¹⁸¹, and the computer poker competition used two winner rules¹⁸².

¹⁷⁶Lanctot, M., Schultz, J., Burch, N., Smith, M. O., Hennes, D., Anthony, T. & Pérolat, J. (2023). “Population-based Evaluation in Repeated Rock-Paper-Scissors as a Benchmark for Multiagent Reinforcement Learning.” *Transactions on Machine Learning Research*; arXiv:2303.03196 — 43 bots, 1,000-throw matches; population return, within-population exploitability and their difference as an aggregate score.

¹⁷⁷Elo, A. E. (1978). *The Rating of Chessplayers, Past and Present*. New York: Arco.

¹⁷⁸Balduzzi, D., Tuyls, K., Pérolat, J. & Graepel, T. (2018). “Re-evaluating Evaluation.” *NeurIPS*; arXiv:1806.02643 — Elo is inflated by copies of beaten agents; Nash averaging is invariant to redundant agents.

¹⁷⁹Omidshafiei, S., Papadimitriou, C., Piliouras, G., Tuyls, K., Rowland, M., Lespiau, J.-B., Czarnecki, W. M., Lanctot, M., Pérolat, J. & Munos, R. (2019). “ α -Rank: Multi-Agent Evaluation by Evolution.” *Scientific Reports* 9, 9937. DOI 10.1038/s41598-019-45619-9.

¹⁸⁰Lanctot, M., Larson, K., Bachrach, Y., Marris, L., Li, Z., Bhoopchand, A., Anthony, T., Tanner, B. & Koop, A. (2023). “Evaluating Agents using Social Choice Theory.” *arXiv:2312.03121* (v4, 2025) — Voting-as-Evaluation; maximal lotteries recommended.

¹⁸¹Moravčík, M. et al. (2017). “DeepStack: Expert-level artificial intelligence in heads-up no-limit poker.” *Science* 356(6337), 508–513; arXiv:1701.01724 — also reports an 85 % reduction in standard deviation with AIVAT.

¹⁸²Bard, N., Hawkin, J., Rubin, J. & Zinkevich, M. (2013). “The Annual Computer Poker Competition.” *AI Magazine* 34(2), 112–114 — total-bankroll and instant-runoff winner rules; duplicate play.

An adaptive agent makes the disagreement structural. It leaves equilibrium on purpose, so exploitability can only count against it however much it gains, and its payoff depends on how long it plays and what it has seen, so it has no single entry in the payoff matrix that population rankings assume. With three or more players the worst case changes meaning: NashConv measures distance from *an* equilibrium¹⁸³, and in three-player Kuhn a player can pass utility to a second at the third’s expense while everyone plays equilibrium strategies¹⁸⁴. The only coalition worst case the literature check found is the team-maxmin value¹⁸⁵ — a solution concept, not an evaluation protocol.

A picture helps. Hiring a negotiator, one reference says “the worst deal a ruthless counterpart could force on this person”, another “how they did in their last twenty negotiations”. Neither says how fast they read a new counterpart, whether they notice a change of tactics, or whether someone who acts naive and then turns can play them — nor, with three parties, what happens when the other two coordinate. The chapter’s protocol asks all of these, with error bars.

14.2 The framework: three layers and a fourth question

Layer 1 is the worst case: exact exploitability in small games, a learned best response in large ones¹⁸⁶. **Layer 2** is the population: round-robin payoffs, Elo, Nash averaging, the meta-Nash (Chapters 9–10), α -Rank, VasE’s maximal lotteries and the “spinning top” decomposition¹⁸⁷¹⁸⁸. **Layer 3** is confidence: seeds, duplicate dealing and AIVAT¹⁸⁹. The fourth question is the chapter’s own: how an adaptive agent behaves *over time*.

¹⁸³Lanctot, M. et al. (2019). “OpenSpiel: A Framework for Reinforcement Learning in Games.” *arXiv:1908.09453* — NashConv as the sum of each player’s incentive to deviate; exploitability = NashConv / n.

¹⁸⁴Szafron, D., Gibson, R. & Sturtevant, N. (2013). “A Parameterized Family of Equilibrium Profiles for Three-Player Kuhn Poker.” *AAMAS*, 247–254.

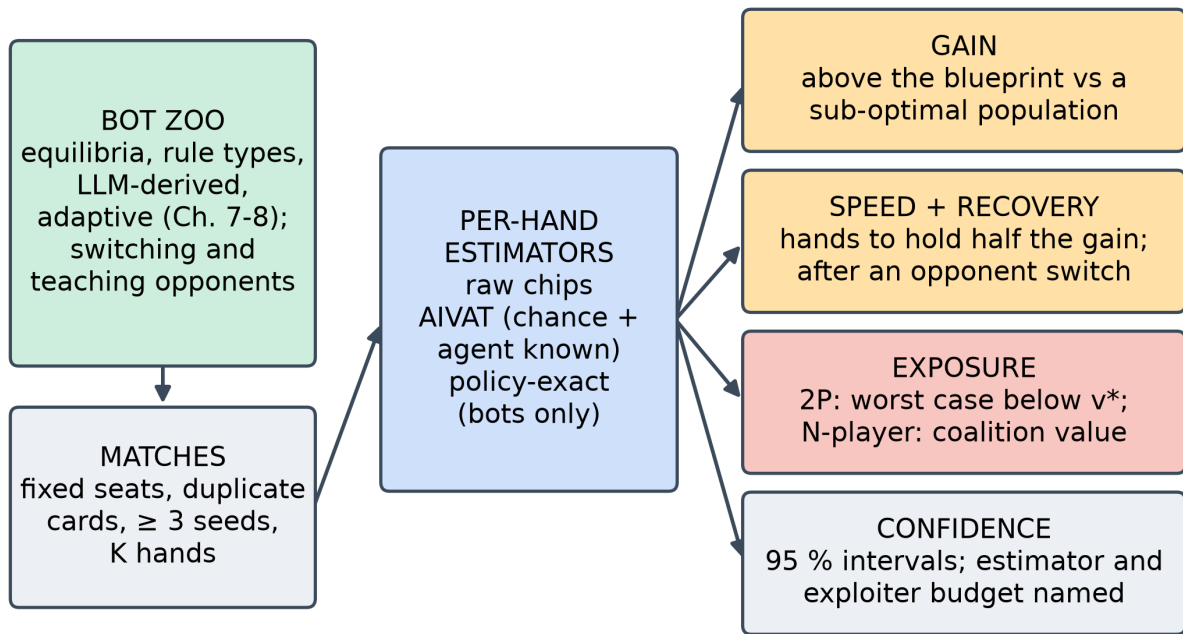
¹⁸⁵Celli, A. & Gatti, N. (2018). “Computational Results for Extensive-Form Adversarial Team Games.” *AAAI* 32(1). DOI 10.1609/aaai.v32i1.11462; see also Zhang, Y., An, B. & Černý, J. (2021). “Computing Ex Ante Coordinated Team-Maxmin Equilibria in Zero-Sum Multiplayer Extensive-Form Games.” *AAAI* 35(6), 5813–5821.

¹⁸⁶Timbers, F., Bard, N., Lockhart, E., Lanctot, M., Schmid, M., Burch, N., Schrittwieser, J., Hubert, T. & Bowling, M. (2022). “Approximate Exploitability: Learning a Best Response.” *IJCAI*; arXiv:2004.09677.

¹⁸⁷Balduzzi, D. et al. (2019). “Open-ended Learning in Symmetric Zero-sum Games.” *ICML*, PMLR 97, 434–443 — the transitive/cyclic decomposition.

¹⁸⁸Czarnecki, W. M., Gidel, G., Tracey, B., Tuyls, K., Omidshafiei, S., Balduzzi, D. & Jaderberg, M. (2020). “Real World Games Look Like Spinning Tops.” *NeurIPS*; arXiv:2004.09468.

¹⁸⁹Burch, N., Schmid, M., Moravčík, M., Morrill, D. & Bowling, M. (2018). “AIVAT: A New Variance Reduction Technique for Agent Evaluation in Imperfect Information Games.” *AAAI* 32(1); arXiv:1612.06915 — unbiased for any value function (Theorem 1); on Leduc, 48–75 % lower standard deviation for dissimilar strategies.



Context, not a verdict: Elo, Nash averaging, α -Rank (α swept), VasE, spinning top

Figure 86: The joint protocol. A bot zoo, including switching and teaching opponents, plays fixed-seat duplicate matches over several seeds; each hand is scored by three estimators and four readouts are reported together.

Everything rests on one exact engine. Each game is enumerated once from OpenSpiel’s definition, so a strategy is a vector and its expected payoff, best response and exposure are matrix operations (Leduc: 9,457 nodes, 468 information sets per player, built in 0.04 s). An adaptive agent is a *sequence* of strategies, one per refit, each scored exactly. The engine reproduces OpenSpiel, Chapter 3 and Chapter 7 to within rounding (`validation.json`).

The four readouts are defined as follows (chips per hand throughout).

- **Gain:** the agent’s expected payoff minus the equilibrium blueprint’s, against the same opponent in the same seat; its **capture** divides this by what the exact best response would gain.
- **Speed and recovery:** *h50* is the first hand at which the capture reaches one half and stays there on average over the next 100 hands; *r50* is the same count after the opponent switches. The adaptive agents refit every 50 hands, so both are resolved to 50 hands.
- **Exposure:** for two players, how far below the game value v^* a best responder could push the policy in force — exploitability applied to the current policy rather than to a fixed agent; plus the realised loss under a white-box teaching attack. For three players, the **coalition value:** the least the other two can hold the agent to with coordinated strategies.
- **Confidence:** 95 % intervals over seeds, with the estimator named. There are three: raw chips;

AIVAT with the agent’s own strategy known; and the *policy-exact* expected payoff of the two policies in force, available only when both are bots.

The zoos have 16 agents on Kuhn and 12 on Leduc: the **equilibrium** blueprint (OpenSpiel’s CFR+); Chapter 7’s **rule-based types** (AlwaysPass, AlwaysBet, TightPassive, LooseAggr, Threshold on Kuhn; CallingStation, Maniac, Rock, LoosePassive on Leduc) plus Random; on Kuhn, three **strategies extracted from language models** in Chapter 12 (Qwen2.5-7B-Instruct, gpt-oss-20b and OpenThinker3-7B, read from action-token log-probabilities; quantisation not recorded; OpenThinker3-7B is a weak stand-in, with 34 % of its mass unmappable to an action); a **specialist**, a static best response to one weak type; and five **adaptive agents** combining Chapter 7’s models with Chapter 8’s responses:

- *TypeBR*: a Bayesian posterior over the types, best-responded to;
- *DirBR*: the continuous Dirichlet model, best-responded to;
- *DirBR-CP*: DirBR with Chapter 7’s change-point reset;
- *RNR(0.5)*: the continuous model inside a restricted Nash response with $p = 0.5$ ¹⁹⁰;
- *BestEq*: the best equilibrium against the model — the most it can earn while never falling below v^* , Ganzfried and Sandholm’s best-equilibrium baseline¹⁹¹.

Chapter 8’s constraint-generation loop for the last two did not converge on full Leduc. Here they are solved with the sequence-form dual linear program^{192,193}: one LP per method solves full Leduc in 0.02–0.04 s and reproduces Chapter 8’s Kuhn results.

14.3 Layer 2 — rankings disagree, in predictable ways

Each pair plays 2,000-hand matches in both seats with duplicate cards over 10 seeds (stationary pairs exactly); Elo is fitted to the probability of finishing a 100-hand session ahead (`population_{kuhn,leduc}.json`).

¹⁹⁰Johanson, M., Zinkevich, M. & Bowling, M. (2007). “Computing Robust Counter-Strategies.” *NIPS 2007* — Restricted Nash Response.

¹⁹¹Ganzfried, S. & Sandholm, T. (2015). “Safe Opponent Exploitation.” *ACM Transactions on Economics and Computation* 3(2), Article 8. DOI 10.1145/2716322 — the best-equilibrium strategy is their baseline; their safe algorithms risk only gifts already won.

¹⁹²Koller, D., Megiddo, N. & von Stengel, B. (1996). “Efficient Computation of Equilibria for Extensive Two-Person Games.” *Games and Economic Behavior* 14(2), 247–259.

¹⁹³von Stengel, B. (1996). “Efficient Computation of Behavior Strategies.” *Games and Economic Behavior* 14(2), 220–246 — the sequence form.

DirBR	10	2	1	2	6	4	4	5
RNR(0.5)	3	1	2	1	5	3	1	4
TypeBR	8	6	3	7	3	5	6	3
BR-Rock	5	5	4	6	4	5	5	7
DirBR-CP	12	7	5	5	8	5	7	6
BestEq	1	3	6	3	1	1	2	2
Nash	1	4	7	4	1	2	3	1
Rock	4	8	8	8	7	8	8	7
LoosePassive	7	9	9	9	10	9	9	7
CallingStation	6	11	10	10	11	11	9	7
Maniac	9	10	11	11	9	10	9	7
Random	11	12	12	12	12	12	9	7

Exploit. Elo Pop. return RRPS score Nash avg. VasE α -Rank 0.1 α -Rank 100
 ranking method (rank 1 = best)

Figure 87: Rank of every Leduc agent under eight methods (1 = best). RNR(0.5) and DirBR lead Elo, population return and RRPS; BestEq and Nash lead exploitability and Nash averaging.

The rankings fall into two camps. **Exploitability, Nash averaging and VasE** favour agents that never lose: on Leduc the meta-game’s maximum-entropy Nash mixture is BestEq 0.88, Nash 0.11, TypeBR 0.01, and RNR(0.5) and DirBR get zero weight despite the highest population return (+0.807 and +0.890 chips/hand). **Elo, population return and the RRPS score** favour agents that beat many opponents: Elo on Leduc is RNR(0.5) 1892, DirBR 1848, BestEq 1779, Nash 1763. The Kendall correlation between the Elo and exploitability rankings is 0.45 on Kuhn and 0.36 on Leduc. For an exploiting agent the two questions have opposite answers, and a single ranking silently picks one.

Three more predicted failure modes appear. **Clones inflate Elo.** Adding up to eight copies of the weakest bot widens DirBR’s Elo lead over Nash from +166 to +181 points on Kuhn and from +84 to +113 on Leduc, while no Nash-averaged skill moves by more than $6e-6^{178}$. **α -Rank depends on α .** As α grows from 0.01 to 100, three different agents lead on Kuhn (DirBR, RNR(0.5), Nash) and four on Leduc (DirBR, RNR(0.5), BestEq, Nash). **The horizon matters.** Truncating the same runs to 100 hands puts TypeBR first in Elo on Kuhn and the *static* specialist BR-Rock first on Leduc; from 300 hands on, DirBR-CP, then DirBR (Kuhn) and RNR(0.5) (Leduc) lead. An adaptive agent’s place in a meta-game is only defined once the match length is fixed.

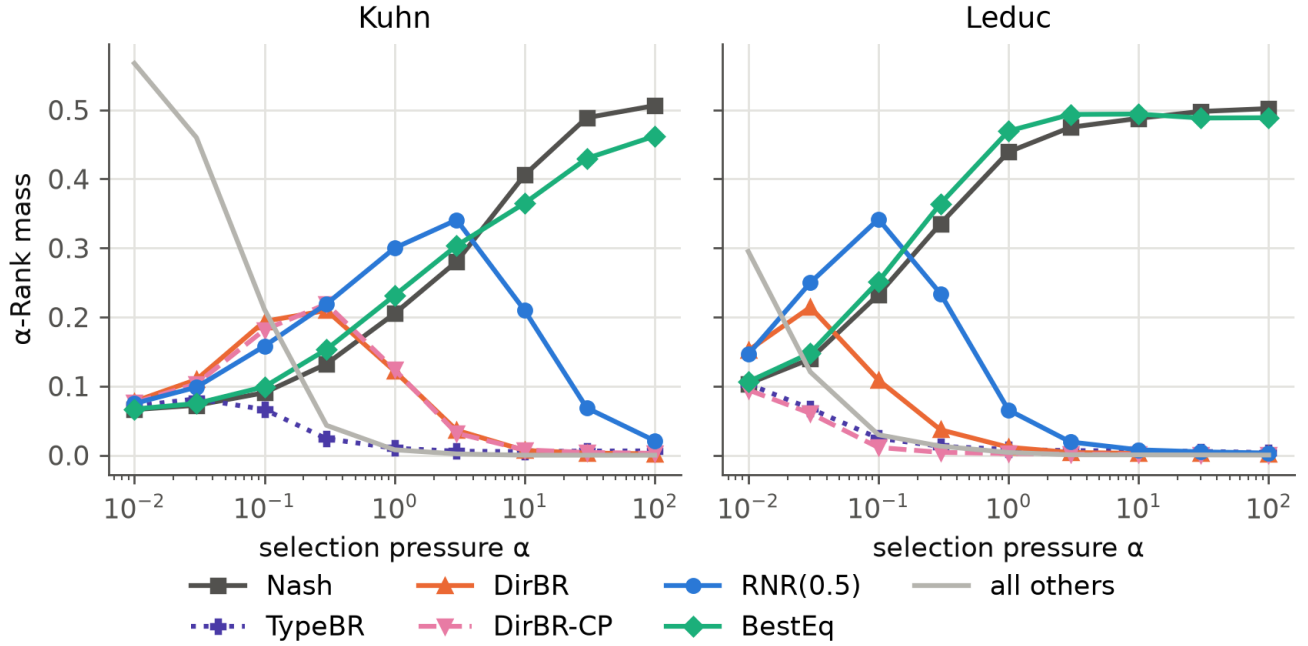


Figure 88: α -Rank mass of the main agents as the selection pressure α grows; the leader moves from DirBR through RNR(0.5) to Nash on Kuhn, and through BestEq to Nash on Leduc.

Ranking confidence is uneven too: over 200 seed resamples Elo keeps its Kuhn winner every time, α -Rank at $\alpha = 10$ only in 22 %, because near-tied equilibrium agents make its top unstable¹⁹⁴. And an arena-style rating misorders the LLM-derived strategies: Elo puts the Qwen2.5-7B strategy 7th of 16 and the Threshold bot 10th, although Qwen’s exploitability is 0.179 chips/hand against Threshold’s 0.118 (Chapter 12’s 0.357 is NashConv; here NashConv/2). LLM arenas that rank by rating alone inherit this.

14.4 Layer 3 — confidence, and what AIVAT taught

AIVAT corrects each hand’s result with control variates for chance events and for the actions of any player whose strategy is known, plus “imaginary observations” over that player’s other possible private cards; the estimate is unbiased for any value function¹⁸⁹. Tabulating the AIVAT value of every terminal once per strategy gives the estimator’s *exact* mean and variance.

The first implementation had a real bug, and the check that caught it is worth keeping: with both strategies known and a perfect value function, AIVAT’s variance must be zero. It was not: the known player’s own card deal had received a chance-correction term, although the imaginary observations already average over that card, so its luck was counted twice. Removing it restored exactly zero variance and made the realistic version, with only the agent’s own strategy known, better than chance-only correction everywhere.

¹⁹⁴Rowland, M., Omidshafiei, S., Tuyls, K., Pérolat, J., Valko, M., Piliouras, G. & Munos, R. (2019). “Multiagent Evaluation under Incomplete Information.” *NeurIPS*; arXiv:1909.09849.

With blueprint self-play values as the value function, the plan’s targets — variance divided by at least 5 on Kuhn and 10 on Leduc — are met for 18 of 20 Kuhn pairs (median factor 48) and 7 of 12 Leduc pairs (median 12.2); the misses are opponents the self-play values predict badly. AIVAT cannot be used for humans, whose strategy is unknown¹⁹⁵, and its value function must be fixed before the data are seen¹⁹⁶ — here, the blueprint.

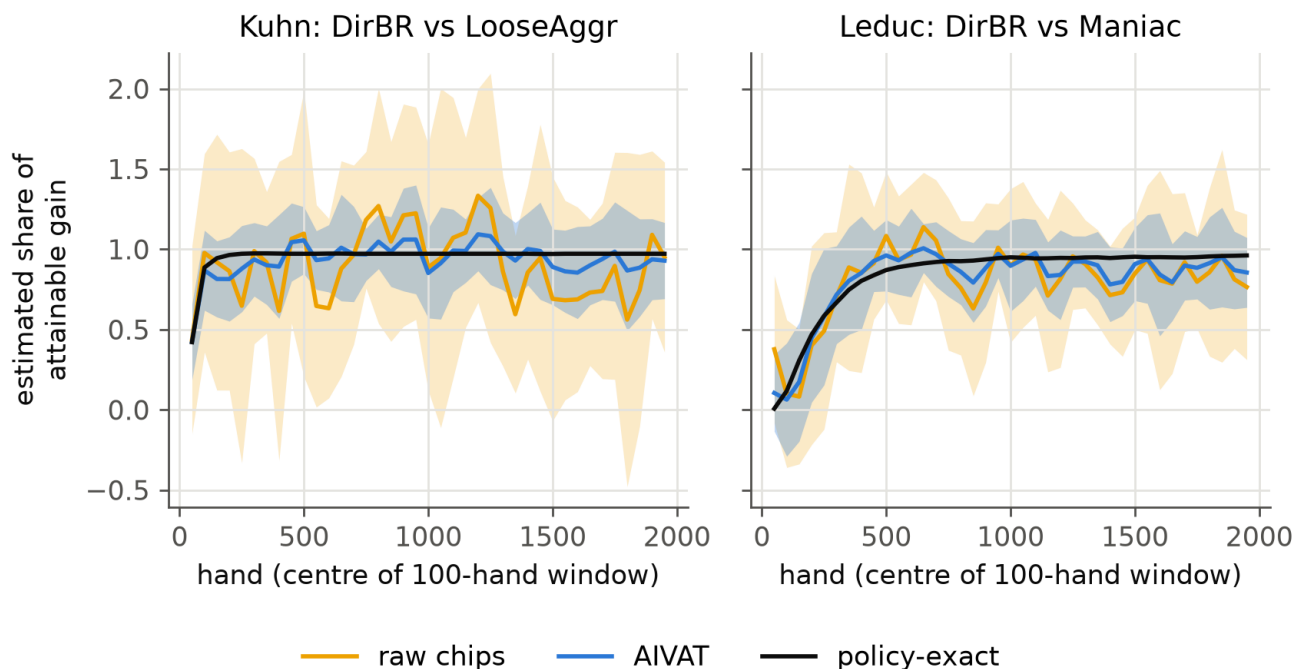


Figure 89: Share of the attainable gain estimated in 100-hand windows from raw chips, AIVAT and exact policy values, for DirBR against LooseAggr (Kuhn) and Maniac (Leduc); mean ± 1 SD over 20 matches.

Measuring *adaptation* rather than a final win rate raises the price sharply, because the quantity of interest lives in short windows. To estimate the capture of one 100-hand window to ± 0.25 with 95 % confidence, the median agent–opponent pair needs 3,383 hands with raw chips on Kuhn and 379 with AIVAT; on Leduc, 1,313 and 354. On real data, the 10,000 released Pluribus hands (Chapter 13) give a raw win rate of -70.9 ± 172.8 mbb/hand (95 % interval), against 48 mbb/game with a standard error of 25 with AIVAT and Pluribus’s strategy¹⁹⁵. A third party holding only the hand histories cannot reach significance on the win rate, let alone on how fast the agent adapts.

¹⁹⁵Brown, N. & Sandholm, T. (2019). “Superhuman AI for multiplayer poker.” *Science* 365(6456), 885–890. DOI 10.1126/science.aay2400 — 48 mbb/game, standard error 25, over 10,000 hands; AIVAT cannot be applied to the human players.

¹⁹⁶Kim, J. & Sandholm, T. (2026). “Heuristic Pathologies and Further Variance Reduction via Uncertainty Propagation in the AIVAT Family of Techniques.” *arXiv:2605.14261*.

14.5 The joint protocol on Kuhn and Leduc

Seven agents were run against every sub-optimal stationary zoo member, three opponents that switch at hand 1,000 of 2,000, and three teaching attacks, each for 10 seeds in both seats (`adaptation_{kuhn,leduc}.json`). A teaching attack plays a bait strategy until hand 1,000 and then the exact best response to the agent’s current policy, refreshed every 50 hands — the “taught then exploited” scenario behind adaptation safety¹⁹⁷.

Table 39: Gain against the sub-optimal population, mean exposure of the policy in force, and loss below v^* after a teaching attack, on Kuhn (K) and Leduc (L), chips per hand. 95 % intervals on the gains are at most 0.007.

Agent	Gain K	Exposure K	Teach loss K	Gain L	Exposure L	Teach loss L
Nash	0	0.000	0.000	0	0.000	0.000
BestEq	+0.020	0.000	0.000	+0.047	0.000	0.000
RNR(0.5)	+0.210	0.090	0.045	+0.797	0.264	0.179
DirBR	+0.273	0.338	0.246	+0.988	2.461	1.241
DirBR-CP	+0.230	0.305	0.199	+0.441	2.459	1.757
TypeBR	+0.229	0.352	0.337	+0.961	1.885	2.000

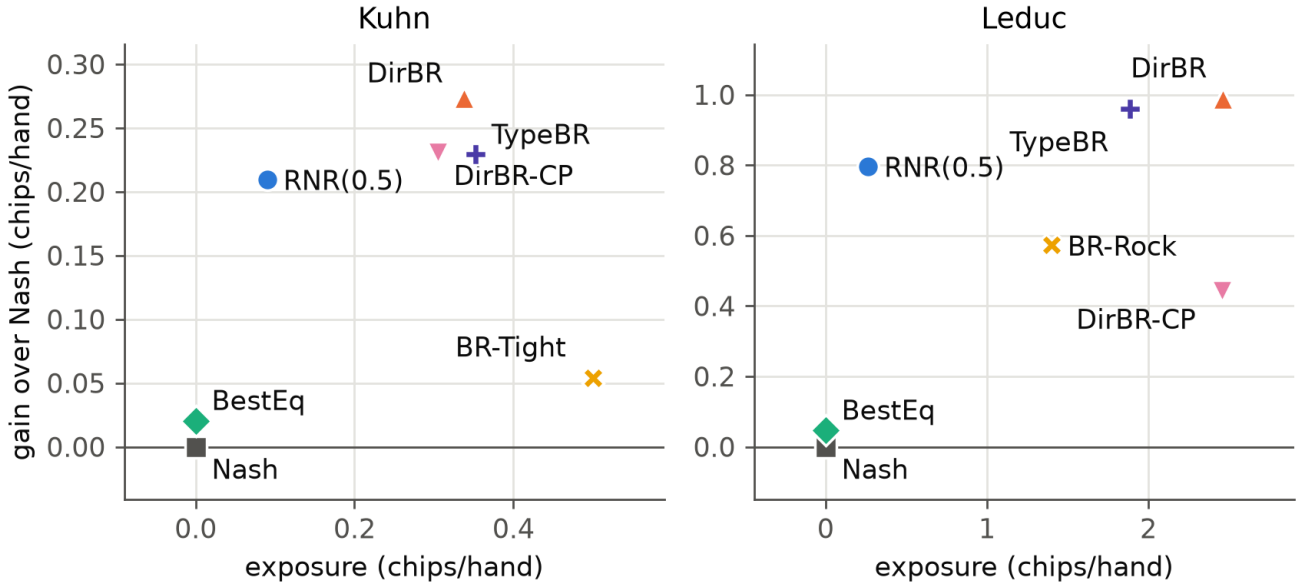


Figure 90: Gain over the Nash blueprint against the sub-optimal population, against exposure. Error bars are 95 % intervals over 10 seeds and are smaller than the markers.

¹⁹⁷Ge, Z., Xu, Z., Ding, T., Meng, L., An, B., Li, W. & Gao, Y. (2024). “Safe and Robust Subgame Exploitation in Imperfect Information Games.” *ICML*, PMLR 235, 15255–15270 — adaptation safety and the “taught and exploited” problem.

Read together, the numbers separate agents that every single ranking conflates. **BestEq** is safe by construction and gains almost nothing: 0.020 and 0.047 chips/hand. **RNR(0.5)** gains 10 to 17 times as much at an exposure of 0.090 on Kuhn and 0.264 on Leduc, and loses at most 0.274 chips/hand to any teaching attack. **DirBR** gains a little more still (0.988 on Leduc) but at an exposure of 2.46 — almost thirty times the game value — and a teaching attacker takes 1.0–1.5 chips/hand back. Exploitability ranks BestEq and Nash first; Elo ranks them the other way. Only the pair (gain, exposure) shows that RNR(0.5) buys most of the gain for a small, measured risk. Exposure is also the upper envelope of the teaching loss: an attacker in step with the refits takes exactly the current policy’s exposure, and the measured losses equal it for every agent except DirBR-CP, whose resets leave the attacker’s response stale.

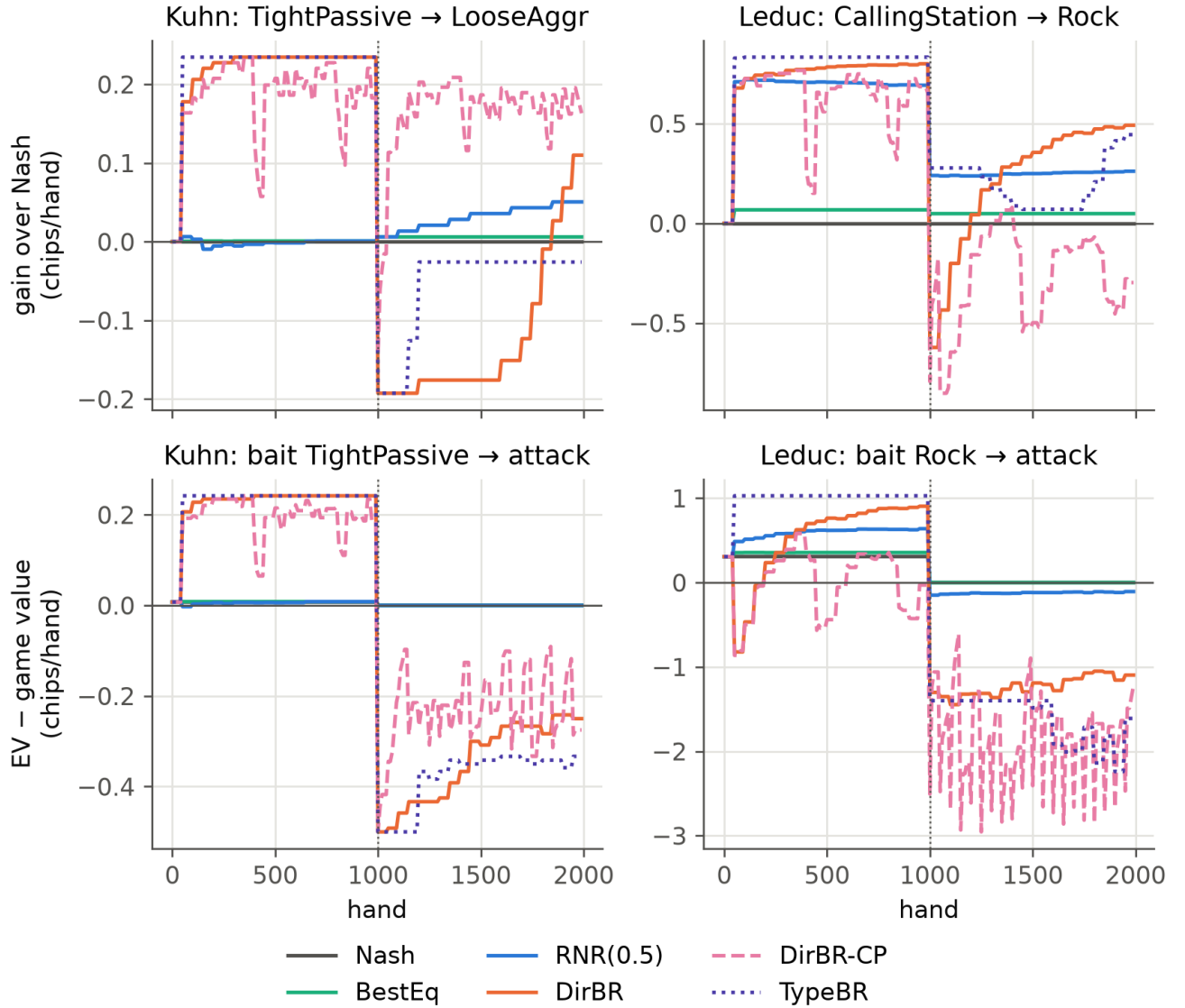


Figure 91: Top: gain over Nash when the opponent switches at hand 1,000. Bottom: expected payoff minus the game value under a teaching attack (bait until hand 1,000, then the exact best response, refreshed every 50 hands).

Speed and recovery separate agents with similar gains. On Kuhn every adaptive agent reaches half the attainable gain at the first refit (median $h50 = 50$ hands), in 100 % of matches for DirBR-CP down to 73 % for RNR(0.5). After the Kuhn switch from TightPassive to LooseAggr, where the old exploit fails, DirBR-CP recovers in a median of 50 hands, RNR(0.5) in 400 (30 % of matches), DirBR in 850 (45 %), and TypeBR, whose posterior has absorbed 1,000 hands of TightPassive, never. The change-point exploiter shows why one game is not enough: on Kuhn it recovers fastest and loses least to teaching among the best-response agents; on Leduc its detector fires on *stationary* opponents, it captures only 0.34 of the attainable gain, and it loses 1.76 chips/hand to teaching against DirBR’s 1.24.

14.6 Approximate best responses and the RRPS score

In games too large to traverse, exploitability is estimated by learning a best response, which gives a lower bound¹⁸⁶¹⁹⁸; Kuhn and Leduc allow an exact check.

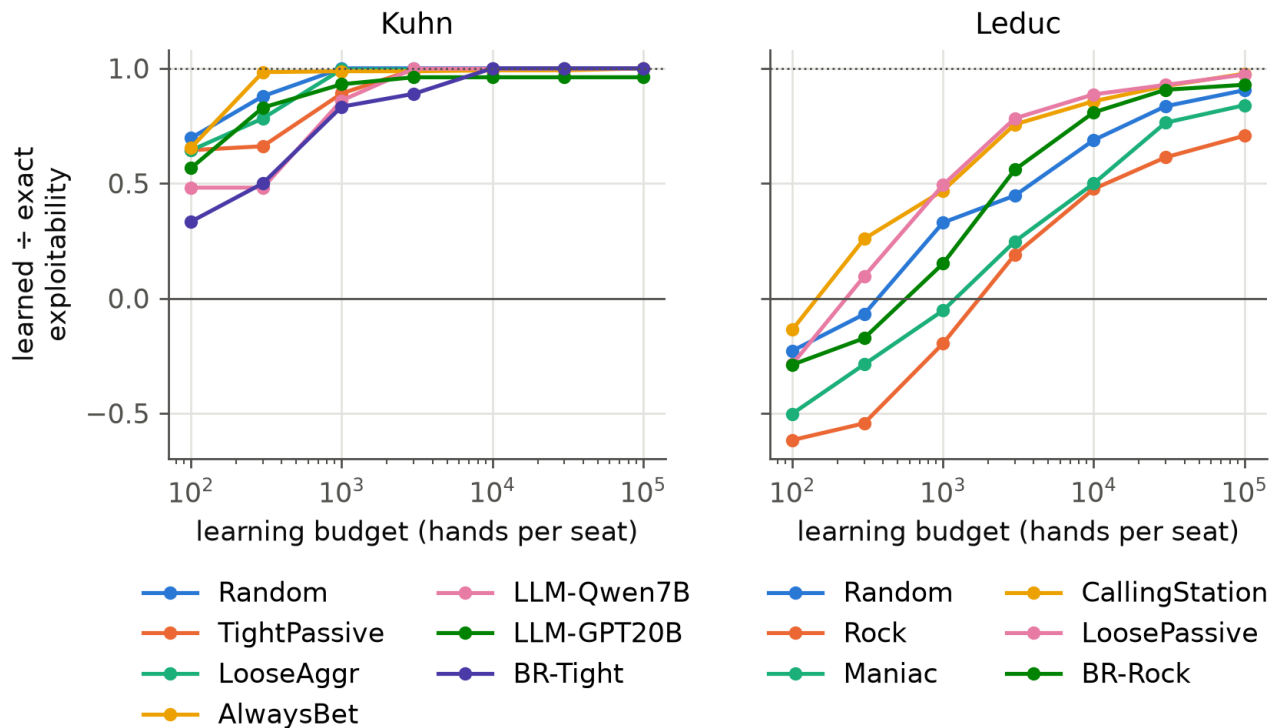


Figure 92: Exploitability found by a learned best response (tabular Monte-Carlo control, $\epsilon = 0.1$) divided by the exact exploitability, as the learning budget grows; mean over 3 seeds and both seats.

On Kuhn the learner comes within 4 % of the exact value by 10⁴ hands for every target; on Leduc it reaches 71 % (Rock) to 98 % (CallingStation) after 10⁵ hands per seat, and after 10³ hands it *loses* to Rock and Maniac, so a budget-limited evaluator would report both as safe. Against the adaptive agents the gap is

¹⁹⁸Lisý, V. & Bowling, M. (2017). “Equilibrium Approximation Quality of Current No-Limit Poker Bots.” *AAAI-17 Workshop on Computer Poker and Imperfect Information Games*; arXiv:1612.07547 — local best response, a lower bound on exploitability.

starker: the same learner, playing a 2,000-hand match online, inflicts no damage on Leduc (the agents earn 0.35–0.86 chips/hand above the game value), while the white-box attack takes 1.0–1.5 from DirBR. The RRPS benchmark’s *within-population* exploitability has the same blind spot: 0.279 chips/hand for DirBR on Leduc against an exposure of 2.461, ranking DirBR second of twelve. Adversarial policies against superhuman Go programs are the large-scale version¹⁹⁹. An exploiter’s number means little without its budget and strength.

14.7 Three and four players

With three players the protocol changes in one place, the worst case. Five approximate equilibria of three-player Kuhn (CFR, CFR+, MCCFR with three seeds) all have near-zero NashConv yet are not interchangeable: utility moves between seats inside the equilibrium set, as Szafron et al. describe¹⁸⁴, and profiles mixing components from different solvers reach a NashConv of 0.125. Worse, an equilibrium component guarantees nothing against a pair. Its coalition value is computed exactly by enumerating one member’s 2^{16} pure strategies and best-responding with the other. For the CFR and CFR+ equilibria it ranges from -0.094 to -0.125 chips/hand, against equilibrium values of -0.029 to $+0.050$ (nplayer_kuhn3.json).

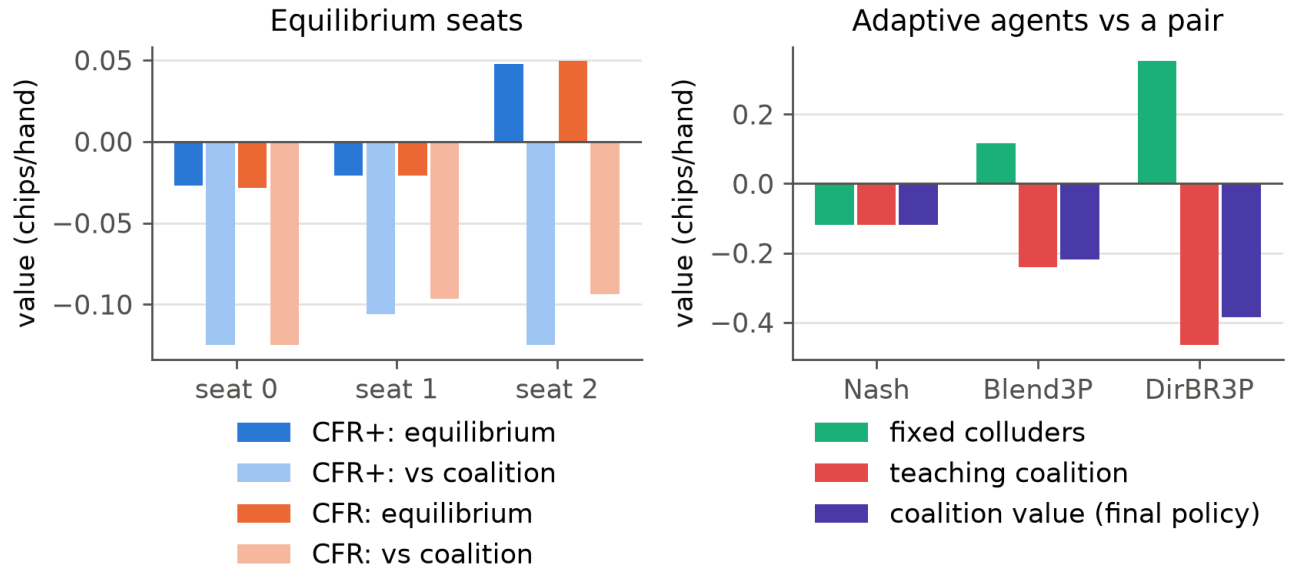


Figure 93: Left: each seat’s equilibrium value and coalition value in the CFR+ and CFR equilibria. Right: Nash, Blend3P and DirBR3P against a fixed colluding pair, under a coalition teaching attack, and their final coalition value.

The three-player adaptive agent DirBR3P extends Chapter 7’s continuous model to two opponents and best-responds to the modelled pair. Against independent sub-optimal pairs it gains $+0.372 \pm 0.038$

¹⁹⁹Wang, T. T., Gleave, A., Tseng, T., Pelrine, K., Belrose, N., Miller, J., Dennis, M. D., Duan, Y., Pogrebniak, V., Levine, S. & Russell, S. (2023). “Adversarial Policies Beat Superhuman Go AIs.” *ICML*, PMLR 202, 35655–35739.

chips/hand over the blueprint (capture 0.99 within 50 hands), and population return, a win-share Elo, α -Rank at $\alpha = 0.1$ and Nash averaging all put it first. The coalition-aware readout disagrees. Against two opponents that bait until hand 1,000 and then play the exact coalition best response to its current policy, DirBR3P earns -0.466 chips/hand, four times the blueprint’s -0.119 ; the coalition value of its final policy is -0.386 . Blending with the blueprint (Blend3P) roughly halves both the gain and the teaching loss ($+0.213$; -0.242). A fixed colluding pair aimed at the blueprint costs the blueprint 0.119 per hand but is itself exploitable: DirBR3P earns $+0.354$ against it. Adaptation is a defence against a *static* coalition and a liability against an adaptive one. No ranking built from independently seated opponents can tell these apart, which is failure mode 8 in the gap analysis.

So Long Sucker supports only the population layer: the pairwise projection is 0.955 transitive and Nash averaging picks the betrayer, but a planted alliance lowers a focal baseline’s win rate by only 0.8–1.9 percentage points, since about 99.5 % of random games end in deadlock (Chapter 11).

14.8 Where existing evaluation breaks — a checklist

Table 40: The failure modes of `lit_evaluation.md` (point 6, generalisation suites, concerns cooperative substrates and is not tested here) and what this chapter measured.

Failure mode (gap analysis)	Evidence in this chapter	What the protocol reports instead
1. Exploitability only punishes adaptation	BestEq ties Nash for first on exploitability with 0.02–0.05 gain	gain and exposure as a pair
2. No N-player meaning of exploitability	NashConv ≈ 0 components lose 0.06–0.17 more to a pair	the exact coalition value
3. Approximate BRs are lower bounds	learner negative after 10^3 hands; online learner finds nothing	exact exposure; exploiter budget stated
4. Rankings over- or under-credit exploitation	clones, α , horizon change the winner	rankings reported as context
5. Variance compounds with adaptation	3,383 vs 379 hands per window; Pluribus ± 173	estimator named; AIVAT where possible
7. Deceptive opponents untested	teaching attack takes 1.0–2.2 chips/hand on Leduc	teaching loss
8. Collusion evaluated apart from play	DirBR3P first in return, Elo and Nash averaging; -0.47 under a coalition	coalition teaching attack
9. LLM arenas inherit the above	Elo ranks the Qwen strategy above a less exploitable bot	the same readouts for LLM agents

14.9 Honest notes, limitations, and where this hands off

What did not hold, and why it matters. Every exact quantity matches an independent reference, but three plan predictions failed and are kept as stated: “random loses to everyone” (on Kuhn it beats AlwaysPass); “AIVAT $\geq 10\times$ on Leduc” (7 of 12 pairs); “approximate within 10 % on Leduc” (4 of 6 targets, only at 10^5 hands). Two of my own implementation errors were caught by checks, not by luck: the AIVAT double count, and a Nash averaging solver that returned infeasible “optimal” points on near-tied agents until a fixed tolerance and support LPs replaced it.

Limitations. These are toy games. The exact worst case and the exact coalition value (2^{16} strategies per member) exist only because the games are tiny; at scale a learned attacker gives only a lower bound on either. The policy-exact estimator exists only for bots. The white-box teaching attacker is a strong adversary but not the worst adaptive one. The adaptive agents are Chapter 7–8 baselines: the protocol *measures* the missing N-player loss bound; it does not provide one. So Long Sucker is weak evidence.

Connections. The engine absorbs Chapters 3, 7, 8 (now as one LP), 10, 11 and 12. For Contribution 2 it supplies the measurement a safe N-player agent must pass — gain against independent pairs, coalition value and coalition teaching loss — and the one-shot LP removes the scaling obstacle Chapter 8 met.

14.10 Key takeaways for the thesis synthesis

- **Exploitability and rankings give opposite answers for exploiting agents.** On the same Leduc zoo, BestEq and Nash lead exploitability and Nash averaging; RNR(0.5) and DirBR lead Elo, population return and the RRPS score. The Elo–exploitability rank correlation is 0.36.
- **Gain and exposure reported together separate what single numbers conflate.** On Leduc, BestEq gains 0.047 at zero exposure, RNR(0.5) +0.797 at 0.264, DirBR +0.988 at 2.461.
- **With three players the worst case must be coalition-aware.** Equilibrium components (Nash-Conv ≈ 0) lose 0.06–0.17 chips/hand more to a coordinated pair; DirBR3P tops the population views and loses 0.47 to a coalition teaching attack.
- **Confidence is the binding constraint on measuring adaptation.** A per-window estimate needs about 3,400 hands with raw chips against about 380 with AIVAT on Kuhn; on Pluribus’s released hands the raw 95 % interval is ± 173 mbb/hand.
- **Reported exploiters need a stated budget.** A learned best response reports two exploitable Leduc bots as safe after 10^3 hands, and within-population exploitability misses DirBR’s exposure by a factor of nine.
- **The C3 claim stays narrow.** The pieces exist; this chapter adds their joint, unchanged application across two- and three-player imperfect-information games and shows where each existing measure breaks — so far on toy games only.